USER DOCUMENTATION

PROGRAM REFERENCE & USER GUIDE

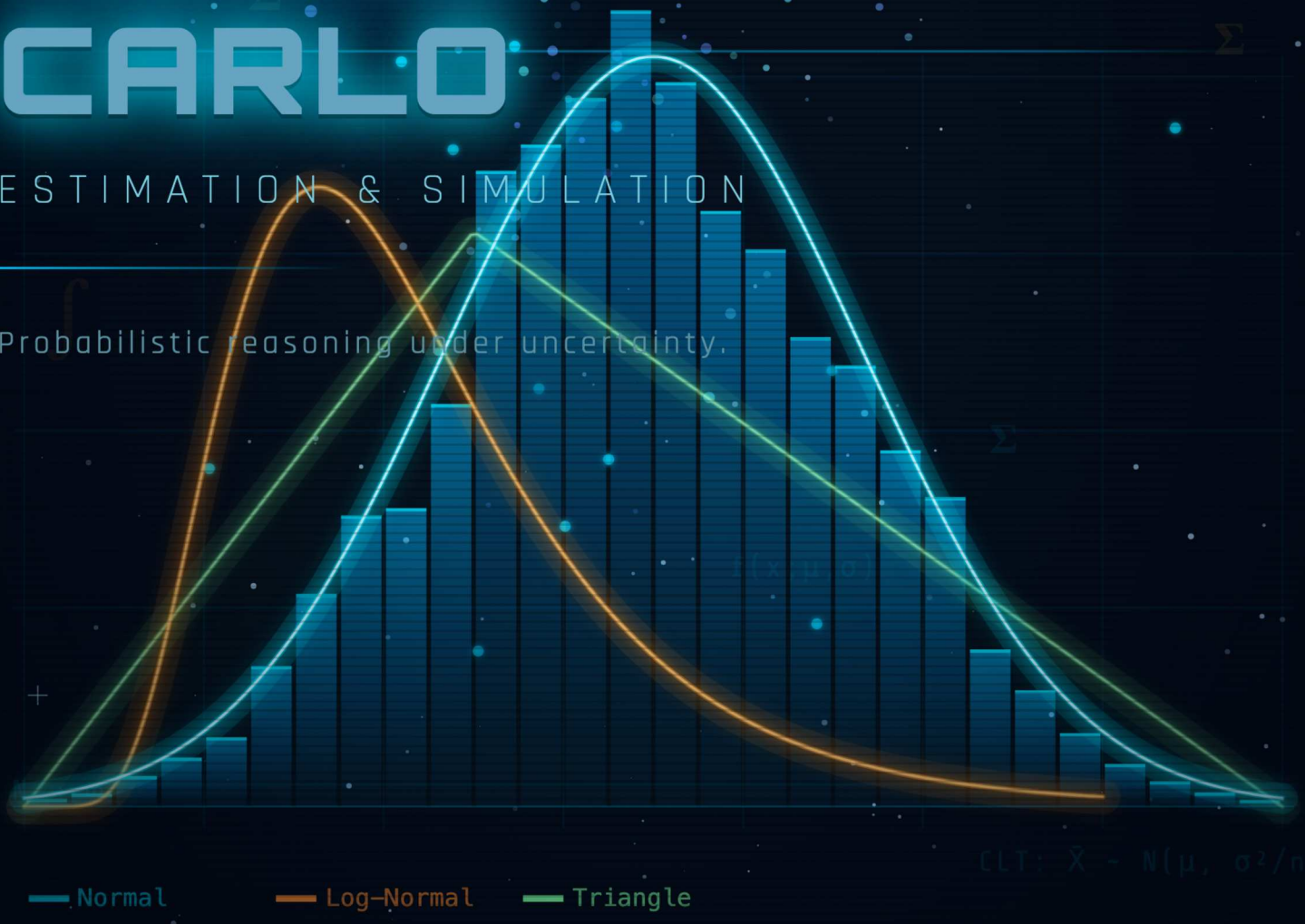
MCProjSim

USER GUIDE

MONTE CARLO

ESTIMATION & SIMULATION

Probabilistic reasoning under uncertainty.



JOHAN PERSSON

FIRST EDITION
2026

MCProjSim User Guide
Program Version: 0.15.2
Copyright © Johan Persson
Generated on May 5, 2026
Prepared with LaTeX from Markdown sources for duplex printing. Sources for documentation available at `https://github.com/johan162/mcprojsim/docs`.

User Guide

`mcprojsim`, Version: 0.15.2

Monte Carlo Project Simulation

Johan Persson <johan162@gmail.com>

Generated on May 5, 2026

Chapters

1	GETTING STARTED	1
2	INTRODUCTION	9
3	YOUR FIRST PROJECT	19
4	UNCERTAINTY FACTORS	37
5	TASK ESTIMATION	43
6	RISKS	63
7	MONETARY COST ESTIMATION	75
8	PROJECT FILES	87
9	SPRINT PLANNING	131
10	CONSTRAINED SCHEDULING	163
11	MULTI-PHASE (TWO-PASS) SIMULATION	179
12	RUNNING SIMULATIONS	189
13	INTERPRETING RESULTS	199
14	CONFIGURATION	219
15	NATURAL LANGUAGE INPUT	237
16	MCP SERVER & NL INPUT	247
17	EXAMPLES	265

Contents

1 Getting Started **1**
- 1.1 What you'll learn . . . 1
- 1.2 Before you start . . . 1
  - 1.2.1 Install Python . . . 1
- 1.3 Install mcprojsim . . . 2
  - 1.3.1 With pipx (recommended) . . . 2
  - 1.3.2 With pip (if you know what you are doing!) . . . 2
  - 1.3.3 Run mcprojsim using a Docker image from `ghcr.io` . . . 2
- 1.4 Verify the installation: . . . 3
- 1.5 Creating the project specification file . . . 3
- 1.6 Generating a YAML project file . . . 4
- 1.7 Validate the file . . . 5
- 1.8 Run your first simulation . . . 5
- 1.9 Export results . . . 6
- 1.10 What the main results mean . . . 6
- 1.11 Useful next commands . . . 7
- 1.12 Where to go next . . . 7
- 1.13 Try these example projects . . . 8

2 Introduction **9**
- 2.1 A brief history of Monte Carlo methods . . . 9
- 2.2 Why software schedule estimates are difficult . . . 9
  - 2.2.1 The problem with single-point estimates . . . 10
- 2.3 How Monte Carlo compares to other estimation approaches . . . 10
- 2.4 From single numbers to structured uncertainty . . . 10
- 2.5 How Monte Carlo simulation uses these ranges . . . 11
- 2.6 From many runs to a probability distribution . . . 11
  - 2.6.1 Common percentiles and what they mean . . . 11
- 2.7 Tasks, dependencies, and project structure . . . 11
- 2.8 The critical path . . . 12
- 2.9 Two sources of schedule variation . . . 12
  - 2.9.1 Uncertainty factors . . . 12
  - 2.9.2 Risks . . . 13
- 2.10 Effort estimation versus calendar scheduling . . . 13
- 2.11 Constrained scheduling: from ideal plans to real-world schedules . . . 14
  - 2.11.1 Resources and capacity . . . 14
  - 2.11.2 Working calendars . . . 14
  - 2.11.3 Planned and unplanned absence . . . 14
  - 2.11.4 Why constrained scheduling changes the picture . . . 15
- 2.12 What the simulation computes, step by step . . . 15
- 2.13 Understanding the results . . . 15

2.13.1 Statistical summary . . . 15
2.13.2 Histograms . . . 15
2.13.3 Sensitivity and criticality . . . 16
2.13.4 Staffing recommendations . . . 16
2.13.5 Reproducibility . . . 16
2.14 How the input files reflect the model . . . 16
2.14.1 Multiple ways to express estimates . . . 16
2.15 When Monte Carlo estimation is most valuable . . . 17
2.16 What comes next in this guide . . . 17

3 Your first project **19**
3.1 Chapter outline . . . 19
3.2 Quick command and flag map . . . 19
3.2.1 Commands at a glance . . . 19
3.2.2 Common `simulate` flags . . . 20
3.3 The smallest possible project file . . . 20
3.3.1 Step 1: one task, one estimate . . . 20
3.4 Add structure: a second task and a dependency . . . 21
3.4.1 Step 2: sequence the work . . . 22
3.4.2 Understanding the critical path output . . . 24
3.5 Add organizational knowledge with a configuration file . . . 24
3.5.1 Step 3: add uncertainty factors . . . 24
3.6 Add explicit risk events . . . 27
3.6.1 Step 4: add project-level and task-level risks . . . 28
3.7 An alternative style: T-shirt sized estimates . . . 29
3.7.1 Step 5: use `t_shirt_size` instead of explicit ranges . . . 29
3.7.2 Categories of T-shirt sizes . . . 30
3.7.3 How T-shirt sizes are configured . . . 31
3.8 Another alternative style: Story Point estimates . . . 32
3.8.1 Step 6: use `story_points` . . . 32
3.8.2 Default Story Point . . . 33
3.8.3 How Story Points are configured . . . 33
3.9 What each stage added . . . 35
3.10 A practical workflow for real projects . . . 36
3.11 How this chapter connects to the shipped examples . . . 36
3.12 What to try next . . . 36

4 Uncertainty Factors **37**
4.1 Supported Uncertainty Factors . . . 37
4.1.1 Team Experience . . . 37
4.1.2 Requirements Maturity . . . 37
4.1.3 Technical Complexity . . . 38
4.1.4 Team Distribution . . . 38
4.1.5 Integration Complexity . . . 38
4.2 How the Combined Multiplier Is Calculated . . . 38
4.3 Validation and Fallback Behavior . . . 39
4.4 Default Values . . . 39
4.5 Defining Uncertainty Factors in the Project File . . . 40
4.6 Configuring Uncertainty Factor Multipliers . . . 40
4.6.1 Default Configuration . . . 41
4.7 Customizing Multipliers . . . 41
4.8 Recommended Authoring Pattern . . . 41
4.9 Summary . . . 41

5 Task Estimation **43**
5.1 How estimation works in the simulation . . . 43
5.2 Estimation methods at a glance . . . 43

5.3 Explicit range estimates . . . 44
5.3.1 Basic example . . . 44
5.3.2 Choosing good range values . . . 44
5.3.3 Asymmetric ranges . . . 45
5.4 Near-deterministic estimates . . . 45
5.5 Probability distributions . . . 45
5.5.1 Triangular distribution (default) . . . 45
5.5.2 Distribution precedence . . . 46
5.5.3 Log-normal distribution . . . 47
5.5.4 Configuring the log-normal high percentile . . . 48
5.5.5 Comparing the two distributions . . . 49
5.5.6 Why is log-normal a good model for software (or project) estimation? . . . 49
5.5.7 Determining log-normal parameters . . . 49
5.5.8 Deriving the formula to translate `low`, `expected`, `high` to log-normal parameters . . . 52
5.6 T-shirt size estimates . . . 52
5.6.1 Default T-shirt size mappings . . . 52
5.6.2 Specifying a T-shirt size estimate . . . 54
5.6.3 Full example: project using T-shirt sizes . . . 55
5.6.4 Customizing T-shirt sizes in the configuration file . . . 55
5.6.5 Choosing the unit for T-shirt sizes . . . 55
5.7 Story point estimates . . . 56
5.7.1 Default story point mappings . . . 56
5.7.2 Specifying a story point estimate . . . 57
5.7.3 Full example: project using story points . . . 57
5.7.4 Customizing story point mappings in the configuration file . . . 57
5.7.5 Choosing the unit for story points . . . 58
5.8 Mixing estimation methods . . . 58
5.9 The `unit` field . . . 59
5.9.1 Supported values . . . 59
5.9.2 Defaults by estimate type . . . 59
5.9.3 The `hours_per_day` setting . . . 60
5.9.4 Unit conversion is automatic . . . 60
5.9.5 Simulation output . . . 60
5.9.6 Unit field for T-shirt sizes and story points . . . 60
5.10 Validation rules . . . 61
5.11 Summary . . . 61

6 Risks **63**
6.1 How Risks Affect the Simulation . . . 63
6.1.1 The Order of Operations . . . 63
6.2 Risk Definitions . . . 63
6.2.1 Specifying Impact . . . 64
6.2.2 Summary of Impact Types . . . 64
6.3 Task-Level Risks . . . 65
6.3.1 When to Use Task-Level Risks . . . 65
6.3.2 Cumulative Evaluation . . . 65
6.3.3 Full Task Example with Risks . . . 66
6.4 Project-Level Risks . . . 66
6.4.1 When to Use Project-Level Risks . . . 66
6.4.2 Percentage vs. Absolute Impact . . . 66
6.4.3 How Project-Level Risks Interact with the Critical Path . . . 67
6.5 Combining Risks at Both Levels . . . 67
6.5.1 Complete Project Example with Both Risk Types . . . 68
6.6 Tasks Without Risks . . . 69
6.7 Examples . . . 69
6.7.1 Example 1: Project Without Uncertainty Factors or Risks . . . 69
6.7.2 Example 2: Same Project With Uncertainty Factors and Risks . . . 70

6.8 Practical Tips for Defining Risks . . . 72
6.9 Summary . . . 73

7 Monetary Cost Estimation 75
7.1 Activating cost estimation . . . 75
7.2 Project-level cost fields . . . 75
7.2.1 `default_hourly_rate` . . . 75
7.2.2 `overhead_rate` . . . 76
7.2.3 `currency` . . . 76
7.3 Per-resource hourly rates . . . 76
7.4 Per-task fixed costs . . . 76
7.5 Risk cost impacts . . . 76
7.5.1 Project-level risks . . . 77
7.6 Full project example . . . 78
7.7 Running a simulation with cost . . . 78
7.8 Reading cost output . . . 79
7.8.1 Table mode (default) . . . 79
7.8.2 Plain CLI mode . . . 79
7.8.3 Cost sensitivity . . . 79
7.9 Budget confidence analysis . . . 79
7.9.1 `--target-budget` . . . 79
7.9.2 Programmatic access (Python API) . . . 79
7.10 Cost in exported reports . . . 79
7.10.1 JSON (`-f json`) . . . 79
7.10.2 CSV (`-f csv`) . . . 80
7.10.3 HTML (`-f html`) . . . 80
7.11 Configuration defaults . . . 80
7.12 Natural-language input . . . 80
7.12.1 Structured risk format . . . 81
7.12.2 Prose risk format . . . 81
7.12.3 Full example . . . 81
7.13 Secondary currencies and FX conversion . . . 82
7.13.1 How it works . . . 82
7.13.2 Adjusted-rate formula . . . 82
7.13.3 Project file fields . . . 82
7.13.4 Manual rate overrides . . . 83
7.13.5 Suppressing FX at run time . . . 83
7.13.6 Console output . . . 83
7.13.7 FX data in exported reports . . . 83
7.13.8 Rate caching . . . 84
7.13.9 Full example with secondary currencies . . . 85
7.14 Known limitations . . . 85
7.15 Example files . . . 85

8 Project files 87
8.1 Quick reference for new users . . . 87
8.2 Creating project files . . . 88
8.2.1 Using `mcprojsim generate` . . . 88
8.3 Supported file formats . . . 88
8.4 Top-level structure . . . 89
8.4.1 Top-level YAML skeleton . . . 89
8.4.2 Top-level TOML skeleton . . . 90
8.5 Formal section order . . . 90
8.6 The `project` section . . . 90
8.6.1 Supported fields . . . 90
8.6.2 Required constraints . . . 91
8.6.3 YAML example . . . 91

8.6.4 TOML example . . . 91
8.7 The `tasks` section . . . 91
8.7.1 Supported task fields . . . 92
8.7.2 Minimal task example . . . 93
8.7.3 TOML task example . . . 93
8.8 The `estimate` section . . . 93
8.8.1 Field name aliases . . . 93
8.8.2 1. Triangular estimate . . . 94
8.8.3 2. Log-normal estimate . . . 94
8.8.4 3. T-shirt-size estimate . . . 95
8.8.5 4. Story Point estimate . . . 96
8.8.6 Symbolic estimate mappings in configuration . . . 97
8.9 The `dependencies` field . . . 97
8.9.1 Supported syntax . . . 97
8.9.2 Validation rules . . . 98
8.9.3 Example . . . 98
8.10 The `uncertainty_factors` field . . . 98
8.10.1 Recognized fields . . . 98
8.10.2 YAML example . . . 98
8.10.3 TOML example . . . 99
8.10.4 Important implementation note . . . 99
8.11 The `resources` field inside a task . . . 99
8.11.1 Resource assignment rule with `max_resources` . . . 99
8.12 The `risks` field inside a task . . . 99
8.12.1 Task-level risk object fields . . . 100
8.12.2 Risk impact syntax . . . 100
8.12.3 Validation rules for risks . . . 101
8.13 The `project_risks` section . . . 101
8.13.1 Example . . . 101
8.14 Cost estimation . . . 101
8.14.1 Cost components . . . 102
8.14.2 Project-level cost fields . . . 102
8.14.3 Task-level cost field . . . 103
8.14.4 Risk `cost_impact` field . . . 104
8.14.5 Resource-level cost field . . . 105
8.14.6 Configuration file cost settings . . . 105
8.14.7 Complete cost example . . . 106
8.14.8 Validation rules for cost fields . . . 108
8.15 The top-level `resources` section . . . 108
8.15.1 Supported fields . . . 108
8.15.2 YAML example . . . 109
8.16 The top-level `calendars` section . . . 109
8.16.1 YAML example . . . 109
8.17 The `sprint_planning` section . . . 109
8.17.1 Top-level sprint planning fields . . . 110
8.17.2 Minimal example . . . 111
8.17.3 Sprint history . . . 111
8.17.4 `future_sprint_overrides` . . . 114
8.17.5 Task fields used by sprint planning . . . 114
8.17.6 Configuration interaction . . . 114
8.18 Full YAML example . . . 115
8.19 Full TOML example . . . 117
8.20 Configuration file reference . . . 119
8.20.1 How configuration loading works . . . 119
8.20.2 Top-level configuration structure . . . 119
8.20.3 Minimal configuration example . . . 119
8.20.4 Full configuration skeleton . . . 120

8.21 The `uncertainty_factors` section . . . 121
8.21.1 YAML structure . . . 121
8.21.2 Supported fields . . . 121
8.21.3 Built-in level names by factor . . . 121
8.21.4 Built-in factor names . . . 121
8.21.5 Built-in defaults . . . 121
8.22 The `t_shirt_sizes` section . . . 122
8.22.1 Supported fields for each category/size entry . . . 122
8.22.2 Built-in category keys . . . 122
8.22.3 Built-in size keys (per category) . . . 122
8.22.4 Built-in `story` defaults . . . 122
8.22.5 Example override . . . 123
8.23 The `t_shirt_size_unit` field . . . 123
8.23.1 Supported values . . . 123
8.23.2 Default . . . 123
8.23.3 Example . . . 123
8.24 The `story_points` section . . . 123
8.24.1 Supported fields for each point value . . . 123
8.24.2 Built-in point values . . . 123
8.24.3 Built-in defaults . . . 124
8.24.4 Example override . . . 124
8.25 The `story_point_unit` field . . . 124
8.25.1 Supported values . . . 124
8.25.2 Default . . . 124
8.26 The `lognormal` section . . . 124
8.26.1 Supported fields . . . 124
8.26.2 Example . . . 124
8.27 The `simulation` section . . . 125
8.27.1 Supported fields . . . 125
8.27.2 Example . . . 125
8.28 The `output` section . . . 125
8.28.1 Supported fields . . . 125
8.28.2 Example . . . 126
8.29 The `staffing` section . . . 126
8.29.1 Supported fields . . . 126
8.29.2 How `min_individual_productivity` affects team-size efficiency . . . 126
8.29.3 Practical interpretation . . . 127
8.29.4 The `experience_profiles` subsection . . . 127
8.29.5 Built-in profile defaults . . . 128
8.29.6 Example . . . 128
8.29.7 Configuration validation summary . . . 128
8.29.8 Recommended authoring style for configuration files . . . 128
8.30 Validation summary . . . 128
8.31 Notes on undocumented keys . . . 129
8.32 Recommended authoring style . . . 129

9 Sprint planning 131
9.1 What sprint planning models . . . 131
9.2 What input is required . . . 132
9.3 Sprint setting precedence . . . 132
9.4 Command workflow . . . 133
9.5 Build up the file gradually . . . 133
9.5.1 Step 1: start from a normal project file . . . 133
9.5.2 Step 2: add the smallest useful `sprint_planning` section . . . 134
9.5.3 Step 3: add richer historical signals . . . 135
9.5.4 Step 4: add forward-looking adjustments . . . 135
9.6 Built on historical real data . . . 136

9.7 All sprint-planning fields . . . 137
9.7.1 Task-level fields used by sprint planning . . . 137
9.7.2 `sprint_planning` fields . . . 137
9.7.3 `sprint_planning.history[]` fields . . . 138
9.7.4 External `sprint_planning.history` descriptor fields . . . 138
9.7.5 `future_sprint_overrides[]` fields . . . 139
9.7.6 `volatility_overlay` fields . . . 139
9.7.7 `spillover` fields . . . 140
9.7.8 `spillover.size_brackets[]` fields . . . 145
9.7.9 `sickness` fields . . . 145
9.8 How the simulation uses these fields . . . 145
9.9 Statistical methods in detail . . . 146
9.9.1 Step 1: normalize the historical data . . . 146
9.9.2 Step 2: build the sampling distribution from history . . . 146
9.9.3 Step 2b: Negative Binomial velocity model (optional) . . . 147
9.9.4 Step 3: cadence matching and weekly fallback . . . 148
9.9.5 Which distributions are used for which quantities . . . 148
9.9.6 Detailed interpretation by entity . . . 149
9.9.7 What is estimated from history, and what is configured directly . . . 153
9.9.8 Percentiles and final forecast distribution . . . 153
9.9.9 Burn-up percentiles . . . 153
9.9.10 Commitment guidance heuristic . . . 154
9.10 Natural-language generation . . . 154
9.10.1 Example 1: story-point-based sprint planning . . . 155
9.10.2 Example 2: task-count throughput mode . . . 156
9.10.3 Example 3: include holiday effects in the text description . . . 157
9.11 Using external CSV or JSON history files . . . 157
9.11.1 JSON example . . . 158
9.11.2 CSV example . . . 158
9.12 Typical command output . . . 158
9.12.1 Validation output . . . 158
9.12.2 Minimal sprint-planning simulation output . . . 159
9.12.3 Richer table output . . . 159
9.12.4 Extended output (non-minimal) . . . 159
9.12.5 HTML and JSON report visibility . . . 160

10 Constrained Scheduling 163
10.1 What constrained scheduling changes . . . 163
10.2 Suggested reading path . . . 163
10.3 How `team_size` affects scheduling (reference) . . . 164
10.3.1 Example A: tasks only (no `team_size`) . . . 164
10.3.2 Example B: same tasks, with `team_size` . . . 164
10.3.3 Example C: explicit `resources` + `team_size` (smaller than resources) . . . 165
10.3.4 Example D: explicit `resources` + `team_size` (larger than resources) . . . 165
10.3.5 Interpretation and policy . . . 165
10.4 Example 1: Baseline (dependency-only) . . . 165
10.5 Resource fields introduced in this chapter . . . 166
10.5.1 First two fields introduced in Example 2 . . . 167
10.6 Task fields related to resources introduced in this chapter . . . 167
10.6.1 `max_resources` semantics (important) . . . 167
10.6.2 How the `16.0` and `3` factors shape assignment . . . 168
10.6.3 How this interacts with actual assignment . . . 168
10.6.4 Assignment timing model . . . 168
10.6.5 Start-now vs wait-for-more-resources behavior . . . 168
10.7 Example 2: Add resources (single-pass automatic assignment) . . . 169
10.7.1 How resources are used in Example 2 . . . 169
10.7.2 Why `experience_level` and `productivity_level` matter here . . . 170

10.7.3 Making assignment explicit (optional) . . . 170
10.8 Example 3: Add working calendars . . . 171
10.8.1 Quick comparison (Examples 1 → 3) . . . 172
10.9 Example 4: Add sickness and planned absence . . . 172
10.9.1 Optional config for default sickness probability . . . 173
10.9.2 Optional config for sickness duration . . . 173
10.10 Example 5: Add task-level resource constraints . . . 173
10.10.1 Compact recap: how assignment works in Example 5 . . . 173
10.11 Example 6: Full constrained project (final build-up) . . . 174
10.12 Single-pass vs two-pass assignment . . . 176
10.12.1 Single-pass automatic assignment (available) . . . 176
10.12.2 Two-pass criticality assignment (available) . . . 176
10.13 CLI options most relevant to constrained runs . . . 176
10.14 Where to configure constrained behavior . . . 177
10.14.1 In the project file (primary for constrained scheduling) . . . 177
10.14.2 In the config file (`--config`) . . . 177
10.15 Related chapters . . . 178

11 Multi-Phase (Two-Pass) Simulation 179
11.1 How It Works . . . 180
11.2 Basic Example . . . 180
11.2.1 Project file . . . 180
11.2.2 Single-pass run . . . 181
11.2.3 Two-pass run . . . 182
11.3 When Is Two-Pass Useful? . . . 183
11.4 Configuration Reference . . . 184
11.4.1 CLI flags . . . 184
11.4.2 Configuration file . . . 184
11.5 More Complex Example: Developer Allocation . . . 184
11.6 Output Formats . . . 186
11.6.1 Console (`-t` table mode) . . . 186
11.6.2 JSON export (`-f json`) . . . 186
11.6.3 CSV export (`-f csv`) . . . 187
11.6.4 HTML export (`-f html`) . . . 187
11.7 Choosing `pass1_iterations` . . . 187
11.8 Interpreting the Delta . . . 188
11.9 Related chapters . . . 188

12 Running simulations 189
12.1 Command overview . . . 189
12.2 `mcprojsim generate` . . . 189
12.2.1 Usage . . . 189
12.2.2 Options . . . 189
12.2.3 Examples . . . 190
12.2.4 Input format . . . 190
12.3 `mcprojsim validate` . . . 190
12.3.1 Usage . . . 190
12.3.2 Options . . . 190
12.3.3 Examples . . . 190
12.4 `mcprojsim simulate` . . . 191
12.4.1 Usage . . . 191
12.4.2 Options . . . 191
12.4.3 Examples . . . 193
12.4.4 Output . . . 194
12.5 `mcprojsim config` . . . 196
12.5.1 Usage . . . 196
12.5.2 Options . . . 196

12.5.3 Examples . . . 196
12.6 Parallel simulation . . . 196
12.6.1 How it works . . . 196
12.6.2 Reproducibility . . . 197
12.6.3 Performance expectations . . . 197
12.6.4 Usage . . . 197

13 Interpreting Results **199**
13.1 The output at a glance . . . 199
13.2 Output modes and flags that change what you see . . . 201
13.3 Project overview fields . . . 201
13.4 Understanding percentiles . . . 201
13.4.1 Choosing the right percentile . . . 202
13.4.2 Working days and delivery dates . . . 202
13.5 Effort confidence intervals . . . 202
13.5.1 What is total effort? . . . 202
13.5.2 Why effort varies across iterations . . . 202
13.5.3 Effort vs. calendar time . . . 203
13.5.4 Practical use . . . 203
13.6 Minimum and maximum . . . 203
13.7 The thermometer chart . . . 203
13.8 Mean vs. median . . . 203
13.9 Standard deviation and coefficient of variation . . . 204
13.9.1 Definition . . . 204
13.9.2 Interpretation . . . 204
13.9.3 What to watch out for . . . 204
13.9.4 Practical use . . . 204
13.10 Skewness . . . 204
13.10.1 Definition . . . 205
13.10.2 Interpretation . . . 205
13.10.3 What to watch out for . . . 205
13.10.4 Practical use . . . 205
13.11 Excess kurtosis . . . 205
13.11.1 Definition . . . 205
13.11.2 Interpretation . . . 206
13.11.3 What to watch out for . . . 206
13.11.4 Practical use . . . 206
13.12 Sensitivity analysis . . . 206
13.12.1 Definition . . . 206
13.12.2 Interpretation . . . 206
13.12.3 What to watch out for . . . 207
13.12.4 Practical use . . . 207
13.12.5 Reading the tornado chart . . . 207
13.13 Schedule slack . . . 208
13.13.1 Definition . . . 208
13.13.2 Interpretation . . . 208
13.13.3 What to watch out for . . . 208
13.13.4 Practical use . . . 208
13.14 Risk impact analysis . . . 208
13.14.1 Reported metrics . . . 208
13.14.2 Interpretation . . . 209
13.14.3 What to watch out for . . . 209
13.14.4 Practical use . . . 209
13.15 Critical path analysis . . . 209
13.15.1 What is the critical path? . . . 209
13.15.2 Why Monte Carlo gives multiple critical paths . . . 209
13.15.3 How to use criticality . . . 210

13.16 Max parallel tasks . . . 210
13.16.1 How it is calculated . . . 210
13.16.2 Interpretation . . . 210
13.16.3 Why this matters . . . 210
13.17 Constrained schedule diagnostics . . . 211
13.18 Two-pass scheduling traceability . . . 211
13.19 Target-date probability . . . 211
13.20 Staffing recommendations . . . 211
13.20.1 The advisory line . . . 211
13.20.2 The `--staffing` flag . . . 212
13.20.3 Communication overhead model (Brooks's Law) . . . 212
13.20.4 Understanding the table columns . . . 213
13.20.5 Experience profiles . . . 213
13.20.6 Choosing an effort basis . . . 213
13.20.7 How efficiency is calculated . . . 214
13.20.8 Interpreting the profile tables together . . . 214
13.20.9 Caveats . . . 215
13.21 Combining the analyses . . . 215
13.22 Reducing volatility . . . 215
13.23 Limitations of the simulation . . . 216
13.23.1 What the simulation does well . . . 216
13.23.2 What the simulation does not do . . . 216
13.23.3 Calibration matters . . . 216
13.24 Quick reference . . . 216
13.25 Sprint-planning mode outputs (when enabled) . . . 217

14 Configuration **219**
14.1 Configuration file format . . . 219
14.2 How configuration loading works . . . 219
14.3 Top-level configuration structure . . . 219
14.4 Full example . . . 221
14.5 Uncertainty factors . . . 222
14.5.1 Defaults . . . 222
14.5.2 Notes . . . 222
14.6 Symbolic estimate mappings . . . 222
14.6.1 T-shirt sizes . . . 223
14.6.2 `t_shirt_size_default_category` . . . 224
14.7 Shifted log-normal configuration . . . 224
14.7.1 `t_shirt_size_unit` . . . 225
14.7.2 Story Points . . . 225
14.7.3 `story_point_unit` . . . 225
14.7.4 Important note . . . 226
14.8 Simulation settings . . . 226
14.8.1 Critical path reporting . . . 226
14.9 Output settings . . . 227
14.9.1 Histogram bins . . . 227
14.10 Staffing settings . . . 227
14.10.1 `effort_percentile` . . . 228
14.10.2 `min_individual_productivity` . . . 228
14.10.3 `experience_profiles` . . . 228
14.11 Sprint planning defaults . . . 229
14.11.1 Field reference . . . 229
14.11.2 Notes on behavioral impact . . . 231
14.12 Viewing current configuration . . . 231
14.13 Sprint planning precedence . . . 231
14.14 Constrained scheduling defaults . . . 232
14.14.1 Field reference . . . 232

14.14.2 `sickness_prob` . . . 232
14.14.3 `assignment_mode` . . . 232
14.14.4 `pass1_iterations` . . . 233
14.15 Sickness duration defaults . . . 233
14.16 Sprint default logistic spillover parameters . . . 233
14.17 Validation rules . . . 234
14.18 Related settings in the project file . . . 234
14.19 Calibrating configuration values . . . 235

15 Natural Language Input **237**
15.1 Introductory example . . . 237
15.2 Project-level metadata . . . 239
15.3 Explicit task headers (`Task N:`) . . . 239
15.3.1 Bullet properties . . . 239
15.4 Auto-detected task lists . . . 240
15.4.1 Supported list formats . . . 240
15.4.2 Continuation lines . . . 240
15.5 Inline properties on task lines . . . 241
15.5.1 Bracketed or parenthesized sizes . . . 241
15.5.2 Fuzzy size hints . . . 241
15.5.3 Inline estimate ranges . . . 241
15.5.4 Approximate point estimates . . . 241
15.5.5 Natural duration phrases . . . 241
15.5.6 Inline dependencies . . . 242
15.5.7 Combined example . . . 242
15.6 T-shirt size aliases . . . 242
15.7 Complete examples . . . 243
15.7.1 Example 1: Numbered list with inline sizes . . . 243
15.7.2 Example 2: Bullet list with fuzzy sizes . . . 243
15.7.3 Example 3: Bracket list with inline ranges . . . 243
15.7.4 Example 4: Mixed inline properties . . . 244
15.7.5 Example 5: Traditional structured format (still fully supported) . . . 244
15.7.6 Example 6: Relative dates and prose-style estimates . . . 244
15.7.7 Example 7: Approximate point estimates . . . 244
15.8 Resources, calendars, and sprint planning . . . 244
15.9 Mixing estimation methods . . . 245
15.10 What is NOT supported in NL input . . . 245

16 MCP Server & NL Input **247**
16.1 What is the MCP server? . . . 247
16.2 Prerequisites and installation . . . 248
16.3 Ways to install the server . . . 248
16.4 Ask your assistant . . . 248
16.5 Download project and install manually . . . 248
16.5.1 Step 1: Install mcprojsim with MCP support . . . 248
16.5.2 Step 2: Verify the installation . . . 248
16.5.3 Step 3: Configure your MCP client . . . 248
16.6 A basic example of a prompt that uses the MCP Server . . . 249
16.7 Workflow overview . . . 250
16.7.1 Two-step workflow: generate then simulate . . . 250
16.7.2 One-step workflow: describe and simulate directly . . . 250
16.7.3 YAML-first workflow: validate/simulate existing project files . . . 250
16.7.4 YAML update workflow: apply natural-language change requests . . . 251
16.8 Translating NL intent to simulation options . . . 253
16.8.1 Example 1: Sprint velocity override . . . 253
16.8.2 Example 2: Disable sickness for a scenario . . . 253
16.8.3 Example 3: Constrained scheduling with two-pass . . . 253

16.8.4 Example 4: Existing YAML, no NL regeneration . . . 253
16.8.5 Example 5: Apply update instructions to an existing YAML . . . 253
16.9 Natural language input format . . . 254
16.9.1 Project-level fields . . . 254
16.9.2 Task definitions . . . 254
16.9.3 Sprint-planning definitions (NL) . . . 254
16.9.4 T-shirt size aliases . . . 255
16.10 Examples . . . 255
16.10.1 Example 1: Minimal description with T-shirt sizes . . . 255
16.10.2 Example 2: Story points with multiple dependencies . . . 257
16.10.3 Example 3: Explicit estimates with different units . . . 258
16.10.4 Example 4: Sloppy formatting (typos, inconsistent punctuation) . . . 260
16.10.5 Example 5: Inline task names and descriptions . . . 260
16.10.6 Example 6: Verbose size labels . . . 261
16.10.7 Example 7: Auto-detected lists with inline properties . . . 261
16.11 Using the parser from the command line . . . 262
16.12 Using the MCP tools directly from Python . . . 262
16.13 Tips & tricks . . . 263
16.13.1 Start simple, then enrich . . . 263
16.13.2 Use the validation tool first . . . 263
16.13.3 Be consistent with task numbering . . . 263
16.13.4 Mix estimation methods across tasks . . . 263
16.13.5 Use a configuration file for T-shirt sizes and story points . . . 263
16.13.6 Generating files for quick “what-if” scenarios . . . 264
16.13.7 Handling the generated YAML output . . . 264
16.13.8 Known limitations . . . 264

17 Examples **265**
17.1 Basic Project with Explicit Estimates . . . 265
17.2 T-Shirt Sizing . . . 267
17.3 Story Points . . . 269
17.4 Complex Project with Risks . . . 271
17.5 Log-Normal Distribution . . . 273
17.6 Constrained Scheduling with `team_size` . . . 274
17.7 Constrained Scheduling with Explicit Resources and Calendars . . . 275
17.8 Natural Language Project Generation . . . 277
17.8.1 Basic text input (dependency-only) . . . 278
17.8.2 Text input with resources and calendars (constrained) . . . 278
17.9 Sprint Planning . . . 280
17.9.1 Minimal story-point forecast . . . 281
17.9.2 Advanced sprint forecast . . . 283
17.9.3 Tasks mode and external history . . . 286
17.10 Running Examples . . . 294
17.10.1 Common CLI options . . . 294
17.10.2 Additional example files . . . 294

CHAPTER 1

Getting Started

Welcome to the Monte Carlo Project Simulator User Guide.

This short, hands-on chapter is meant to quickly show you mcprojsim in action. If you want a quick, practical tour, follow along: you'll create a sample project file, run a Monte Carlo simulation, and learn how to read the key results and reports. We keep theory to a minimum here — the goal is to spark your curiosity and get you producing real outputs quickly so you can explore the deeper chapters with a bit of context where more advanced concepts are introduced.

1.1 What you'll learn

- Quick generation of a valid project file from a plain-text description.
- Running a reproducible Monte Carlo simulation and exporting report files.
- Using sensitivity and critical-path outputs to find the tasks that drive schedule risk.

1.2 Before you start

You need:

- **Python 3.13 or newer**
- A terminal
- `pipx` for the easiest CLI install (recommended), or `pip`

1.2.1 Install Python

macOS

The recommended way is to use the official installer from python.org. Download the latest macOS package and run it.

Alternatively, use Homebrew:

```
brew install python@3.13
```

After installation, verify the version:

```
python3 --version
```

Linux

Most distributions ship with Python 3 but may not include 3.13 yet. Use your package manager:

Debian / Ubuntu:

```
sudo apt update
sudo apt install python3.13 python3.13-venv python3-pip
```

Fedora / RHEL:

```
sudo dnf install python3.13
```

Arch Linux:

```
sudo pacman -S python
```

If your distribution's package manager does not offer 3.13 yet, install it from source or use pyenv:

```
curl https://pyenv.run | bash
pyenv install 3.13
pyenv global 3.13
```

After installation, verify:

```
python3 --version
```

If you do not have `pipx` yet:

```
python3 -m pip install --user pipx
python3 -m pipx ensurepath
```

1.3 Install mcprojsim

1.3.1 With pipx (recommended)

`pipx` installs command-line tools in their own isolated virtual environments while still making the command globally available on your `PATH`. This is important for a tool like `mcprojsim` for several reasons:

Dependency isolation `pip install` drops packages directly into your system or user Python environment. If any dependency conflicts with something already installed — another tool, a framework, or a library your own code depends on — one of them will break silently or produce subtle wrong-version behaviour. `pipx` eliminates this entirely: every tool lives in its own throwaway environment that no other tool can disrupt.

No accidental breakage of system Python On macOS and most Linux distributions, the system Python interpreter manages OS-level utilities. Polluting its site-packages with `pip install` can degrade or break unrelated system tools. `pipx` never touches the system interpreter.

Easy upgrades and rollbacks

```
pipx upgrade mcprojsim       # upgrade to latest
pipx install mcprojsim==0.10 # pin an exact version
pipx uninstall mcprojsim     # clean uninstall, no residue
```

Plain `pip` leaves orphaned package files behind on uninstall; `pipx` removes the entire environment atomically.

Safe alongside virtual environments If you are already working inside a project `venv` or a Conda environment, `pipx` still installs into a separate location. Your project dependencies and the `mcprojsim` CLI remain fully decoupled.

In summary: use `pipx` when you want a CLI tool that just works, stays out of your way, and does not drift your Python environment over time.

```
pipx install mcprojsim
```

1.3.2 With pip (if you know what you are doing!)

```
pip install mcprojsim
```

1.3.3 Run mcprojsim using a Docker image from `ghcr.io`

If you prefer not to install Python or want to try `mcprojsim` without installing, you can run it using the official Docker image.

You can run the image directly with `docker run`, but it is a bit more complex to set up and use.

```
docker run --rm -v "$(pwd)":/app ghcr.io/johan162/mcprojsim:latest --help
```

In the source distribution there is a helper script `mcprojsim.sh` that simplifies running the Docker image. It handles mounting the current directory and passing through command-line arguments, so you can use it as if `mcprojsim` were installed locally. It is available in the `bin/` directory of the repository, so you can copy it to a location on your `PATH` for easy access.

First pull the latest image:

```
docker pull ghcr.io/johan162/mcprojsim:latest
```

Then you can run `mcprojsim` commands using the `mcprojsim.sh` script (just be sure to put it in your `PATH` or always run it with `./mcprojsim.sh` from the directory where it is located). For example, to see the help message:

```
./mcprojsim.sh --help
```

1.4 Verify the installation:

```
mcprojsim --version
mcprojsim --help
```

or , if you installed with Docker:

```
./mcprojsim.sh --version
./mcprojsim.sh --help
```

Note: You need to add the location of `mcprojsim.sh` to your `PATH` or always run it with `./mcprojsim.sh` from the directory where it is located

1.5 Creating the project specification file

Project files describe the project to be simulated and is written in a (fairly) easy to read YAML format. Here we will only touch on the essentials needed to simulate a basic project.

See the Project Files reference for all available fields.

The project file below defines a basic project `"Website Refresh"` with two tasks where the second task depends on the first one finishing. Most fields should be easy to understand but the `confidence_levels` deserves a closer explanation.

The project simulation results are statistical analysis. This means the result is not single number when the project is done. Instead the result is a range of numbers with an associated confidence of success. With the numbers given in the project file the result will be the `50%`, `80%`, and `90%` confidence levels.

```
project:
  name: "Website Refresh"
  description: "Small example project"
  start_date: "2026-04-01"
  confidence_levels: [50, 80, 90]
  distribution: "lognormal"

tasks:
  - id: "task_001"
    name: "Design updates"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"

  - id: "task_002"
    name: "Frontend changes"
    estimate:
      low: 4
      expected: 6
      high: 10
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"

  - id: "task_003"
    name: "Backend changes"
    estimate:
      low: 4
      expected: 6
      high: 20
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"
```

1.6 Generating a YAML project file

An alternative way than get all project file details correct is to have `mcprojsim` generate the YAML projec file for you with the `generate` command verb. Describe your project in a structured plain text and let `mcprojsim generate` command produce the detailed YAML project specification from a less strict textual description of a project.

Create a file named `description.txt` and write the following (indentation and blank rows has no meaning)

```
Project name: Website Refresh
Description: Small example project
Start date: 2026-04-01

Task 1:
- Design updates
- Size: S

Task 2:
- Frontend changes
- Depends on Task 1
- Size: M
```

Generate the project file:

```
mcprojsim generate description.txt -o project.yaml
```

That is it — the generated `project.yaml` is ready for validation and simulation. You can use T-shirt sizes (`XS`, `S`, `M`, `L`, `XL`, `XXL`), story points, or explicit `low/expected/high` estimates. See Running Simulations for the full `generate` command reference.

Flexible input formats

You don't have to use the `Task N:` header format. The parser also accepts plain numbered lists, bullet lists, and even inline sizes — great for copy-paste from meeting notes or planning tools:

```
Project name: Website Refresh
Start date: 2026-04-01

1. Design updates [S]
2. Frontend changes (M) depends on Task 1
```

See Natural Language Input for the full range of supported formats.

1.7 Validate the file

Before simulating, check the file for errors:

```
mcprojsim validate project.yaml
```

Expected result:

```
Validating project.yaml...
✓ Project file is valid!
```

If validation fails, read the reported field name and fix the YAML file before continuing.

Tip: common validation issues are missing `id` fields on tasks, invalid date formats, or incorrect field names — see Project Files for the full field reference and examples.

1.8 Run your first simulation

Simulation is done with the `simulate` command as shown below. The optional flag `--seed` is used to specify a seed for the simulation. This is useful to be able to get repeatable results or to see how specific changes in the project (such as adding risks) alters the result of the simulation while keeping everything else the same. If no seed is specified an automatic random seed is used.

```
mcprojsim simulate project.yaml --seed 42 --minimal
```

What this does:

- Runs the default 10 000 Monte Carlo iterations
- Uses `--seed 42` so the result is reproducible
- Uses `--minimal` prints only the most important results to the terminal (try without to get a more extensive output)

Tip: increase precision with `--iterations` (tradeoff: runtime vs accuracy) and use `--seed` for reproducible runs; see Running Simulations for full CLI options.

Depending on the version of `mcprojsim` used the output will look something like the following (numbers will vary by version and platform):

```
mcprojsim v0.15.2

Project Overview:
  Project: Website Refresh
  Start Date: 2026-04-01
  Number of Tasks: 3
  Effective Default Distribution: lognormal
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 2
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 126.41 hours (16 working days)
  Median (P50): 111.31 hours
  Std Dev: 53.21 hours
  Minimum: 68.61 hours
  Maximum: 1115.92 hours

Project Effort Statistical Summary:
  Mean: 187.83 person-hours (24 person-days)
  Median (P50): 173.24 person-hours
  Std Dev: 57.56 person-hours
  Minimum: 110.54 person-hours
  Maximum: 1199.36 person-hours

Calendar Time Confidence Intervals:
  P50: 111.31 hours (14 working days)  (2026-04-21)
  P80: 147.18 hours (19 working days)  (2026-04-28)
  P90: 181.17 hours (23 working days)  (2026-05-04)
```

1.9 Export results

To generate report files, add `-f` with the desired formats:

```
mcprojsim simulate project.yaml --seed 42 -f json,csv,html
```

If you want to control where the files are written and what base name they use, add `-o` as well:

```
mcprojsim simulate project.yaml --seed 42 -o results/website_refresh -f json,csv,html
```

This creates files like:

```
Website Refresh_results.json
Website Refresh_results.csv
Website Refresh_results.html
```

With `-o results/website_refresh`, the exported files would instead use that path prefix:

```
results/website_refresh.json
results/website_refresh.csv
results/website_refresh.html
```

The HTML report is the best starting point — open it in your browser:

```
open "Website Refresh_results.html"     # macOS
xdg-open "Website Refresh_results.html" # Linux
```

Tip: export HTML first (`-f html`) to inspect sensitivity, critical paths, and charts in the rendered report — it is the easiest way to explore results visually.

1.10 What the main results mean

The output is organised into sections. Here is what each section means:

Project Overview shows that this is a dependency-only simulation (no resource constraints). `Max Parallel Tasks` is the peak number of tasks that ran in parallel across all iterations.

Calendar Time Statistical Summary describes the *elapsed project duration* distribution. This is how long the project takes from start to finish on the calendar, based on the 10,000 simulated runs.

Project Effort Statistical Summary describes total *person-hours* across all tasks. For a single-developer project with sequential tasks the two summaries are identical; they diverge once tasks run in parallel or resources are constrained.

The confidence interval table, like this:

Percentile	Meaning	Typical use
P50	Half of simulated runs finish earlier	Internal discussion target
P80	80% of runs finish earlier	Common management target
P90	90% of runs finish earlier	Conservative commitment

is shown twice — once for calendar time and once for effort — with projected delivery dates (weekends excluded) shown alongside each calendar row.

The other sections in the sample output are worth a quick first-pass explanation too:

- **Sensitivity Analysis** shows which tasks are most strongly associated with the overall finish date moving later. Higher positive values usually mean those tasks are stronger schedule drivers.
- **Schedule Slack** shows how much a task can slip before it starts delaying the overall project. A task marked **Critical** has essentially no slack.
- **Most Frequent Critical Paths** shows the dependency chains that most often determine the final finish date across the Monte Carlo runs.
- **Staffing** is an advisory recommendation for team size derived from the simulated effort and parallelism.

At this level, treat these as decision aids:

- percentiles help you choose a date target
- sensitivity helps you see where uncertainty matters most
- critical paths and slack help you see where sequencing risk is concentrated

1.11 Useful next commands

Run with more iterations for higher precision:

```
mcprojsim simulate project.yaml --iterations 50000 --seed 42
```

Use a custom configuration file:

```
mcprojsim simulate project.yaml --config my_config.yaml --seed 42
```

Suppress progress output (useful in CI/CD):

```
mcprojsim simulate project.yaml --quiet
```

1.12 Where to go next

- Introduction — the ideas behind Monte Carlo project estimation
- Your First Project — build a richer project file step by step
- Task Estimation — T-shirt sizes, story points, and explicit ranges
- Project Files — complete project file reference
- Configuration — customize uncertainty factors and mappings
- Examples — working example projects
- Constrained Scheduling, for modeling resource limits and their impact on schedules
- MCP Server, for AI-assisted project file generation and integration

1.13 Try these example projects

- `examples/sample_project.yaml` — full sample with risks and resources
- `examples/quickstart_project.yaml` — minimal quickstart example used in this chapter
- `examples/resource_cap_small_task.yaml` — a small resource-constrained example to try constrained scheduling

CHAPTER 2

Introduction

This chapter introduces the ideas behind probabilistic project estimation in a gradual way. The goal is not to jump directly into commands or file syntax, but to build a clear mental model of what Monte Carlo simulation is, why it is useful for project planning, and how the main concepts appear in `mcprojsim`.

The intended reader is generally knowledgeable about software projects and planning, but not necessarily familiar with Monte Carlo methods. For that reason, the discussion starts with familiar project problems and then moves step by step toward the simulation model used by the tool.

2.1 A brief history of Monte Carlo methods

The Monte Carlo method was born from a moment of restless curiosity. In 1946, the physicist Stanislaw Ulam was recovering from illness and passing the time playing solitaire. He wondered what the probability of winning a particular Canfield layout might be. Rather than attempting a combinatorial proof, he realized it would be far simpler to play many hands and count the wins.

That insight — replacing analytical complexity with repeated random sampling — turned out to be profoundly useful. Ulam shared the idea with John von Neumann, and together they developed it into a computational technique for the nuclear weapons research at Los Alamos National Laboratory. The physicist Nicholas Metropolis suggested naming the method after the Monte Carlo Casino in Monaco where Ulam's uncle would borrow money from relatives to gamble. The name was also a nod to the role of chance at its core.

Since then, Monte Carlo methods (and more specifically its advanced cousin Markov Chain Monte Carlo (MCMC) methods) have found applications across nearly every quantitative field: nuclear physics, financial risk modeling, climate science, engineering reliability, drug development, and — closer to home — project estimation. The fundamental principle remains the same one that occurred to Ulam over a card game: when a problem is too complex to solve analytically, simulate it many times and study the results.

Project estimation is a natural fit for Monte Carlo methods because of the inherent uncertainty and complexity of software projects. By modeling tasks as probability distributions rather than fixed numbers, we can capture the range of possible outcomes and make more informed decisions about scheduling and risk management.

2.2 Why software schedule estimates are difficult

Software projects are not repetitive production processes. They involve discovery, design trade-offs, unknown technical constraints, communication overhead, integration problems, and changing assumptions. Even when the team is experienced and the work is well understood, the true effort is rarely captured well by a single number.

This is not because software teams are uniquely poor at estimating. It is because the work itself is uncertain. A realistic planning method should therefore begin by acknowledging uncertainty rather than trying to hide it.

2.2.1 The problem with single-point estimates

A single-point estimate such as "this task will take five days" appears precise, but it compresses several different possibilities into one number. It says nothing about how optimistic that estimate is, how much variation is plausible, or how likely delays are.

In practice, a task may finish early if the work is straightforward, finish near the expected time if things go normally, or finish much later if complications arise. A single-point estimate does not distinguish between these cases, which makes it difficult to reason about overall project risk.

2.3 How Monte Carlo compares to other estimation approaches

Monte Carlo simulation is not the only way to handle uncertainty in project schedules, but it offers distinct advantages over the alternatives.

Approach	How it works	Strengths	Limitations
Single-point estimate	Each task gets one fixed duration	Simple, familiar	Hides uncertainty; no basis for confidence levels
Expert judgment (Delphi)	Experts discuss and converge on estimates	Captures experience and intuition	Subjective; hard to combine across many tasks systematically
PERT	Uses optimistic, likely, and pessimistic values in a weighted formula	Accounts for range; well-established method	Assumes independent tasks; uses simplified beta approximation; limited to one critical path
Monte Carlo simulation	Samples from distributions thousands of times, respecting dependencies and risks	Full probability distribution; handles dependencies, risks, and compounding uncertainty naturally	Requires more input data; results require interpretation

The key advantage of Monte Carlo simulation is that it does not reduce uncertainty to a single number or a simple formula. Instead, it builds an entire distribution of possible outcomes. This makes it possible to answer questions like "what is the probability of finishing by a given date?" rather than only "what is the expected finish date?"

2.4 From single numbers to structured uncertainty

Monte Carlo estimation begins with a shift in thinking: instead of asking for one exact duration, we ask for a structured description of plausible outcomes. In its most direct form, a task is described using three values:

Parameter	Meaning	Example
Minimum	Optimistic but plausible outcome	3 days
Most likely	Expected outcome under normal conditions	5 days
Maximum	Difficult but credible outcome if things go wrong	10 days

These three values describe the shape of belief about the task. The minimum is not a best-case fantasy; it is the shortest duration that is still realistic. The maximum is not a worst-case catastrophe; it is the longest duration that is credibly possible without extraordinary events.

A range estimate contains more information than a single number. It tells us not only what outcome seems typical, but also how much variation surrounds it. That variation is essential at project level — even if every task has only moderate uncertainty, the combined uncertainty across many tasks can create a wide spread of overall completion dates.

2.5 How Monte Carlo simulation uses these ranges

Once tasks are described as ranges, the simulator repeatedly samples possible values and computes the resulting project outcome each time. Each run of the simulation is one plausible version of how the project might unfold. By repeating the process many times, the tool builds a distribution of outcomes instead of a single answer.

Consider a single task estimated at 3 / 5 / 10 days. Across several iterations, the sampled durations might look like this:

Iteration	Sampled duration
1	4.7 days
2	6.2 days
3	5.1 days
4	8.3 days
5	4.9 days
…	…
10,000	5.8 days

No single run is "the truth". The value of the method comes from looking at the collection of results. In `mcprojsim`, the default is 10,000 iterations, which is usually enough to provide a stable picture for most planning tasks.

2.6 From many runs to a probability distribution

When we aggregate the results of many iterations, we can see not only the central tendency of the schedule, but also its spread. Instead of saying "the project ends in 73 days", we can say things like "there is a 50% chance of finishing by this date" or "there is an 80% chance of finishing by that date".

This is the point at which simulation becomes valuable for management decisions. Planning is rarely about the most likely outcome alone. It is often about deciding how much confidence is required before committing to a date.

2.6.1 Common percentiles and what they mean

Percentile	Interpretation	Typical use
P50	Half of simulated runs finish earlier	Neutral planning target; even split of risk
P75	Three quarters of runs finish earlier	Moderate confidence
P80	80% of runs finish earlier	Common management target
P90	90% of runs finish earlier	Conservative; appropriate when lateness is costly
P95	95% of runs finish earlier	High confidence; useful for contractual commitments
P99	99% of runs finish earlier	Very high confidence, useful for mission critical commiments

The important point is not that one percentile is always correct. The simulation allows stakeholders to choose a confidence level deliberately rather than inheriting one implicitly from an optimistic single-point estimate.

2.7 Tasks, dependencies, and project structure

Real projects consist of many tasks, not one. Each task has its own estimate, and the tasks are not independent in a scheduling sense. Some can start immediately, while others must wait for predecessors to finish.

Dependencies determine the order in which work can occur. A task cannot begin simply because a team is ready to work on it; it may also require certain other tasks to be complete first. This matters because uncertainty propagates through the dependency network — a delay in one upstream task can delay many downstream tasks even if those later tasks are estimated accurately.

Project duration is not computed by adding all task durations together. Some tasks can run in parallel, while others form chains that must happen in sequence. The final project duration emerges from both the estimated task durations and the structure of the task network:

Scenario	Tasks A and B (each 5 days)	Project duration
Sequential — B depends on A	A then B	10 days
Parallel — no dependency	A alongside B	5 days
Partial overlap — C depends on A, not B	A then C, B runs alongside	Depends on relative durations

This is why task scheduling is a central part of project simulation. The same set of task estimates can lead to different overall project durations depending on how the work is connected.

2.8 The critical path

In any given simulated run, some sequence of dependent tasks determines the final completion time. This sequence is the critical path.

Not every task contributes equally to schedule risk. Some tasks are important but not schedule-dominant. Others repeatedly determine whether the project finishes early or late.

A common mistake is to assume there is one fixed critical path. In deterministic planning this may be a useful simplification, but in a simulation context it is often wrong. Because task durations vary from run to run, the schedule-dominating path can also change. One of the strengths of Monte Carlo simulation is that it can show which tasks are *frequently* critical, not just which tasks are critical in one nominal plan. `mcprojsim` tracks this across all iterations and reports a criticality index for each task — the percentage of iterations in which that task appeared on the critical path.

2.9 Two sources of schedule variation

Beyond the estimate range itself, `mcprojsim` models two additional sources of schedule variation. Understanding the distinction between them is important for building accurate project models.

Source	Nature	How it works	Example
Uncertainty factors	Persistent conditions	Multiply the sampled duration	An inexperienced team is 30% slower
Risks	Discrete events	Add time if triggered	A security audit adds 5 days with 30% probability

Uncertainty factors describe conditions. Risks describe events. A difficult architecture is an uncertainty factor. A failed security audit that adds rework is a risk. This distinction helps the model stay understandable — not every source of delay should be represented as a probabilistic event.

2.9.1 Uncertainty factors

Uncertainty factors are known characteristics of the working environment that systematically affect how long work takes. `mcprojsim` supports five configurable factors:

Factor	What it captures
`team_experience`	Familiarity with the technology and domain
`requirements_maturity`	How well-defined and stable requirements are
`technical_complexity`	How technically challenging the work is
`team_distribution`	Colocated vs. distributed teams
`integration_complexity`	Degree of integration with other systems

These factors are multiplicative. When a task specifies several of them, the multipliers are combined by multiplication. This means adverse factors compound — even moderate individual adjustments can produce a significant combined effect.

The numeric multipliers are defined in a separate configuration file, not in the project file. This allows teams to calibrate the meaning of labels like "high complexity" or "low maturity" to match their own experience, without changing the project definition. See Uncertainty Factors for the full list of default multiplier values and guidance on calibration.

2.9.2 Risks

Risks describe events that may or may not happen. Unlike uncertainty factors, which always apply, a risk triggers probabilistically in each simulation iteration. If it triggers, its impact is added to the duration.

Risks can be defined at two levels:

- **Task-level risks** apply to a specific task. For example, a database migration might have a 20% chance of encountering schema conflicts that add 2 days.
- **Project-level risks** apply to the project as a whole. For example, there might be a 15% chance that a key developer leaves, delaying the entire project by 20%.

Every risk combines two dimensions: how likely the event is, and how large the consequence is if it happens. A low-probability, high-impact event behaves very differently from a high-probability, low-impact event. Monte Carlo simulation captures both naturally without forcing them into a single oversimplified score.

2.10 Effort estimation versus calendar scheduling

Everything discussed so far — task estimates, uncertainty factors, risks — describes how much *work* a task requires. This is the effort dimension, measured in hours of productive labor. But effort alone does not answer the question that stakeholders actually ask: "When will it be done?"

The answer to that question lives in a different dimension: calendar time. Calendar time is the elapsed clock time from project start to project finish, and it depends on much more than effort. Two projects with identical total effort can finish weeks or even months apart depending on how many people are available, when they work, and what interruptions occur along the way.

Dimension	What it measures	Example
Effort	Total productive work required	80 person-hours
Calendar time	Elapsed wall-clock duration	15 working days (3 calendar weeks)

The distinction matters because most planning conversations conflate the two. When someone estimates "this will take two weeks", they usually mean effort — but the listener hears a calendar commitment. A simulation that only models effort without considering how that effort maps onto real calendars will produce misleadingly optimistic dates.

Converting effort into calendar time requires knowing the capacity available at each point in time. If two people work full-time on a task, 80 hours of effort can finish in 40 clock-hours — five working days. If one person works half-time, the same effort stretches to 20 working days. The conversion is not a simple division; it must account for varying capacity across time as people come and go, work different schedules, and encounter interruptions.

2.11 Constrained scheduling: from ideal plans to real-world schedules

Dependency-only scheduling assumes that any task can start immediately once its predecessors are complete, as if an unlimited number of equally skilled people were always available. This is a useful simplification for understanding project structure, but it does not reflect reality. Real projects have a finite number of people, and those people have different skills, work different hours, take holidays, and occasionally get sick.

Constrained scheduling bridges this gap. When the simulation knows about the actual team — who is available, when they work, and what skills they have — it can produce calendar dates that reflect genuine capacity rather than theoretical parallelism.

2.11.1 Resources and capacity

In a constrained simulation, each team member is modeled as a resource with specific characteristics:

Characteristic	What it captures	Why it matters
Availability	Fraction of the workday this person is available	A 50%-available resource takes twice as long in calendar time
Productivity	Relative throughput compared to baseline	A senior developer may complete work faster than a junior one
Experience level	Skill tier of the resource	Some tasks require a minimum experience level

These properties determine how quickly a resource converts effort into progress. The scheduler integrates effort over actual capacity windows — it does not simply divide effort by a fixed rate. When multiple resources work on the same task, their capacities combine, but practical limits prevent unrealistic compression (assigning eight people to an eight-hour task does not make it finish in one hour).

Because resources are finite, tasks sometimes cannot start even when their dependencies are satisfied. A task may be ready to go, but all qualified resources are occupied elsewhere. This waiting time — invisible in dependency-only mode — is a real and often significant contributor to project duration.

2.11.2 Working calendars

People do not work around the clock. A working calendar defines the rhythm of productive time: which days of the week are working days, how many hours per day, and which dates are holidays. Different resources can follow different calendars — a full-time developer on a Monday-to-Friday schedule, a part-time contractor working four shorter days.

The effect on scheduling is direct. An eight-hour task that begins on Friday afternoon cannot finish until Monday. A task assigned to someone on a four-day week takes longer in calendar time than the same task assigned to someone working five days. Holidays create gaps where no progress occurs. All of these effects compound across the project, and they are especially visible in the tail of the schedule distribution — the dates at P90 and beyond that matter most for planning commitments.

2.11.3 Planned and unplanned absence

Even on working days, people are not always available. Planned absences — vacations, conferences, training — are known in advance and can be specified per resource. The simulation treats these dates as non-working days for that individual, reducing their capacity to zero on those dates.

Unplanned absence is inherently stochastic. People get sick, have family emergencies, or encounter other interruptions that cannot be predicted. `mcprojsim` models this by assigning each resource a daily sickness probability. On each simulated working day, the simulator rolls a random check for each resource. If sickness triggers, the resource is unavailable for a realistic duration drawn from a probability distribution. Because this happens independently in each simulation iteration, the aggregate effect of unplanned absence emerges

naturally in the results — some iterations see little disruption, others see more, and the overall distribution captures the range.

This stochastic modeling of absence is one of the reasons Monte Carlo simulation is particularly well suited to schedule planning. Deterministic methods either ignore absence entirely or account for it with a crude average discount. A simulation can model the actual variability: the possibility that two key people happen to be sick during the same critical week, or that the project runs smoothly with minimal disruption. Both scenarios contribute to the final probability distribution.

2.11.4 Why constrained scheduling changes the picture

The combined effect of resource constraints, calendars, and absences is often substantial. A project that finishes in 130 hours under dependency-only scheduling might take 550 calendar-hours — over four times longer — when resource constraints and calendars are applied. The effort has not changed; the calendar reality has.

This is not a failure of estimation. It is a more honest picture of when work actually completes. The gap between effort-based estimates and calendar-based schedules is where many real-world planning surprises originate. By modeling resources, calendars, and absences explicitly, the simulation surfaces these effects before the project starts rather than after commitments have been made.

2.12 What the simulation computes, step by step

For each iteration, the simulation follows this sequence:

Step	Action	Scope
1	**Sample** a base duration for each task from its estimate distribution	Per task
2	**Adjust** each duration by multiplying in the task's uncertainty factors	Per task
3	**Evaluate** each task's risks — if triggered, add the impact	Per task
4	**Schedule** all tasks respecting dependency constraints	Whole project
5	**Calculate** the project duration as the latest task completion time	Whole project
6	**Evaluate** project-level risks — if triggered, add the impact	Whole project

This process is repeated for every iteration. The final output is not the result of one deterministic calculation, but the statistical summary of many plausible project histories.

2.13 Understanding the results

2.13.1 Statistical summary

Simulation results should be read as statements about confidence, spread, and likelihood. A median outcome tells you what happens in the middle of the simulated runs. Percentiles tell you the duration that a given percentage of runs finish within.

These numbers are decision aids, not guarantees. They help users choose targets that reflect a chosen level of planning confidence.

2.13.2 Histograms

A histogram gives a visual summary of how outcomes are distributed. A narrow histogram suggests relatively stable outcomes. A wide histogram suggests greater uncertainty. Skewness can indicate that the project has more downside risk than upside potential.

Even users who are not statistically trained can often read histograms effectively once they understand that the horizontal axis represents duration and the vertical axis represents frequency.

2.13.3 Sensitivity and criticality

One of the most useful questions in project planning is not only "how long will this take?" but "what drives the answer?" Sensitivity analysis identifies which tasks contribute most to schedule variation. Criticality analysis shows which tasks most often appear on the critical path. Together, these views tell you where to focus mitigation and management attention.

2.13.4 Staffing recommendations

Simulation results also help answer the question "how many people should work on this?" Adding people to a project increases capacity but also increases communication overhead — a trade-off famously described by Brooks's Law. `mcprojsim` uses the simulated total effort, the critical-path duration, and the peak parallelism in the schedule to recommend an optimal team size for different experience profiles. The recommendation is an advisory layer on top of the simulation, not a constraint on the simulation itself.

2.13.5 Reproducibility

Although Monte Carlo simulation relies on randomness, it can be made reproducible. By setting a random seed, users can repeat the same simulation and obtain the same results. This is useful for analysis, review, automation, and auditability — it allows teams to discuss the same output without worrying that the numbers changed simply because the simulation was run again.

2.14 How the input files reflect the model

`mcprojsim` uses two types of input files, each with a distinct purpose:

File	Purpose	Contains
Project file (YAML or TOML)	Describes the specific project	Metadata, tasks, dependencies, estimates, risks
Configuration file (YAML)	Defines organizational assumptions	Uncertainty factor multipliers, T-shirt size mappings, story point mappings, simulation defaults

This separation means the project file describes what is being planned, while the configuration file describes how the organization interprets uncertainty. Teams can share a single configuration across many projects, keeping individual project files focused and readable.

Before learning every schema detail, it helps to read a project file as a narrative. The project block says what is being planned. The task list says what must be done. Dependencies say in what order work becomes possible. Risks and uncertainty factors explain why durations may vary.

2.14.1 Multiple ways to express estimates

`mcprojsim` supports several ways to express effort uncertainty. These are not competing approaches — they are different ways of expressing the same underlying concept, suited to different stages of planning.

Estimate type	When to use it	Example
Explicit range (low / expected / high)	When the team can provide numeric estimates	`low: 3, expected: 5, high: 10`
T-shirt size (XS through XXL)	For early-stage or relative estimation	`t_shirt_size: "M"`
Story points (1, 2, 3, 5, 8, 13, 21)	When the team uses story point estimation	`story_points: 5`

T-shirt sizes and story points are mapped to numeric ranges in the configuration file, so they ultimately feed into the same simulation machinery as explicit estimates. They provide a natural bridge from familiar agile estimation practices into probabilistic simulation.

2.15 When Monte Carlo estimation is most valuable

Monte Carlo methods are particularly helpful when projects have:

- Multiple tasks with dependencies that create compounding uncertainty
- Meaningful risk events that may or may not occur
- Stakeholders who need to reason in terms of confidence rather than optimistic commitments
- Significant consequences for late delivery, making conservative planning worthwhile

The method is less valuable when the work is trivial, highly repetitive, or too poorly defined to estimate in any structured way. Like any planning method, it works best when the inputs are thoughtful and the model is used with judgment.

2.16 What comes next in this guide

The rest of the user guide builds on the concepts introduced here. Later chapters cover the project file format, configuration, risk and uncertainty factor modeling, and interpreting the generated outputs in detail.

By the time you move into those chapters, the goal is that the fields in the input files no longer feel arbitrary. They should feel like natural answers to the questions that this introduction has raised.

CHAPTER 3

Your first project

This chapter shows how to build a project input file gradually.

The goal is not to start with a large "realistic" file and explain it all at once. Instead, we begin with the smallest useful example and then add one idea at a time: more tasks, dependencies, configuration, uncertainty factors, and risks.

That approach mirrors how most teams actually learn the tool. First they want to know the minimum needed to run a simulation. After that, they want to enrich the model with the information they already have about the work, the team, and the main sources of schedule risk.

The examples in this chapter are intentionally small, but they follow the same style as the more complete files in `examples/sample_project.yaml`, `examples/sample_config.yaml`, `examples/tshirt_sizing_project.yaml`, and `examples/story_points_walkthrough_project.yaml`.

3.1 Chapter outline

In this chapter we will build the input in six stages:

1. Start with the smallest possible project file.
2. Add a second task and a dependency.
3. Introduce a configuration file and uncertainty factors.
4. Add task-level and project-level risks.
5. Express work with T-shirt sizes.
6. Express work with Story Points.

At each step we will run the simulation and look at how the results change.

3.2 Quick command and flag map

This chapter focuses on building the project file step-by-step, but it helps to know the full CLI surface area up front.

3.2.1 Commands at a glance

Command	Purpose
`mcprojsim generate INPUT_FILE`	Generate a project YAML file from plain-text input
`mcprojsim validate PROJECT_FILE`	Validate a project file without simulating
`mcprojsim simulate PROJECT_FILE`	Run Monte Carlo simulation and optionally export reports
`mcprojsim config`	Show active configuration

3.2.2 Common `simulate` flags

Flag	What it controls
`--iterations`, `-n`	Number of Monte Carlo iterations
`--seed`, `-s`	Reproducibility seed
`--config`, `-c`	Custom config file
`--target-date`	Probability of finishing on/before a target date
`--output-format`, `-f`	Export format(s): json, csv, html
`--output`, `-o`	Export output base path
`--critical-paths`	Number of critical path sequences shown/exported
`--table`, `-t`	Render CLI tabular sections as ASCII tables
`--minimal`, `-m`	Minimal console output mode
`--verbose`, `-v`	Show detailed informational messages
`--quiet`, `-q`, `-qq`	Reduce or suppress normal CLI output
`--staffing`	Expanded staffing recommendation tables
`--tshirt-category`	Override default T-shirt category
`--velocity-model`	Override sprint velocity model for how historic data are used (`empirical`/`neg_binomial`)
`--no-sickness`	Disable sprint sickness modeling
`--two-pass`	Enable criticality two-pass scheduling (resource-constrained mode)
`--pass1-iterations`	Pass-1 iterations used by `--two-pass`
`--include-historic-base`	Add Historic Base section/series in HTML+JSON exports

For full details, defaults, and examples, see Running simulations.

This chapter intentionally uses only a small subset of these flags in each step.

3.3 The smallest possible project file

The simplest useful project file contains two top-level sections:

- `project`: basic metadata about the project.
- `tasks`: at least one task with an estimate.

That is enough to validate the file and run a simulation.

3.3.1 Step 1: one task, one estimate

Start with a single task:

```
project:
  name: "Tiny Landing Page"
  description: "The smallest possible example"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Create landing page"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"
```

This file does not yet say anything about dependencies, team characteristics, or risks. It simply says: “we have one task, and we believe it is likely to take around three days, but with some uncertainty around that value”.

Before simulating, it is good practice to validate the file:

```
mcprojsim validate first-project-step-1.yaml
```

Expected result:

```
Validating first-project-step-1.yaml...
✓ Project file is valid!
```

Now run the simulation:

```
mcprojsim simulate first-project-step-1.yaml --iterations 5000 --seed 42 --minimal
```

Example result:

```
mcprojsim v0.15.2

Project Overview:
  Project: Tiny Landing Page
  Start Date: 2026-03-01
  Number of Tasks: 1
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 26.61 hours (4 working days)
  Median (P50): 26.14 hours
  Std Dev: 4.99 hours
  Minimum: 16.05 hours
  Maximum: 39.67 hours

Project Effort Statistical Summary:
  Mean: 26.61 person-hours (4 person-days)
  Median (P50): 26.14 person-hours
  Std Dev: 4.99 person-hours
  Minimum: 16.05 person-hours
  Maximum: 39.67 person-hours

Calendar Time Confidence Intervals:
  P50: 26.14 hours (4 working days)  (2026-03-05)
  P80: 31.23 hours (4 working days)  (2026-03-05)
  P90: 33.68 hours (5 working days)  (2026-03-06)
```

The important thing here is not the exact numbers. The important thing is that even a one-task project produces a range of likely outcomes rather than one fixed answer. Results are reported in hours (the canonical internal unit) with working days shown alongside.

3.4 Add structure: a second task and a dependency

A one-task example is useful for learning the mechanics, but real projects are interesting because tasks interact. The next step is therefore to add a second task that depends on the first one.

3.4.1 Step 2: sequence the work

```
project:
  name: "Tiny Landing Page"
  description: "Adding sequencing"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]
tasks:
  - id: "task_001"
    name: "Create landing page"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"
  - id: "task_002"
    name: "Deploy site"
    estimate:
      low: 1
      expected: 2
      high: 4
      unit: "days"
    dependencies: ["task_001"]
```

The new information is in the `dependencies` field. It says that deployment cannot start until the landing page work is complete.

Run it:

```
mcprojsim simulate first-project-step-2.yaml --iterations 5000 --seed 42
```

Example result summary:

```
──────────────────────────────────────────────
 mcprojsim v0.15.2   Monte Carlo Simulator
 Project : Tiny Landing Page
 Run     : 2026-05-05 22:52:06
──────────────────────────────────────────────

Project Overview:
  Project: Tiny Landing Page
  Start Date: 2026-03-01
  Number of Tasks: 2
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 45.13 hours (6 working days)
  Median (P50): 44.88 hours
  Std Dev: 6.91 hours
  Minimum: 26.24 hours
  Maximum: 66.85 hours
  Coefficient of Variation: 0.1531
  Skewness: 0.1895
  Excess Kurtosis: -0.3088

Project Effort Statistical Summary:
  Mean: 45.13 person-hours (6 person-days)
  Median (P50): 44.88 person-hours
  Std Dev: 6.91 person-hours
  Minimum: 26.24 person-hours
  Maximum: 66.85 person-hours
  Coefficient of Variation: 0.1531
  Skewness: 0.1895
  Excess Kurtosis: -0.3088

Calendar Time Confidence Intervals:
  P50: 44.88 hours (6 working days)  (2026-03-09)
  P80: 51.04 hours (7 working days)  (2026-03-10)
  P90: 54.18 hours (7 working days)  (2026-03-10)

Effort Confidence Intervals:
  P50: 44.88 person-hours (6 person-days)
  P80: 51.04 person-hours (7 person-days)
  P90: 54.18 person-hours (7 person-days)

Schedule Slack:
  task_001: 0.00 hours (Critical)
  task_002: 0.00 hours (Critical)

Most Frequent Critical Paths:
  1. task_001 -> task_002 (5000/5000, 100.0%)

Staffing (based on mean effort): 1 people recommended (mixed team), 7 working days
  Total effort: 45 person-hours (6 person-days) | Parallelism ratio: 1.0
```

Compared with the first example, the schedule is now longer because the project contains more work. More importantly, the dependency means the tasks form a chain rather than happening independently. This is the beginning of a project network.

This is also the point where the input file starts to describe more than estimates. It now describes project structure.

3.4.2 Understanding the critical path output

Starting from this step, the CLI and exports also report full critical path sequences.

That means the tool now shows:

- **task-level criticality**, which answers: "how often was this task on a critical path?"
- **full path sequences**, which answer: "which complete dependency chains were most often critical?"

For this small example there is only one dependency chain, so the reported path is always:

```
task_001 -> task_002 (5000/5000, 100.0%)
```

In larger projects, several different paths may become critical across the Monte Carlo iterations. The simulator aggregates those paths and shows the most common ones.

You can request more than the default two paths in the CLI:

```
mcprojsim simulate first-project-step-2.yaml --iterations 5000 --seed 42 --critical-paths 4
```

For this small project there is only one critical path, so specifying a larger report limit does not change the output.

And you can control how many are stored and reported by default through `config.yaml`:

```
simulation:
  max_stored_critical_paths: 20

output:
  critical_path_report_limit: 2
```

3.5 Add organizational knowledge with a configuration file

So far, the estimates came entirely from the task ranges themselves. In practice, teams usually know more than that. They often know whether the team is experienced, whether the requirements are mature, or whether the technical environment is unusually complex.

In `mcprojsim`, that sort of shared interpretation belongs in a configuration file.

3.5.1 Step 3: add uncertainty factors

Uncertainty factors are a way to convert known project conditions into tangible schedule effects. For example, requirements may be immature and likely to change after work begins, or the team may be distributed across countries and time zones, creating additional communication overhead and a higher risk of misunderstandings.

These types of know risks can be modelled by multiplicative factors. The program supports the following uncertaintly factors

Factor	Default Level
`team_experience`	`medium`
`requirements_maturity`	`high`
`technical_complexity`	`low`
`team_distribution`	`colocated`
`integration_complexity`	`low`

The name of the factors should be self-explanatory. The values of the factors are heuristically determined and can be adjusted for company or team specific circumstances in the config file.

The uncertainty factors can be applied both on project- and task level. In the following example we add them for each task.

```yaml
project:
  name: "Tiny Landing Page"
  description: "Adding uncertainty factors"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]
tasks:
  - id: "task_001"
    name: "Create landing page"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "medium"

  - id: "task_002"
    name: "Deploy site"
    estimate:
      low: 1
      expected: 2
      high: 4
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"
```

Run it with this configuration file:

```
mcprojsim simulate first-project-step-3.yaml --seed 42 --minimal
```

Example result summary:

```
mcprojsim v0.15.2

Project Overview:
  Project: Tiny Landing Page
  Start Date: 2026-03-01
  Number of Tasks: 2
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 49.98 hours (7 working days)
  Median (P50): 49.67 hours
  Std Dev: 7.88 hours
  Minimum: 27.41 hours
  Maximum: 76.84 hours

Project Effort Statistical Summary:
  Mean: 49.98 person-hours (7 person-days)
  Median (P50): 49.67 person-hours
  Std Dev: 7.88 person-hours
  Minimum: 27.41 person-hours
  Maximum: 76.84 person-hours

Calendar Time Confidence Intervals:
  P50: 49.67 hours (7 working days)  (2026-03-10)
  P80: 56.79 hours (8 working days)  (2026-03-11)
  P90: 60.43 hours (8 working days)  (2026-03-11)
```

This is an important modeling step. As can be seen from the simulation these uncertainty factors add another day effort to be 80% certain.

To modify these factors according to the project specific circumstances add the specific values in separate config file that are used in the simulation. For example we can use slightly higher values for the uncertainty factors than the program uses by default as:

```
uncertainty_factors:
  team_experience:
    high: 0.90
    medium: 1.1
    low: 1.50
  requirements_maturity:
    high: 1.0
    medium: 1.40
    low: 1.80
  technical_complexity:
    low: 1.0
    medium: 1.15
    high: 1.40
simulation:
  default_iterations: 10000
  random_seed: null
output:
    formats: ["json"]
    include_histogram: true
    number_bins: 30
```

To see the programs default values use

```
mcprojsim config
```

The interesting part of the output is

```
Uncertainty Factors:
  team_experience:
    high: 0.9
    medium: 1.0
    low: 1.3
  requirements_maturity:
    high: 1.0
    medium: 1.15
    low: 1.4
  technical_complexity:
    low: 1.0
    medium: 1.2
    high: 1.5
  team_distribution:
    colocated: 1.0
    distributed: 1.25
  integration_complexity:
    low: 1.0
    medium: 1.15
    high: 1.35
  ...
```

Save these updated values as `first-project-step-3-config.yaml` and then run the simulation again using these values as

```
mcprojsim simulate first-project-step-3.yaml --seed 42 --minimal --config first-project-step-3-config.yaml
```

We do not repeat the full output but show only the summary as

```
<-- SKIP -->
Calendar Time Statistical Summary:
  Mean: 57.19 hours (8 working days)
  Median (P50): 56.85 hours
  Std Dev: 8.87 hours
  Minimum: 31.76 hours
  Maximum: 87.52 hours
Project Effort Statistical Summary:
  Mean: 57.19 person-hours (8 person-days)
  Median (P50): 56.85 person-hours
  Std Dev: 8.87 person-hours
  Minimum: 31.76 person-hours
  Maximum: 87.52 person-hours
```

As expected, the simulation shows slightly higher effort and longer calendar time.

3.6 Add explicit risk events

Uncertainty factors model persistent conditions around the work. Risks model events that may or may not happen.

That distinction matters. A distributed team is not an event; it is a condition. A late stakeholder change is an event; it may happen, or it may not.

3.6.1 Step 4: add project-level and task-level risks

```
project:
  name: "Tiny Landing Page"
  description: "Adding risk events"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]
project_risks:
  - id: "risk_001"
    name: "Late stakeholder changes"
    probability: 0.20
    impact:
      type: "absolute"
      value: 2
      unit: "days"
tasks:
  - id: "task_001"
    name: "Create landing page"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "medium"
  - id: "task_002"
    name: "Deploy site"
    estimate:
      low: 1
      expected: 2
      high: 4
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"
    risks:
      - id: "risk_002"
        name: "Hosting problem"
        probability: 0.15
        impact: 1.5     # bare float = hours; use impact.type/value/unit for other units
```

Again, run this as:

```
mcprojsim simulate first-project-step-4.yaml \
  --config first-project-step-3-config.yaml \
  --iterations 5000 --seed 42
```

Example result summary:

```
mcprojsim v0.15.2

Project Overview:
  Project: Tiny Landing Page
  Start Date: 2026-03-01
  Number of Tasks: 2
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 60.52 hours (8 working days)
  Median (P50): 59.67 hours
  Std Dev: 10.90 hours
  Minimum: 35.15 hours
  Maximum: 103.52 hours

Project Effort Statistical Summary:
  Mean: 57.36 person-hours (8 person-days)
  Median (P50): 57.08 person-hours
  Std Dev: 8.84 person-hours
  Minimum: 31.76 person-hours
  Maximum: 87.52 person-hours

Calendar Time Confidence Intervals:
  P50: 59.67 hours (8 working days)  (2026-03-11)
  P80: 69.69 hours (9 working days)  (2026-03-12)
  P90: 75.18 hours (10 working days)  (2026-03-13)
```

Notice what happened relative to Step 3:

- the project mean increased
- the upper percentiles increased,
- the spread increased.

In this example the change was the added overall project risk and we kept the task risks the same. For that reason we do not see any change in the task effort but only in the overall calendar time for the project.

A new section, **Risk Impact Analysis**, also appears. It shows how often each risk triggered and how much it added on average. In this example, `risk_002` ("Hosting problem") triggered in about 15% of iterations, adding 1.5 hours when it did — matching the 0.15 probability and 1.5-hour impact we specified.

That is exactly what we expect when we add plausible delay events to the model. The file is not just more detailed. It is representing more of the actual uncertainty that the team believes exists.

3.7 An alternative style: T-shirt sized estimates

So far, every task has used an explicit estimate range with `low`, `expected`, and `high`. That is often the best choice when the team is comfortable expressing estimates directly in days.

However, some teams think more naturally in relative sizing. They can say that one task is `S`, another is `M`, and another is `XL`, even when they do not yet want to commit to detailed numeric ranges.

`mcprojsim` supports that style directly.

3.7.1 Step 5: use `t_shirt_size` instead of explicit ranges

In the source code, a task estimate may specify the field `t_shirt_size` instead of `expected` and the related range values. Validation accepts that form, and the simulation engine later resolves the symbolic size to concrete `low`, `expected`, and `high` values from the active configuration.

That means a T-shirt-sized task still becomes an ordinary probabilistic range during simulation. The symbolic label is just a more convenient way to express the estimate in the input file.

Important: T-shirt size estimates must not include a `unit` field in the project file. The unit is determined by the configuration file's `t_shirt_size_unit` setting (default: `"hours"`).

Here is a small example:

```
project:
  name: "Tiny Landing Page"
  description: "T-shirt sizing example"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Design page"
    estimate:
      t_shirt_size: "S"
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "high"
  - id: "task_002"
    name: "Build page"
    estimate:
      t_shirt_size: "M"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"
  - id: "task_003"
    name: "Deploy page"
    estimate:
      t_shirt_size: "XS"
    dependencies: ["task_002"]
```

Note that no `unit` field appears on any task — the unit is taken from the configuration.

3.7.2 Categories of T-shirt sizes

T-shirt sizes are organized into categories so that the same label (`M`, `L`, etc.) can have different meanings depending on the type of work. An `M` story and an `M` epic represent vastly different amounts of work, and the category structure keeps those scales separate.

The built-in categories cover the common levels of planning granularity in software delivery:

Category	Typical use	Example `M` expected (hours)
`bug`	Individual defects and small fixes	40
`story`	User-facing features and tasks (default)	60
`epic`	Groups of related stories	240
`business/program`	Larger business capabilities or programmes	4 000
`initiative`	Strategic initiatives spanning multiple programmes	20 000

The default category is `story`. When a task specifies a bare size like `t_shirt_size: "M"`, it is resolved against the `story` category unless you override `t_shirt_size_default_category` in the configuration file.

Use a qualified form such as `t_shirt_size: "epic.M"` to reference a size from a specific category directly, regardless of the default.

3.7.3 How T-shirt sizes are configured

If you want to use a custom configuration file with T-shirt estimates, add a `t_shirt_sizes` section like this to the configuration file:

```
t_shirt_sizes:
  story:
    XS:
      low: 3
      expected: 5
      high: 15
    S:
      low: 5
      expected: 16
      high: 40
    M:
      low: 40
      expected: 60
      high: 120

t_shirt_size_default_category: story
```

The application ships with built-in mappings for multiple categories (`bug`, `story`, `epic`, `business`, `initiative`) and defaults bare sizes like `M` to the `story` category. In this example, `t_shirt_size_default_category: story` makes bare sizes use story-scale values instead.

If you use your own configuration file, it is merged onto built-in defaults. That means you do **not** need to redefine `t_shirt_sizes` unless you want to override defaults.

The simulation engine resolves each T-shirt label against the active merged configuration and raises an error only when the requested category/size is unknown.

This detail is important: `t_shirt_size` is not just a comment or a reporting label. It is a real input that must be backed by a configuration mapping.

In other words, you have two common options:

- run without `--config` and use built-in size mappings, or
- run with `--config` to override only the categories/sizes you want to change.

Run the T-shirt-sized example like this:

```
mcprojsim simulate examples/tshirt_walkthrough_project.yaml \
  --config examples/tshirt_walkthrough_config.yaml \
  --seed 42 --iterations 5000 --minimal
```

Example result summary:

```
mcprojsim v0.15.2

Project Overview:
  Project: Tiny Landing Page
  Start Date: 2026-03-01
  Number of Tasks: 3
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 9.65 hours (2 working days)
  Median (P50): 9.56 hours
  Std Dev: 1.39 hours
  Minimum: 5.83 hours
  Maximum: 14.08 hours

Project Effort Statistical Summary:
  Mean: 9.65 person-hours (2 person-days)
  Median (P50): 9.56 person-hours
  Std Dev: 1.39 person-hours
  Minimum: 5.83 person-hours
  Maximum: 14.08 person-hours

Calendar Time Confidence Intervals:
  P50: 9.56 hours (2 working days)  (2026-03-03)
  P80: 10.85 hours (2 working days)  (2026-03-03)
  P90: 11.54 hours (2 working days)  (2026-03-03)
```

The interpretation is exactly the same as for explicit ranges. The only difference is how the task effort was expressed in the input file.

T-shirt sizes are especially useful early in planning, when the team can compare tasks relative to each other more easily than they can assign precise numeric ranges. Later, if needed, the team can replace those symbolic sizes with explicit estimate ranges for finer control.

3.8 Another alternative style: Story Point estimates

Some teams prefer to estimate backlog items in Story Points rather than T-shirt sizes or explicit day ranges.

`mcprojsim` supports that style as another symbolic estimate form. In the input file you provide a `story_points` value, and during simulation the active configuration resolves it to a numeric `(low, expected, high)` range in the configured unit (default: days).

Important: Story point estimates must not include a `unit` field in the project file. The unit is determined by the configuration file's `story_point_unit` setting (default: `"days"`).

Story points was originally conceived as a way to model actual working time where 1 SP was eqal to 1 uninterrupted day of work (something that rarely happens in real life). Now, SP is an estimat calibrated to a specific team and story points are only valid within the same and stable team as their own "currency" were each team have different exchange rates between SP <–> days. The program have *one* exchange rate by default but that can easily be adjusted by supplying a different exchange rate in a config file as shown below

3.8.1 Step 6: use `story_points`

Here is a small Story Point example:

```
project:
  name: "Tiny Landing Page"
  description: "Story Point sizing example"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Design page"
    estimate:
      story_points: 2
    dependencies: []

  - id: "task_002"
    name: "Build page"
    estimate:
      story_points: 5
    dependencies: ["task_001"]

  - id: "task_003"
    name: "Deploy page"
    estimate:
      story_points: 1
    dependencies: ["task_002"]
```

Note that no `unit` field appears on any task — the unit is taken from the configuration.

3.8.2 Default Story Point

The built-in Story Points have the following mapping to time (in days)

```
Story Points (unit: days):
  1:
    low: 0.5, expected: 1.0, high: 3.0
  2:
    low: 1.0, expected: 2.0, high: 4.0
  3:
    low: 1.5, expected: 3.0, high: 5.0
  5:
    low: 3.0, expected: 5.0, high: 8.0
  8:
    low: 5.0, expected: 8.0, high: 15.0
  13:
    low: 8.0, expected: 13.0, high: 21.0
  21:
    low: 13.0, expected: 21.0, high: 34.0
```

3.8.3 How Story Points are configured

The built-in story point values cover the Fibonacci sequence 1, 2, 3, 5, 8, 13, 21. If you want to customize any of those mappings, add a `story_points` section to the configuration file:

```
story_points:
  1:
    low: 0.5
    expected: 1
    high: 3
  2:
    low: 1
    expected: 2
    high: 4
  3:
    low: 1.5
    expected: 3
    high: 5
  5:
    low: 3
    expected: 5
    high: 8
  8:
    low: 5
    expected: 8
    high: 15
```

These are built-in defaults. If you provide a custom configuration file, you may override only the Story Point values you want to recalibrate for your team; the remaining built-in defaults stay available.

As with T-shirt sizes, a custom config file does not need to redefine `story_points` unless you are changing them.

Run the Story Point example like this:

```
mcprojsim simulate first-project-step-6.yaml \
--config  first-project-step-6-config.yaml \
--iterations 5000 --seed 42  --minimal
```

Example result summary:

```
mcprojsim v0.15.2

Project Overview:
  Project: Tiny Landing Page
  Start Date: 2026-03-01
  Number of Tasks: 3
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 80.46 hours (11 working days)
  Median (P50): 80.02 hours
  Std Dev: 11.57 hours
  Minimum: 47.68 hours
  Maximum: 116.54 hours

Project Effort Statistical Summary:
  Mean: 80.46 person-hours (11 person-days)
  Median (P50): 80.02 person-hours
  Std Dev: 11.57 person-hours
  Minimum: 47.68 person-hours
  Maximum: 116.54 person-hours

Calendar Time Confidence Intervals:
  P50: 80.02 hours (11 working days)  (2026-03-16)
  P80: 90.41 hours (12 working days)  (2026-03-17)
  P90: 95.87 hours (12 working days)  (2026-03-17)
```

Story Points are useful when the team has a stable internal understanding of what `1`, `2`, `3`, `5`, `8`, `13`, or `21` mean, but that understanding does not translate directly to raw hours or days. The config file is where that team-specific calibration belongs.

3.9 What each stage added

It is useful to summarize what changed at each step:

Step	New concept	Why it matters
1	Single task with a range estimate	Gives the smallest valid simulation input
2	Dependencies	Turns isolated tasks into a schedule network
3	Configuration and uncertainty factors	Adds team and context knowledge without cluttering the project file
4	Risks	Models probabilistic events that may push the schedule later
5	T-shirt sizes	Lets teams express effort symbolically while still simulating numeric ranges
6	Story Points	Lets agile teams use calibrated relative sizing while still simulating numeric ranges

This is a good way to think about building real project files. Start with the structure that is definitely known. Then add more realism in layers.

3.10 A practical workflow for real projects

When building your own project file, the following sequence usually works well:

1. List the tasks.
2. Add simple effort ranges.
3. Add dependencies.
4. Validate the file.
5. Run an initial simulation.
6. Add uncertainty factors from a shared configuration file.
7. Add the most important task-level and project-level risks.
8. Re-run the simulation and compare the percentiles.

This approach helps prevent over-modeling too early. A simple, valid project file that you understand is more useful than a large, complicated file that nobody fully trusts.

3.11 How this chapter connects to the shipped examples

The tiny examples in this chapter are teaching examples. The files in the `examples/01_first_project` directory show how the same ideas scale up.

If you have understood the stages in this chapter, those larger example files should now feel much easier to read.

3.12 What to try next

Once you are comfortable with the basic structure, good next experiments are:

- change a dependency and see how the forecast moves,
- add a task-level risk and compare P80 or P90,
- move uncertainty-factor labels from one level to another,
- try T-shirt sizes instead of explicit estimate ranges,
- try Story Points with a team-calibrated config mapping,
- export JSON or HTML output and inspect the richer reports.

The next chapters go deeper into the input format and configuration options. The purpose of this chapter was simply to get you from “empty file” to “first useful simulation” in a gradual and understandable way.

CHAPTER 4

Uncertainty Factors

Uncertainty factors are persistent conditions that systematically stretch or compress estimated effort. An inexperienced team, poorly defined requirements, or high technical complexity are uncertainty factors. They are not random events; they are known characteristics of the working environment that affect how long tasks are likely to take.

Understanding both **risks** and **uncertainty factors** — and when to use each — is essential for building realistic project models.

Every task can specify its own set of uncertainty factors. The simulator reads these labels, looks up the corresponding numeric multipliers from the configuration file, and multiplies them together to produce a single adjustment that scales the sampled base duration.

In the task pipeline, uncertainty factors are applied **after estimate sampling** and **before risk impacts** are added.

4.1 Supported Uncertainty Factors

`mcprojsim` supports five uncertainty factors out of the box. Each factor maps qualitative labels to numeric multipliers defined in the configuration file.

4.1.1 Team Experience

Reflects how familiar the team is with the technology and domain.

Level	Default Multiplier	Effect
`high`	0.90	10% faster than baseline
`medium`	1.00	No adjustment (baseline)
`low`	1.30	30% slower than baseline

An experienced team working in a familiar domain is likely to move faster. An inexperienced team facing unfamiliar technology needs more time for learning, mistakes, and rework.

4.1.2 Requirements Maturity

Captures how well-defined and stable the requirements are.

Level	Default Multiplier	Effect
`high`	1.00	Clear, stable requirements
`medium`	1.15	Some ambiguity or gaps

Level	Default Multiplier	Effect
`low`	1.40	High ambiguity, likely changes

Poorly defined requirements lead to rework, misunderstandings, and scope changes. Even a skilled team slows down when the target keeps moving.

4.1.3 Technical Complexity

Indicates how technically challenging the work is.

Level	Default Multiplier	Effect
`low`	1.00	Well-understood technology
`medium`	1.20	Moderate complexity
`high`	1.50	Novel or cutting-edge work

Work involving unfamiliar architectures, new frameworks, or complex algorithms takes longer than routine development even for experienced teams.

4.1.4 Team Distribution

Reflects whether the team is colocated or distributed.

Level	Default Multiplier	Effect
`colocated`	1.00	Same location, easy communication
`distributed`	1.25	Remote/distributed, overhead applies

Distributed teams face communication delays, timezone differences, and coordination overhead. This factor accounts for that systematic slowdown.

4.1.5 Integration Complexity

Measures how much integration work is involved with other systems.

Level	Default Multiplier	Effect
`low`	1.00	Minimal integration needed
`medium`	1.15	Moderate integration effort
`high`	1.35	Complex multi-system integration

Tasks that require coordination with external APIs, databases, or third-party services carry additional overhead from interface changes, testing environments, and compatibility issues.

4.2 How the Combined Multiplier Is Calculated

When a task specifies multiple uncertainty factors, their multipliers are multiplied together to produce a single combined adjustment. The sampled base duration is then scaled by this combined multiplier.

Formula:

$$\text{adjusted_duration} = \text{base_duration} \times \prod_{i=1}^{n} \text{multiplier}_i$$

Example calculation:

Consider a task with the following uncertainty factors:

Factor	Level	Multiplier
`team_experience`	`medium`	1.00
`requirements_maturity`	`low`	1.40
`technical_complexity`	`high`	1.50
`team_distribution`	`distributed`	1.25
`integration_complexity`	`medium`	1.15

Combined multiplier: $1.00 \times 1.40 \times 1.50 \times 1.25 \times 1.15 = 3.0188$

If the sampled base duration is 5 days, the adjusted duration becomes $5 \times 3.0188 \approx 15.1$ days.

This illustrates why multiple adverse factors compound quickly. Even moderate individual adjustments can produce a significant overall effect when multiplied together.

4.3 Validation and Fallback Behavior

The current implementation distinguishes between project-file schema validation and runtime multiplier lookup:

- The task model defines five named uncertainty fields: `team_experience`, `requirements_maturity`, `technical_complexit` `team_distribution`, `integration_complexity`.
- When these labels are looked up at runtime, missing factor names or unknown levels in the active configuration fall back to multiplier `1.0`.

Practical implications:

- If a configured factor name is missing entirely, that factor contributes no scaling (`1.0`).
- If a level string is not present under a configured factor, that factor also contributes `1.0`.
- In other words, unknown or unmatched labels do not raise an error in multiplier lookup; they behave like neutral multipliers.

For predictable behavior, keep project-file labels aligned with your configured level names.

4.4 Default Values

If a task does not specify a particular uncertainty factor, the default level is applied. The defaults are:

Factor	Default Level
`team_experience`	`medium`
`requirements_maturity`	`high`
`technical_complexity`	`low`
`team_distribution`	`colocated`
`integration_complexity`	`low`

A task that omits the `uncertainty_factors` block entirely still receives these defaults, resulting in a combined multiplier based on the default levels. Each default level maps to a multiplier of exactly `1.0` in the default configuration, so the combined multiplier for a task with no explicit uncertainty factors is:

$$1.00 \times 1.00 \times 1.00 \times 1.00 \times 1.00 = 1.0$$

This means a task with no explicit uncertainty factors will have its sampled base duration used as-is, with no stretching applied before risks are evaluated.

4.5 Defining Uncertainty Factors in the Project File

Uncertainty factors are specified per task. You only need to include the factors you want to set explicitly; any omitted factors use their default level.

```
tasks:
  - id: "task_001"
    name: "Database schema design"
    estimate:
      low: 3
      expected: 5
      high: 10
      unit: "days"
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "medium"
      technical_complexity: "low"
      team_distribution: "colocated"
      integration_complexity: "low"
```

You may also specify only the factors that differ from the defaults:

```
tasks:
  - id: "task_010"
    name: "Prototype ML pipeline"
    estimate:
      low: 8
      expected: 14
      high: 25
      unit: "days"
    uncertainty_factors:
      team_experience: "low"
      technical_complexity: "high"
```

Here, `requirements_maturity`, `team_distribution`, and `integration_complexity` all take their default values.

4.6 Configuring Uncertainty Factor Multipliers

The numeric multipliers for each factor and level are defined in the configuration file (`config.yaml`), not in the project file. This separation means the project file describes the project conditions, while the configuration file defines how the organization interprets those conditions.

4.6.1 Default Configuration

```
uncertainty_factors:
  team_experience:
    high: 0.90      # Experienced team is 10% faster
    medium: 1.0     # Baseline
    low: 1.30       # Inexperienced team is 30% slower
  requirements_maturity:
    high: 1.0       # Well-defined requirements
    medium: 1.15    # Some ambiguity
    low: 1.40       # High ambiguity
  technical_complexity:
    low: 1.0        # Simple, well-understood technology
    medium: 1.20    # Moderate complexity
    high: 1.50      # High complexity, cutting-edge tech
  team_distribution:
    colocated: 1.0  # Team in same location
    distributed: 1.25  # Distributed team with communication overhead

  integration_complexity:
    low: 1.0        # Minimal integration
    medium: 1.15    # Moderate integration
    high: 1.35      # Complex integration with multiple systems
```

4.7 Customizing Multipliers

You can adjust the multipliers to match your organization's experience. For example, if your organization has found that distributed teams incur a higher overhead than the default 25%, you might change that value:

```
uncertainty_factors:
  team_distribution:
    colocated: 1.0
    distributed: 1.40  # 40% overhead for distributed work
```

Any values you specify in your configuration file are merged with the defaults. You only need to include the factors or levels you want to override.

If you provide a factor with only some levels (for example only `distributed` under `team_distribution`), unspecified levels continue to use defaults.

If you add additional factor names in configuration that are not referenced by the project task schema, they are effectively unused by the current simulation pipeline.

4.8 Recommended Authoring Pattern

- Keep task files focused on the five supported factor fields.
- Keep organization-specific multiplier calibration in `config.yaml`.
- Override only the factor levels that differ from defaults.
- Recheck combined multiplier magnitude for heavily adverse combinations to avoid unintentionally extreme schedules.

4.9 Summary

Uncertainty factor are persistent working conditions affecting the team and specific task involved in the project.

- Uncertainty factors are **multiplicative**: they scale the sampled base duration before risks are evaluated.
- Multiple uncertainty factors **compound**: their multipliers are multiplied together, which can produce significant combined effects.

CHAPTER 5

Task Estimation

This chapter explains how individual tasks are estimated in `mcprojsim` and how those estimates drive the Monte Carlo simulation. Every task in a project file needs an effort estimate. That estimate is not a single fixed number — it is a structured description of plausible effort that the simulator samples from in each iteration.

The goal of this chapter is to explain all supported estimation methods, the probability distributions behind them, and how to specify each one in a project file.

5.1 How estimation works in the simulation

In each Monte Carlo iteration, the simulator processes every task as follows:

1. **Read the estimate** from the task definition.
2. **Resolve symbolic estimates** (T-shirt sizes or story points) into numeric ranges using the configuration file.
3. **Sample a base duration** from the specified probability distribution.
4. **Apply uncertainty factors** (multiplicative adjustment).
5. **Evaluate task-level risks** (additive impact if triggered).

The result is the task's effective duration for that iteration. Over thousands of iterations, the sampled values form a distribution that captures the inherent uncertainty in the estimate.

This chapter focuses on steps 1 through 3. Uncertainty factors and risks are covered in Uncertainty Factors and Risks.

5.2 Estimation methods at a glance

`mcprojsim` supports four ways to express task effort. Each method provides the same information to the simulator — a probability distribution to sample from — but is suited to different estimation contexts.

Method	Input	Resolved to	Best for
Explicit range (triangular)	`low`, `expected`, `high`	Used directly	Teams comfortable giving numeric estimates with bounded outcomes
Explicit range (log-normal)	`low`, `expected`, `high`, `distribution: "lognormal"`	Used directly	Tasks with significant right-tail risk or open-ended uncertainty

Method	Input	Resolved to	Best for
T-shirt size	`t_shirt_size` (e.g., `"M"` or `"epic.M"`)	Looked up in config → `low`, `expected`, `high`	Early-stage or relative estimation
Story points	`story_points` (e.g., `5`)	Looked up in config → `low`, `expected`, `high`	Teams using story point estimation practices

Table I:

All four methods ultimately feed into the same simulation machinery. T-shirt sizes and story points are convenience mappings that resolve to explicit ranges before sampling begins.

5.3 Explicit range estimates

The most direct way to estimate a task is to provide three values that describe the range of plausible effort:

Parameter	Meaning	Required
`low`	The shortest plausible duration (optimistic)	Yes
`expected`	The expected duration under normal conditions	Yes
`high`	The longest plausible duration (pessimistic)	Yes
`unit`	The time unit: `"hours"`, `"days"`, or `"weeks"` (default: `"hours"`)	No

Table II:

The three values must satisfy: `low ≤ expected ≤ high`.

5.3.1 Basic example

```
tasks:
  - id: "task_001"
    name: "Database schema design"
    estimate:
      low: 3
      expected: 5
      high: 10
      unit: "days"
```

This tells the simulator: in the best case, the task takes about 3 days; most likely it takes about 5 days; in a difficult scenario, it could take up to 10 days. Because `unit: "days"` is specified, the simulator converts these values to hours internally (using the project's `hours_per_day` setting, which defaults to 8). The simulator then samples from this range in every iteration, producing a distribution of task durations across the full simulation.

5.3.2 Choosing good range values

The three values are not arbitrary guesses. They carry specific meaning:

- **`low`** is not the theoretical fastest time if everything goes perfectly. It is the shortest duration that is still realistic given the scope of work. Ask: "If things go smoothly, how fast could this realistically be done?"
- **`expected`** is the duration you would expect under normal conditions. It represents the mode — the single most probable outcome. Ask: "If I had to pick one number for this task, what would it be?"
- **`high`** is not a catastrophic worst case. It is the longest duration that is credibly possible if several things go wrong, but without extraordinary events like losing the entire team. Ask: "If this task hits significant headwinds, how long could it take?"

The spread between `low` and `high` reflects how uncertain you are. A narrow range (e.g., 4 / 5 / 6) means high confidence. A wide range (e.g., 3 / 5 / 15) means substantial uncertainty, and the simulation results will reflect that.

5.3.3 Asymmetric ranges

In practice, most software tasks have more upside risk than downside opportunity. It is common for the distance between `expected` and `high` to be larger than the distance between `low` and `expected`. This naturally produces a right-skewed distribution — reflecting the reality that tasks are more likely to run over than to finish early by the same margin.

```
estimate:
  low: 3
  expected: 5
  high: 15
  unit: "days"
```

Here, the best-case savings is 2 days (5 minus 3), but the worst-case overrun is 10 days (15 minus 5). This kind of asymmetry is realistic and the triangular distribution handles it naturally.

5.4 Near-deterministic estimates

Sometimes a task has very little uncertainty — for example, a well-understood routine task with a known duration. You might be tempted to set `low`, `expected`, and `high` to the same value.

However, the triangular distribution requires `low` < `high`. Setting all three values equal will produce a validation error at sampling time. To model a near-deterministic task, use a very narrow range:

```
estimate:
  low: 4.9
  expected: 5
  high: 5.1
  unit: "days"
```

This effectively produces a duration of approximately 5 days in every iteration, with negligible variation. The sampled values will be very close to 5.0 but not exactly identical, which is usually close enough for practical purposes.

5.5 Probability distributions

The estimate range defines the inputs. The probability distribution defines how values are sampled from that range. `mcprojsim` supports two distributions.

5.5.1 Triangular distribution (default)

The triangular distribution is the default and most commonly used distribution in project estimation. It is defined by three parameters — minimum, mode, and maximum — which correspond directly to the `low`, `expected`, and `high` fields in the estimate.

Properties:

Property	Description
Shape	Triangle-shaped probability density
Parameters	`low`, `expected` (mode), `high`
Support	Values between `low` and `high` only
Skewness	Determined by position of `expected` within the range
Implementation	`numpy.random.triangular`

Table III:

When to use it:

- When the team can provide three-point estimates (optimistic, likely, pessimistic)
- When you want a bounded distribution — no sampled value can fall outside the stated range
- When you want an intuitive model that is easy to explain to stakeholders
- For most software estimation tasks — this is the recommended default

Characteristics:

The triangular distribution concentrates probability around the `expected` value and tapers linearly toward the extremes. It guarantees that no sample will be less than `low` or greater than `high`, which can be reassuring when the team has confidence in the boundaries.

If `expected` is centered between `low` and `high`, the distribution is symmetric. If `expected` is closer to `low` (common in software estimation), the distribution is right-skewed — producing a longer tail toward the high end.

Specification:

The triangular distribution is the default, so you do not need to specify it explicitly. Both of the following are equivalent:

```
# Implicit (triangular is the default)
estimate:
  low: 3
  expected: 5
  high: 10
  unit: "days"

# Explicit
estimate:
  distribution: "triangular"
  low: 3
  expected: 5
  high: 10
  unit: "days"
```

5.5.2 Distribution precedence

The distribution used for a task is resolved in this order, from highest to lowest priority:

1. **Per-task `distribution` field** — set directly on a task's `estimate` block. Always takes precedence.
2. **Project-level `distribution` field** — set under `project:` in the project file. Acts as the default for all tasks in that project.
3. **Built-in default** — `triangular` when nothing else is specified.

This means you can set `distribution: lognormal` once at the project level and get log-normal sampling for every task without touching each individual task, while still overriding specific tasks back to `triangular` (or another distribution) where needed.

There is currently no global `--distribution` CLI override for `simulate`.

Example — project-wide lognormal with one triangular override:

```yaml
project:
  name: "Research Project"
  start_date: "2026-04-01"
  distribution: "lognormal"          # all tasks default to lognormal

tasks:
  - id: "task_001"
    name: "Write specification"
    estimate:                         # inherits lognormal from project
      low: 1
      expected: 3
      high: 8
      unit: "days"

  - id: "task_002"
    name: "Set up CI pipeline"
    estimate:
      distribution: "triangular"     # overrides project default for this task only
      low: 0.5
      expected: 1
      high: 2
      unit: "days"
```

5.5.3 Log-normal distribution

The log-normal distribution is an alternative that produces a right-skewed, unbounded distribution. It is useful for tasks where the upside risk (overrun) is potentially much larger than the downside opportunity (early finish).

`mcprojsim` implements this as a **shifted log-normal** so end users can keep working with the same `low`, `expected`, and `high` fields they already use for triangular estimates.

Properties:

Property	Description
Shape	Right-skewed, long tail toward high values
Parameters	`low`, `expected`, `high`, plus configured percentile for `high`
Support	All values greater than `low`
Skewness	Always right-skewed; heavier tail with larger fitted sigma
Implementation	`numpy.random.lognormal`

Table IV:

Internally the simulator defines a shifted variable:

$$Y = X - \text{low}$$

where X is the actual duration and Y follows a standard log-normal distribution.

The inputs are interpreted as:

- `low`: hard minimum / shift
- `expected`: mode of X
- `high`: configured percentile of X (P95 by default)

This gives:

$$e^{\mu - \sigma^2} = \text{expected} - \text{low}$$

and

$$e^{\mu + z_p \sigma} = \text{high} - \text{low}$$

where z_p is the z-score for the configured percentile. For the default P95, $z_p \approx 1.645$.

Subtracting the two equations yields:

$$\sigma^2 + z_p \sigma = \ln(\text{high} - \text{low}) - \ln(\text{expected} - \text{low})$$

The simulator solves that quadratic for the positive σ, then computes:

$$\mu = \ln(\text{expected} - \text{low}) + \sigma^2$$

When to use it:

- For tasks with significant right-tail risk — where overruns could be much larger than expected
- When you are uncertain about the upper bound — the log-normal has no maximum, which may better represent truly open-ended tasks
- For research-oriented work, prototyping, or integration tasks with high unknowns
- When historical data suggests log-normal effort patterns (this is supported by research in some software estimation contexts)

When not to use it:

- When you have clear, defensible bounds on the task duration — in that case, a triangular distribution is more appropriate
- When stakeholders need guarantees that the sampled value will not exceed a certain threshold - the log-normal can produce extreme outliers
- When the team is not comfortable with unbounded estimates

Specification:

```
estimate:
  distribution: "lognormal"
  low: 2
  expected: 5
  high: 14
  unit: "days"
```

Here, 2 days is the minimum, 5 days is the most likely outcome, and 14 days is interpreted as the configured high percentile (P95 by default). Wider gaps between `expected` and `high` produce a heavier right tail.

5.5.4 Configuring the log-normal high percentile

The `high` value for log-normal estimates is interpreted using the configuration setting `lognormal.high_percentile`.

```
lognormal:
  high_percentile: 95
```

Allowed values are: `70`, `75`, `80`, `85`, `90`, `95`, `99`.

- Lower values (for example `80`) treat `high` as a less extreme percentile and produce a lighter right tail.
- Higher values (for example `99`) treat `high` as a more extreme percentile and produce a heavier right tail.

Example: a research task with high uncertainty

```
tasks:
  - id: "prototype"
    name: "Prototype ML model"
    estimate:
      distribution: "lognormal"
      low: 4
      expected: 10
      high: 30
      unit: "days"
    dependencies: []
```

Here, the most likely duration is 10 days, but the long right tail means that in some iterations the sampled value could be 20 or 30 days, reflecting the genuine uncertainty in exploratory work.

5.5.5 Comparing the two distributions

Aspect	Triangular	Log-normal
Bounded	Yes — samples always within [min, max]	No — no upper bound
Parameters	`low`, `expected`, `high`	`low`, `expected`, `high` + configured high percentile
Skewness	Depends on parameter placement	Always right-skewed
Intuition	Easy to explain to non-technical stakeholders	Requires more statistical background
Extreme values	Impossible beyond stated bounds	Possible — long tail
Recommended for	Most estimation tasks	Tasks with open-ended risk

Table V:

For most projects, the triangular distribution is the right choice. It is intuitive, bounded, and maps naturally to how teams think about estimates. The log-normal distribution is a specialized tool for tasks where the unbounded right tail better reflects reality.

5.5.6 Why is log-normal a good model for software (or project) estimation?

Individual task durations are often observed to follow a log-normal distribution in empirical studies of projects and software projects in particular. This can be explained by noticing that task durations are influenced by many small, multiplicative factors — such as the complexity of the work, the efficiency of the team, the quality of requirements, and unforeseen issues that arise during execution. When you have a process that is influenced by many independent multiplicative factors, the resulting distribution of outcomes tends to be log-normal. This can be easily seen by observing that if an estimate can be written as

$$X = X_0 \times F_1 \times F_2 \times ... \times F_n$$

where X_0 is the base estimate and F_i are the multiplicative factors, then taking the logarithm gives:

$$\ln(X) = \ln(X_0) + \ln(F_1) + \ln(F_2) + ... + \ln(F_n)$$

If we assume the $\ln(F_i)$ are independent and identically distributed, then by the Central Limit Theorem, $\ln(X)$ will tend to be normally distributed, which means that X is log-normally distributed.

5.5.7 Determining log-normal parameters

An example will clarify this.

Assume we want to make a log normal distribution from the three point estimate

low	expected	high
5	9	16

Table VI:

To use a log-normal distribution based on a three-point estimate we need a way to fit those three point to the log-normal distribution.

Since log-normal doesn't have a natural "minimum," the standard approach is to shift the distribution by your minimum, then fit μ and σ to the remaining two constraints.

For the high estimate we also need to define which confidence level of the log-normal distribution it should correspond to. A common value is to use the 90%- or 95%- percentile. To make this relative to the distribution we use the z-value, i.e. the fraction number of standard deviation from the mean value.

Confidence level	z-score
80th percentile	0.842
85th percentile	1.036
90th percentile	1.282
95th percentile	1.645
97.5th percentile	1.960
99th percentile	2.326

Table VII:

Great practical question. Since log-normal doesn't have a natural "minimum," the standard approach is to **shift the distribution** by your minimum, then fit μ and σ to the remaining two constraints.

Derivation

Define the shifted variable:

$Y = X - 5$ (where X is actual duration, Y is the excess above minimum)

Y follows a standard log-normal, so $\texttt{ln}(Y) \sim \texttt{Normal}(\mu, \sigma^2)$

The constraints become: - Mode of $X = 9 \implies$ mode of $Y = 4 \implies e^{(\mu - \sigma^2)} = 4$ - 95th percentile of $X = 16 \implies$ 95th percentile of $Y = 11 \implies e^{(\mu + 1.645\sigma)} = 11$

Solving

From the two equations:

$$\mu - \sigma^2 = \texttt{ln}(4) \approx 1.386 \qquad (1)$$

$$\mu + 1.645\sigma = \texttt{ln}(11) \approx 2.398 \qquad (2)$$

Subtract (1) from (2):

$$\sigma^2 + 1.645\sigma - 1.012 = 0$$

Solving the quadratic gives

$$\sigma \approx 0.472$$

Then from equation (1):

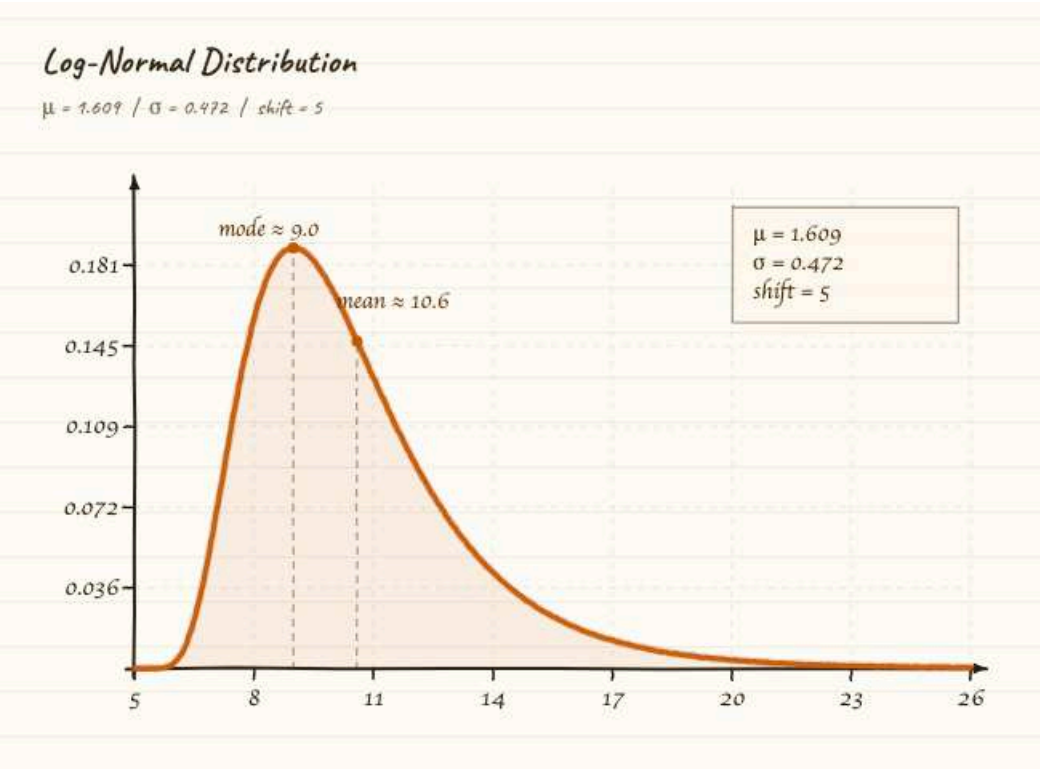


Figure 5.1: Log-normal task duration (5,9,16)

$$\mu = \ln(4) + \sigma^2 \approx 1.386 + 0.223 \approx 1.609$$

Resulting Parameters

Parameter	Value
Shift (minimum)	5 days
μ (log-mean of Y)	**1.609**
σ (log-std of Y)	**0.472**

Table VIII:

Sanity Check

With these parameters, the distribution of X gives:

Statistic	Value
Minimum (hard floor)	5 days
Mode	$\sim$ 9 days
Median	$5 + e^{\mu} = 5 + 5.0 \approx 10\,\texttt{days}$
Mean	$5 + e^{(\mu+\sigma^2/2)} \approx 5 + 5.6 \approx 10.6\,\texttt{days}$
95th percentile	$\sim$ 16 days

Table IX:

The median and mean both sitting above the mode is the classic log-normal skew in action — reinforcing that even with a "9 day" estimate, you should plan for closer to **10.5–11 days** on average.

5.5.8 Deriving the formula to translate `low`, `expected`, `high` to log-normal parameters

Starting from the two equations:

$$\mu - \sigma^2 = \ln(\text{expected} - \text{low})$$
$$\mu + z \cdot \sigma = \ln(\text{high} - \text{low})$$

Then using the same reasoning as in the previous example we express the log-normal parameters in closed form as

$$\sigma = \frac{-z + \sqrt{z^2 + 4R}}{2}$$

$$\mu = \ln(\text{expected} - \text{low}) + \sigma^2$$

where

$$R = \ln\left[\frac{\text{high} - \text{low}}{\text{expected} - \text{low}}\right]$$

z = z-score for the chosen confidence level for high estimate

and where **z** is the z-score corresponding to your chosen confidence level for the high estimate as seen in ***Table VII:***

5.6 T-shirt size estimates

T-shirt sizing is a relative estimation technique where tasks are classified into sizes such as `XS`, `S`, `M`, `L`, `XL`, and `XXL`, scoped by named categories (`bug`, `story`, `epic`, `business`, `initiative`). This is useful when teams need different calibration levels across planning horizons.

In `mcprojsim`, each T-shirt size is mapped to a numeric range (`low`, `expected`, `high`) inside a category in the configuration file. During simulation, a bare value like `M` resolves through `t_shirt_size_default_category` (default: `story`), while a qualified value like `epic.M` resolves directly.

5.6.1 Default T-shirt size mappings

The default unit for T-shirt sizes is **hours** (configurable via `t_shirt_size_unit` in the configuration file).

The built-in default category is `story`, so a bare value like `M` resolves as `story.M` unless you change `t_shirt_size_default_categor` or pass `--tshirt-category` on the CLI.

Table 5.10: Table: Bug - category

Category	Size	low (hours)	expected (hours)	high (hours)
`bug`	`XS`	0.5	1	4
`bug`	`S`	1	3	10
`bug`	`M`	3	8	24

Category	Size	low (hours)	expected (hours)	high (hours)
bug	L	8	20	60
bug	XL	20	40	100
bug	XXL	40	80	200

Table 5.11: Table: Story - category

Category	Size	low (hours)	expected (hours)	high (hours)
story	XS	3	5	15
story	S	5	16	40
story	M	40	60	120
story	L	160	240	500
story	XL	320	400	750
story	XXL	400	500	1200

Table 5.12: Table: Epic - category

Category	Size	low (hours)	expected (hours)	high (hours)
epic	XS	20	40	60
epic	S	60	120	170
epic	M	120	240	400
epic	L	290	480	700
epic	XL	600	1000	1500
epic	XXL	1200	2000	3200

Table 5.13: Table: Business/Program - category

Category	Size	low (hours)	expected (hours)	high (hours)
business	XS	400	800	2000
business	S	800	2000	5000
business	M	2000	4000	10000
business	L	4000	8000	20000
business	XL	8000	16000	40000
business	XXL	16000	32000	80000

Table 5.14: Table: Initiative - category

Category	Size	low (hours)	expected (hours)	high (hours)
initiative	XS	2000	4000	10000
initiative	S	4000	10000	25000
initiative	M	10000	20000	50000
initiative	L	20000	40000	100000
initiative	XL	40000	80000	200000
initiative	XXL	80000	160000	400000

These category-specific defaults let you keep the same symbolic size scale while tuning absolute magnitude for different planning scopes.

5.6.2 Specifying a T-shirt size estimate

T-shirt size estimates must **not** include a `unit` field in the project file. The unit is determined by the configuration file's `t_shirt_size_unit` setting (default: `"hours"`).

```
tasks:
  - id: "design_ui"
    name: "UI/UX Design"
    estimate:
      t_shirt_size: "M"
    dependencies: []
```

You can also use a qualified value:

```
estimate:
  t_shirt_size: "epic.M"
```

Bare long-form aliases are accepted too, for example:

```
estimate:
  t_shirt_size: "Medium"
```

With the built-in defaults, the bare `"M"` example above is equivalent to writing:

```
estimate:
  low: 120
  expected: 240
  high: 400
  unit: "hours"
```

If you want story-scale values instead, either use a qualified value (`story.M`) or set `t_shirt_size_default_category: story` in your configuration.

The resolution happens automatically when the simulation runs. The resolved values are then converted to hours internally (if they are not already in hours) using the project's `hours_per_day` setting.

Including `unit` on a T-shirt size estimate is a validation error:

```
# INVALID — will produce a validation error
estimate:
  t_shirt_size: "M"
  unit: "days"   # ERROR: T-shirt size estimates must not specify 'unit'
```

5.6.3 Full example: project using T-shirt sizes

```
project:
  name: "Tiny Landing Page"
  description: "T-shirt sizing example"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]
  hours_per_day: 8
tasks:
  - id: "task_001"
    name: "Design page"
    estimate:
      t_shirt_size: "S"
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "high"

  - id: "task_002"
    name: "Build page"
    estimate:
      t_shirt_size: "M"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"

  - id: "task_003"
    name: "Deploy page"
    estimate:
      t_shirt_size: "XS"
    dependencies: ["task_002"]
```

Note that no `unit` field appears on any task — the unit is taken from the configuration.

5.6.4 Customizing T-shirt sizes in the configuration file

The default mappings can be overridden in the configuration file. This allows organizations to calibrate the sizes to match their own team velocity and task granularity. For example, a team that works in shorter cycles might define smaller ranges:

```
# config.yaml
t_shirt_sizes:
  story:
    M:
      low: 45
      expected: 65
      high: 130
  epic:
    M:
      low: 240
      expected: 520
      high: 1400
t_shirt_size_default_category: epic
```

Any category/size values you define in the configuration file override those defaults while untouched categories and sizes remain available.

5.6.5 Choosing the unit for T-shirt sizes

The numeric ranges in the T-shirt size mappings are interpreted in the unit specified by `t_shirt_size_unit` in the configuration file. The default is `"hours"`:

```
# config.yaml — T-shirt sizes in hours (the default)
t_shirt_size_unit: "hours"

t_shirt_sizes:
  story:
    S:
      low: 1
      expected: 2
      high: 4       # 1-4 hours
    M:
      low: 3
      expected: 5
      high: 8       # 3-8 hours
    L:
      low: 5
      expected: 8
      high: 13      # 5-13 hours
```

If your team thinks about T-shirt sizes in terms of working days rather than hours, set `t_shirt_size_unit` to `"days"`. The simulator will then convert the ranges to hours using `hours_per_day`:

```
# config.yaml — T-shirt sizes in days
t_shirt_size_unit: "days"

t_shirt_sizes:
  story:
    S:
      low: 0.5
      expected: 1
      high: 2       # 0.5-2 days -> 4-16 hours (at 8 hours/day)
    M:
      low: 1
      expected: 2
      high: 4       # 1-4 days -> 8-32 hours
    L:
      low: 2
      expected: 4
      high: 7       # 2-7 days -> 16-56 hours
```

The unit setting applies to **all** T-shirt sizes in the configuration. You cannot mix units across individual sizes — the `t_shirt_size_unit` value governs the entire mapping. The project file never specifies a unit for T-shirt size estimates; the unit is always determined by this configuration setting.

5.7 Story point estimates

Story points are a common estimation unit in agile teams. In `mcprojsim`, story point values are mapped to day-based numeric ranges in the configuration file, similar to T-shirt sizes.

5.7.1 Default story point mappings

The default unit for story points is **days** (configurable via `story_point_unit` in the configuration file).

Story points	low (days)	expected (days)	high (days)
1	0.5	1	3
2	1	2	4
3	1.5	3	5
5	3	5	8
8	5	8	15
13	8	13	21
21	13	21	34

The allowed values are: **1, 2, 3, 5, 8, 13, 21**. Other values are rejected during validation.

5.7.2 Specifying a story point estimate

Story point estimates must **not** include a `unit` field in the project file. The unit is determined by the configuration file's `story_point_unit` setting (default: `"days"`).

```
tasks:
  - id: "task_001"
    name: "Design page"
    estimate:
      story_points: 5
    dependencies: []
```

Including `unit` on a story point estimate is a validation error:

```
# INVALID — will produce a validation error
estimate:
  story_points: 5
  unit: "days"   # ERROR: Story Point estimates must not specify 'unit'
```

5.7.3 Full example: project using story points

```
project:
  name: "Tiny Landing Page"
  description: "Story point estimation example"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]
  hours_per_day: 8

tasks:
  - id: "task_001"
    name: "Design page"
    estimate:
      story_points: 2
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "high"
  - id: "task_002"
    name: "Build page"
    estimate:
      story_points: 5
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"
  - id: "task_003"
    name: "Deploy page"
    estimate:
      story_points: 1
    dependencies: ["task_002"]
```

Note that no `unit` field appears on any task — the unit is taken from the configuration (`story_point_unit`, which defaults to `"days"`).

5.7.4 Customizing story point mappings in the configuration file

Different teams have different velocity patterns. The configuration file lets you adjust the day-based ranges that each story point value maps to:

```
# config.yaml
story_points:
  1:
    low: 0.5
    expected: 1
    high: 2.5
  2:
    low: 1
    expected: 2
    high: 3.5
  5:
    low: 3.5
    expected: 5.5
    high: 9
```

You only need to include the story point values you want to override. Any values not specified in your configuration will use the built-in defaults. Note that you can only use values from the allowed set (1, 2, 3, 5, 8, 13, 21) — the allowed values are defined in the code, not in configuration.

5.7.5 Choosing the unit for story points

The numeric ranges in the story point mappings are interpreted in the unit specified by `story_point_unit` in the configuration file. The default is `"days"`:

```
story_point_unit: "days"
story_points:
  1:
    low: 0.5
    expected: 1
    high: 3       # 0.5–3 days → 4–24 hours (at 8 hours/day)
  5:
    low: 3
    expected: 5
    high: 8       # 3–8 days → 24–64 hours
```

If your team calibrates story points directly in hours, set `story_point_unit` to `"hours"`:

```
story_point_unit: "hours"
story_points:
  1:
    low: 1
    expected: 3
    high: 6       # 1–6 hours
  5:
    low: 8
    expected: 16
    high: 32      # 8–32 hours
```

As with T-shirt sizes, the unit setting applies to **all** story point mappings in the configuration. The project file never specifies a unit for story point estimates — the unit is always determined by `story_point_unit`.

> **Note:** The `t_shirt_size_unit` and `story_point_unit` settings are independent. It is perfectly valid to have T-shirt sizes in hours and story points in days (the defaults), or any other combination.

5.8 Mixing estimation methods

A single project can use different estimation methods for different tasks. This is useful when some tasks are well-understood enough for explicit day estimates while others are better expressed as relative sizes.

```yaml
tasks:
  - id: "research"
    name: "Technology research"
    estimate:
      distribution: "lognormal"
      low: 3
      expected: 8
      high: 20
      unit: "days"
    dependencies: []

  - id: "design"
    name: "Architecture design"
    estimate:
      t_shirt_size: "L"
    dependencies: ["research"]
  - id: "implementation"
    name: "Core implementation"
    estimate:
      low: 10
      expected: 15
      high: 25
      unit: "days"
    dependencies: ["design"]
  - id: "testing"
    name: "Integration testing"
    estimate:
      story_points: 8
    dependencies: ["implementation"]
```

All four estimation methods are combined in one project. Each task's estimate is resolved and sampled independently using the same simulation pipeline. The simulator converts all values to hours internally using the appropriate unit for each estimate method.

Implementation detail: if a symbolic estimate (`t_shirt_size` or `story_points`) is present together with explicit numeric fields (`low` / `expected` / `high`), the symbolic estimate path is used and numeric fields are effectively ignored.

Recommended authoring style: avoid mixing symbolic and explicit fields on the same task. Use exactly one method per task to keep intent unambiguous.

5.9 The `unit` field

The `unit` field specifies the time unit for numeric estimate values. The simulator uses this field to convert all values to **hours** internally, which is the canonical unit used throughout the simulation.

5.9.1 Supported values

The `unit` field accepts exactly three values:

Value	Meaning	Conversion to hours
`"hours"`	Working hours	Used as-is (1 hour = 1 hour)
`"days"`	Working days	Multiplied by `hours_per_day` (default 8)
`"weeks"`	Working weeks	Multiplied by `hours_per_day × 5` (default 40)

Any other value is a validation error.

5.9.2 Defaults by estimate type

Estimate type	Default unit	`unit` field allowed?
Explicit range	`"hours"`	Yes — `"hours"`, `"days"`, or `"weeks"`
T-shirt size	*(from config)*	**No** — specifying `unit` is a validation error
Story points	*(from config)*	**No** — specifying `unit` is a validation error

For explicit range estimates, the default unit is `"hours"`. If you omit the `unit` field, your numeric values are interpreted as hours.

For T-shirt sizes and story points, the unit is controlled by the configuration file (`t_shirt_size_unit` and `story_point_unit` respectively). The project file must not include a `unit` field for these estimate types.

5.9.3 The `hours_per_day` setting

The project metadata supports an `hours_per_day` field (default: `8.0`) that controls how days and weeks are converted to hours:

```
project:
  name: "My Project"
  start_date: "2026-03-01"
  hours_per_day: 6    # 6-hour productive work days
```

This setting affects:

- conversion of `unit: "days"` values (multiplied by `hours_per_day`)
- conversion of `unit: "weeks"` values (multiplied by `hours_per_day × 5`)
- conversion of simulation results from hours back to working days for reporting

5.9.4 Unit conversion is automatic

The simulator automatically converts all estimate values to hours before simulation begins. This means you can safely mix units across tasks in the same project:

```
tasks:
  - id: "quick_fix"
    estimate:
      low: 2
      expected: 4
      high: 8
      unit: "hours"    # Small task estimated in hours
  - id: "major_feature"
    estimate:
      low: 2
      expected: 4
      high: 8
      unit: "days"     # Large task estimated in days
```

The simulator converts the second task's values to hours (2×8=16, 4×8=32, 8×8=64) before sampling. The two tasks are compared on a common scale.

5.9.5 Simulation output

All simulation results are reported in hours. The CLI and export formats also show working days (computed as `ceil(hours / hours_per_day)`) and projected delivery dates (skipping weekends, based on `start_date`).

5.9.6 Unit field for T-shirt sizes and story points

When a T-shirt size or story point estimate is resolved during simulation, the numeric values come from the configuration file. The unit for those values is also defined in the configuration:

- `t_shirt_size_unit`: defaults to `"hours"`
- `story_point_unit`: defaults to `"days"`

The resolved values are then converted to hours using the same conversion logic. This means a story point estimate of `5` with default configuration resolves to `min=3, expected=5, max=8` in days, which the simulator converts to `min=24, expected=40, max=64` in hours.

5.10 Validation rules

`mcprojsim` validates every task estimate before the simulation runs. Understanding the validation rules helps avoid errors:

Rule	Applies to	Error if violated
`low` $\leq$ `expected` $\leq$ `high`	Triangular distribution	Yes — values must be in order
`low` $<$ `high`	Triangular distribution	Yes — NumPy requires strict inequality
`expected` > 0	All explicit estimates	Yes — zero or negative not allowed
`low` $\quad 0$	All explicit estimates	Yes — negative values not allowed
`low` $<$ `expected` $<$ `high`	Log-normal distribution	Yes — shifted fit requires a strict range
`low`, `expected`, `high` all provided	Log-normal distribution	Yes — all three required
`low`, `expected`, `high` all provided	Triangular distribution	Yes — all three required
Only one symbolic type	T-shirt / story point	Yes — cannot use both on one task
At least one estimate mode provided (`expected` or symbolic)	All tasks	Yes — estimate cannot be empty
Story point value in allowed set	Story points	Yes — must be 1, 2, 3, 5, 8, 13, or 21
`unit` must be `"hours"`, `"days"`, or `"weeks"`	Explicit estimates	Yes — free-form strings not accepted
No `unit` on symbolic estimates	T-shirt / story point	Yes — unit comes from config
`t_shirt_size` token shape must be `<size>` or `<category>.<size>`	T-shirt size	Yes — invalid token formats fail validation
`t_shirt_size` category and size must exist in active config	T-shirt size	Yes — unknown category/size fails resolution

5.11 Summary

Estimation method	Required fields	Distribution	Good for
Explicit range (triangular)	`low`, `expected`, `high`	Triangular (bounded)	Well-understood tasks with clear bounds

Estimation method	Required fields	Distribution	Good for
Explicit range (log-normal)	`low`, `expected`, `high`, `distribution: "lognormal"`	Shifted log-normal	Exploratory tasks with open-ended risk
T-shirt size	`t_shirt_size`	Inherits project/task distribution after config lookup	Relative estimation, early planning
Story points	`story_points`	Inherits project/task distribution after config lookup	Agile teams using story point practices

All methods produce the same kind of output: a sampled duration for each iteration. The choice of method depends on what information the team is comfortable providing, not on any difference in simulation behavior.

CHAPTER 6

Risks

This chapter explains how `mcprojsim` models schedule variation due to **risks**. (The other source of schedule variation comes from **uncertainty factors** — see Uncertainty Factors.) Both influence the simulated project duration, but they represent fundamentally different ideas and are handled differently by the simulation engine.

Risks are discrete events that may or may not happen. A failed security audit, the departure of a key developer, or a sudden change in requirements are all risks. Each risk has a probability of occurring and an impact if it does.

6.1 How Risks Affect the Simulation

During each Monte Carlo iteration, every risk is independently evaluated. The simulator draws a random number and compares it to the risk's probability. If the risk triggers, its impact is added to the relevant duration. If it does not trigger, no penalty is applied.

Because risks are evaluated independently in each iteration, the overall project duration distribution naturally reflects the combined likelihood of various risk combinations. Some iterations will have no risks trigger at all. Others will see several risks fire simultaneously. Over thousands of iterations, the statistical summary captures the full range of possibilities.

6.1.1 The Order of Operations

For each iteration, the simulation processes each task as follows:

1. **Sample the base duration** from the task's estimate distribution (triangular or log-normal).
2. **Apply uncertainty factor multipliers** to produce an adjusted duration.
3. **Evaluate task-level risks** against the adjusted duration; any triggered risk adds its impact.
4. **Schedule tasks** respecting dependency constraints.
5. **Evaluate project-level risks** against the overall project duration; any triggered risk adds its impact to the total.

This order matters. Uncertainty factors scale the base estimate before risks are evaluated. A percentage-based task risk therefore acts on the already-adjusted duration, not on the raw sampled value. Project-level risks are applied after scheduling, so they affect the final project completion time but do not change which tasks appear on the critical path for that iteration.

6.2 Risk Definitions

Every risk in `mcprojsim` has four key properties:

Property	Required	Description
`id`	Yes	A unique identifier for the risk (e.g., `"risk_001"`)
`name`	Yes	A human-readable label
`probability`	Yes	The likelihood of the risk occurring, between `0.0` and `1.0`
`impact`	Yes	The time penalty if the risk triggers (absolute value or structured object)
`description`	No	An optional free-text explanation

6.2.1 Specifying Impact

Risk impact can be expressed in two ways:

Simple absolute impact — a single number representing hours of effort. When `impact` is given as a plain number without a `type`, it is always treated as an absolute value in hours:

```
risks:
  - id: "task_risk_001"
    name: "Schema migration issues"
    probability: 0.20
    impact: 2
```

If this risk triggers, it adds exactly 2 hours to the task duration. There is no percentage interpretation; a plain number is always an absolute time penalty in hours.

Structured impact — an object with explicit type, value, and optional unit. This form is required when you want percentage-based impact; there is no way to get percentage behavior without specifying `type: "percentage"` explicitly:

```
risks:
  - id: "risk_001"
    name: "Key developer leaves"
    probability: 0.15
    impact:
      type: "percentage"
      value: 20
```

If this risk triggers, it adds 20% of the current duration. The percentage is calculated against the base duration passed to the risk evaluator — for task-level risks, this is the uncertainty-adjusted task duration; for project-level risks, it is the scheduled project duration.

!!! note The `unit` field is not applicable to percentage impacts; if provided it is silently ignored. Percentage impacts are always dimensionless fractions of the base duration.

```
risks:
  - id: "risk_002"
    name: "Requirements change"
    probability: 0.30
    impact:
      type: "absolute"
      value: 10
      unit: "days"
```

The structured form with `type: "absolute"` behaves the same as the simple numeric form but is more explicit. When `unit` is omitted, it defaults to `"hours"` — the same as the simple numeric form. When a `unit` is specified (e.g., `"days"`), the value is converted to hours using the project's `hours_per_day` setting.

6.2.2 Summary of Impact Types

Form	Meaning
`impact: 5`	Adds 5 hours if triggered
`impact: { type: "absolute", value: 5 }`	Adds 5 hours if triggered (explicit form)

Form	Meaning
`impact: { type: "absolute", value: 2, unit: "days" }`	Adds 2 days (converted to hours) if triggered
`impact: { type: "percentage", value: 20 }`	Adds 20% of current duration if triggered

6.3 Task-Level Risks

Task-level risks represent events tied to a specific piece of work. They are defined inside the `risks` list of a task and are evaluated once per iteration for that task.

6.3.1 When to Use Task-Level Risks

Use task-level risks for events that are specific to one task or activity. Typical examples include:

- Schema migration problems during a database redesign
- Browser compatibility issues in frontend development
- Security audit findings requiring rework on an authentication task
- Performance targets not being met during an optimization phase
- Infrastructure provisioning failures for a deployment task

6.3.2 Cumulative Evaluation

If a task defines multiple risks, each risk is evaluated independently. When more than one risk triggers in the same iteration, their impacts are cumulative — they are simply added together.

When percentage-based risks are combined, each percentage is calculated against the **same original adjusted duration** — not against a running cumulative total. Two 20% risks on a 100-hour task each contribute 20 hours (total +40 hours), not a compounded 44 hours.

For example, if a task has two risks:

```
risks:
  - id: "risk_a"
    name: "Data format issues"
    probability: 0.25
    impact: 3
  - id: "risk_b"
    name: "Vendor API downtime"
    probability: 0.10
    impact: 5
```

In a given iteration, neither, one, or both may trigger. If both trigger, the task receives an additional 8 hours on top of its adjusted duration.

6.3.3 Full Task Example with Risks

```
tasks:
  - id: "task_004"
    name: "Authentication & Authorization"
    description: "Implement OAuth2 and role-based access control"
    estimate:
      low: 4
      expected: 6
      high: 12
      unit: "days"
    dependencies: ["task_002"]
    uncertainty_factors:
      team_experience: "medium"
      requirements_maturity: "high"
      technical_complexity: "high"
      team_distribution: "colocated"
      integration_complexity: "high"
    risks:
      - id: "task_risk_003"
        name: "Security audit findings"
        probability: 0.30
        impact: 5
```

In each iteration, the simulator:

1. Samples a base duration between 4 and 12 days (most likely around 6).
2. Multiplies by the combined uncertainty factor (in this case the product of multipliers for medium experience, high requirements maturity, high technical complexity, colocated distribution, and high integration complexity).
3. Evaluates the "Security audit findings" risk: with 30% probability, adds 5 hours.
4. Uses the resulting value as the task duration for scheduling.

6.4 Project-Level Risks

Project-level risks represent events that affect the project as a whole rather than any single task. They are defined in the top-level `project_risks` list and are evaluated once per iteration after the schedule has been computed.

6.4.1 When to Use Project-Level Risks

Use project-level risks for events whose impact is not easily attributed to one task:

- Loss of a key team member mid-project
- Major requirements changes that affect multiple work streams
- Organizational restructuring or budget reductions
- Regulatory or compliance surprises
- Vendor contract delays

6.4.2 Percentage vs. Absolute Impact

Project-level risks are particularly well suited to percentage-based impact, because the absolute impact of losing a developer or changing requirements is often proportional to the project size.

```
project_risks:
  - id: "risk_001"
    name: "Key developer leaves"
    probability: 0.15
    impact:
      type: "percentage"
      value: 20
    description: "Risk of losing senior developer mid-project"

  - id: "risk_002"
    name: "Requirements change"
    probability: 0.30
    impact:
      type: "absolute"
      value: 10
      unit: "days"
```

In this example, if the first risk triggers in an iteration where the scheduled project duration is 60 days, it adds 12 days (20% of 60). The second risk, if triggered, always adds a flat 10 days regardless of project length.

6.4.3 How Project-Level Risks Interact with the Critical Path

Project-level risks are applied after the schedule is computed. This means they increase the final project duration but do not change which tasks are identified as being on the critical path for that iteration. The critical path is determined purely by task durations, uncertainty factors, task-level risks, and dependency structure.

Total duration can exceed the longest critical chain

Because project-level risks add time that is not tied to any specific task, the total project duration shown in the confidence interval table can be longer than the sum of durations along the identified critical path. The difference represents accumulated project-level risk impact — for example, an organisation-wide event such as an unplanned freeze or a key person becoming unavailable — which cannot be attributed to a specific task sequence.

6.5 Combining Risks at Both Levels

A realistic project model often uses both task-level and project-level risks. There is no conflict between them — they operate at different stages of the simulation pipeline.

6.5.1 Complete Project Example with Both Risk Types

```
project:
  name: "Payment Gateway Integration"
  description: "Integrate new payment provider into existing platform"
  start_date: "2026-05-01"
  confidence_levels: [50, 80, 90, 95]

project_risks:
  - id: "proj_risk_001"
    name: "Vendor API contract delay"
    probability: 0.20
    impact:
      type: "absolute"
      value: 15
      unit: "days"
    description: "Payment provider delays final API specification delivery"
  - id: "proj_risk_002"
    name: "Compliance review extends timeline"
    probability: 0.25
    impact:
      type: "percentage"
      value: 15
    description: "PCI-DSS compliance review reveals additional work"

tasks:
  - id: "pay_001"
    name: "API client library"
    description: "Build client library for vendor payment API"
    estimate:
      low: 5
      expected: 8
      high: 14
      unit: "days"
    dependencies: []
    risks:
      - id: "pay_risk_001"
        name: "Incomplete vendor documentation"
        probability: 0.35
        impact: 4
  - id: "pay_002"
    name: "Transaction processing engine"
    description: "Core payment transaction handling logic"
    estimate:
      low: 8
      expected: 12
      high: 20
      unit: "days"
    dependencies: ["pay_001"]
    risks:
      - id: "pay_risk_002"
        name: "Edge cases in currency conversion"
        probability: 0.20
        impact: 3
      - id: "pay_risk_003"
        name: "Retry logic complexity"
        probability: 0.15
        impact: 2
  - id: "pay_003"
    name: "Integration testing with sandbox"
    description: "End-to-end testing against vendor sandbox environment"
    estimate:
      low: 3
      expected: 5
      high: 10
      unit: "days"
```

In this project, each iteration might unfold differently. In one run the vendor documentation risk triggers on `pay_001`, adding 4 days. In another, the compliance review project risk fires, stretching the total duration by 15%. The Monte Carlo process captures all these possibilities and summarizes them statistically.

6.6 Tasks Without Risks

Not every task needs risks. Many tasks have uncertainty captured sufficiently by their estimate range and uncertainty factors alone. If a task has no meaningful discrete risk events, simply omit the `risks` field or set it to an empty list:

```
tasks:
  - id: "task_007"
    name: "Documentation"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"
    dependencies: ["task_002", "task_003"]
    risks: []
```

6.7 Examples

The following two examples demonstrate the same small project defined with and without uncertainty factors and risks. Comparing them illustrates the effect these mechanisms have on simulation input.

6.7.1 Example 1: Project Without Uncertainty Factors or Risks

This project relies solely on the estimate ranges to model uncertainty. No uncertainty factors are specified (defaults will apply), and no risks are defined.

```
project:
  name: "Internal Dashboard"
  description: "Build an internal reporting dashboard"
  start_date: "2026-06-01"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "dash_001"
    name: "Data model design"
    estimate:
      low: 2
      expected: 4
      high: 7
      unit: "days"
    dependencies: []
```

```
  - id: "dash_002"
    name: "Backend API"
    estimate:
      low: 5
      expected: 8
      high: 14
      unit: "days"
    dependencies: ["dash_001"]
  - id: "dash_003"
    name: "Frontend charts"
    estimate:
      low: 4
      expected: 6
      high: 11
      unit: "days"
    dependencies: ["dash_001"]

  - id: "dash_004"
    name: "Integration and deployment"
    estimate:
      low: 2
      expected: 3
      high: 6
      unit: "days"
    dependencies: ["dash_002", "dash_003"]
```

In this project, the only source of variation across iterations is the random sampling from each task's triangular distribution. The simulation will produce a distribution of project durations shaped entirely by the estimate ranges and the dependency structure.

6.7.2 Example 2: Same Project With Uncertainty Factors and Risks

The same project, enriched with uncertainty factors and risks to capture additional real-world conditions.

```
project:
  name: "Internal Dashboard"
  description: "Build an internal reporting dashboard"
  start_date: "2026-06-01"
  confidence_levels: [50, 80, 90]

project_risks:
  - id: "proj_risk_001"
    name: "Stakeholder changes reporting requirements"
    probability: 0.25
    impact:
      type: "percentage"
      value: 15
    description: "New metrics requested after development starts"
```

```yaml
  - id: "proj_risk_002"
    name: "Data warehouse migration overlap"
    probability: 0.10
    impact:
      type: "absolute"
      value: 8
      unit: "days"
    description: "Data source temporarily unavailable during warehouse migration"

tasks:
  - id: "dash_001"
    name: "Data model design"
    estimate:
      low: 2
      expected: 4
      high: 7
      unit: "days"
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "low"
      technical_complexity: "low"
    risks:
      - id: "dash_risk_001"
        name: "Source data schema undocumented"
        probability: 0.30
        impact: 3

  - id: "dash_002"
    name: "Backend API"
    estimate:
      low: 5
      expected: 8
      high: 14
      unit: "days"
    dependencies: ["dash_001"]
    uncertainty_factors:
      team_experience: "medium"
      requirements_maturity: "medium"
      technical_complexity: "medium"
      integration_complexity: "high"
    risks:
      - id: "dash_risk_002"
        name: "Unexpected data volume"
        probability: 0.20
        impact: 4
```

```yaml
  - id: "dash_003"
    name: "Frontend charts"
    estimate:
      low: 4
      expected: 6
      high: 11
      unit: "days"
    dependencies: ["dash_001"]
    uncertainty_factors:
      team_experience: "low"
      technical_complexity: "medium"
    risks:
      - id: "dash_risk_003"
        name: "Chart library limitations"
        probability: 0.15
        impact: 2

  - id: "dash_004"
    name: "Integration and deployment"
    estimate:
      low: 2
      expected: 3
      high: 6
      unit: "days"
    dependencies: ["dash_002", "dash_003"]
    uncertainty_factors:
      team_experience: "medium"
      team_distribution: "distributed"
      integration_complexity: "high"
    risks:
      - id: "dash_risk_004"
        name: "Staging environment unavailable"
        probability: 0.20
        impact: 2
```

Compared to Example 1, this version will produce a wider and generally longer distribution of project durations. The uncertainty factors shift task durations systematically (for instance, the low team experience on `dash_003` multiplies that task's sampled duration by 1.30), and the risks introduce discrete jumps when they trigger. The two project-level risks add further spread to the overall distribution.

6.8 Practical Tips for Defining Risks

- **Be specific.** A risk named "something goes wrong" is not actionable. Prefer names like "Security audit findings" or "Vendor API downtime".
- **Calibrate probability honestly.** A probability of `0.50` means you expect the event to happen roughly half the time. If you are unsure, `0.20` to `0.30` is a reasonable starting range for events that are plausible but not expected.
- **Do not double-count.** If a condition is already captured by an uncertainty factor (e.g., high technical complexity), do not also add a risk for the same effect. Risks are for events; uncertainty factors are for conditions.
- **Use percentage impact for scale-dependent events.** If a risk tends to hurt larger tasks more than smaller ones, percentage-based impact is a better fit.
- **Use absolute impact for fixed-cost events.** If a risk has a known cost regardless of the task size (e.g., a mandatory 3-day wait for an external review), absolute impact is appropriate.
- **Keep risk lists manageable.** Most tasks should have zero to three risks. If a task has many risks, consider whether some of them are better captured as uncertainty factors or whether the task should be broken into smaller pieces.

6.9 Summary

Concept	What It Models	When It Applies	How It Affects Duration
Task-level risk	A discrete event tied to one task	Per task, per iteration	Adds time to that task
Project-level risk	A discrete event affecting the whole project	Per project, per iteration	Adds time to total project duration

- Risks are **additive**: triggered risks add time to the duration.
- Task-level risks are evaluated **after** uncertainty factors are applied, so percentage-based task risks act on the adjusted duration.
- Project-level risks are evaluated **after** the schedule is computed, so they do not affect critical path membership.

CHAPTER 7

Monetary Cost Estimation

Monte Carlo simulation gives you a probabilistic view of *when* a project finishes. `mcprojsim` can extend that same probabilistic reasoning to *how much* it will cost — turning every iteration's sampled schedule into a cost estimate and surfacing a full cost distribution alongside the duration distribution.

7.1 Activating cost estimation

Cost estimation is opt-in: it activates automatically as soon as any cost-related field is present in your project file. Specifically, cost is computed when **at least one** of the following is set:

Field	Where	Activates because…
`default_hourly_rate > 0`	`project:` section	Labor cost is non-zero
`hourly_rate` (any value)	a `resources:` entry	Resource has an explicit rate
`fixed_cost`	a task	Task has a fixed fee regardless of duration
`cost_impact`	a risk	Risk has a monetary consequence

A `default_hourly_rate: 0` alone does **not** activate cost — zero rate produces zero labor cost, so there is nothing to report. But `hourly_rate: 0` on a named resource *does* activate it (an explicit zero is different from "not set").

7.2 Project-level cost fields

Add these to the `project:` metadata block in your YAML file.

```
project:
  name: My Project
  start_date: "2026-03-01"
  default_hourly_rate: 150     # fallback rate (per hour) for any task with no resource
  overhead_rate: 0.15          # 15% markup applied to total labor cost
  currency: EUR                # currency code displayed in output (default: EUR)
```

7.2.1 `default_hourly_rate`

The hourly rate (in whatever currency you choose) applied to every task that has no named resource, or whose resource has no explicit `hourly_rate`.

- Must be `≥ 0`
- If omitted, defaults to `0` (no labor cost for unrated tasks)

7.2.2 `overhead_rate`

A fractional multiplier added on top of total labour cost to represent management overhead, tooling, office, etc.

```
overhead = labor_cost × overhead_rate
total_cost = labor_cost + overhead + fixed_costs + risk_cost_impacts
```

- Range: `0.0` – `3.0` (i.e., up to 300 % markup)
- Overhead is applied to **labor only** — fixed costs and risk cost impacts are not marked up again

7.2.3 `currency`

An arbitrary string label (e.g. `EUR`, `USD`, `GBP`, `SEK`) displayed next to all cost figures. It is **not** used for conversion — it is purely cosmetic.

- If omitted, falls back to the `currency` setting in your configuration file (default: `EUR`)

7.3 Per-resource hourly rates

When you have named resources, you can assign each a specific rate:

```
resources:
  - name: alice
    hourly_rate: 240    # senior engineer — overrides the project default
  - name: bob
    hourly_rate: 160    # mid-level engineer
  - name: carol         # no hourly_rate → uses project default_hourly_rate
    experience_level: 1
```

`hourly_rate` on a resource **overrides** `default_hourly_rate` for tasks assigned to that resource. If a task is assigned to multiple resources the engine uses the **mean** of their rates as a Phase 1 simplifying assumption (see Known limitations below).

7.4 Per-task fixed costs

Some tasks carry a cost regardless of how long they take — a software license, a cloud-infrastructure setup fee, a hardware purchase:

```
tasks:
  - id: cloud_setup
    name: Cloud Infrastructure Setup
    estimate:
      low: 8
      expected: 16
      high: 30
    fixed_cost: 3500    # flat €3,500 added every iteration, regardless of duration
```

- `fixed_cost` is an unrestricted float — **negative values represent credits or rebates** (e.g. a committed-spend discount or an early-delivery bonus)
- Fixed costs are included in total cost but are **not** subject to `overhead_rate`

7.5 Risk cost impacts

Risks can carry a monetary consequence in addition to (or instead of) a schedule impact. When a risk triggers in an iteration, its `cost_impact` is added to that iteration's total cost.

```
risks:
  - name: Vendor support escalation
    probability: 0.25
    impact: 5           # +5 hours schedule impact when triggered
    cost_impact: 4000   # +€4,000 cost impact when triggered
  - name: Early sign-off bonus
    probability: 0.20
    impact: -8          # saves 8 hours
    cost_impact: -1500  # saves €1,500
  - name: Compliance audit fee
    probability: 0.10
    impact: 0           # no schedule impact
    cost_impact: 8000   # pure cost event — affects cost distribution only
```

Both task-level and project-level risks support `cost_impact`.

7.5.1 Project-level risks

Project-level risks are defined in a separate `project_risks:` section at the top level of the YAML file (not nested under `project:` or `tasks:`). They apply globally to the whole project:

```
project_risks:
  - id: funding_delay
    name: Funding approval delay
    probability: 0.15
    impact:
      type: absolute
      value: 80
      unit: hours
    cost_impact: 25000    # resubmission fees + additional consultant time

  - id: vendor_bankruptcy
    name: Key vendor goes bankrupt
    probability: 0.05
    impact: 0              # no schedule impact — cost-only
    cost_impact: 50000     # emergency vendor replacement
```

7.6 Full project example

```yaml
project:
  name: API Platform
  start_date: "2026-06-01"
  default_hourly_rate: 120
  overhead_rate: 0.18
  currency: EUR

resources:
  - name: alice
    hourly_rate: 200
    experience_level: 3
  - name: bob
    hourly_rate: 140
    experience_level: 2

tasks:
  - id: design
    name: API Design
    estimate: { low: 16, expected: 24, high: 40 }
    resources: [alice]
    risks:
      - name: Scope creep
        probability: 0.30
        impact: 8
        cost_impact: 1500
  - id: backend
    name: Backend Implementation
    estimate: { low: 60, expected: 100, high: 160 }
    dependencies: [design]
    resources: [alice, bob]
    fixed_cost: -2000   # tooling credit

  - id: deploy
    name: Deployment
    estimate: { low: 8, expected: 12, high: 24 }
    dependencies: [backend]
    fixed_cost: 500     # cloud provisioning fee
```

7.7 Running a simulation with cost

```
# Standard run — cost output appears automatically when cost fields are present
mcprojsim simulate project.yaml

# Set a budget target to see the probability of staying within it
mcprojsim simulate project.yaml --target-budget 40000

# Include full per-task cost breakdown in JSON export
mcprojsim simulate project.yaml -f json --full-cost-detail
```

7.8 Reading cost output

7.8.1 Table mode (default)

```
┌──────────────────────────────────────────────────────────┐
│ Cost Distribution (EUR)                                  │
├──────────┬──────────┬──────────┬─────────────────────────┤
│ P50      │ P80      │ P90      │ P95                     │
│ €34,200  │ €41,800  │ €46,500  │ €50,100                 │
└──────────┴──────────┴──────────┴─────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│ Budget Analysis                                          │
├──────────────────────────────────────────────────────────┤
│ Target budget:     €40,000                               │
│ Probability within budget: 74.3%                         │
│ 80% confidence interval:   €29,800 – €41,800             │
└──────────────────────────────────────────────────────────┘
```

7.8.2 Plain CLI mode

When no `--table` flag is used, the cost section is printed in plain text after the standard duration summary, showing the same percentile breakdown plus a brief sensitivity note.

7.8.3 Cost sensitivity

When cost estimation is active, `mcprojsim` computes a **Spearman rank correlation** between each task's cost contribution and the total project cost. High-correlation tasks are the biggest cost drivers.

7.9 Budget confidence analysis

7.9.1 `--target-budget`

Pass `--target-budget <amount>` to see:

- The probability that total cost stays within the budget
- The 80 % confidence interval for total cost

```
mcprojsim simulate project.yaml --target-budget 45000
```

7.9.2 Programmatic access (Python API)

```
results = engine.run()

# Probability of finishing within €45,000
p = results.probability_within_budget(45000)

# Budget needed for 90% confidence
budget_90 = results.budget_for_confidence(0.90)

# Confidence interval (point estimate, lower, upper)
point, lo, hi = results.budget_confidence_interval(45000, confidence_level=0.80)
```

7.10 Cost in exported reports

7.10.1 JSON (`-f json`)

The cost section appears under `"cost"` in the JSON output:

```
"cost": {
  "currency": "EUR",
  "mean": 36450.0,
  "std_dev": 6200.0,
  "percentiles": { "50": 35800.0, "80": 41900.0, "90": 46200.0, "95": 49800.0 },
  "overhead_rate": 0.18,
  "task_costs": {
    "design": { "mean": 4200.0, "p50": 4050.0, "p90": 5800.0 },
    "backend": { "mean": 28100.0, "p50": 27500.0, "p90": 34200.0 },
    "deploy": { "mean": 3600.0, "p50": 3400.0, "p90": 4800.0 }
  },
  "sensitivity": { "backend": 0.82, "design": 0.35, "deploy": 0.18 },
  "duration_correlation": 0.97
}
```

Use `--full-cost-detail` to include the complete per-task breakdown.

7.10.2 CSV (`-f csv`)

A `cost_statistics` section is appended at the end of the CSV with one row per percentile plus rows for mean and standard deviation.

7.10.3 HTML (`-f html`)

The HTML report gains:

- A **Cost Summary** card showing mean, median, std dev, and the percentile table
- A **Cost Histogram** visualising the cost distribution across iterations
- A **Cost Sensitivity Tornado Chart** showing which tasks drive cost variance (Spearman rank correlation, top 15 tasks)
- A **Cost Breakdown by Task** table listing each task's mean cost, std dev, min, max, and percentage of total project cost
- A **Budget Confidence** card (if `--target-budget` was provided) with a probability gauge, percentile table, and cost CDF S-curve chart
- A **Cost Sensitivity** table ranking tasks by their cost correlation

7.11 Configuration defaults

Override cost defaults in a config file so you don't have to repeat them in every project file:

```
cost:
  default_hourly_rate: 130   # applied when project omits default_hourly_rate
  overhead_rate: 0.10        # default 10% overhead
  currency: EUR              # default currency label
  include_in_output: true    # set false to suppress cost sections globally
```

`include_in_output: false` suppresses cost output in all reports and console output, but does **not** suppress the underlying computation — budget methods on `SimulationResults` still return values.

7.12 Natural-language input

The `generate` command understands cost fields in plain-text project descriptions. Supported cost-related patterns:

Input pattern	Maps to
`Default rate: €120/hour`, `Hourly rate: 150`	`default_hourly_rate`
`Overhead: 15%`, `Overhead rate: 0.15`	`overhead_rate`
`Currency: EUR`	`currency`
`Rate: €200/hour` (under a resource)	resource `hourly_rate`
`Fixed cost: $5000`, `One-time cost: 5000`	task `fixed_cost`

Input pattern	Maps to
`Risks:` header + structured bullets	task `risks` (see below)
Prose risk sentences	task `risks` with `cost_impact`

7.12.1 Structured risk format

Indent a `Risks:` header under a task, then list the risk name, probability, impact, and optional cost impact as sub-bullets:

```
Task 3: Data Transformation
- Estimate: 40/80/140 hours
- Depends on Task 2
- Risks:
  - Third-Party API Changes
  - Probability: 0.25
  - Impact: 3/5 days
  - Cost impact: 8000
```

7.12.2 Prose risk format

The parser recognises risk sentences that include a probability and at least one consequence (schedule delay or cost impact). Keywords like *penalty*, *fee*, or *surcharge* produce a positive `cost_impact`; keywords like *bonus*, *reward*, *saving*, or *credit* produce a negative `cost_impact`:

```
Task 6: Go-Live
- Estimate: 10/20/35 hours
- There is a 25% probability of a deployment rollback penalty of $5000 and 2 days delay
- There is a 15% chance the client rewards early delivery with a $4000 bonus
```

7.12.3 Full example

```
Project name: My API
Start date: 2026-06-01
Default rate: €120/hour
Overhead: 15%
Currency: EUR

Resource 1: Alice
- Experience: 3
- Rate: €200/hour
Task 1:
- API Design
- Size: M
- Resources: Alice
- Fixed cost: 1000

Task 2:
- Backend
- Size: XL
- Depends on Task 1
- There is a 20% risk of integration delays costing a $3000 fee and adding 2 days
```

```
mcprojsim generate project_description.txt > project.yaml
mcprojsim simulate project.yaml --target-budget 50000
```

See `examples/cost_nl_prose_example.txt`, `examples/cost_nl_example.txt`, and `examples/cost_nl_risk_example.txt` for full natural-language examples with cost and risk definitions.

7.13 Secondary currencies and FX conversion

When your team spans multiple countries, or when you report costs to a client in a different currency, `mcprojsim` can automatically convert every cost distribution to one or more **secondary currencies** and include the results in all outputs.

7.13.1 How it works

1. You nominate up to **5 secondary currencies** in the project file.
2. At simulation time, `mcprojsim` fetches the current mid-market exchange rate from Frankfurter — a free, key-less service backed by official sources.
3. Rates are **cached on disk** for 24 hours so that repeated runs do not hit the network (see Rate caching below).
4. You can optionally mark up the official rate by a **conversion cost** and an **overhead rate** to model the real cost of buying foreign currency.
5. The adjusted rate is applied element-wise to the entire cost distribution — every simulation iteration is converted — so the output is a proper probabilistic distribution in the target currency.

7.13.2 Adjusted-rate formula

```
r_adj = (1 + fx_conversion_cost + fx_overhead_rate) × r_official
```

Term	Meaning
`r_official`	Mid-market rate fetched from Frankfurter (e.g. 1 EUR = 11.20 SEK)
`fx_conversion_cost`	Bank bid-ask spread as a fraction, e.g. `0.015` = 1.5 %
`fx_overhead_rate`	Additional overhead (hedging, admin) as a fraction, e.g. `0.005` = 0.5 %
`r_adj`	Effective rate actually paid, used for all conversions

Example: EUR → SEK with a 2 % bank spread and 0.5 % admin fee:

```
r_official = 11.20        # live mid-market
r_adj = (1 + 0.02 + 0.005) × 11.20 = 1.025 × 11.20 = 11.48 SEK/EUR
```

A project with a P80 cost of €42,000 would show **SEK 482,160** at P80.

7.13.3 Project file fields

Add the following to your `project:` section:

```
project:
  name: API Platform
  currency: EUR               # primary (base) currency
  default_hourly_rate: 150
  overhead_rate: 0.15

  secondary_currencies: [SEK, USD, GBP]   # up to 5 ISO 4217 codes
  fx_conversion_cost: 0.015               # 1.5% bank spread
  fx_overhead_rate: 0.005                 # 0.5% internal overhead
```

Field	Type	Range	Default	Description
`secondary_currencies`	list of strings	up to 5 entries	`[]`	ISO 4217 codes to convert to
`fx_conversion_cost`	float	0 – 0.50	`0.0`	Bank spread fraction
`fx_overhead_rate`	float	0 – 1.0	`0.0`	Additional overhead fraction

Field	Type	Range	Default	Description
`fx_rates`	mapping	any positive float	`{}`	Manual rate overrides (bypass live fetch)

All four fields are optional. If `secondary_currencies` is empty (the default), no FX fetching or conversion is performed.

7.13.4 Manual rate overrides

If you have a contractually locked exchange rate, or simply want reproducible results without a network dependency, set `fx_rates` to bypass the live fetch:

```
project:
  currency: EUR
  secondary_currencies: [SEK, USD]
  fx_rates:
    SEK: 11.50    # use this fixed rate — no markups applied
    USD: 1.08     # manual rates are always used as-is
```

Manual rates take precedence over anything fetched from Frankfurter. **No markups are applied to manual rates** — they are used exactly as specified.

7.13.5 Suppressing FX at run time

Pass `--no-fx` to any `simulate` call to skip secondary-currency output even if the project file defines `secondary_currencies`:

```
mcprojsim simulate project.yaml --no-fx
```

This is useful for quick local runs where network access is unavailable or undesired.

7.13.6 Console output

When secondary currencies are configured, a **Cost in Secondary Currencies** section is appended after the primary cost table. For each currency it shows:

- The effective exchange rate used (official and adjusted, if markups were applied)
- The same confidence-level percentiles as the primary cost table

```
Cost in Secondary Currencies:
  SEK  1 EUR = 11.48 SEK  (official: 11.20, bank: +1.5%, overhead: +0.5%)
       P50: SEK 391,360  |  P80: SEK 481,160  |  P90: SEK 527,100  |  P95: SEK 563,400

  USD  1 EUR = 1.09 USD  (official: 1.068, bank: +1.5%, overhead: +0.5%)
       P50: $37,200  |  P80: $45,700  |  P90: $50,100  |  P95: $53,500
```

If the rate source is a manual override, `(manual)` appears instead of the markup breakdown. If a rate is unavailable (network failure, unknown currency), that currency is silently skipped and a warning is emitted.

7.13.7 FX data in exported reports

JSON (`-f json`)

A `secondary_currencies` array is added inside the `"cost"` section:

```
"cost": {
  "currency": "EUR",
  "mean": 36450.0,
  "percentiles": { "50": 34200, "80": 41800, "90": 46500, "95": 50100 },
  "secondary_currencies": [
    {
      "currency": "SEK",
      "official_rate": 11.2,
      "adjusted_rate": 11.48,
      "fx_conversion_cost": 0.015,
      "fx_overhead_rate": 0.005,
      "source": "live",
      "fetched_at": "2026-06-01T09:14:33+00:00",
      "mean": 418000.0,
      "percentiles": { "50": 392400, "80": 479700, "90": 533700, "95": 575100 }
    }
  ]
}
```

The `source` field is one of `"live"`, `"disk_cache"`, or `"manual_override"`, allowing you to audit which data source was used.

CSV and HTML

Both exporters receive the same `fx_provider` and include secondary-currency data alongside the primary cost section — extra rows in the CSV cost block and an additional card in the HTML report.

7.13.8 Rate caching

Fetching live rates for every simulation run would be wasteful and fragile. `mcprojsim` maintains a disk cache at:

```
~/.mcprojsim/fx_rates_cache.json
```

Cache behaviour:

Property	Value
Location	`~/.mcprojsim/fx_rates_cache.json`
TTL	24 hours per entry
Granularity	Per base-currency / target-currency pair
Format	JSON: `{ "EUR": { "SEK": { "rate": 11.2, "fetched_at": "..." } } }`

Entries from different base currencies coexist in the same file. On startup, `ExchangeRateProvider` loads all fresh entries for the current base currency into memory. Stale entries (older than 24 h) are skipped and re-fetched on next use.

All disk I/O errors (permissions, corrupt file, etc.) are silently swallowed — the cache is best-effort. If the cache cannot be read or written, live rates are still fetched and used for the current run; they just will not be persisted.

7.13.9 Full example with secondary currencies

```
project:
  name: International API Platform
  start_date: "2026-06-01"
  default_hourly_rate: 150
  overhead_rate: 0.15
  currency: EUR

  # Show costs in Swedish kronor and US dollars as well
  secondary_currencies: [SEK, USD]
  fx_conversion_cost: 0.015   # 1.5% bank spread
  fx_overhead_rate: 0.005     # 0.5% internal admin

resources:
  - name: alice
    hourly_rate: 200
    experience_level: 3
  - name: bob
    hourly_rate: 140
    experience_level: 2

tasks:
  - id: design
    name: API Design
    estimate: { low: 16, expected: 24, high: 40 }
    resources: [alice]

  - id: backend
    name: Backend Implementation
    estimate: { low: 60, expected: 100, high: 160 }
    dependencies: [design]
    resources: [alice, bob]
    fixed_cost: -2000   # tooling credit

  - id: deploy
    name: Deployment
    estimate: { low: 8, expected: 12, high: 24 }
    dependencies: [backend]
    fixed_cost: 500
```

```
mcprojsim simulate project.yaml --target-budget 45000
```

7.14 Known limitations

Multi-resource equal-effort-split (Phase 1 simplification)

When a task is assigned to multiple resources, the engine uses the **mean** of their hourly rates multiplied by the sampled elapsed duration. This assumes each resource contributes an equal share of the effort. If resources truly work in parallel for the full elapsed duration, the actual cost is higher than what the simulation reports.

Example: two resources at €100/hr and €200/hr on a 32-hour task. The engine computes `32 × mean(100, 200) = 32 × 150 = €4,800`, which equals the true cost only when each person works 16 hours (equal split). If both work the full 32 hours in parallel the true cost is `32×100 + 32×200 = €9,600`.

This limitation will be addressed in a future release with explicit effort tracking per resource.

7.15 Example files

File	Description
`examples/cost_simple.yaml`	`default_hourly_rate` + `overhead_rate` + one `fixed_cost` + risk `cost_impact`
`examples/cost_constrained.yaml`	Per-resource rates with resource-constrained scheduling
`examples/cost_advanced.yaml`	Full feature set: per-resource rates, multiple fixed costs and credits, risks with cost-only impacts
`examples/cost_nl_example.txt`	Structured natural-language input with cost fields
`examples/cost_nl_prose_example.txt`	Prose-style natural-language project brief

CHAPTER 8

Project files

This chapter is the authoritative reference for the project definition file format used by `mcprojsim`. It documents every supported field, explains the motivation for each feature, and includes examples for every accepted syntax form.

For the formal EBNF grammar that precisely defines what is syntactically valid, see the Formal Grammar Specification. This reference uses human-readable prose and worked examples; the grammar is the machine-precise complement.

The parser layer is intentionally simple:

- `.yaml` and `.yml` files are loaded with the YAML parser,
- `.toml` files are loaded with the TOML parser,
- both are then validated against the same `Project` model.

Because of that design, the logical schema is the same regardless of whether you write the file in YAML or TOML.

8.1 Quick reference for new users

The table below summarises every top-level section. The two mandatory sections — `project` and `tasks` — are enough to run your first simulation. Everything else is optional and can be added incrementally.

Section	Required	Activates
`project`	**Yes**	Project name, start date, and reporting settings
`tasks`	**Yes**	The work items to simulate
`project_risks`	No	Cross-cutting risks applied to the whole project
`resources`	No	Resource-constrained scheduling (replaces unlimited-workforce mode)
`calendars`	No	Custom working hours, holidays, non-standard weeks
`sprint_planning`	No	Sprint-based forecasting from historical velocity data

Inside each task, the only truly required field beyond `id` and `name` is `estimate`. The minimal estimate is a numeric range:

```
estimate:
  low: 1
  expected: 3
  high: 8
```

T-shirt sizes and story points are shorthand alternatives — they map to numeric ranges via the configuration file:

```
estimate:
  t_shirt_size: "M"          # resolved from config, no unit needed
```

```
estimate:
  story_points: 5            # resolved from config, no unit needed
```

For a complete worked example see the Full YAML example at the end of this chapter.

8.2 Creating project files

There are three ways to create a project file:

Method	Best for
Write YAML or TOML by hand	Full control over every field
`mcprojsim generate`	Quick creation from a natural language description
MCP server	AI-assisted generation through an MCP client

The `generate` command converts a plain-text project description into a valid YAML file without requiring any AI service — it runs locally using a built-in pattern-based parser.

8.2.1 Using `mcprojsim generate`

Write a description in a text file (e.g., `description.txt`):

```
Project name: Website Redesign
Start date: 2026-04-15
Task 1:
- Gather requirements
- Size: S
Task 2:
- Create wireframes
- Depends on Task 1
- Size: M
Task 3:
- Build frontend
- Depends on Task 2
- Size: XL
```

Generate the YAML project file:

```
mcprojsim generate description.txt -o project.yaml
```

See Running Simulations — `mcprojsim generate` for the full command reference and all options. The MCP Server page covers the complete input format and more examples.

!!! note The `generate` command creates a minimal but valid project file. After generating, you can manually add uncertainty factors, risks, resources, and other fields documented in this chapter.

8.3 Supported file formats

The validator and CLI currently recognize these project file extensions:

- `.yaml`
- `.yml`

- `.toml`

Any other extension is rejected as an unsupported file format.

8.4 Top-level structure

At the highest level, a project file may contain the following sections:

- `project` — required
- `tasks` — required
- `project_risks` — optional
- `resources` — optional. When present, constrained scheduling is activated.
- `calendars` — optional. Used by constrained scheduling when resources reference calendars.
- `sprint_planning` — optional. Activates sprint-based simulation mode.

If `project.team_size` is greater than zero, default resources are generated up to that size (after validating explicit resources), which also makes scheduling resource-constrained.

The smallest valid project file therefore looks like this:

```
project:
  name: "My Project"
  start_date: "2026-03-01"

tasks:
  - id: "task_001"
    name: "First task"
    estimate:
      low: 1
      expected: 2
      high: 3
```

8.4.1 Top-level YAML skeleton

```
project:
  ...

tasks:
  - ...
  - ...

project_risks:
  - ...

resources:
  - ...

calendars:
  - ...
```

8.4.2 Top-level TOML skeleton

```
[project]
# ...

[[tasks]]
# ...

[[project_risks]]
# ...

[[resources]]
# ...

[[calendars]]
# ...
```

8.5 Formal section order

The parser does not require a specific order for top-level sections, but this is the clearest and most conventional order:

1. `project`
2. `project_risks`
3. `tasks`
4. `resources`
5. `calendars`
6. `sprint_planning`

That is also the order used in most examples and in this reference.

8.6 The `project` section

The `project` section is required. It contains project-level metadata and reporting settings.

Every simulation starts here: the `name` and `start_date` are the two fields the engine cannot work without. Everything else in this section controls how results are interpreted and displayed — which confidence percentiles appear in the output, what colour thresholds mark a date as red or green, and what distribution model is used by default across all tasks. You set these once at project level and every task inherits them automatically, which keeps individual task definitions short.

8.6.1 Supported fields

Field	Required	Type	Default	Notes
`name`	Yes	string	—	Project display name
`description`	No	string	`null`	Optional descriptive text
`start_date`	Yes	ISO date string	—	Must parse as `YYYY-MM-DD`
`currency`	No	string	`"USD"`	Stored as metadata
`confidence_levels`	No	list of integers	`[10, 25, 50, 75, 80, 85, 90, 95, 99]`	Controls reported percentiles
`hours_per_day`	No	float	`8.0`	Hours in a working day; used for day/week conversion

Field	Required	Type	Default	Notes
`distribution`	No	`"triangular"` or `"lognormal"`	`"triangular"`	Default estimate distribution for tasks that do not specify one
`team_size`	No	integer	`null`	If > `0`, target total resources after validation (may auto-create defaults)
`probability_red_threshold`	No	float	`0.50`	Must be between `0.0` and `1.0`
`probability_green_threshold`	No	float	`0.90`	Must be between `0.0` and `1.0`

8.6.2 Required constraints

The implementation currently enforces these rules for `project`:

- `start_date` must be a valid ISO-format date string or a date object,
- `probability_red_threshold` must be less than `probability_green_threshold`,
- both thresholds must be in the range `0.0` to `1.0`,
- `distribution`, if provided, must be either `triangular` or `lognormal`,
- if provided, `team_size` must be `>= 0`,
- if `team_size > 0` and explicit `resources` are fewer, default resources are added up to `team_size`,
- if explicit `resources` exceed `team_size`, validation fails.

8.6.3 YAML example

```yaml
project:
  name: "Customer Portal Redesign"
  description: "Next-generation customer portal with enhanced features"
  start_date: "2025-11-01"
  currency: "USD"
  distribution: "triangular"
  confidence_levels: [10, 25, 50, 75, 80, 85, 90, 95, 99]
  probability_red_threshold: 0.50
  probability_green_threshold: 0.90
```

8.6.4 TOML example

```toml
[project]
name = "Customer Portal Redesign"
description = "Next-generation customer portal with enhanced features"
start_date = "2025-11-01"
currency = "USD"
distribution = "triangular"
confidence_levels = [10, 25, 50, 75, 80, 85, 90, 95, 99]
probability_red_threshold = 0.50
probability_green_threshold = 0.90
```

8.7 The `tasks` section

The `tasks` section is required and must contain at least one task.

A **task** is any unit of schedulable work. You give each task an estimate of its duration, list its dependencies (other tasks that must complete before it can start), and optionally add uncertainty factors, resource requirements, and task-level risks. The simulation samples every task's duration in every iteration and sequences

them according to the dependency graph — or, when resources are present, according to resource availability too. The more accurately each task is described, the more representative the resulting uncertainty distribution will be.

Each task is validated as a `Task` object with the following fields.

8.7.1 Supported task fields

Field	Required	Type	Default	Notes
`id`	Yes	string	—	Must be unique across all tasks
`name`	Yes	string	—	Human-readable task name
`description`	No	string	`null`	Optional task description
`estimate`	Yes	object	—	One of the supported estimate syntaxes
`dependencies`	No	list of strings	`[]`	Each entry must match another task `id`
`uncertainty_factors`	No	object	defaults applied	Recognized factor fields described below
`resources`	No	list of strings	`[]`	Task-level resource names
`max_resources`	No	integer	`1`	Max number of resources that may be assigned concurrently
`min_experience_level`	No	integer	`1`	Minimum resource experience allowed (`1`, `2`, `3`)
`planning_story_points`	No	integer > 0	`null`	Story point size used for sprint planning; overrides `estimate.story_points` when set
`priority`	No	integer	`null`	Scheduling priority hint used in some sprint-planning modes
`spillover_probability_override`	No	float 0.0–1.0	`null`	Per-task override for the probability that incomplete work spills into the next sprint
`risks`	No	list of risk objects	`[]`	Task-level probabilistic risks

8.7.2 Minimal task example

```
tasks:
  - id: "task_001"
    name: "Design database schema"
    estimate:
      low: 3
      expected: 5
      high: 8
```

8.7.3 TOML task example

```
[[tasks]]
id = "task_001"
name = "Design database schema"

[tasks.estimate]
min = 3
expected = 5
max = 8
```

8.8 The `estimate` section

Every task must define an `estimate` object. The estimate is the engine's primary input: it describes the uncertain duration of a task (or, in sprint planning mode, its size in story points or tasks) so the simulator can draw a different sample in each Monte Carlo iteration.

Choosing the right estimate style matters for accuracy:

- **Triangular** is the simplest and most common choice. It matches the way team members typically think — "at best X, most likely Y, at worst Z" — and is well suited to tasks with a finite worst case.
- **Log-normal** is appropriate for tasks where extreme over-runs are more likely than a symmetric range suggests — for example, research-heavy work, integration tasks with unknown third-party behaviour, or anything where "two to three times longer than expected" is a realistic tail scenario.
- **T-shirt sizes** are useful when teams do not have enough information to produce numeric estimates, or when they want to use a consistent sizing vocabulary across many tasks without converting each size into hours by hand.
- **Story points** are the standard agile unit and are mapped to numeric ranges through the configuration file, with a default mapping included out of the box.

The implementation supports four estimate styles:

1. triangular estimate,
2. log-normal estimate,
3. T-shirt-size estimate,
4. Story Point estimate.

The `distribution` field defaults to `triangular` when omitted. Setting it at task level overrides the project-level default for that task only — useful when most tasks warrant a triangular estimate but a few specific tasks benefit from a log-normal tail model.

For the formal grammar of all estimate forms see Formal Grammar — `<estimate_spec>`.

8.8.1 Field name aliases

The model accepts both long-form and short-form names for the three range fields:

Canonical name	Accepted alias
`low`	`min`
`expected`	`most_likely`
`high`	`max`

Both forms are valid in YAML and TOML. Examples in this chapter use both; they are equivalent.

8.8.2 1. Triangular estimate

This is the default and most common form.

Supported fields

Field	Required	Type	Default
`distribution`	No	`"triangular"` or `"lognormal"`	`null` (inherits project default)
`min`	Yes for triangular	number 0	`null`
`expected`	Yes	number > 0	`null`
`max`	Yes for triangular	number 0	`null`
`unit`	No	`"hours"`, `"days"`, or `"weeks"`	`"hours"`

Validation rules

For a triangular estimate:

- `expected` must be present,
- `min` must be present,
- `max` must be present,
- `low <= expected <= high`.
- `unit` must be one of `"hours"`, `"days"`, or `"weeks"` if specified.

YAML example

```
estimate:
  low: 3
  expected: 5
  high: 10
  unit: "days"
```

TOML example

```
[tasks.estimate]
min = 3
expected = 5
max = 10
unit = "days"
```

8.8.3 2. Log-normal estimate

The implementation also supports **shifted** log-normal estimates. In this mode you still provide `low`, `expected`, and `high`, but they are interpreted differently:

- `low` is the hard shift / minimum,
- `expected` is the mode,
- `high` is interpreted as the configured percentile (P95 by default; see `lognormal.high_percentile` in the configuration file).

Supported fields

Field	Required	Type	Notes
`distribution`	Yes	`"lognormal"`	Must be set explicitly
`low`	Yes	number 0	Required

Field	Required	Type	Notes
`expected`	Yes	number > 0	Required
`high`	Yes	number $\geq$ `low`	Required
`unit`	No	`"hours"`, `"days"`, or `"weeks"`	`"hours"`

Validation rules

For a log-normal estimate:

- `distribution` must be `lognormal`,
- `low` must be present,
- `expected` must be present,
- `high` must be present,
- `low < expected < high`,
- `unit` must be one of `"hours"`, `"days"`, or `"weeks"` if specified.

YAML example

```yaml
estimate:
  distribution: "lognormal"
  low: 3
  expected: 8
  high: 20
  unit: "days"
```

TOML example

```toml
[tasks.estimate]
distribution = "lognormal"
low = 3
expected = 8
high = 20
unit = "days"
```

8.8.4 3. T-shirt-size estimate

This form lets the task refer to a symbolic size token.

Supported token forms:

- bare size: `M`
- qualified category/size: `epic.M`
- long-form size alias: `Medium`, `Epic.Large`
- full long-form aliases: `EXTRA_SMALL`, `SMALL`, `MEDIUM`, `LARGE`, `EXTRA_LARGE`, `EXTRA_EXTRA_LARGE`

For the complete token grammar see Formal Grammar — `<tshirt_size>`.

Supported fields

Field	Required	Type	Default
`t_shirt_size`	Yes	string	—
`distribution`	No	enum	Defaults to `triangular`

Validation behavior

When `t_shirt_size` is present:

- the `TaskEstimate` validator accepts the estimate immediately,
- explicit `min`, `expected`, and `max` are not required,

- **`unit` must not be specified** — the unit comes from the configuration file's `t_shirt_size_unit` setting (default: `"hours"`),
- the simulation engine resolves the size to actual `min`, `expected`, and `max` values from the active configuration,
- if `distribution` is omitted, the task inherits the project-level default distribution.

If the chosen size does not exist in the active configuration, simulation raises an error.

YAML example

```yaml
estimate:
  t_shirt_size: "M"
```

```yaml
estimate:
  t_shirt_size: "epic.M"
```

TOML example

```toml
[tasks.estimate]
t_shirt_size = "M"
```

Important precedence note

As implemented today, if `t_shirt_size` is present together with explicit numeric estimate fields, the T-shirt-size path takes precedence during validation and resolution.

In other words, this is technically accepted by the model:

```yaml
estimate:
  t_shirt_size: "M"
  low: 1
  expected: 2
  high: 3
```

But it should be treated as ambiguous and avoided in real project files. If you use `t_shirt_size`, prefer to omit the explicit numeric range fields.

8.8.5 4. Story Point estimate

This form lets a task use agile-style relative sizing while still simulating an effort range.

Supported fields

Field	Required	Type	Default
`story_points`	Yes	integer	—
`distribution`	No	enum	Defaults to `triangular`

Validation behavior

When `story_points` is present:

- the value must currently be one of `1`, `2`, `3`, `5`, `8`, `13`, or `21`,
- **`unit` must not be specified** — the unit comes from the configuration file's `story_point_unit` setting (default: `"days"`),
- the simulation engine resolves the Story Point value to actual `min`, `expected`, and `max` values from the active configuration.

If the chosen Story Point value does not exist in the active configuration, simulation raises an error.

YAML example

```yaml
estimate:
  story_points: 5
```

TOML example

```
[tasks.estimate]
story_points = 5
```

8.8.6 Symbolic estimate mappings in configuration

Both `t_shirt_size` and `story_points` are symbolic estimate forms. They are converted to numeric ranges by the active configuration, using the unit specified by `t_shirt_size_unit` (default: `"hours"`) and `story_point_unit` (default: `"days"`) respectively. All values are then converted to hours internally.

Built-in defaults exist for both styles, and a custom configuration file may override all or only some of those mappings.

Example configuration

```
t_shirt_sizes:
  story:
    M:
      low: 45
      expected: 65
      high: 130
  epic:
    M:
      low: 240
      expected: 520
      high: 1400

t_shirt_size_default_category: epic

story_points:
  5:
    low: 4
    expected: 6
    high: 9
  8:
    low: 6
    expected: 9
    high: 16
```

If a custom configuration overrides only some T-shirt sizes or Story Point values, the remaining built-in defaults stay available.

8.9 The `dependencies` field

`dependencies` is a list of task IDs that must complete before the current task can start.

Dependencies drive the critical-path analysis and scheduling. When the dependency graph is shallow (few dependencies), many tasks can run in parallel, and the project duration is limited mainly by the longest single task. When the graph is deep (long chains of dependent tasks), the critical path grows and the overall schedule becomes more sensitive to individual task delays. Expressing dependencies accurately is therefore important: underspecifying them produces over-optimistic forecasts; overspecifying them produces unnecessarily conservative ones.

8.9.1 Supported syntax

```
dependencies: []
dependencies: ["task_001"]
dependencies: ["task_001", "task_002", "task_003"]
```

8.9.2 Validation rules

The project validator enforces that:

- every dependency ID must match an existing task ID,
- task IDs must be unique,
- no circular dependency may exist,
- at least one task must exist overall.

8.9.3 Example

```
tasks:
  - id: "task_001"
    name: "Backend design"
    estimate:
      low: 2
      expected: 4
      high: 6

  - id: "task_002"
    name: "API implementation"
    estimate:
      low: 5
      expected: 8
      high: 12
    dependencies: ["task_001"]
```

8.10 The `uncertainty_factors` field

At task level, `uncertainty_factors` is modeled as a structured object with a fixed set of recognized fields.

Uncertainty factors apply a multiplier to a task's sampled duration before it enters the schedule. They let you express qualitative risk signals — "the team is junior", "the requirements are immature", "this task integrates with multiple external systems" — without converting those signals into numeric estimates by hand. The multipliers for each factor level are defined in the configuration file; the project file simply assigns a level to each factor.

All five factors are **optional** and each defaults to its baseline level, so you only need to specify factors where a task deviates from a typical baseline. Omitting `uncertainty_factors` entirely applies the full set of baseline defaults.

8.10.1 Recognized fields

Field	Default	Valid values
`team_experience`	`"medium"`	`"high"`, `"medium"`, `"low"`
`requirements_maturity`	`"medium"`	`"high"`, `"medium"`, `"low"`
`technical_complexity`	`"medium"`	`"low"`, `"medium"`, `"high"`
`team_distribution`	`"colocated"`	`"colocated"`, `"distributed"`
`integration_complexity`	`"medium"`	`"low"`, `"medium"`, `"high"`

8.10.2 YAML example

```
uncertainty_factors:
  team_experience: "high"
  requirements_maturity: "medium"
  technical_complexity: "high"
  team_distribution: "distributed"
  integration_complexity: "medium"
```

8.10.3 TOML example

```
[tasks.uncertainty_factors]
team_experience = "high"
requirements_maturity = "medium"
technical_complexity = "high"
team_distribution = "distributed"
integration_complexity = "medium"
```

8.10.4 Important implementation note

Only the five fields above are represented by the current project model and used by the simulation engine.

If you add other names under `uncertainty_factors`, they are not part of the supported project-file reference and should not be relied on as active inputs.

The numeric multiplier that each level applies (e.g. `"high"` for `team_experience` → `0.90×`) is defined in the configuration file. See The `uncertainty_factors` section later in this chapter for the built-in defaults and how to override them.

8.11 The `resources` field inside a task

Each task may list resource names as strings:

```
resources: ["backend_dev", "database_admin"]
```

Task-level `resources` is typed as a list of strings in the current model.

8.11.1 Resource assignment rule with `max_resources`

When `resources` lists multiple names, the scheduler may still assign fewer resources:

- assignment at task start is capped by `max_resources` (default `1`),
- scheduler applies an automatic practical cap:
  - `granularity_cap = max(1, floor(task_effort_hours / 16.0))`
  - `coordination_cap = 3`
  - `practical_cap = min(granularity_cap, coordination_cap)`
- effective start-time assignment is:
  - `min(max_resources, practical_cap, eligible_available_resources_now)`.

This avoids unrealistic over-assignment on short tasks while still permitting parallelization on larger tasks.

Important behavior for schema users:

- assignment happens at task start only,
- assigned resources remain fixed for the task execution,
- no mid-task reassignment/swapping is performed.

Also, if you explicitly list resource names and set `min_experience_level`, each named resource must meet that minimum or validation fails.

8.12 The `risks` field inside a task

Each task may have zero or more task-level risks.

A **risk** models a discrete event that may or may not occur during the task. In each Monte Carlo iteration, the engine draws a Bernoulli sample for each risk using its `probability`. When the risk fires, the `impact` is added to the task's sampled duration. This cleanly separates planned estimation uncertainty (captured by the estimate range) from identified discrete risks (captured here). It lets you quantify the schedule impact of a risk without baking it unconditionally into the estimate.

Task-level risks are appropriate for events that affect only one task — for example, "the third-party API may introduce a breaking change during this task" at probability 0.15. For events that could affect the whole

project — for example, "key stakeholder may request a scope change" — use `project_risks` instead.

For the complete risk grammar see Formal Grammar — `<risk_properties>`.

8.12.1 Task-level risk object fields

Field	Required	Type	Notes
`id`	Yes	string	Risk identifier
`name`	Yes	string	Display name
`probability`	Yes	float	Must be between `0.0` and `1.0`
`impact`	Yes	number or object	See below
`description`	No	string	Optional

8.12.2 Risk impact syntax

The `impact` field can be specified in two ways but is normally specified as as structured object. The two ways to specify risk impact are:

1. Structured impact object (recommended)

This may be either of type `"absolute"` or type `"percentage"` and is the clearest way to specify a risk as it is unambigous. A unit can only be specified if the type `"absolute"` since the percentage is applied to the already specified unit for the simulation result either for the task or the overall effort for the project. The following example makes it clear:

```
risks:
  - id: "risk_001"
    name: "Architecture rework"
    probability: 0.20
    impact:
      type: "absolute"
      value: 5
      unit: "days"
risks:
  - id: "risk_002"
    name: "Approval delay"
    probability: 0.10
    impact:
      type: "percentage"
      value: 15
```

In the first case `risk_001` the risk adds 5 working days (`=hours_per_day*5`). Unit can be one of `"hours"`, `"days"` or `"weeks"`.

2. Short form

This is interpreted as an absolute time penalty in hours and is supplied as a "quick-and-dirty" way to specify a risk impact. However, we strongly recommend that the full structured object way is used!

```
risks:
  - id: "risk_001"
    name: "Integration issues"
    probability: 0.25
    impact: 3  # Interpretated as an absolute impact in hours
```

Caution:

```
 Correct:
risks:
  - id: "risk_001"
    name: "Delay"
    probability: 0.2
    impact:
      type: "absolute"
      value: 3
      unit: "days"

 Wrong:
risks:
  - id: "risk_001"
    name: "Delay"
    probability: 0.2
    impact: 3
    unit: "days"  ← unit goes inside impact, not outside
```

8.12.3 Validation rules for risks

The model enforces:

- `probability` must be between `0.0` and `1.0`,
- `impact.value` must be greater than `0` when object syntax is used,
- numeric impacts are converted to floats,
- structured impacts must use `type: "absolute"` or `type: "percentage"`.
- a unit only makes sense for `"absolute"`type of impact

8.13 The `project_risks` section

`project_risks` has exactly the same syntax as task-level `risks`, but it appears at top level and applies to the project as a whole.

When a project-level risk fires in a Monte Carlo iteration, its impact is added to the total elapsed project duration (on top of the scheduled task chain). Use `project_risks` for cross-cutting uncertainties — late design freezes, vendor delays, or regulatory response times — that cannot be cleanly attributed to a single task.

8.13.1 Example

```
project_risks:
  - id: "proj_risk_001"
    name: "Requirements change"
    probability: 0.30
    impact:
      type: "absolute"
      value: 10
      unit: "days"
    description: "Late business scope change"
```

Note: Be careful if you add an impact type of percentage as a top level risk as it is added as a multiplicative percentage to the overall simulated effort.

8.14 Cost estimation

`mcprojsim` can model monetary costs alongside schedule duration. When cost estimation is active, each Monte Carlo iteration produces a total project cost in addition to an elapsed duration. The output includes a cost distribution with the same percentile structure as the schedule output.

Cost estimation is **opt-in**: it activates automatically as soon as any of the following are present in the project file:

- `project.default_hourly_rate` is set to a value greater than zero,
- any `resources` entry has a non-null `hourly_rate`,
- any task has a non-null `fixed_cost`,
- any task-level or project-level risk has a non-null `cost_impact`.

If none of these conditions hold, the simulation runs without cost tracking and no cost section appears in the output.

8.14.1 Cost components

Each iteration's total project cost is the sum of four components:

$$\text{total_cost} = \text{labor_cost} + \text{fixed_cost} + \text{risk_cost} + \text{overhead}$$

Labor cost is `sum over all tasks of (sampled_task_duration_hours × effective_hourly_rate)`. The effective rate for a task is:

- the **resource's** `hourly_rate` when the task is assigned to a named resource with a rate,
- the **mean rate** across all assigned resources when multiple resources are assigned,
- the **project-level** `default_hourly_rate` for tasks in dependency-only mode, or for tasks with no assigned resource or no resource rate.

Fixed cost is the sum of `fixed_cost` across all tasks. Fixed costs are applied once per iteration regardless of sampled duration. They may be negative (to represent subsidies, credits, or rebates).

Risk cost is the sum of all `cost_impact` values for risks that trigger in that iteration — both task-level and project-level risks. A risk's cost impact shares the same probability roll as its time impact: if the risk fires, both the time delay and the cost penalty are applied in the same iteration.

Overhead is `labor_cost × overhead_rate`. It is applied **only to labor** — fixed costs and risk costs are not marked up by overhead. Per-task cost breakdowns in the output therefore do not include overhead; overhead is shown separately at the project level.

8.14.2 Project-level cost fields

These fields are all optional and appear inside the `project` section.

Field	Type	Default	Constraints	Notes
`default_hourly_rate`	float	`null`	`≥ 0`	Blended hourly rate for all tasks when no per-resource rate is available; activates cost estimation when `> 0`
`overhead_rate`	float	`0.0`	`0.0` – `3.0`	Fractional overhead markup on labor cost (e.g. `0.20` = 20%)
`currency`	string	config default (`"EUR"`)	ISO 4217	Currency label used in output and exports; non-standard codes produce a warning
`secondary_currencies`	list of strings	`[]`	max 5 entries, ISO 4217	Additional currencies for secondary conversion output in JSON exports

Field	Type	Default	Constraints	Notes
`fx_conversion_cost`	float	`0.0`	`0.0` – `0.50`	Bank conversion spread fraction added to the official mid-market exchange rate
`fx_overhead_rate`	float	`0.0`	`0.0` – `1.0`	Additional overhead fraction for hedging or admin costs, applied on top of `fx_conversion_cost`
`fx_rates`	mapping	`{}`	—	Manual exchange rate overrides by ISO 4217 target code; bypasses live rate fetch and applies no markups

YAML example

```yaml
project:
  name: "API Rewrite"
  start_date: "2026-06-01"
  default_hourly_rate: 150.0
  overhead_rate: 0.15
  currency: "EUR"
  secondary_currencies: ["USD", "SEK"]
  fx_conversion_cost: 0.015
  fx_overhead_rate: 0.005
```

8.14.3 Task-level cost field

Field	Type	Default	Constraints	Notes
`fixed_cost`	float	`null`	any finite float	One-time monetary cost for this task, independent of effort; may be negative

`fixed_cost` is applied once per iteration—it does not scale with sampled duration. Use it to represent predictable one-time fees attached to a task, such as cloud infrastructure provisioning charges, software licence costs, or vendor setup fees. A negative value models a credit or rebate.

YAML example

```
tasks:
  - id: "infra_setup"
    name: "Cloud Infrastructure Setup"
    estimate:
      low: 16
      expected: 24
      high: 40
      unit: "hours"
    fixed_cost: 3500.0      # one-time provisioning fee
  - id: "data_migration"
    name: "Data Migration"
    estimate:
      low: 8
      expected: 16
      high: 32
      unit: "hours"
    fixed_cost: -1200.0     # vendor credit applied on migration completion
```

8.14.4 Risk `cost_impact` field

Both task-level and project-level risks support an optional `cost_impact` field.

Field	Type	Default	Constraints	Notes
`cost_impact`	float	`null`	any finite float	Monetary cost added when this risk triggers; may be negative for cost savings; shares the probability roll of the time impact

`cost_impact` is independent of the time `impact` field. You can specify:

- a time impact only (schedule delay, no cost),
- a `cost_impact` only (cost penalty without schedule delay, using `impact: 0` for the time part),
- or both together.

When a risk fires, both the time impact and the cost impact are applied in the same iteration. The shared probability roll means cost and schedule impacts are always correlated: a risk either affects both, or neither.

YAML example — task-level risk with both impacts

```
risks:
  - id: "risk_api_change"
    name: "Third-Party API Breaking Change"
    probability: 0.20
    impact:
      type: "absolute"
      value: 2
      unit: "days"
    cost_impact: 4500.0     # emergency vendor support fee
```

YAML example — cost-only risk

```
risks:
  - id: "risk_audit_fee"
    name: "Regulatory Audit Surcharge"
    probability: 0.10
    impact: 0               # no time delay
    cost_impact: 8000.0     # external audit costs if triggered
```

YAML example — negative cost impact (saving)

```
risks:
  - id: "risk_early_signoff"
    name: "Early Stakeholder Sign-off"
    probability: 0.15
    impact: 0
    cost_impact: -1500.0    # early-completion bonus reduces total cost
```

8.14.5 Resource-level cost field

When named resources are defined (see The top-level `resources` section), each resource may specify its own hourly rate.

Field	Type	Default	Constraints	Notes
`hourly_rate`	float	`null`	`≥ 0`	Per-resource hourly cost rate; overrides `project.default_hourly_rate` for tasks assigned to this resource

When a task is assigned to resources, the effective rate used for labor cost is the mean of the `hourly_rate` values of the assigned resources. Resources with `hourly_rate: null` fall back to `project.default_hourly_rate`. Setting `hourly_rate: 0.0` is valid and models volunteer or no-cost resources.

YAML example

```
resources:
  - name: "alice"
    experience_level: 3
    hourly_rate: 240.0      # principal engineer; overrides project default
  - name: "bob"
    experience_level: 2
    hourly_rate: 160.0
  - name: "carol"
    experience_level: 2
    hourly_rate: 105.0
  - name: "intern"
    experience_level: 1
    hourly_rate: 0.0        # no cost
```

8.14.6 Configuration file cost settings

The `cost` section of the configuration file supplies defaults that apply when the project file does not specify a value.

Field	Type	Default	Constraints	Notes
`default_hourly_rate`	float	`null`	`≥ 0`	Fallback hourly rate when project does not set one
`overhead_rate`	float	`0.0`	`0.0` – `3.0`	Fallback overhead markup when project does not set one
`currency`	string	`"EUR"`	ISO 4217	Fallback currency when `project.currency` is not set
`include_in_output`	boolean	`true`	—	Whether to include cost sections in CLI output and exports when cost data is present

YAML example

```yaml
cost:
  default_hourly_rate: 130.0
  overhead_rate: 0.15
  currency: "USD"
  include_in_output: true
```

8.14.7 Complete cost example

The following project file uses all cost features: per-resource rates, a project fallback rate for unassigned tasks, fixed costs, positive and negative risk cost impacts, and overhead.

```yaml
project:
  name: "E-Commerce Platform Launch"
  start_date: "2026-08-01"
  hours_per_day: 8
  default_hourly_rate: 130.0   # fallback for tasks with no named resource rate
  overhead_rate: 0.18          # 18 % markup on total labor
  currency: "USD"
  confidence_levels: [50, 80, 90, 95]

resources:
  - name: "alice"
    experience_level: 3
    hourly_rate: 240.0         # principal engineer
  - name: "bob"
    experience_level: 3
    hourly_rate: 160.0         # senior engineer
  - name: "carol"
    experience_level: 2
    hourly_rate: 105.0         # mid-level engineer

project_risks:
  - id: "proj_risk_001"
    name: "Regulatory Delay"
    probability: 0.12
    impact:
      type: "absolute"
      value: 1
      unit: "weeks"
    cost_impact: 12000.0       # legal fees if triggered

tasks:
  - id: "cloud_infra"
    name: "Cloud Infrastructure"
    estimate:
      low: 16
      expected: 24
      high: 40
      unit: "hours"
    resources: ["alice"]
    fixed_cost: -5000.0        # AWS credit (negative = rebate)
  - id: "checkout"
    name: "Checkout & Payments"
    estimate:
      low: 40
      expected: 80
      high: 140
      unit: "hours"
    resources: ["bob"]
    dependencies: ["cloud_infra"]
    fixed_cost: 1200.0         # Stripe setup fee
    risks:
      - id: "risk_pci"
        name: "PCI Scope Expansion"
        probability: 0.20
        impact:
          type: "absolute"
          value: 4
          unit: "days"
        cost_impact: 9500.0    # external QSA assessment

  - id: "admin_tasks"
    name: "Documentation and Handover"
    estimate:
      low: 8
      expected: 16
      high: 24
```

8.14.8 Validation rules for cost fields

- `default_hourly_rate` and `hourly_rate` must be `≥ 0` (zero is allowed; negative is rejected).
- `overhead_rate` must be in the range `0.0` to `3.0`.
- `fx_conversion_cost` must be in the range `0.0` to `0.50`.
- `fx_overhead_rate` must be in the range `0.0` to `1.0`.
- `secondary_currencies` may contain at most 5 entries.
- Non-standard ISO 4217 currency codes produce a `UserWarning` rather than an error.
- `fixed_cost` and `cost_impact` may be negative; no lower-bound constraint is applied.
- `cost_impact: 0` on a risk is valid when a time-only impact is intended (the `cost_impact` field may simply be omitted in that case).

8.15 The top-level `resources` section

Adding a `resources` section switches the scheduler from **dependency-only** mode to **resource-constrained** mode. In dependency-only mode the scheduler assumes unlimited workforce and sequences tasks purely by dependencies. In resource-constrained mode each task can only start when both its dependencies are satisfied *and* a suitable resource is available. This produces longer but more realistic schedules whenever the team is genuinely the bottleneck.

If you prefer not to enumerate individual resources, you can instead set `project.team_size` to a number of default resources to generate automatically.

Each `resources` entry supports the following fields:

8.15.1 Supported fields

Field	Required	Type	Default	Constraints
`name`	No	string	auto-generated	Unique across all resolved resource names
`id`	No	string	`null`	Legacy fallback for `name`; still accepted
`experience_level`	No	integer	`2`	Must be `1`, `2`, or `3`
`productivity_level`	No	float	`1.0`	0.1 to 2.0
`sickness_prob`	No	float	`0.0`	0.0 to 1.0
`planned_absence`	No	list of dates	`[]`	ISO `YYYY-MM-DD` format
`calendar`	No	string	`"default"`	Must match a calendar `id` if calendars are defined
`availability`	No	float	`1.0`	Must be in (0, 1]

!!! note If `name` is omitted, the engine auto-generates a unique name (`resource_001`, `resource_002`, …). The legacy `id` field is still accepted and used as a fallback for `name`.

8.15.2 YAML example

```
resources:
  - name: "backend_dev"
    experience_level: 3
    productivity_level: 1.1
    sickness_prob: 0.05
    planned_absence:
      - "2026-07-01"
      - "2026-07-02"
  - experience_level: 2
    productivity_level: 0.9
    sickness_prob: 0.08
```

8.16 The top-level `calendars` section

Calendars control when resources are available. Without a calendar definition, the scheduler uses an 8-hour / 5-day working week. Defining a calendar lets you model public holidays, reduced-hour days, or non-standard working weeks. Each resource can reference a specific calendar by name; resources that do not specify one fall back to the `default` calendar.

Each `calendars` entry supports the following fields:

Field	Required	Type	Default	Notes
`id`	No	string	`"default"`	Must be unique across all calendars
`work_hours_per_day`	No	float > 0	`8.0`	Number of working hours in a day
`work_days`	No	list of integers	`[1, 2, 3, 4, 5]`	Days of the week; 1 = Monday, 7 = Sunday
`holidays`	No	list of ISO dates	`[]`	Dates in `YYYY-MM-DD` format

8.16.1 YAML example

```
calendars:
  - id: "standard"
    work_hours_per_day: 8
    work_days: [1, 2, 3, 4, 5]
    holidays:
      - "2026-12-25"
      - "2026-12-26"
```

8.17 The `sprint_planning` section

The optional `sprint_planning` section activates sprint-based simulation mode. When present and `enabled: true`, the engine models work as a sequence of fixed-length sprints rather than a single elapsed duration. Each sprint draws a velocity from the historical distribution, places as many backlog items as will fit, and carries the rest forward to the next sprint. The output is a distribution over the number of sprints (and hence calendar weeks) needed to complete the backlog, rather than a raw elapsed-hours distribution.

Use sprint planning when the team works in fixed-length iterations and tracks velocity 2014 whether measured in story points or tasks completed per sprint. The historical sprint data you provide is the primary input: the simulator fits a velocity distribution to it and samples from that distribution in each Monte Carlo iteration.

!!! note Sprint planning requires at least two usable historical sprint entries (entries with a positive delivery signal). The `capacity_mode` field controls whether the unit of delivery is story points or task counts, and **must be consistent** across all history entries and task backlogs.

For a comprehensive walkthrough of sprint planning features, configuration options, and interpretation of results, see Sprint Planning.

8.17.1 Top-level sprint planning fields

Field	Required	Type	Default	Notes
`enabled`	No	boolean	`false`	When `false`, the section is parsed but not simulated
`sprint_length_weeks`	Yes	integer > 0	—	Duration of each sprint in calendar weeks
`capacity_mode`	Yes	`story_points` | `tasks`	—	Unit family for all velocity and backlog measurements
`planning_confidence_level`	No	float (0, 1)	`0.80`	Percentile of the sprint count distribution reported
`velocity_model`	No	`empirical` | `neg_binomial`	`empirical`	Distribution fitted to historical velocity observations
`removed_work_treatment`	No	`churn_only` | `reduce_backlog`	`churn_only`	How removed sprint items affect the net backlog
`history`	No	list or external descriptor	`[]`	Historical sprint data; see below
`future_sprint_overrides`	No	list	`[]`	Forward-looking capacity adjustments
`volatility_overlay`	No	object	disabled	Sprint-level disruption model
`spillover`	No	object	disabled	Task-level spillover model
`sickness`	No	object	disabled	Per-person sickness model

`velocity_model` values

Value	Meaning
`empirical`	Resample directly from observed historical velocities
`neg_binomial`	Fit a negative-binomial distribution to history and sample from that

`removed_work_treatment` values

Value	Meaning
`churn_only`	Removed items are treated as pure churn with no net effect on remaining backlog
`reduce_backlog`	Removed items reduce the total backlog to be completed

8.17.2 Minimal example

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 10
      spillover_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 9
      spillover_story_points: 2
    - sprint_id: "SPR-003"
      completed_story_points: 11
```

For a `tasks`-based project, replace `completed_story_points` with `completed_tasks` and set `capacity_mode: tasks`.

8.17.3 Sprint history

Historical sprint data is the empirical foundation of the sprint planning simulation. Rather than asking you to specify a single velocity, `mcprojsim` fits a probability distribution to your team's observed sprint outcomes and samples from that distribution in each Monte Carlo iteration. The result is a forecast of "how many sprints will this backlog require?" expressed as a confidence interval, not a single number.

Why history matters

Two sprints of history give the simulator just enough to estimate the spread of the velocity distribution. More history reduces sampling uncertainty and produces tighter, more reliable confidence intervals. A minimum of two usable entries — entries where the delivery signal (completed points or tasks) is greater than zero — is required when `enabled: true`.

The velocity the simulator works with is not simply `completed_story_points`. For each historical sprint it computes the **effective velocity** as:

```
effective_velocity = completed + spillover - added + removed  (treatment-dependent)
```

This means `spillover_story_points`, `added_story_points`, and `removed_story_points` all influence the distribution the simulator samples from. The more accurately you record them, the more representative the velocity distribution will be.

Unit-family consistency

All history entries in a project file must belong to the same unit family as `capacity_mode`. You cannot mix `completed_story_points` entries with `completed_tasks` entries in a single `history` list.

Inline history format

Each `history` list entry represents one completed sprint. Fields marked *Conditional* are required only for the chosen `capacity_mode`.

Field	Required	Type	Default	Notes
`sprint_id`	Yes	string	—	Unique string identifier for the sprint; must be non-empty
`sprint_length_weeks`	No	integer > 0	inherits top-level `sprint_length_weeks`	Override sprint length for this individual entry

Field	Required	Type	Default	Notes
`completed_story_points`	Conditional	float ≥ 0	—	Completed capacity; required when `capacity_mode: story_points`
`completed_tasks`	Conditional	integer ≥ 0	—	Completed capacity; required when `capacity_mode: tasks`
`spillover_story_points`	No	float ≥ 0	`0`	Story points that carried over from the previous sprint into this one
`spillover_tasks`	No	integer ≥ 0	`0`	Tasks that carried over from the previous sprint into this one
`added_story_points`	No	float ≥ 0	`0`	Story points added to the sprint backlog mid-sprint
`added_tasks`	No	integer ≥ 0	`0`	Tasks added to the sprint backlog mid-sprint
`removed_story_points`	No	float ≥ 0	`0`	Story points removed from the sprint backlog mid-sprint
`removed_tasks`	No	integer ≥ 0	`0`	Tasks removed from the sprint backlog mid-sprint
`holiday_factor`	No	float > 0	`1.0`	Capacity reduction from public holidays; `0.8` means 20% reduction
`end_date`	No	ISO date string	`null`	Date the sprint ended; used for timeline charts and calendar alignment
`team_size`	No	integer ≥ 0	`null`	Actual team headcount during this sprint
`notes`	No	string	`null`	Free-text annotation; not used by the simulator

!!! note Story-point and task-count fields are mutually exclusive within a single entry. Do not mix `completed_story_points` with `spillover_tasks`, `added_tasks`, or `removed_tasks` in the same entry, or vice versa. The validator will reject it.

Story-point mode example

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 34
      spillover_story_points: 2
    - sprint_id: "SPR-002"
      completed_story_points: 28
      spillover_story_points: 4
      added_story_points: 3
    - sprint_id: "SPR-003"
      completed_story_points: 31
      removed_story_points: 2
      holiday_factor: 0.9
      end_date: "2026-03-14"
      notes: "Public holiday reduced capacity"
```

Task-count mode example

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 1
  capacity_mode: tasks
  history:
    - sprint_id: "WK-01"
      completed_tasks: 7
      spillover_tasks: 1
    - sprint_id: "WK-02"
      completed_tasks: 6
      spillover_tasks: 2
      added_tasks: 1
    - sprint_id: "WK-03"
      completed_tasks: 8
```

External history format

For teams that maintain sprint data in a separate file, `mcprojsim` can load history from an external JSON or CSV source instead of an inline list:

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  history:
    format: json
    path: "sprint_planning_history.json"
```

Supported formats:

Value	Description
`json`	A JSON file containing either a top-level array of sprint objects, or an object with a `sprints` key whose value is that array
`csv`	A CSV file with a header row; column names must match the field names in the table above

The external file must use the same field names as the inline entries. See Sprint Planning for complete JSON

and CSV shape examples.

8.17.4 `future_sprint_overrides`

Forward-looking sprint overrides let you express known capacity variations in upcoming sprints 2014 for example, a holiday week that will reduce capacity by 20%, or a sprint where the team is larger.

Each override must identify its target sprint via at least one of `sprint_number` or `start_date`.

Field	Required	Type	Default	Notes
`sprint_number`	Conditional	integer > 0	`null`	1-based sprint number relative to the start of simulation
`start_date`	Conditional	date string	`null`	ISO date of the sprint start
`holiday_factor`	No	float > 0	`1.0`	Capacity scaling due to holidays
`capacity_multiplier`	No	float > 0	`1.0`	Additional scaling factor (e.g. `0.5` for a half-team sprint)
`notes`	No	string	`null`	Annotation

The effective capacity for an overridden sprint is `holiday_factor \u00d7 capacity_multiplier` of the baseline.

```
future_sprint_overrides:
  - sprint_number: 3
    holiday_factor: 0.8
    notes: "Easter week"
  - start_date: "2026-06-15"
    capacity_multiplier: 0.5
    notes: "Half the team at off-site"
```

8.17.5 Task fields used by sprint planning

When `sprint_planning` is active, three additional task fields become relevant:

Field	Type	`capacity_mode`	Notes
`planning_story_points`	integer > 0	`story_points`	Story point size for sprint planning; overrides `estimate.story_points` when set
`priority`	integer	both	Scheduling priority hint; lower values are allocated to sprints first
`spillover_probability_override`	float 0.020131.0	both	Per-task spillover probability, overrides the model default

8.17.6 Configuration interaction

The `sprint_defaults` section in the configuration file supplies default values for every `sprint_planning` parameter. Any value set directly in the project file's `sprint_planning` section overrides the corresponding config default. This means you can tune velocity models, sickness parameters, and spillover behaviour once in a shared config file and override only sprint-specific values in each project file.

8.18 Full YAML example

The following example demonstrates every currently recognized project-file section in one file.

```yaml
project:
  name: "Reference Example"
  description: "Comprehensive project file example"
  start_date: "2026-04-01"
  currency: "USD"
  confidence_levels: [50, 75, 80, 90, 95]
  probability_red_threshold: 0.45
  probability_green_threshold: 0.90
project_risks:
  - id: "proj_risk_001"
    name: "Late stakeholder change"
    probability: 0.20
    impact:
      type: "absolute"
      value: 4
      unit: "days"
tasks:
  - id: "task_001"
    name: "Design"
    description: "Design the feature set"
    estimate:
      low: 2
      expected: 4
      high: 7
      unit: "days"
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "medium"
    resources: ["designer"]
    risks:
      - id: "task_risk_001"
        name: "Clarification delay"
        probability: 0.15
        impact: 1.5
  - id: "task_002"
    name: "Implementation"
    estimate:
      distribution: "lognormal"
      low: 3
      expected: 8
      high: 20
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "high"
      team_distribution: "distributed"
    resources: ["backend_dev", "frontend_dev"]
  - id: "task_003"
    name: "Deployment"
    estimate:
      t_shirt_size: "S"
    dependencies: ["task_002"]

resources:
  - name: "designer"
  - name: "backend_dev"
  - name: "frontend_dev"

calendars:
  - id: "standard"
    work_days: [1, 2, 3, 4, 5]
    holidays:
      - "2026-12-25"
```

8.19 Full TOML example

The same logical content can be written in TOML. The most important syntax difference is that repeated entries such as tasks and risks use array-of-table syntax.

```
[project]
name = "Reference Example"
description = "Comprehensive project file example"
start_date = "2026-04-01"
currency = "USD"
confidence_levels = [50, 75, 80, 90, 95]
probability_red_threshold = 0.45
probability_green_threshold = 0.90

[[project_risks]]
id = "proj_risk_001"
name = "Late stakeholder change"
probability = 0.20
[project_risks.impact]
type = "absolute"
value = 4
unit = "days"

[[tasks]]
id = "task_001"
name = "Design"
description = "Design the feature set"
dependencies = []
resources = ["designer"]
[tasks.estimate]
min = 2
expected = 4
max = 7
unit = "days"

[tasks.uncertainty_factors]
team_experience = "high"
requirements_maturity = "medium"

[[tasks.risks]]
id = "task_risk_001"
name = "Clarification delay"
probability = 0.15
impact = 1.5

[[tasks]]
id = "task_002"
name = "Implementation"
dependencies = ["task_001"]
resources = ["backend_dev", "frontend_dev"]

[tasks.estimate]
distribution = "lognormal"
low = 3
expected = 8
high = 20
unit = "days"

[tasks.uncertainty_factors]
team_experience = "medium"
technical_complexity = "high"
team_distribution = "distributed"

[[tasks]]
id = "task_003"
name = "Deployment"
dependencies = ["task_002"]

[tasks.estimate]
t_shirt_size = "S"
[[resources]]
```

8.20 Configuration file reference

The project file defines the work being simulated. The configuration file defines how `mcprojsim` interprets uncertainty, symbolic estimates, reporting defaults, and staffing analysis.

Use a configuration file when you want to:

- change uncertainty multipliers,
- override T-shirt size mappings,
- override Story Point mappings,
- set simulation and report defaults,
- tune staffing analysis behavior.

Unlike project files, the configuration file is currently loaded as YAML.

8.20.1 How configuration loading works

When you pass `--config config.yaml`, the loader:

1. starts from the built-in default configuration,
2. reads your YAML file,
3. merges your values into the defaults recursively,
4. validates the result against the `Config` model.

That means you can override only the values you care about. For example, if you provide only `t_shirt_sizes.M`, the built-in definitions for `XS`, `S`, `L`, `XL`, and `XXL` remain available.

8.20.2 Top-level configuration structure

The current configuration schema supports these top-level sections:

- `uncertainty_factors`
- `t_shirt_sizes`
- `t_shirt_size_unit`
- `t_shirt_size_default_category`
- `story_points`
- `story_point_unit`
- `lognormal`
- `simulation`
- `output`
- `staffing`
- `constrained_scheduling`
- `sprint_defaults`

8.20.3 Minimal configuration example

```
simulation:
  default_iterations: 5000

staffing:
  effort_percentile: 80
```

8.20.4 Full configuration skeleton

```
uncertainty_factors:
  team_experience:
    high: 0.90
    medium: 1.0
    low: 1.30
  requirements_maturity:
    high: 1.0
    medium: 1.15
    low: 1.40
  technical_complexity:
    low: 1.0
    medium: 1.20
    high: 1.50
  team_distribution:
    colocated: 1.0
    distributed: 1.25
  integration_complexity:
    low: 1.0
    medium: 1.15
    high: 1.35
t_shirt_sizes:
  story:
    XS:
      low: 3
      expected: 5
      high: 15
    M:
      low: 40
      expected: 60
      high: 120
  epic:
    M:
      low: 200
      expected: 480
      high: 1200
t_shirt_size_unit: "hours"
t_shirt_size_default_category: "epic"
story_points:
  1:
    low: 0.5
    expected: 1
    high: 3
  5:
    low: 3
    expected: 5
    high: 8
story_point_unit: "days"

lognormal:
  high_percentile: 95

simulation:
  default_iterations: 10000
  random_seed: null
  max_stored_critical_paths: 20

output:
  formats: ["json", "csv", "html"]
  include_histogram: true
  number_bins: 50
  critical_path_report_limit: 2

staffing:
  min_individual_productivity: 0.25
```

8.21 The `uncertainty_factors` section

This section maps uncertainty factor names to per-level multipliers.

8.21.1 YAML structure

```
uncertainty_factors:
  factor_name:
    level_name: 1.0
    another_level: 1.0
```

8.21.2 Supported fields

Field	Required	Type	Default	Notes
factor name	No	mapping	built-in defaults	Outer keys are factor names
level name	No	float	none at the schema level	Inner keys depend on the factor, for example `high`, `medium`, `low`, `colocated`, or `distributed`

8.21.3 Built-in level names by factor

Factor	Built-in level names
`team_experience`	`high`, `medium`, `low`
`requirements_maturity`	`high`, `medium`, `low`
`technical_complexity`	`low`, `medium`, `high`
`team_distribution`	`colocated`, `distributed`
`integration_complexity`	`low`, `medium`, `high`

8.21.4 Built-in factor names

The default configuration defines these factor names:

- `team_experience`
- `requirements_maturity`
- `technical_complexity`
- `team_distribution`
- `integration_complexity`

These are also the names used by the current project-file model under `tasks[].uncertainty_factors`.

8.21.5 Built-in defaults

Factor	High / low-side values	Medium / baseline
`team_experience`	`high: 0.90`, `low: 1.30`	`medium: 1.0`
`requirements_maturity`	`high: 1.0`, `low: 1.40`	`medium: 1.15`
`technical_complexity`	`low: 1.0`, `high: 1.50`	`medium: 1.20`
`team_distribution`	`colocated: 1.0`, `distributed: 1.25`	not applicable
`integration_complexity`	`low: 1.0`, `high: 1.35`	`medium: 1.15`

!!! note The configuration model can parse arbitrary nested dictionaries here, but the current project-file schema only exposes the recognized uncertainty-factor names listed above. Extra factor names in the config file are not useful unless the source model and simulation logic also reference them.

8.22 The `t_shirt_sizes` section

This section maps symbolic T-shirt sizes to numeric effort ranges by category. The typical structure looks like this:

```
t_shirt_sizes:
  story:
    M:
      low: 40
      expected: 60
      high: 120
  epic:
    M:
      low: 200
      expected: 480
      high: 1200

t_shirt_size_default_category: epic
```

8.22.1 Supported fields for each category/size entry

Field	Required	Type	Default	Constraints
`low`	Yes when that size is defined	float	—	`> 0`
`expected`	Yes	float	—	`> 0`
`high`	Yes	float	—	`> 0`

8.22.2 Built-in category keys

- `bug`
- `story`
- `epic`
- `business`
- `initiative`

8.22.3 Built-in size keys (per category)

- `XS`
- `S`
- `M`
- `L`
- `XL`
- `XXL`

8.22.4 Built-in `story` defaults

Size	`low`	`expected`	`high`
`XS`	3	5	15
`S`	5	16	40
`M`	40	60	120
`L`	160	240	500
`XL`	320	400	750
`XXL`	400	500	1200

8.22.5 Example override

```
t_shirt_sizes:
  story:
    M:
      low: 45
      expected: 65
      high: 130

t_shirt_size_default_category: epic
```

With this override, only `story.M` changes. Other built-in categories and sizes remain available.

8.23 The `t_shirt_size_unit` field

This field controls the unit used for all values in `t_shirt_sizes`.

8.23.1 Supported values

- `"hours"`
- `"days"`
- `"weeks"`

8.23.2 Default

`"hours"`

8.23.3 Example

```
t_shirt_size_unit: "days"
```

If a task uses `estimate.t_shirt_size: "M"`, the simulator resolves it through `t_shirt_size_default_category`. A qualified value like `estimate.t_shirt_size: "epic.M"` resolves directly to that category.

8.24 The `story_points` section

This section maps Story Point values to numeric effort ranges.

8.24.1 Supported fields for each point value

Field	Required	Type	Default	Constraints
`low`	Yes when that point value is defined	float	—	`> 0`
`expected`	Yes	float	—	`> 0`
`high`	Yes	float	—	`> 0`

8.24.2 Built-in point values

- `1`
- `2`
- `3`
- `5`
- `8`
- `13`
- `21`

8.24.3 Built-in defaults

Points	low	expected	high
1	0.5	1	3
2	1	2	4
3	1.5	3	5
5	3	5	8
8	5	8	15
13	8	13	21
21	13	21	34

8.24.4 Example override

```
story_points:
  8:
    low: 6
    expected: 9
    high: 16
```

8.25 The `story_point_unit` field

This field controls the unit used for all values in `story_points`.

8.25.1 Supported values

- `"hours"`
- `"days"`
- `"weeks"`

8.25.2 Default

`"days"`

8.26 The `lognormal` section

This section controls how the simulation interprets log-normal estimates.

8.26.1 Supported fields

Field	Required	Type	Default	Constraints	Notes
`high_percentile`	No	integer	`95`	one of `70`, `75`, `80`, `85`, `90`, `95`, `99`	The percentile that `high` is treated as in a log-normal estimate

8.26.2 Example

```
lognormal:
  high_percentile: 90
```

When a task uses `distribution: lognormal`, the `high` value is fitted as this percentile of the resulting distribution. A lower value makes the tail shorter; a higher value widens it.

8.27 The `simulation` section

This section controls default simulation behavior.

8.27.1 Supported fields

Field	Required	Type	Default	Constraints	Notes
`default_iterations`	No	integer	`10000`	`> 0`	Used by commands that rely on config defaults
`random_seed`	No	integer or `null`	`null`	none	Set for reproducible runs
`max_stored_critical_paths`	No	integer	`20`	`> 0`	Number of full critical path sequences retained in results

8.27.2 Example

```
simulation:
  default_iterations: 20000
  random_seed: 42
  max_stored_critical_paths: 50
```

8.28 The `output` section

This section controls reporting and export defaults.

8.28.1 Supported fields

Field	Required	Type	Default	Constraints	Notes
`formats`	No	list of strings	`["json", "csv", "html"]`	each entry must be `json`, `csv`, or `html`; list must not be empty	Default export formats for config-driven workflows
`include_histogram`	No	boolean	`true`	—	Whether histogram data should be included where supported
`number_bins`	No	integer	`50`	`> 0`	Number of bins for histogram generation
`critical_path_report_limit`	No	integer	`2`	`> 0`	Number of stored full critical paths shown in reports by default

8.28.2 Example

```
output:
  formats: ["json", "html"]
  include_histogram: true
  number_bins: 80
  critical_path_report_limit: 5
```

8.29 The `staffing` section

This section controls the staffing analysis added to CLI output and exports.

8.29.1 Supported fields

Field	Required	Type	Default	Constraints	Notes
`effort_percentile`	No	integer	omitted	`1..99` when set	Uses that effort percentile instead of the mean for staffing calculations
`min_individual_productivity`	No	float	`0.25`	`> 0`, `<= 1`	Lower bound on each person's productivity after communication overhead is applied
`experience_profiles`	No	mapping	built-in defaults	profile values validated individually	Defines named team profiles

When `effort_percentile` is omitted, staffing uses the mean total effort and mean elapsed time. When it is set, staffing uses the matching percentile for both effort and elapsed time, for example P80 effort with P80 elapsed time.

8.29.2 How `min_individual_productivity` affects team-size efficiency

The staffing model assumes that each additional person creates some communication overhead. For a team of size n, the model first calculates a raw per-person productivity:

$$P_{raw}(n) = 1 - c(n-1)$$

where c is the `communication_overhead` for the selected experience profile.

That raw value is then floored by `min_individual_productivity`:

$$P(n) = \max(P_{min}, P_{raw}(n))$$

where P_{min} is `min_individual_productivity`.

This means `min_individual_productivity` is not a bonus. It is a safety floor. It prevents the model from predicting that people become almost useless, or literally zero-productivity, as team size increases.

The model then converts per-person productivity into total effective capacity:

$$E(n) = n \cdot P(n) \cdot f$$

where f is the profile's `productivity_factor`.

Calendar duration for that team size is then:

$$T(n) = \max\left(T_{cp}, \frac{W}{E(n)}\right)$$

where:

- T_{cp} is the critical-path elapsed time,
- W is the total effort in person-hours,
- $E(n)$ is the effective capacity of the team.

Finally, the **Efficiency** shown in the staffing table is calculated relative to the fastest team size found for that profile:

$$\text{Efficiency}(n) = \frac{T_{min}}{T(n)}$$

So `min_individual_productivity` affects efficiency indirectly:

- if the floor is **lower**, very large teams lose more effective capacity as communication overhead grows,
- if the floor is **higher**, large teams retain more capacity and the efficiency drop-off on the right side of the staffing table is less severe,
- if the project is already near the critical-path floor, changing this value may have little visible effect, because no team can compress the schedule below T_{cp} anyway.

8.29.3 Practical interpretation

- **Small team sizes**: `min_individual_productivity` usually does nothing, because raw productivity is still above the floor.
- **Medium team sizes**: the value may begin to matter if communication overhead becomes significant.
- **Large team sizes**: this setting determines how harshly the model penalises oversized teams.

For example, with `communication_overhead: 0.06`, raw individual productivity is:

- 1 person: 1.00
- 3 people: $1 - 0.06 \cdot 2 = 0.88$
- 8 people: $1 - 0.06 \cdot 7 = 0.58$
- 15 people: $1 - 0.06 \cdot 14 = 0.16$

If `min_individual_productivity` is `0.25`, the 15-person team is floored to 0.25 instead of dropping to 0.16. That keeps the team from looking unrealistically ineffective, while still showing diminishing returns.

In practice:

- use a **lower** value when you want the model to penalise oversized teams more aggressively,
- use a **higher** value when you believe communication overhead is real but should not collapse individual output too sharply,
- keep in mind that this value mainly shapes the **right-hand side** of the efficiency curve, where teams are larger than the optimal size.

8.29.4 The `experience_profiles` subsection

Each profile name maps to an object with these fields:

Field	Required	Type	Default	Constraints
`productivity_factor`	Yes when that profile is defined	float	—	`> 0`
`communication_overhead`	Yes when that profile is defined	float	—	`0..1`

8.29.5 Built-in profile defaults

Profile	`productivity_factor`	`communication_overhead`
`senior`	1.00	0.04
`mixed`	0.85	0.06
`junior`	0.65	0.08

8.29.6 Example

```
staffing:
  effort_percentile: 80
  min_individual_productivity: 0.30
  experience_profiles:
    senior:
      productivity_factor: 1.0
      communication_overhead: 0.03
    contractor:
      productivity_factor: 0.75
      communication_overhead: 0.05
```

8.29.7 Configuration validation summary

The current configuration model validates these rules directly:

- `t_shirt_size_unit` must be one of `hours`, `days`, or `weeks`,
- `story_point_unit` must be one of `hours`, `days`, or `weeks`,
- `lognormal.high_percentile` must be one of `70`, `75`, `80`, `85`, `90`, `95`, `99`,
- all configured estimate ranges require positive `min`, `expected`, and `max`,
- `output.formats` must be a non-empty list; each entry must be `json`, `csv`, or `html`,
- `simulation.default_iterations` must be greater than 0,
- `simulation.max_stored_critical_paths` must be greater than 0,
- `output.number_bins` must be greater than 0,
- `output.critical_path_report_limit` must be greater than 0,
- `staffing.effort_percentile`, when set, must be between 1 and 99,
- `staffing.min_individual_productivity` must be greater than 0 and at most 1,
- `experience_profiles[*].productivity_factor` must be greater than 0,
- `experience_profiles[*].communication_overhead` must be between 0 and 1.

8.29.8 Recommended authoring style for configuration files

- override only the values you need,
- keep symbolic estimate mappings consistent with your team's estimation conventions,
- set `random_seed` only when you want reproducible runs,
- use `effort_percentile` when staffing recommendations should be conservative,
- add custom experience profiles only when they correspond to real planning scenarios.

8.30 Validation summary

The current implementation validates the following rules directly:

- file extension must be `.yaml`, `.yml`, or `.toml`,
- `project` must be present,
- `tasks` must be present,
- there must be at least one task,
- task IDs must be unique,
- all dependencies must point to existing tasks,
- task dependencies must not be circular,
- `start_date` must parse as an ISO date,
- probability thresholds must be in range and ordered correctly,
- triangular estimates must satisfy `low <= expected <= high`,
- log-normal estimates must include `low`, `expected`, and `high`, and must satisfy `low < expected < high`,
- risks must have probabilities in `0.0..1.0`,
- structured risk impacts must use positive values.

8.31 Notes on undocumented keys

This reference documents the supported keys that are explicitly modeled by the current source code.

Keys outside these structures are not part of the formal project-file reference. In the current implementation, undeclared keys are not a reliable extension mechanism and should not be used to represent important semantics unless and until the source model explicitly supports them.

8.32 Recommended authoring style

Although multiple forms are accepted, the clearest project files usually follow these practices:

- keep the `project` section concise and metadata-focused,
- use triangular estimates unless log-normal or T-shirt sizing is clearly justified,
- use task IDs that are stable and machine-friendly,
- keep dependency lists explicit,
- use only the recognized uncertainty-factor names,
- use structured risk impacts when you need `percentage` or explicit units,
- treat `resources` and `calendars` as advanced sections whose detailed internal schema may evolve.

If you want to see the reference syntax used in practice, compare this chapter with `examples/sample_project.yaml`, `examples/tshirt_walkthrough_project.yaml`, `examples/story_points_walkthrough_project.yaml`, and `examples/project_wi`

CHAPTER 9

Sprint planning

Sprint planning adds a second forecasting layer on top of the normal duration simulation.

It is optional.

If your project file does not include a `sprint_planning` section, or if `sprint_planning.enabled` is `false`, `mcprojsim simulate` runs only the standard duration forecast.

The standard simulation answers:

- How long will the project probably take in calendar time and effort?

Sprint planning answers:

- How many sprints will it probably take to finish the current backlog?
- What is a conservative commitment level for the next sprint?
- How much spillover, scope churn, and sprint-to-sprint disruption should we expect?

This chapter introduces sprint planning gradually:

1. what it models
2. what input it needs
3. the smallest working file
4. richer fields for more realistic forecasting
5. how to generate these files from natural language
6. how external CSV and JSON history files work
7. how to interpret the output

9.1 What sprint planning models

Sprint planning is built on real historical sprint outcomes. Instead of asking you to guess an abstract team velocity, MCProjSim samples from past sprint results and projects those patterns forward.

The model works with one of two capacity units:

- `story_points`: forecast completion based on historical delivered story points
- `tasks`: forecast completion based on historical completed task counts

Which capacity mode to choose: Use `story_points` when your backlog items have explicit story-point estimates and your team tracks points. It produces a more stable forecast when tasks vary in size. Use `tasks` when your backlog items are roughly comparable in scope (e.g., a bug-fix queue) and you want a simple throughput count. If tasks vary widely in size, `story_points` will generally produce a more reliable forecast than `tasks`.

From your sprint history it models these signals:

- completed work: how many story points or tasks were actually delivered

- spillover: work that was started but carried into a later sprint
- scope added: new work introduced during a sprint
- scope removed: work taken out during a sprint
- holiday factor: reduced effective capacity for a historical sprint
- future overrides: explicit planned reductions or increases for known upcoming sprints
- volatility: random sprint-level disruption, such as support load or incident response

Optional task-level spillover modeling can also simulate the case where a pulled item is only partially finished and returns as carryover into the next sprint.

9.2 What input is required

To run sprint planning you need three things:

1. A normal project file with `project` and `tasks`
2. A `sprint_planning` section with a sprint length and capacity mode
3. At least two usable historical sprint observations with positive delivery signal

There is no separate sprint-planning CLI command. Sprint planning is activated during `mcprojsim simulate` when the project file contains `sprint_planning.enabled: true`.

You can define company-wide sprint defaults in `config.yaml` under `sprint_defaults` (for example sickness probabilities, velocity model, and planning confidence). When a project relies on built-in sprint defaults, `mcprojsim` applies values from `sprint_defaults`.

9.3 Sprint setting precedence

When multiple sources define sprint-planning behavior, `mcprojsim` resolves values in this order (highest to lowest priority):

1. CLI flags
2. Project `sprint_planning` fields
3. Global `sprint_defaults` in `config.yaml`
4. Built-in defaults

Examples:

- `--velocity-model` overrides both project and config values
- a value set in project `sprint_planning` overrides `sprint_defaults`
- `sprint_defaults` provides company-wide defaults when project fields remain at built-in values

Practical detail: sprint defaults are applied when a project value still equals the built-in default. In other words, this behavior is value-based rather than strictly "field omitted"-based.

A historical row is usable when all of the following are true:

- it uses the correct unit family for the selected `capacity_mode`
- it includes exactly one delivery field: `completed_story_points` or `completed_tasks`
- its effective delivery signal is positive

In plain language, positive delivery signal means the sprint delivered or carried some work. The implementation counts this as:

- `completed_story_points + spillover_story_points > 0` in `story_points` mode
- `completed_tasks + spillover_tasks > 0` in `tasks` mode

There are a few important rules:

- In `story_points` mode, every task must have a resolvable story-point size. You can provide it either as `estimate.story_points` or as `planning_story_points`.
- In `tasks` mode, the backlog is forecast in item counts instead of points. This works best when tasks are roughly similar in size.
- If sprint spillover modeling is enabled, every task must have a resolvable story-point size, even if capacity

mode is not `story_points`.

9.4 Command workflow

Sprint planning uses the same CLI flow as the rest of the tool:

```
mcprojsim validate project.yaml
mcprojsim simulate project.yaml --seed 42 --iterations 10000
```

The important detail is that `simulate` is still the command that produces sprint-planning results. There is no separate `mcprojsim sprint-plan` command.

Typical workflow:

1. write the project YAML or TOML by hand, or generate it from natural language
2. validate it
3. run the simulation
4. inspect both the normal duration forecast and the sprint-planning summary

For example:

```
mcprojsim validate sprint_project.yaml
mcprojsim simulate sprint_project.yaml --seed 42 --iterations 10000
mcprojsim simulate sprint_project.yaml --seed 42 --iterations 10000 --table
mcprojsim simulate sprint_project.yaml --seed 42 --iterations 10000 --velocity-model neg_binomial
```

Sprint-planning flags that matter most in practice:

- `--velocity-model empirical|neg_binomial` overrides the project file velocity model for the current run
- `--no-sickness` disables sprint sickness modeling without editing the project file
- `--minimal` keeps the CLI output short and suppresses the detailed diagnostic sections
- `--table` formats the sprint summary and confidence intervals as ASCII tables
- `--output-format json,html,csv` writes export files in addition to CLI output
- `--include-historic-base` adds the historic baseline section to HTML and JSON exports when sprint history is available

Example export workflow:

```
mcprojsim simulate sprint_project.yaml \
  --seed 42 \
  --output out/sprint_forecast \
  --output-format json,html \
  --include-historic-base
```

Important export rule:

- `--include-historic-base` only works when `--output-format` includes `json` or `html`
- unsupported `--output-format` values now fail fast instead of being ignored

See Running Simulations for the full command reference.

9.5 Build up the file gradually

9.5.1 Step 1: start from a normal project file

Sprint planning does not replace the normal project definition. It extends it.

```
project:
  name: "Website refresh"
  start_date: "2026-05-04"

tasks:
  - id: "task_001"
    name: "Foundation"
    estimate:
      story_points: 3
  - id: "task_002"
    name: "Backend API"
    estimate:
      story_points: 5
  - id: "task_003"
    name: "Frontend"
    estimate:
      story_points: 8
```

At this point the file is valid for normal Monte Carlo simulation, but it does not yet contain any sprint-planning forecast.

That is the default and optional case: a standard project file without `sprint_planning` is still fully valid.

9.5.2 Step 2: add the smallest useful `sprint_planning` section

Now add the sprint cadence, the unit family, and at least two historical sprint rows.

```
project:
  name: "Website refresh"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Foundation"
    estimate:
      story_points: 3
  - id: "task_002"
    name: "Backend API"
    estimate:
      story_points: 5
  - id: "task_003"
    name: "Frontend"
    estimate:
      story_points: 8
  - id: "task_004"
    name: "Integration"
    estimate:
      story_points: 5
  - id: "task_005"
    name: "Release prep"
    estimate:
      story_points: 3
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 10
    - sprint_id: "SPR-002"
      completed_story_points: 9
```

This is the smallest practical setup:

- `enabled: true` turns sprint planning on
- `sprint_length_weeks: 2` says one sprint is two weeks long
- `capacity_mode: story_points` means backlog is measured in story points
- `history` supplies the real sprint outcomes the model will resample

9.5.3 Step 3: add richer historical signals

The next improvement is to capture spillover and scope churn explicitly.

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  planning_confidence_level: 0.8
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 10
      spillover_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 9
      spillover_story_points: 2
      added_story_points: 1
    - sprint_id: "SPR-003"
      completed_story_points: 11
      spillover_story_points: 1
      removed_story_points: 1
```

This gives the simulation more behavioral information:

- `spillover_story_points` records started-but-not-finished work
- `added_story_points` records scope growth during a sprint
- `removed_story_points` records scope removed during a sprint
- `planning_confidence_level` controls how conservative the guidance should be

9.5.4 Step 4: add forward-looking adjustments

Once the basic history is working, you can model upcoming known events and optional spillover/disruption behavior.

```
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  planning_confidence_level: 0.85
  removed_work_treatment: reduce_backlog
  future_sprint_overrides:
    - sprint_number: 2
      capacity_multiplier: 0.7
      notes: "Shared release sprint"
    - start_date: "2026-06-29"
      holiday_factor: 0.8
      notes: "Summer vacation period"
  volatility_overlay:
    enabled: true
    disruption_probability: 0.25
    disruption_multiplier_low: 0.6
    disruption_multiplier_expected: 0.8
    disruption_multiplier_high: 1.0
  spillover:
    enabled: true
    model: logistic
    size_reference_points: 5
    consumed_fraction_alpha: 3.25
    consumed_fraction_beta: 1.75
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 12
      spillover_story_points: 1
      added_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 10
      spillover_story_points: 3
      added_story_points: 2
      removed_story_points: 1
      holiday_factor: 0.9
    - sprint_id: "SPR-003"
      completed_story_points: 11
      spillover_story_points: 2
    - sprint_id: "SPR-004"
      completed_story_points: 13
      spillover_story_points: 1
      removed_story_points: 1
```

At this stage the forecast can answer more realistic planning questions:

- what if sprint 2 will have reduced capacity?
- what if summer vacations affect a later sprint?
- what if some sprints are randomly disrupted?
- what if larger items often spill over?

9.6 Built on historical real data

Sprint planning is intentionally empirical.

The model does not require you to declare a fixed team velocity like “10 points per sprint”. Instead, it uses your actual sprint history and samples from it.

Each historical row describes what really happened in one sprint:

- delivered work
- spillover
- scope change

- optional holiday reduction

That means the forecast learns from the behavior of the team and the backlog as they were actually executed.

If your team historically delivers 9 to 12 points with occasional spillover and moderate churn, the model will project a future distribution consistent with those observations.

If the team changes substantially, the input history should change too. The forecast is only as representative as the data you feed it.

9.7 All sprint-planning fields

This section is a full field reference for sprint planning and the task fields it depends on.

9.7.1 Task-level fields used by sprint planning

Field	Location	Default	Explanation
`estimate.story_points`	`tasks[].estimate`	none	Story-point size used directly for story-point planning when present
`planning_story_points`	`tasks[]`	omitted	Optional explicit planning size. Overrides `estimate.story_points` for sprint planning only
`priority`	`tasks[]`	omitted	Lower values are pulled earlier when several tasks are ready in the same sprint
`spillover_probability_override`	`tasks[]`	omitted	Per-task override for spillover probability when spillover modeling is enabled

9.7.2 `sprint_planning` fields

Field	Default if omitted	Explanation
`enabled`	`false`	Enables sprint-planning simulation and reporting
`sprint_length_weeks`	none	Required sprint cadence in weeks
`capacity_mode`	none	Required. Either `story_points` or `tasks`
`history`	`[]`	Historical sprint observations. If `enabled` is true, at least two usable rows are required
`planning_confidence_level`	`0.80`	Confidence level used for commitment guidance and conservative heuristics
`removed_work_treatment`	`churn_only`	Whether removed work is tracked only as churn or also reduces remaining backlog
`future_sprint_overrides`	`[]`	Explicit adjustments for specific future sprints

Field	Default if omitted	Explanation
`volatility_overlay`	default object	Optional random disruption model applied on top of sampled historical capacity
`spillover`	default object	Optional task-level spillover model
`velocity_model`	`empirical`	How future sprint velocity is sampled. Either `empirical` (resample historical rows) or `neg_binomial` (fit a Negative Binomial distribution to history)
`sickness`	default object	Optional per-person sickness-absence model that reduces sprint capacity stochastically

9.7.3 `sprint_planning.history[]` fields

Field	Default if omitted	Explanation
`sprint_id`	none	Required unique sprint identifier
`sprint_length_weeks`	inherited from `sprint_planning.sprint_length_weeks`	Historical sprint length. Useful when your old data mixes cadences
`completed_story_points`	none	Required in `story_points` mode
`completed_tasks`	none	Required in `tasks` mode
`spillover_story_points`	`0`	Carryover in story points
`spillover_tasks`	`0`	Carryover in tasks
`added_story_points`	`0`	Scope added during the sprint
`added_tasks`	`0`	Scope added during the sprint
`removed_story_points`	`0`	Scope removed during the sprint
`removed_tasks`	`0`	Scope removed during the sprint
`holiday_factor`	`1.0`	Historical effective-capacity multiplier. Use values below `1.0` for reduced-capacity sprints
`end_date`	omitted	Optional metadata only
`team_size`	omitted	Optional metadata only
`notes`	omitted	Optional metadata only

9.7.4 External `sprint_planning.history` descriptor fields

When you keep sprint history in a separate file, `history` is not a list of rows in the project file. Instead it becomes a small descriptor object with these fields:

Field	Default if omitted	Explanation
`format`	none	Required external history format. Supported values are `json` and `csv`

Field	Default if omitted	Explanation
`path`	none	Required path to the external history file. Relative paths are resolved from the project file location

9.7.5 `future_sprint_overrides[]` fields

Field	Default if omitted	Explanation
`sprint_number`	omitted	Optional future sprint locator by number
`start_date`	omitted	Optional future sprint locator by boundary date
`holiday_factor`	`1.0`	Capacity multiplier for that future sprint
`capacity_multiplier`	`1.0`	Additional explicit multiplier for that future sprint
`notes`	omitted	Optional descriptive text

Notes:

- At least one of `sprint_number` or `start_date` must be present.
- If both are present, they must point to the same future sprint.
- `start_date` must align to a sprint boundary derived from `project.start_date` and `sprint_length_weeks`.
- The effective multiplier for a future sprint is `holiday_factor × capacity_multiplier`. Both are combined multiplicatively.

Validation rules:

- `future_sprint_overrides` must be a list of objects
- each override must resolve to exactly one simulated future sprint
- two overrides must not target the same sprint after resolution
- `sprint_number` must be a positive integer
- `start_date` must be a valid ISO date in `YYYY-MM-DD` format
- `holiday_factor` and `capacity_multiplier` must both be greater than `0`
- use `holiday_factor` for known calendar reduction, `capacity_multiplier` for any additional explicit adjustment, or combine both when needed

9.7.6 `volatility_overlay` fields

Field	Default if omitted	Explanation
`enabled`	`false`	Enables sprint-level disruption sampling
`disruption_probability`	`0.0`	Probability that a given sprint is disrupted
`disruption_multiplier_low`	`1.0`	Lower bound of the disruption multiplier
`disruption_multiplier_expected`	`1.0`	Most likely disruption multiplier
`disruption_multiplier_high`	`1.0`	Upper bound of the disruption multiplier

The multipliers must satisfy `low <= expected <= high`.

9.7.7 `spillover` fields

Field	Default if omitted	Explanation
`enabled`	`false`	Enables task-level spillover simulation
`model`	`table`	Spillover probability model. Either `table` or `logistic`
`size_reference_points`	`5.0`	Reference point scale used by the logistic model
`size_brackets`	built-in defaults	Table-model mapping from item size to spillover probability
`consumed_fraction_alpha`	`3.25`	Alpha parameter of the beta distribution for consumed fraction
`consumed_fraction_beta`	`1.75`	Beta parameter of the beta distribution for consumed fraction
`logistic_slope`	`1.9`	Logistic model slope
`logistic_intercept`	`-1.9924301646902063`	Logistic model intercept

Task-level spillover modeling in depth

Background

In any agile sprint, a story or task pulled into the sprint may not be fully finished by sprint end. The team ran out of time, the item turned out to be bigger than estimated, or an unexpected blocker appeared mid-sprint. The unfinished work carries into the next sprint. This is called *spillover* or *carryover*.

This is distinct from the sprint-level spillover already captured in `sprint_planning.history[]`. Those historical `spillover_story_points` rows describe the aggregate carryover the team observed after each historical sprint. The task-level spillover model described here operates at a different granularity: it simulates what happens to *each individual backlog item* during a future simulated sprint.

What problem does task-level spillover solve?

A simulation without task-level spillover treats each sprint's pulled work as binary: an item either fits into the sprint's capacity and completes, or it doesn't fit and stays in the backlog. This misses a common real-world pattern: a 13-point item pulled into a sprint might be 70 % complete by sprint end, leaving only 4 points of work as carryover. Without this model, that item either disappears cleanly or never starts.

Enabling task-level spillover produces more realistic forecast distributions because it:

- prolongs the forecast tail when many large items are in the backlog — large items fragment across sprints more often
- generates partial-completion carryover items that inflate future sprint loads
- creates a correlation between item size, sprint fragmentation, and overall sprint count that a pure velocity model cannot capture

How the two-stage simulation works

When an item is pulled into a simulated sprint, the engine applies a two-stage process:

1. **Spillover trial.** The engine computes a spillover probability $p(x)$ for the item based on its planning story points x. It then draws a Bernoulli sample: with probability $p(x)$ the item spills, otherwise it completes normally.
2. **Consumed fraction.** If the item spills, the engine draws a consumed fraction f from a Beta(α, β) distribution (clamped to $(10^{-6}, 0.999999)$). The sprint records $x \cdot f$ as completed work and creates a new backlog item of size $x \cdot (1 - f)$ to carry into a later sprint.

The `consumed_fraction_alpha` and `consumed_fraction_beta` parameters control the shape of this fraction. The

default values ($\alpha = 3.25$, $\beta = 1.75$) produce a mean consumed fraction of about 65 %, meaning the team typically finishes roughly two-thirds of a spilling item before the sprint ends.

Choosing between `table` and `logistic`

The `table` model assigns a flat spillover probability to each size band. It is easy to reason about: a 6-point item and an 8-point item receive exactly the same probability because they share the same bracket. This makes calibration intuitive but creates step discontinuities at bracket boundaries.

The `logistic` model replaces those steps with a smooth, continuously increasing sigmoid curve. Every distinct story-point value gets a distinct probability. It is better suited when:

- the backlog contains items with fine-grained story-point estimates (e.g., a Fibonacci scale of 1, 2, 3, 5, 8, 13)
- you want the forecast to be sensitive to precise size differences rather than coarse buckets
- you have retrospective data that tells you the spillover rate at a representative item size (the reference point)

For a new team without calibration data, the `table` model is simpler to start with. The default `table` brackets (5 %, 12 %, 25 %, 40 %) reflect typical observed spillover rates across size categories.

How the logistic model computes probability

The logistic model maps each item's planning story points x to a probability $p(x)$ through a sigmoid function applied in log-space:

$$p(x) = \frac{1}{1 + e^{-\left(s \cdot \ln\left(\frac{x}{r}\right) + b\right)}}$$

Three parameters jointly define the curve:

Parameter	Field	Role
r	`size_reference_points`	The anchor size. The curve passes through `sigmoid(b)` at $x = r$
b	`logistic_intercept`	Sets the probability at the reference size. Calibrate with $b = \ln(p_0/(1 - p_0))$
s	`logistic_slope`	Controls how steeply probability grows as size increases above (or below) the reference

The slope and intercept are independent controls. Changing the intercept shifts the entire curve up or down while preserving its shape. Changing the slope stretches or compresses the curve without moving the probability at the reference size. This separation makes calibration predictable: if your retrospectives tell you that 5-point items spill 12 % of the time, set the intercept to $\ln(0.12/0.88) \approx -1.992$ and use the reference size of 5. The slope then governs only how fast risk grows for items above 5 points.

Extended example: comparing `table` and `logistic` on a Fibonacci backlog

Consider a team building a SaaS backend. Their backlog uses a Fibonacci story-point scale: 1, 2, 3, 5, 8, 13. After several retrospectives they have observed:

- 3-point items almost never spill (~5 % of the time)
- 5-point items spill occasionally (~12 %)
- 8-point items spill frequently (~25 %)

These observations match the default `table` brackets almost exactly. However, the team also estimates many items as 4, 6, or 7 points and notices that the `table` model treats them identically to the bracket ceiling: a 4-point item gets the same 12 % probability as a 5-point item, and a 6-point item gets the same 25 % probability as an 8-point item.

Switching to `logistic` with the default parameters gives item-level differentiation without changing the cali-

bration at the anchor sizes:

```
spillover:
  enabled: true
  model: logistic
  size_reference_points: 5.0       # 5-point item is the anchor
  logistic_intercept: -1.9924      # 12 % at the reference size
  logistic_slope: 1.9              # default steepness
  consumed_fraction_alpha: 3.25
  consumed_fraction_beta: 1.75
```

Resulting probabilities compared to the `table` model:

Item size (pts)	`table` model	`logistic` (defaults)	Note
1	5 %	1 %	logistic treats very small items as nearly safe
2	5 %	2 %	logistic differentiates within the 2 bracket
3	12 %	5 %	logistic separates 3-pt from 5-pt; table treats them equally
4	12 %	8 %	logistic differentiates 4-pt from 5-pt
5 (reference)	**12 %**	**12 %**	both models agree
6	25 %	16 %	logistic separates 6-pt from 8-pt; table treats them equally
7	25 %	21 %	logistic differentiates 7-pt from 8-pt
8	**25 %**	**25 %**	both models agree at the 8-pt bracket boundary
10	40 %	34 %	logistic gives a smooth curve above the highest bracket
13	40 %	46 %	logistic continues rising; table stays flat at 40 %

Key observations:

- The two models agree exactly at the reference size (5 pts) and at the 8-pt bracket boundary, because the default logistic parameters were calibrated to match those points.
- For items smaller than the reference, the logistic curve descends rapidly. A 1-point item receives only 1 % spillover probability versus 5 % in the `table` model.
- For items larger than the highest `table` bracket (8 pts), the logistic model continues increasing past 40 %, while the `table` model plateaus. Whether a 13-pt item should be assigned 40 % or 46 % is a judgment call — if your team's largest items spill more than 40 % of the time, `logistic` is the more honest representation.
- The choice of model has the greatest impact on forecasts when the backlog contains a mix of very small (1–2 pt) and very large (10–13 pt) items. For homogeneous backlogs where most items cluster near the reference size, the two models produce similar results.

The `logistic_slope` used in this example (1.9) is the default. It is *not derived from* the 12 % observation at 5 points — it is an independent assumption about the steepness of the size-to-risk relationship. See the parameter reference section below for guidance on fitting the slope from multi-size retrospective data.

Understanding `logistic_slope` and `logistic_intercept`

These two parameters define the shape of the continuous sigmoid curve that maps item size to spillover probability when `model: logistic` is selected. Together they answer two independent questions: *how likely is a reference-size item to spill over?* and *how fast does that probability grow as items get larger?*

The complete formula (also given in Item spillover in the statistical methods section) is:

$$p(x) = \frac{1}{1 + e^{-\left(s \cdot \ln\left(\frac{x}{r}\right) + b\right)}}$$

where x is the item's planning story points, r is `size_reference_points`, s is `logistic_slope`, and b is `logistic_intercept`.

`logistic_intercept` controls the probability at the reference size.

When $x = r$, the $\ln(x/r)$ term equals zero and the formula reduces to:

$$p(r) = \frac{1}{1 + e^{-b}} = \text{sigmoid}(b)$$

So `logistic_intercept` is the logit of your desired baseline spillover probability. To target a specific probability p_0 at the reference size, compute:

$$b = \ln\left(\frac{p_0}{1 - p_0}\right)$$

Calibration reference:

Desired p at reference size	`logistic_intercept` value
5 %	−2.944
10 %	−2.197
12 % (default)	**−1.992**
15 %	−1.735
20 %	−1.386
30 %	−0.847

The default −1.9924 produces exactly 12 % at the reference size of 5 points, which matches the built-in `table` bracket for items up to 5 points.

`logistic_slope` controls how steeply probability grows with item size.

Because the formula operates in log-space ($\ln(x/r)$), the slope governs a power-law-like sensitivity. A higher slope makes probability rise sharply as items grow beyond the reference size; a lower slope produces a more gradual curve. The probability at the reference size ($x = r$) is always `sigmoid(logistic_intercept)` regardless of slope, because the ln term vanishes there.

Comparison with `size_reference_points = 5` and the default `logistic_intercept = -1.9924`:

Item size (pts)	`logistic_slope = 1.0`	`logistic_slope = 1.9` (default)	`logistic_slope = 3.0`
1	3 %	1 %	<1 %
2	5 %	2 %	1 %
5 (reference)	**12 %**	**12 %**	**12 %**
8	18 %	25 %	36 %

Item size (pts)	`logistic_slope = 1.0`	`logistic_slope = 1.9` (default)	`logistic_slope = 3.0`
13	26 %	46 %	71 %

All three curves pass through 12 % at 5 points because `logistic_intercept` is the same. The curves diverge only as items move away from the reference.

How the defaults relate to the built-in `table` breakpoints.

The default parameters ($s = 1.9$, $b = -1.9924$, $r = 5$) were chosen so the logistic curve closely matches the four built-in `table` bracket probabilities:

Size (pts)	Built-in `table`	Logistic (defaults)
2	5 %	2 %
5	12 %	12 %
8	25 %	25 %
13	—	46 %

The `table` model applies a flat probability across each size band. The logistic model produces a smooth monotone curve — use it when you want item-level precision rather than step-wise jumps between brackets.

Practical example: team with higher baseline spillover.

Suppose your retrospective data shows that 5-point items spill about 20 % of the time. That single data point lets you set `logistic_intercept` precisely — but it tells you nothing about `logistic_slope`. The slope is an independent assumption about how steeply spillover risk grows with item size. In this example `logistic_slope` is kept at its default of 1.9, which means the 38 % and 61 % figures below are consequences of that slope choice, not of the 20 % observation.

If you have size-stratified retrospective data — for example, you know that 8-point items spilled 35 % of the time — you can fit the slope directly by solving the logistic equation at two (size, probability) pairs. Otherwise, keep the default slope and treat probabilities for sizes far from the reference as order-of-magnitude estimates rather than precise predictions.

Set `logistic_intercept` to `logit(0.20) = -1.386` and keep `logistic_slope` at its default:

```
spillover:
  enabled: true
  model: logistic
  size_reference_points: 5.0
  logistic_intercept: -1.386    # sigmoid(-1.386) ≈ 20 % at x = r
  logistic_slope: 1.9           # default; not derived from the 20 % figure
```

With these settings:

Item size (pts)	Spillover probability	Derivation
2	4 %	$1.9 \cdot \ln(0.4) - 1.386 \Rightarrow p \approx 0.04$
5 (reference)	**20 %**	anchored by `logistic_intercept`
8	38 %	$1.9 \cdot \ln(1.6) - 1.386 \Rightarrow p \approx 0.38$
13	61 %	$1.9 \cdot \ln(2.6) - 1.386 \Rightarrow p \approx 0.61$ — slope assumption

To verify a calibration before using it in production, compute $p(x)$ for a few representative item sizes by substituting your values of s, b, and r into the formula above.

9.7.8 `spillover.size_brackets[]` fields

Field	Default if omitted	Explanation
`max_points`	omitted	Inclusive upper bound of the bracket. Omit on the last row for the unbounded catch-all bracket
`probability`	none	Spillover probability for items in that bracket

Built-in default brackets:

`max_points`	`probability`
`2.0`	`0.05`
`5.0`	`0.12`
`8.0`	`0.25`
omitted	`0.40`

9.7.9 `sickness` fields

Field	Default if omitted	Explanation
`enabled`	`false`	Enable per-person sickness absence modeling
`team_size`	project `team_size`	Number of team members. Falls back to the top-level project `team_size` if omitted
`probability_per_person_per_week`	`0.058`	Probability that any one person starts a sickness episode in a given week
`duration_log_mu`	`0.693`	μ of the LogNormal duration distribution (log-scale). Default gives a median of 2 days. **Shared with constrained scheduling.**
`duration_log_sigma`	`0.75`	σ of the LogNormal duration distribution (log-scale). Default gives mean 2.6 days, P90 5.5 days. **Shared with constrained scheduling.**

9.8 How the simulation uses these fields

Once the input is loaded, the simulation uses the fields in this order:

1. normalize historical rows
2. choose the correct unit family (`story_points` or `tasks`)
3. sample future sprint outcomes from history
4. apply volatility, future overrides, and sickness multiplier
5. plan ready backlog items into each sprint
6. optionally create spillover remainder items
7. repeat until the backlog is done

More concretely:

- `holiday_factor` normalizes historical completed and spillover work to an equivalent full-capacity sprint before resampling
- `future_sprint_overrides` scale future sampled capacity for specific sprints
- when sickness modeling is enabled, a per-sprint sickness multiplier reduces capacity based on simulated team absences (duration parameters are shared with constrained scheduling, if both are used)
- `removed_work_treatment: churn_only` records removed work as churn but does not shrink the remaining backlog
- `removed_work_treatment: reduce_backlog` subtracts removed work from the remaining backlog
- `priority` affects which ready tasks are pulled first
- `spillover_probability_override` replaces the configured spillover model for one task

If `sprint_planning` is absent or disabled, none of these sprint-specific steps run and the tool falls back to the standard project duration simulation only.

9.9 Statistical methods in detail

The sprint-planning forecast uses a small number of explicit statistical building blocks. Some quantities are sampled from named probability distributions. Others are sampled directly from the empirical historical data without fitting a parametric family.

The important distinction is:

- historical sprint capacity is modeled empirically from your past data
- disruption and spillover behavior use explicit probability distributions or deterministic probability formulas
- the final sprint-count forecast is the Monte Carlo output distribution produced by repeatedly simulating the backlog to completion

9.9.1 Step 1: normalize the historical data

Before sampling, each sprint-history row is converted into a canonical internal row:

- `completed_units`
- `spillover_units`
- `added_units`
- `removed_units`
- `sprint_length_weeks`

The chosen unit family depends on `capacity_mode`:

- in `story_points` mode, the row uses the story-point fields
- in `tasks` mode, the row uses the task-count fields

Historical `holiday_factor` is then used to normalize delivered and spillover work to an equivalent full-capacity sprint:

$$\text{normalized completed} = \frac{\text{completed}}{\text{holiday factor}}$$

$$\text{normalized spillover} = \frac{\text{spillover}}{\text{holiday factor}}$$

This means a sprint with reduced availability is not treated as evidence that the team permanently has lower underlying capacity.

9.9.2 Step 2: build the sampling distribution from history

The default velocity model is `empirical`.

For sprint capacity itself, the default implementation does not fit a Normal, Lognormal, Gamma, or any other named parametric distribution.

Instead, it uses an empirical discrete distribution over the normalized historical rows.

In practice that means:

- every usable historical row is one possible future sprint outcome
- each row has equal probability mass
- one row is sampled uniformly at random with replacement for each simulated sprint

So if the normalized completed capacities in your history are `[8, 10, 12]`, the model for a future sprint's nominal velocity is an empirical discrete distribution supported exactly on those values, each with probability 1/3.

This approach has two important consequences:

- no distribution parameters such as mean/variance are estimated and then fed into a fitted curve for sprint velocity
- the observed joint behavior inside a historical row is preserved

That second point matters. When a row is sampled, its `completed_units`, `spillover_units`, `added_units`, and `removed_units` travel together. If historically high-delivery sprints also tended to have low spillover, that empirical relationship remains present in the sampled rows.

9.9.3 Step 2b: Negative Binomial velocity model (optional)

When `velocity_model` is set to `neg_binomial`, the engine fits a Negative Binomial distribution to the normalized completed-units history using method-of-moments estimation.

The Negative Binomial (NB) provides a smooth parametric model for count-like sprint velocity data that naturally accounts for overdispersion — situations where the variance of historical velocity exceeds the mean.

Parameter estimation:

The engine computes the sample mean μ and sample variance s^2 of the normalized completed-units from the diagnostic rows, then derives:

$$k = \frac{\mu^2}{s^2 - \mu}$$

where k is the dispersion parameter. If $s^2 \leq \mu$ (no overdispersion), k is treated as infinite and the distribution falls back to a Poisson model.

Sampling:

For each simulated sprint, the engine draws a completed-units value from:

$$\text{completed} \sim \text{NegBinomial}\left(k,\ \frac{k}{k+\mu}\right)$$

This distribution has mean μ and variance $\mu + \mu^2/k$.

When k is infinite (Poisson fallback), the draw is from Poisson(μ) instead.

Spillover, added scope, and removed scope are still sampled empirically from a random historical row for each sprint. That means the NB model changes only how velocity is generated — churn behavior continues to reflect observed historical patterns.

When to use each model:

- Use `empirical` (the default) when you have moderate history and want the forecast to reflect exact observed delivery patterns, including correlations between velocity and churn.
- Use `neg_binomial` when you want a smooth parametric velocity model, for example when you have enough history that the observed values look overdispersed and you want the simulation to interpolate between observed outcomes rather than only replaying them.

Example:

```yaml
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  velocity_model: neg_binomial
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 10
      spillover_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 9
      spillover_story_points: 2
      added_story_points: 1
    - sprint_id: "SPR-003"
      completed_story_points: 14
      spillover_story_points: 1
    - sprint_id: "SPR-004"
      completed_story_points: 8
      spillover_story_points: 3
    - sprint_id: "SPR-005"
      completed_story_points: 12
```

The CLI flag `--velocity-model neg_binomial` can also override the project file setting without editing the YAML:

```
mcprojsim simulate sprint_project.yaml --seed 42 --velocity-model neg_binomial
```

The output summary will show the fitted NB parameters:

```
Velocity Model: neg_binomial
NB mu: 10.6000
NB dispersion k: 18.7333
```

9.9.4 Step 3: cadence matching and weekly fallback

The engine first tries to sample from historical rows whose `sprint_length_weeks` exactly match the requested future cadence.

That is the normal case.

If no rows match the requested cadence, the engine falls back to weekly normalization:

1. each historical row is converted into a one-week rate by dividing by its sprint length
2. the simulation samples one weekly row per future week
3. those weekly samples are summed to form the future sprint outcome

Statistically, that fallback is a sum of independent draws from the empirical weekly-rate distribution. It is still empirical, but now at the weekly level rather than at the full-sprint level.

9.9.5 Which distributions are used for which quantities

The table below lists the actual modeling family used by each entity.

Quantity	Source	Distribution / model used
Nominal sprint velocity (`completed_units`)	historical rows	Empirical discrete distribution over normalized historical rows (default), or Negative Binomial distribution fitted by method of moments when `velocity_model: neg_binomial`

Quantity	Source	Distribution / model used
Historical spillover units in sampled sprint outcome	historical rows	Empirical discrete distribution, sampled jointly with the same historical row as velocity (empirical model) or from a random historical row (NB model)
Historical scope added units	historical rows	Empirical discrete distribution, sampled jointly with the same historical row
Historical scope removed units	historical rows	Empirical discrete distribution, sampled jointly with the same historical row
Future override effect	`future_sprint_overrides`	Deterministic multiplier, no random distribution
Whether a sprint disruption occurs	`volatility_overlay.disruption_probability`	Bernoulli trial with probability p
Disruption multiplier given a disruption	`volatility_overlay` multipliers	Triangular distribution
Number of people falling sick per week	`sickness.probability_per_person_per_week`, `sickness.team_size`	Binomial(n, p) per sprint week where n = team size, p = weekly probability
Days lost per sickness event	`sickness.duration_log_mu`, `sickness.duration_log_sigma`	LogNormal(μ, σ) capped at total sprint working days (`sprint_length_weeks × 5`)
Spillover probability in `table` mode	`spillover.size_brackets`	Deterministic step function by story-point size
Spillover probability in `logistic` mode	`spillover` logistic fields	Logistic probability curve over story-point size
Whether a pulled item spills over	spillover probability	Bernoulli trial with item-specific probability p_i
Fraction of a spilled item completed before carryover	`consumed_fraction_alpha`, `consumed_fraction_beta`	Beta distribution
Final sprint-count forecast	full Monte Carlo simulation	Empirical Monte Carlo output distribution over iterations

9.9.6 Detailed interpretation by entity

Sprint velocity

For a particular future sprint, nominal velocity is the sampled `completed_units` from one historical row (empirical model) or drawn from the fitted Negative Binomial distribution (NB model).

- In `story_points` mode, this is a story-point velocity.
- In `tasks` mode, this is a throughput count.

With the default `empirical` model, this is a discrete distribution based on the normalized history. With the `neg_binomial` model, the distribution is parametric with mean and dispersion estimated from history.

If volatility, future overrides, or sickness modeling are enabled, the effective velocity becomes:

$$\text{effective velocity} = \text{nominal velocity} \times \text{volatility multiplier} \times \text{future override multiplier} \times \text{sickness multiplier}$$

Spillover, added scope, and removed scope at the sprint level

In the empirical model, these are sampled from the same historical row as velocity, not independently from separate fitted distributions. That preserves empirical row-level dependence between these quantities.

In the `neg_binomial` model, velocity is drawn from the fitted NB distribution. Spillover, added scope, and removed scope are then drawn from a randomly selected historical row, independently of the velocity draw. This breaks the joint row-level correlation but produces smooth parametric velocity variation.

Disruption frequency and impact

The volatility overlay is a two-stage model:

1. sample whether disruption happens using a Bernoulli trial with probability `disruption_probability`
2. if disruption occurs, sample the multiplicative impact from a triangular distribution with:

- lower bound = `disruption_multiplier_low`
- mode = `disruption_multiplier_expected`
- upper bound = `disruption_multiplier_high`

If all three multipliers are equal, the impact becomes deterministic.

Sickness absence

The sickness model is a two-stage per-person simulation that runs once per simulated sprint:

1. **Occurrence stage.** For each week in the sprint, draw the number of new sickness events from a Binomial distribution:

$$\text{sick_count}_w \sim \text{Binomial}(n,\, p)$$

   where n is the team size and p is `probability_per_person_per_week`.

2. **Duration stage.** For each sickness event, draw the duration from a LogNormal distribution:

$$d_i \sim \text{LogNormal}(\mu,\, \sigma)$$

   Each draw is capped at the total sprint working days (`sprint_length_weeks × 5`). The aggregate lost-days total is further capped at `team_size × sprint_length_weeks × 5` to ensure the result stays in a valid range.

3. **Capacity multiplier.** Total lost days are summed and converted to a fractional multiplier:

$$\text{sickness multiplier} = \max\left(0,\; 1 - \frac{\sum d_i}{n \times w \times 5}\right)$$

   where w is `sprint_length_weeks` and 5 is the assumed working days per week.

This multiplier is applied multiplicatively alongside the volatility and future-override multipliers.

Default calibration and EU/OECD motivation. The three default parameters are derived from published European and OECD labour-market statistics:

- The average EU worker takes roughly **7–8 sick days per year** (Eurostat, *Absence from work* data, 2019–2023; OECD, *Health at a Glance*, 2023).
- Those days arise from approximately **3 sickness episodes per year** (Eurofound, *Sixth European Working Conditions Survey*, 2015; UK ONS, *Sickness absence in the labour market*, 2024).
- Across EU-27, average sickness-absence rates cluster around **2.5–3.8 %** of scheduled working days (Eurostat, table `hsw_abs_typ`, 2019–2022).

Converting to per-week parameters:

Parameter	Value	Derivation
`probability_per_person_per_week`	0.058	3 episodes/year ÷ 52 weeks ≈ 0.058
`duration_log_mu`	0.693	ln(2), giving a median duration of **2 working days**

Parameter	Value	Derivation
`duration_log_sigma`	0.75	Produces mean 2.6 days, P90 5.5 days, consistent with the right-skewed shape of sickness-duration data

With these defaults and a team of 6 over a 2-week sprint, the expected capacity loss per sprint is roughly **2–4 %**, consistent with the EU-27 aggregate sickness-absence rate.

You can override any of these parameters to match your team's or region's reality.

Example:

```
project:
  name: "Sprint with sickness"
  team_size: 6

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  sickness:
    enabled: true
    # team_size falls back to project.team_size (6) if not set here
  history:
    - sprint_id: "S1"
      completed_story_points: 10
    - sprint_id: "S2"
      completed_story_points: 12
    - sprint_id: "S3"
      completed_story_points: 9
```

To disable sickness from the CLI without editing the file:

```
mcprojsim simulate project.yaml --seed 42 --no-sickness
```

Item spillover

Item spillover is also a two-stage model:

1. compute the spillover probability for the item
2. if spillover occurs, sample the consumed fraction from a Beta distribution

The spillover probability p_i is determined in this order:

1. `spillover_probability_override` on the task, if present
2. otherwise the configured global spillover model

The global model can be:

- `table`: a deterministic lookup based on the item's planning story points
- `logistic`: a logistic curve

Logistic mode uses exactly this sequence:

1. compute scaled points with a lower clamp:

$$z = \max\left(\frac{x}{r}, 10^{-6}\right)$$

2. compute the logit value:

$$\ell = s \cdot \ln(z) + b$$

3. convert to probability:

$$p_i = \frac{1}{1 + e^{-\ell}}$$

where:

- x is the item's planning story points
- r is `size_reference_points`
- s is `logistic_slope`
- b is `logistic_intercept`

The clamp on z is important: it prevents invalid $\ln(0)$ or negative-log inputs when very small values appear.

Interpretation of slope and intercept:

- `logistic_slope` controls sensitivity to item size in log-space. Larger values make probability rise faster as x increases.
- `logistic_intercept` shifts the curve up or down. At the reference size ($x = r$), the probability is:

$$p(x = r) = \frac{1}{1 + e^{-b}}$$

With the built-in defaults ($s = 1.9$, $b = -1.9924301646902063$), this baseline is approximately 0.12.

If spillover occurs, the completed fraction is sampled from:

$$\text{Beta}(\alpha, \beta)$$

with:

- α = `consumed_fraction_alpha`
- β = `consumed_fraction_beta`

Then the sampled fraction is clamped to:

$$\text{consumed fraction} \in [10^{-6}, 0.999999]$$

This guarantees both consumed and remaining work are strictly positive in the spillover branch.

The remaining fraction becomes carryover into the next sprint.

In implementation terms, if an item with size u spills over:

$$u_{\text{consumed}} = u \cdot f$$

$$u_{\text{remaining}} = u \cdot (1 - f)$$

where f is the clamped Beta sample. If $u_{\text{remaining}} \leq 10^{-9}$, no carryover item is created.

How `consumed_fraction_alpha` and `consumed_fraction_beta` affect behavior:

- Mean consumed fraction:

$$\mathbb{E}[f] = \frac{\alpha}{\alpha + \beta}$$

- Variance:

$$\text{Var}(f) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$$

Interpretation:

- higher `consumed_fraction_alpha` shifts spillover events toward "mostly completed before carryover"
- higher `consumed_fraction_beta` shifts spillover events toward "mostly carried over"
- larger values of both parameters (with similar ratio) produce tighter clustering around the mean

With built-in defaults (`alpha=3.25`, `beta=1.75`):

$$\mathbb{E}[f] = \frac{3.25}{3.25 + 1.75} = 0.65$$

So, on average, a spillover event consumes about 65% of an item and carries about 35% into the next sprint (before clamping and tiny-remainder guard).

Spillover itself is a Bernoulli event using the computed probability p_i: if a uniform draw is below p_i, spillover happens; otherwise the item completes normally.

9.9.7 What is estimated from history, and what is configured directly

This is the practical summary:

Estimated from historical data:

- empirical sprint velocity distribution (empirical model), or Negative Binomial mu and dispersion k (NB model)
- empirical spillover-unit distribution at sprint level
- empirical scope-added distribution
- empirical scope-removed distribution
- empirical correlations between those sprint-level quantities, because they are sampled as whole rows (empirical model only; the NB model samples churn independently)

Configured directly by the user rather than estimated from history:

- velocity model choice (`empirical` or `neg_binomial`)
- sickness absence parameters (probability, duration distribution, team size)
- volatility probability and triangular multiplier bounds
- future sprint override multipliers
- spillover probability table or logistic curve parameters
- beta-distribution parameters for partial completion of spilled items

So the historical data primarily determines the sprint-level capacity and churn distribution, while the optional advanced controls determine how that empirically sampled capacity is adjusted or how individual items may fragment across sprints.

9.9.8 Percentiles and final forecast distribution

After many Monte Carlo iterations, the engine does not fit a named distribution to the resulting sprint counts. Instead it stores the simulated sprint counts directly and computes empirical percentiles such as P50, P80, and P90 from those samples.

- P50 is the median forecast
- P80 is a more conservative delivery target
- P90 is more conservative still

The same idea applies to the burn-up traces and to the displayed summary statistics.

9.9.9 Burn-up percentiles

The engine records a cumulative delivery trace for each Monte Carlo iteration. After all iterations are complete, these traces are aligned by sprint number and summarized as percentile bands (P50, P80, P90) at each sprint boundary.

The result is a burn-up chart in tabular form: for each sprint, you can see the median cumulative delivery and the more conservative estimates. This lets you answer questions like "by sprint 3, how much work is likely to be done?" without committing to a single deterministic plan.

9.9.10 Commitment guidance heuristic

The planned commitment guidance combines historical delivery and churn behavior at the configured confidence level.

The implementation first computes historical ratio series such as:

- spillover ratio = `spillover / (completed + spillover)`
- removal ratio = `removed / (completed + spillover + removed)`

It then combines historical percentiles into a conservative planning heuristic.

For confidence level q, the current implementation is approximately:

$$\text{guidance} = P50(\text{completed}) \times (1 - P_q(\text{spillover ratio})) \times (1 - P_q(\text{removal ratio})) - P_q(\text{scope added})$$

This produces a conservative "what should we plan for" number in the current capacity unit.

It is important to read this as a heuristic derived from historical percentiles, not as the mean of a fitted probability distribution.

How to use the commitment guidance in practice:

The output line `Planned Commitment Guidance: 7.55` (in story-point mode) means: based on your history at the configured confidence level, loading the sprint with about 7.55 story points gives a reasonable chance of completing them without carry-over. Use it as the starting point for your sprint backlog cap, not as a hard limit. Teams usually round to the nearest whole item, then apply judgment about priority. A higher `planning_confidence_level` (e.g., `0.90`) makes the guidance more conservative; a lower value (e.g., `0.65`) makes it more optimistic.

9.10 Natural-language generation

You do not have to hand-write the YAML from scratch. The local `generate` command can create a sprint-planning project file from semi-structured text.

This is also optional. You can:

- write sprint-planning YAML or TOML by hand
- generate a starting file with `mcprojsim generate`
- skip sprint planning entirely and use only the normal project simulation

9.10.1 Example 1: story-point-based sprint planning

```
Project: Sprint Planning from Text
Start date: 2026-05-04
Task 1:
- Discovery
- Story points: 3
Task 2:
- API implementation
- Story points: 5
Task 3:
- Frontend integration
- Story points: 8
Sprint planning:
- Sprint length: 2
- Capacity mode: story points
- Planning confidence level: 80%
Sprint history SPR-001:
- Done: 10 points
- Carryover: 1 points
Sprint history SPR-002:
- Done: 9 points
- Carryover: 2 points
- Scope added: 1 points
```

Command:

```
mcprojsim generate sprint_description.txt -o sprint_project.yaml
```

Typical generated YAML:

```yaml
project:
  name: "Sprint Planning from Text"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90, 95]

tasks:
  - id: "task_001"
    name: "Discovery"
    estimate:
      story_points: 3
    dependencies: []

  - id: "task_002"
    name: "API implementation"
    estimate:
      story_points: 5
    dependencies: []

  - id: "task_003"
    name: "Frontend integration"
    estimate:
      story_points: 8
    dependencies: []

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: "story_points"
  planning_confidence_level: 0.8
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 10
      spillover_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 9
      spillover_story_points: 2
      added_story_points: 1
```

9.10.2 Example 2: task-count throughput mode

```
Project: Maintenance Queue
Start date: 2026-05-04
Task 1:
- Fix login issue
- Size: S
Task 2:
- Improve audit logs
- Size: S
Task 3:
- Add export endpoint
- Size: S
Sprint planning:
- Sprint length: 1
- Capacity mode: tasks
Sprint history S1:
- Done: 4 tasks
- Carryover: 1 tasks
Sprint history S2:
- Done: 5 tasks
```

Use `tasks` mode only when backlog items are roughly comparable. If task sizes vary a lot, story points usually give a more stable forecast.

The CLI will print a warning if it detects heterogeneous task sizes in `tasks` mode. For example:

```
Warning: Sprint planning is using 'tasks' capacity mode, but task
planning sizes are heterogeneous. Throughput-based forecasting is
most reliable when items are roughly comparable in size.
```

9.10.3 Example 3: include holiday effects in the text description

```
Project: Summer Release
Start date: 2026-06-01
Task 1:
- Release prep
- Story points: 5
Sprint planning:
- Sprint length: 2
- Capacity mode: story points
Sprint history SPR-001:
- Done: 12 points
- Holiday factor: 90%
Sprint history SPR-002:
- Done: 9 points
- Carryover: 2 points
```

9.11 Using external CSV or JSON history files

Inline history is convenient for short examples. For real teams, history often lives better in a separate data file.

The project file can point to external sprint history like this:

```yaml
sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  history:
    format: json
    path: sprint_history.json
```

Or in TOML:

```toml
[sprint_planning]
enabled = true
sprint_length_weeks = 2
capacity_mode = "story_points"

[sprint_planning.history]
format = "csv"
path = "sprint_history.csv"
```

Supported formats:

- `json`
- `csv`

For JSON, both of these top-level shapes are supported:

- an array of sprint rows
- an object containing a `sprints` array

The history path may be relative. Relative paths are resolved from the project file location.

9.11.1 JSON example

```
{
  "metricDefinitions": {
    "committed_StoryPoints": "Sum of story points at sprint start",
    "completed_StoryPoints": "Sum of story points completed in sprint"
  },
  "sprints": [
    {
      "sprintUniqueID": "SPR-001",
      "startDate": "2026-01-01T09:00:00.000+01:00",
      "endDate": "2026-01-15T09:00:00.000+01:00",
      "metrics": {
        "committed_StoryPoints": 12,
        "completed_StoryPoints": 8,
        "spilledOver_StoryPoints": 1
      }
    },
    {
      "sprintUniqueID": "SPR-002",
      "endDate": "2026-01-29T09:00:00.000+01:00",
      "metrics": {
        "committed_StoryPoints": 14,
        "completed_StoryPoints": 10,
        "addedIntraSprint_StoryPoints": 2
      }
    }
  ]
}
```

9.11.2 CSV example

```
sprintUniqueID,committed_StoryPoints,completed_StoryPoints,addedIntraSprint_StoryPoints,removedInSprint_StoryPoints,spilledOver_
SPR-001,12,8,0,0,1,2026-01-01T09:00:00.000+01:00,2026-01-15T09:00:00.000+01:00
SPR-002,14,10,2,0,0,2026-01-15T09:00:00.000+01:00,2026-01-29T09:00:00.000+01:00
```

The external file is loaded before validation, so the same schema rules still apply.

The same history-row rules still apply after loading:

- the rows must use the correct unit family
- sprint identifiers must be unique
- if sprint planning is enabled, at least two usable rows are still required

9.12 Typical command output

The examples below were produced from real runs against working example files.

The `--minimal` (`-m`) flag controls how much output is displayed. With `--minimal`, only the summary block, sprint count statistics, and confidence intervals are shown. Without it, additional diagnostic sections are included: historical sprint series, ratio summaries, correlations, and burn-up percentiles.

When you also request file exports with `--output-format`, sprint-planning results are written into dedicated sprint sections in JSON and HTML output. The CLI summary focuses on forecast results; the richer planning-assumption details for future sprint overrides are surfaced in the exported reports.

9.12.1 Validation output

```
Validating .build/doc-examples/sprint_planning_minimal.yaml...
✓ Project file is valid!
```

9.12.2 Minimal sprint-planning simulation output

```
Sprint Planning Summary:
Sprint Length: 2 weeks
Planning Confidence Level: 80%
Removed Work Treatment: churn_only
Velocity Model: empirical
Planned Commitment Guidance: 7.55
Historical Sampling Mode: matching_cadence
Historical Observations: 3
Carryover Mean: 0.00
Aggregate Spillover Rate: 0.0000
Observed Disruption Frequency: 0.0000

Sprint Count Statistical Summary:
Mean: 3.00 sprints
Median (P50): 3.00 sprints
Std Dev: 0.00 sprints
Minimum: 3.00 sprints
Maximum: 3.00 sprints
Coefficient of Variation: 0.0000

Sprint Count Confidence Intervals:
  P50: 3 sprints  (2026-06-01)
  P80: 3 sprints  (2026-06-01)
  P90: 3 sprints  (2026-06-01)
```

`removed_work_treatment` is shown directly as `churn_only` or `reduce_backlog` in current CLI output.

9.12.3 Richer table output

```
Sprint Planning Summary:
┌──────────────────────────────┬──────────────────────────────────────┐
│ Field                        │ Value                                │
├──────────────────────────────┼──────────────────────────────────────┤
│ Sprint Length                │ 2 weeks                              │
│ Planning Confidence Level    │ 85%                                  │
│ Removed Work Treatment       │ reduce_backlog                       │
│ Velocity Model               │ empirical                            │
│ Planned Commitment Guidance  │ 7.13                                 │
│ Historical Sampling Mode     │ matching_cadence                     │
│ Historical Observations      │ 4                                    │
│ Carryover Mean               │ 2.10                                 │
│ Aggregate Spillover Rate     │ 0.1981                               │
│ Observed Disruption Frequency │ 0.7250                              │
└──────────────────────────────┴──────────────────────────────────────┘

Sprint Count Confidence Intervals:
┌──────────────┬────────────┬──────────────────────────┐
│ Percentile   │ Sprints    │ Projected Delivery Date  │
├──────────────┼────────────┼──────────────────────────┤
│ P50          │ 4.00       │ 2026-06-15               │
│ P80          │ 5.00       │ 2026-06-29               │
│ P90          │ 7.00       │ 2026-07-27               │
└──────────────┴────────────┴──────────────────────────┘
```

9.12.4 Extended output (non-minimal)

When `--minimal` is not set, the CLI appends several diagnostic sections after the main summary. These sections appear in both table and plain output modes.

Historical Sprint Series

Descriptive statistics for each normalized historical quantity:

```
Historical Sprint Series:
  completed_units: mean=10.75, median=10.50, std=1.09, min=10.00, max=12.00
  spillover_units: mean=1.75, median=1.75, std=0.83, min=1.00, max=2.67
  added_units: mean=1.00, median=0.75, std=0.87, min=0.00, max=2.00
  removed_units: mean=0.50, median=0.50, std=0.50, min=0.00, max=1.00
```

These reflect the normalized rows the sampler actually uses. Use them to sanity-check that holiday-factor normalization has not introduced unexpected values.

Historical Ratio Summaries

```
Historical Ratio Summaries:
  scope_addition_ratio: mean=0.0834, median=0.0675, std=0.0686, P50=0.0675, P80=0.1349, P90=0.1519
  scope_removal_ratio: mean=0.0396, median=0.0396, std=0.0354, P50=0.0396, P80=0.0659, P90=0.0738
  spillover_ratio: mean=0.1380, median=0.1433, std=0.0404, P50=0.1433, P80=0.1678, P90=0.1799
```

These ratios feed the commitment-guidance heuristic. Higher spillover or scope-addition ratios will push guidance lower.

Historical Correlations

```
Historical Correlations:
  added_units|completed_units: -0.8036
  added_units|removed_units: 0.7906
  added_units|spillover_units: 0.8242
  completed_units|removed_units: -0.2747
  completed_units|spillover_units: -0.7656
  removed_units|spillover_units: 0.2747
```

Pearson correlations between normalized historical series. They can highlight patterns such as "sprints with high added scope also tend to have high spillover".

Burn-up Percentiles

```
Burn-up Percentiles:
  Sprint 1: P50=10.50, P80=12.00, P90=12.00
  Sprint 2: P50=21.00, P80=23.33, P90=24.00
  Sprint 3: P50=31.50, P80=34.67, P90=36.00
```

Cumulative delivery bands per sprint. Read these as "by the end of sprint N, how many units are likely to be done". The P80 and P90 columns reflect slower scenarios, so their cumulative values rise more slowly.

9.12.5 HTML and JSON report visibility

Future sprint capacity adjustments are recorded explicitly in exported reports.

In HTML output, future overrides appear in a dedicated section named `Planning Assumptions: Future Sprint Capacity Adjustments`. Each row shows:

- the targeted sprint number or boundary date
- `holiday_factor`
- `capacity_multiplier`
- effective multiplier
- optional notes

In JSON output, the same data appears under:

- `sprint_planning.planning_assumptions.future_sprint_overrides`

The JSON planning-assumptions block also includes a short note explaining that the effective multiplier is:

$$\text{effective multiplier} = \text{holiday factor} \times \text{capacity multiplier}$$

The CLI summary does not currently list every override row individually. It shows the resulting forecast and diagnostics, while HTML and JSON exports preserve the detailed future-sprint assumptions for review and audit.

If you want those report sections, use an export command such as:

```
mcprojsim simulate sprint_project.yaml \
  --output out/sprint_forecast \
  --output-format json,html
```

These outputs are best read as ranges, not promises. The goal is to make sprint planning explicit, data-driven, and auditable.

CHAPTER 10

Constrained Scheduling

This chapter shows how to model and run **resource- and calendar-constrained** simulations in `mcprojsim`, starting from a simple project and building up to a full-featured example.

Constrained scheduling is activated automatically when the validated project has resources available for assignment—either from a top-level `resources` section or from `project.team_size > 0` (which auto-generates default resources up to that size).

!!! note Constrained schedules are often much longer than dependency-only estimates!

```
**Example:** A 3-person team working on 10 tasks in parallel (dependency-only)
might complete in 100 hours. The same team with resource constraints might
need 400 hours because resources unavailability creates wait time.

This is NOT a bug; it's a more realistic picture of how work actually gets done.
```

10.1 What constrained scheduling changes

In dependency-only mode, tasks start as soon as dependencies are complete.

In constrained mode, task start and completion times also depend on:

- resource availability,
- resource experience and productivity,
- task-level resource requirements,
- working calendars and holidays,
- planned absences and sickness probability.

The CLI and exporters report this explicitly via:

- `Schedule Mode` (`dependency_only` or `resource_constrained`),
- constrained diagnostics (resource wait time, utilization, calendar delay contribution).

10.2 Suggested reading path

For first-time readers, follow this order:

1. Example 1 through Example 6 (progressive walkthrough)
2. `team_size` rules (reference section)
3. CLI/config tuning knobs (`--two-pass`, `--pass1-iterations`, and config defaults)

If you already know the core flow, jump directly to the `team_size` and two-pass sections.

10.3 How `team_size` affects scheduling (reference)

`team_size` and top-level `resources` interact during validation before simulation starts.

Current rules:

- `team_size` omitted or `0`: use only explicitly listed `resources`.
- `team_size > 0` and explicit resources **exceed** `team_size`: validation error.
- `team_size > 0` and explicit resources are **fewer** than `team_size`: default resources are auto-created up to `team_size`.
- no `resources` and no `team_size` (or `team_size: 0`): scheduler remains dependency-only.

10.3.1 Example A: tasks only (no `team_size`)

```
project:
  name: "Team Size Demo"
  start_date: "2026-04-01"
  hours_per_day: 8

tasks:
  - id: "task_001"
    name: "Task 1"
    estimate: { low: 8, expected: 16, high: 24 }
  - id: "task_002"
    name: "Task 2"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]
```

10.3.2 Example B: same tasks, with `team_size`

```
project:
  name: "Team Size Demo"
  start_date: "2026-04-01"
  hours_per_day: 8
  team_size: 10

tasks:
  - id: "task_001"
    name: "Task 1"
    estimate: { low: 8, expected: 16, high: 24 }
  - id: "task_002"
    name: "Task 2"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]
```

Sample output excerpt (seed `42`, `200` iterations):

```
Schedule Mode: resource_constrained
Median (P50): 529.86 hours
```

`team_size: 10` auto-creates 10 default resources, so this run is constrained.

10.3.3 Example C: explicit `resources` + `team_size` (smaller than resources)

```
project:
  name: "Team Size + Resources"
  start_date: "2026-04-01"
  hours_per_day: 8
  team_size: 1

tasks:
  - id: "task_001"
    name: "Task 1"
    estimate: { low: 8, expected: 16, high: 24 }
  - id: "task_002"
    name: "Task 2"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]

resources:
  - name: "alice"
    experience_level: 3
    productivity_level: 1.0
  - name: "bob"
    experience_level: 2
    productivity_level: 0.9
```

Sample output excerpt:

```
Error: Invalid project file format: 1 validation error for Project
  Value error, team_size is smaller than explicitly specified resources: team_size=1, resources=2
```

10.3.4 Example D: explicit `resources` + `team_size` (larger than resources)

Same as Example C but with `team_size: 20`.

Sample output excerpt (same run settings):

```
Schedule Mode: resource_constrained
Median (P50): 529.86 hours
```

`team_size: 20` is valid here: two explicit resources are kept and the remaining capacity is auto-filled with defaults.

10.3.5 Interpretation and policy

- `team_size` is a capacity target when greater than zero.
- Explicit `resources` can be combined with `team_size`, but they must not exceed it.
- If explicit resources are fewer than `team_size`, generated defaults fill the gap.
- If both are omitted (or `team_size: 0` with no resources), the run is dependency-only.

10.4 Example 1: Baseline (dependency-only)

Start with a minimal project that has dependencies but no resource section.

```
project:
  name: "Onboarding Portal"
  start_date: "2026-04-01"
  hours_per_day: 8

tasks:
  - id: "task_001"
    name: "Requirements"
    estimate: { low: 8, expected: 16, high: 24 }

  - id: "task_002"
    name: "Implementation"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]
```

Run:

```
mcprojsim simulate baseline.yaml --seed 42 --table
```

You should see `Schedule Mode: dependency_only`.

Sample output excerpt (seed `42`, `200` iterations):

```
Schedule Mode: dependency_only
Calendar Time Confidence Intervals:
  P50: 129.86 hours (17 working days)  (2026-04-24)
  P90: 155.97 hours (20 working days)  (2026-04-29)
  P99: 170.61 hours (22 working days)  (2026-05-01)
```

10.5 Resource fields introduced in this chapter

Before Example 2, here is a quick reference for the `resources` fields used throughout the walkthrough.

Field	Required	Default	What it controls
`name`	No	auto-generated (`resource_001`, …)	Stable resource identifier used by tasks
`availability`	No	`1.0`	Fractional availability `(0.0, 1.0]` (for example `0.8` for 80%); values > 1.0 are not allowed
`experience_level`	No	`2`	Skill tier `1`, `2`, or `3`; matched against task `min_experience_level`
`productivity_level`	No	`1.0`	Relative throughput multiplier (valid range `0.1`–`2.0`); `1.5` means 50% more productive than baseline
`calendar`	No	`"default"`	Which calendar (`calendars[*].id`) the resource follows
`sickness_prob`	No	`0.0`	Probability of a sickness event starting on any given working day (see note below)
`planned_absence`	No	`[]`	Explicit non-working dates (`"YYYY-MM-DD"`) for that resource

!!! note “Choosing `sickness_prob`” `sickness_prob` is checked independently on **every working day**, so even

small values have a cumulative effect. A value of `0.01` (1%) implies roughly a 5% chance of sickness in any given work week. Typical starting values: `0.005`–`0.02`. Leave at `0.0` (the default) to disable stochastic sickness entirely.

10.5.1 First two fields introduced in Example 2

- `experience_level`: the resource's capability tier (`1`, `2`, `3`).
- `productivity_level`: how quickly the resource converts effort into progress relative to baseline `1.0`. A value of `0.9` means 10% slower than baseline; `1.2` means 20% faster.

In Example 2, Alice (`experience_level: 3`, `productivity_level: 1.0`) is modeled as the most senior resource at baseline capacity, while Bob (`experience_level: 2`, `productivity_level: 0.9`) works at 90% of baseline speed.

10.6 Task fields related to resources introduced in this chapter

Field	Required	Default	What it controls
`resources`	No	(all pool resources)	Explicit list of resource names eligible for this task; omit to allow any available resource
`max_resources`	No	1	Maximum number of resources assigned to a task
`min_experience_level`	No	1	Minimum `experience_level` a resource must have to be eligible for this task

10.6.1 `max_resources` semantics (important)

`max_resources` is an upper bound on concurrent assignees for a task, but the scheduler also applies an automatic practical cap.

- If `resources` lists multiple names and `max_resources` is smaller than that list, only up to `max_resources` resources are assigned at task start.

Automatic practical cap heuristic:

- `granularity_cap = max(1, floor(task_effort_hours / 16.0))`
- `coordination_cap = 3`
- `practical_cap = min(granularity_cap, coordination_cap)`

Effective assignment count at start is:

```
min(max_resources, practical_cap, currently_available_eligible_resources)
```

- If `max_resources` is omitted, the default is `1`.

Why this exists:

- It prevents unrealistic compression of small tasks (for example, assigning eight people to an ~8–24 hour task).
- It keeps behavior deterministic while limiting over-parallelization on large tasks.

Practical examples:

- Task effort `8h` → `granularity_cap = max(1, floor(8/16)) = 1` → at most 1 assignee.
- Task effort `24h` → `granularity_cap = max(1, floor(24/16)) = 1` → at most 1 assignee.
- Task effort `80h` → `granularity_cap = floor(80/16) = 5`, but coordination cap limits to 3.

Why these constants were selected:

- `MIN_EFFORT_PER_ASSIGNEE_HOURS = 16.0` means each assignee should have about two working days of effort before the heuristic adds another concurrent person.
- `MAX_ASSIGNEES_PER_TASK = 3` is a hard global coordination ceiling, even for very large tasks.
- The pair balances realism and runtime simplicity: the scheduler remains deterministic and fast while avoiding implausible near-linear speedups.

10.6.2 How the `16.0` and `3` factors shape assignment

The heuristic has two stages:

1. **Granularity stage (`16.0`)**
   - Compute `floor(task_effort_hours / 16.0)`.
   - This is a rough "how many meaningful chunks" estimate for parallel work.
   - `max(1, ...)` guarantees at least one assignee.
2. **Coordination stage (`3`)**
   - Apply `min(granularity_cap, 3)`.
   - This caps parallelism to avoid unrealistic linear speedup from many assignees.

Combined effect by effort range:

- `0 <= effort < 32` hours: practical cap is `1`
- `32 <= effort < 48` hours: practical cap is `2`
- `effort >= 48` hours: practical cap is `3` (ceiling reached)

So `16.0` controls when you *earn* additional assignees, while `3` controls the *maximum* you can ever get from the heuristic.

10.6.3 How this interacts with actual assignment

The practical cap is only one limiter. Final assigned count is:

```
min(max_resources, practical_cap, currently_available_eligible_resources)
```

Implications:

- If `max_resources` is lower than the practical cap, task-level config wins.
- If fewer eligible resources are free, availability wins.
- If both are high, heuristic limits still apply (`3` max from practical cap).

Quick examples:

- Effort `40h`, `max_resources: 5`, 4 eligible free resources: `granularity_cap = floor(40/16) = 2`, `practical_cap = 2` -> assign up to `2`.
- Effort `80h`, `max_resources: 2`, 5 eligible free resources: `granularity_cap = 5`, `practical_cap = 3`, then `max_resources` lowers final cap to `2`.
- Effort `120h`, `max_resources: 5`, only 1 eligible free resource: heuristic allows `3`, but availability lowers final assignment to `1`.

If you need stricter or looser behavior in your environment, these constants can be adjusted in the scheduler implementation and validated with scenario-specific simulations.

10.6.4 Assignment timing model

Resource assignment is performed only when the task starts.

- Assigned resources remain fixed for that task run (non-preemptive execution).
- The scheduler does not swap or add resources mid-task.

10.6.5 Start-now vs wait-for-more-resources behavior

Current automatic assignment is greedy:

- If at least one eligible resource is available, the task can start immediately.

- The scheduler does not delay task start to wait for additional resources that might become available later.

This is a deliberate trade-off for deterministic and scalable simulation on large projects. This avoids combinatorial explosion and keeps runtime predictable for large projects. It is a heuristic, not globally optimal schedule optimization.

10.7 Example 2: Add resources (single-pass automatic assignment)

Now add a top-level `resources` section. This enables constrained scheduling.

```
project:
  name: "Onboarding Portal"
  start_date: "2026-04-01"
  hours_per_day: 8
tasks:
  - id: "task_001"
    name: "Requirements"
    estimate: { low: 8, expected: 16, high: 24 }

  - id: "task_002"
    name: "Implementation"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]

resources:
  - name: "alice"
    experience_level: 3
    productivity_level: 1.0

  - name: "bob"
    experience_level: 2
    productivity_level: 0.9
```

Run:

```
mcprojsim simulate resources-basic.yaml --seed 42 --table
```

Sample output excerpt (seed `42`, `200` iterations):

```
Schedule Mode: resource_constrained
Calendar Time Confidence Intervals:
  P50: 529.86 hours (67 working days)  (2026-07-03)
  P90: 651.97 hours (82 working days)  (2026-07-24)
  P99: 698.61 hours (88 working days)  (2026-08-03)

Constrained Schedule Diagnostics:
  Calendar Delay Contribution: 404.16 hours
```

10.7.1 How resources are used in Example 2

In this example, tasks do **not** define `tasks[*].resources`, so each ready task can use the full resource pool:

- `alice`
- `bob`

Assignment behavior in this case is:

1. A task becomes eligible only after all dependencies are finished.
2. If the task has no explicit `resources` list, the scheduler considers **all** project resources.
3. Resources already busy on other tasks are not available (a resource can only work on one task at a time).
4. The task receives up to `max_resources` resources (default is `1` if omitted).
5. If no eligible resource is free, the task waits; this contributes to `Average Resource Wait` in diagnostics.

Because `max_resources` defaults to `1`, each task in Example 2 is effectively assigned one resource at a time unless you override it on the task.

10.7.2 Why `experience_level` and `productivity_level` matter here

- `experience_level` on each resource is matched against task `min_experience_level` (default task minimum is `1`).
- `productivity_level` affects effective capacity, so two resources with different productivity can produce different calendar durations even for the same effort.

10.7.3 Making assignment explicit (optional)

If you want explicit control instead of automatic pooling, set `tasks[*].resources`:

```
tasks:
  - id: "task_001"
    name: "Requirements"
    estimate: { low: 8, expected: 16, high: 24 }
    resources: ["alice"]

  - id: "task_002"
    name: "Implementation"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]
    resources: ["alice", "bob"]
    max_resources: 2
```

In that explicit form, only listed resources are considered for each task.

Look for:

- `Schedule Mode: resource_constrained`
- `Constrained Schedule Diagnostics`
  - **Average Resource Wait** — average hours tasks spent waiting for an eligible resource to become free after their dependencies finished. A high value indicates resource contention.
  - **Effective Resource Utilization** — ratio of total task effort to total available resource capacity over the project duration, capped at 1.0 (100%). Near 1.0 means the resource pool is fully loaded; near 0 means resources were idle most of the time.
  - **Calendar Delay Contribution** — total hours lost to non-working periods (weekends, holidays, planned absences, sickness) while resources were carrying work.

!!! note “Zero resource wait alongside large calendar delay is not a contradiction” A project can show **Average Resource Wait: 0.00h** and a large **Calendar Delay Contribution** at the same time. This simply means resources were always available when tasks needed them (no queuing), but the project spanned many non-working periods — evenings, weekends, holidays, or absences — during which work was paused. Calendar delay measures elapsed clock time lost to non-working hours, not resource contention.

10.8 Example 3: Add working calendars

Attach resources to calendars and define working patterns.

Calendar fields quick reference:

Field	Default	What it controls
`id`	`"default"`	Calendar identifier referenced by `resources[*].calendar`
`work_hours_per_day`	`8.0`	Number of productive hours in a working day
`work_days`	`[1, 2, 3, 4, 5]`	Working weekdays as integers: **1 = Monday … 7 = Sunday**
`holidays`	`[]`	Dates (ISO 8601 `"YYYY-MM-DD"`) that are non-working regardless of `work_days`

`work_days: [1, 2, 3, 4, 5]` is the standard Mon–Fri schedule. `work_days: [1, 2, 3, 4]` is Mon–Thu (for a four-day working week).

```
project:
  name: "Onboarding Portal"
  start_date: "2026-04-01"
  hours_per_day: 8

tasks:
  - id: "task_001"
    name: "Requirements"
    estimate: { low: 8, expected: 16, high: 24 }
  - id: "task_002"
    name: "Implementation"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]

resources:
  - name: "alice"
    calendar: "default"
    experience_level: 3
    productivity_level: 1.0
  - name: "bob"
    calendar: "part_time"
    experience_level: 2
    productivity_level: 0.9
calendars:
  - id: "default"
    work_hours_per_day: 8
    work_days: [1, 2, 3, 4, 5]
    holidays: ["2026-04-10"]

  - id: "part_time"
    work_hours_per_day: 6
    work_days: [1, 2, 3, 4]
    holidays: []
```

This introduces calendar-driven delays automatically (weekends, holidays, shorter days).

Sample output excerpt (seed `42`, `200` iterations):

```
Schedule Mode: resource_constrained
Calendar Time Confidence Intervals:
  P50: 553.86 hours (70 working days)  (2026-07-08)
  P90: 675.97 hours (85 working days)  (2026-07-29)
  P99: 722.61 hours (91 working days)  (2026-08-06)

Constrained Schedule Diagnostics:
  Calendar Delay Contribution: 437.28 hours
```

Compared with Example 2, the added holiday and part-time calendar increase calendar-time percentiles and calendar-delay contribution.

10.8.1 Quick comparison (Examples 1 → 3)

The table below compares calendar-time percentiles from the sample runs above (seed `42`, `200` iterations):

Example	Schedule Mode	P50 (hours)	P90 (hours)	P99 (hours)
Example 1 (dependency-only)	`dependency_only`	129.86	155.97	170.61
Example 2 (resources only)	`resource_constrained`	529.86	651.97	698.61
Example 3 (resources + calendars)	`resource_constrained`	553.86	675.97	722.61

This progression highlights how resource constraints and then calendar constraints increase elapsed calendar time, even when effort distributions are unchanged.

10.9 Example 4: Add sickness and planned absence

Sickness has three related configuration layers:

- `constrained_scheduling.sickness_prob` is the default per-resource sickness probability used when a resource omits `sickness_prob`,
- `resources[*].sickness_prob` controls/overrides probability for that specific resource,
- `sprint_defaults.sickness.duration_log_mu` and `sprint_defaults.sickness.duration_log_sigma` control the shared log-normal duration model used when sickness occurs.

Explicit days off are set with `planned_absence`.

```
resources:
  - name: "alice"
    calendar: "default"
    experience_level: 3
    productivity_level: 1.0
    sickness_prob: 0.02
    planned_absence: ["2026-04-22"]

  - name: "bob"
    calendar: "part_time"
    experience_level: 2
    productivity_level: 0.9
    sickness_prob: 0.04
    planned_absence: ["2026-04-15", "2026-04-16"]
```

!!! note `resources[*].sickness_prob` is configured in the project file per resource. If omitted, constrained scheduling falls back to `constrained_scheduling.sickness_prob` from the config file. If neither is set, the effective default remains `0.0`.

10.9.1 Optional config for default sickness probability

```
constrained_scheduling:
  sickness_prob: 0.03
```

With this setting:

- resources with explicit `sickness_prob` keep their own value,
- resources without `sickness_prob` use `0.03`,
- omitting this config block keeps the legacy `0.0` default.

10.9.2 Optional config for sickness duration

If you want the constrained scheduler to assume shorter or longer sickness episodes, add this to your config file and run with `--config`:

```
sprint_defaults:
  sickness:
    duration_log_mu: 1.10
    duration_log_sigma: 0.90
```

This does not change who gets sick. It changes how long an absence tends to last once a sickness event occurs.

!!! note The duration parameters (`duration_log_mu` and `duration_log_sigma`) are configured once under `sprint_defaults.sickness` and apply to **both constrained scheduling and sprint planning**. This ensures consistent absence duration modeling across both forecasting modes. Sickness probabilities, however, have separate mode-specific defaults: `constrained_scheduling.sickness_prob` (constrained mode only) and `sprint_defaults.sickness.probability_per_person_per_week` (sprint planning only).

10.10 Example 5: Add task-level resource constraints

Use task fields to control assignment behavior:

- `resources`: restrict eligible resources by name,
- `max_resources`: cap parallel resources assigned to a task,
- `min_experience_level`: minimum allowed experience (`1`, `2`, `3`).

```
tasks:
  - id: "task_003"
    name: "Data migration"
    estimate: { low: 24, expected: 40, high: 64 }
    dependencies: ["task_002"]
    resources: ["alice"]
    max_resources: 1
    min_experience_level: 3

  - id: "task_004"
    name: "System testing"
    estimate: { low: 16, expected: 24, high: 40 }
    dependencies: ["task_003"]
    resources: ["alice", "bob"]
    max_resources: 2
    min_experience_level: 2
```

10.10.1 Compact recap: how assignment works in Example 5

- `task_003` can only use `alice` (`resources: ["alice"]`) and is capped at one resource (`max_resources: 1`).
- `task_004` can use either `alice` or `bob`, and can run with up to two resources in parallel (`max_resources: 2`) when both are available.
- `min_experience_level` filters the eligible set before assignment (for example, tasks requiring `3` cannot use level-`2` resources).
- If fewer resources are available than requested by `max_resources`, the task starts with what is available (if at least one eligible resource exists) and continues with reduced effective capacity.

This is the first point in the walkthrough where `max_resources` is actively overridden above the default (`1`) to allow multi-resource task execution.

10.11 Example 6: Full constrained project (final build-up)

This example combines core project fields, risks, uncertainty factors, resource constraints, calendars, absences, and sickness.

```
project:
  name: "Commerce Platform Upgrade"
  description: "Backend modernization with staged rollout"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90, 95]
  hours_per_day: 8
  probability_red_threshold: 0.5
  probability_green_threshold: 0.9

project_risks:
  - id: "risk_vendor"
    name: "Vendor API latency"
    probability: 0.2
    impact: 16
    impact_unit: "hours"

tasks:
  - id: "task_001"
    name: "Architecture design"
    estimate: { low: 16, expected: 24, high: 40 }
    uncertainty_factors:
      team_experience: medium
      technical_complexity: medium
  - id: "task_002"
    name: "Core implementation"
    estimate: { low: 80, expected: 120, high: 180 }
    dependencies: ["task_001"]
    resources: ["alice", "bob", "carol"]
    max_resources: 2
    min_experience_level: 2
    risks:
      - id: "risk_rework"
        name: "Unexpected rework"
        probability: 0.25
        impact: 24
        impact_unit: "hours"
  - id: "task_003"
    name: "Migration"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_002"]
    resources: ["alice"]
    max_resources: 1
    min_experience_level: 3

  - id: "task_004"
    name: "Verification and rollout"
    estimate: { low: 24, expected: 40, high: 64 }
    dependencies: ["task_003"]
    resources: ["alice", "bob", "carol"]
    max_resources: 2

resources:
  - name: "alice"
    calendar: "default"
    availability: 1.0
    experience_level: 3
    productivity_level: 1.1
    sickness_prob: 0.02
    planned_absence: ["2026-05-15"]
  - name: "bob"
    calendar: "default"
    availability: 0.8
    experience_level: 2
    productivity_level: 1.0
    sickness_prob: 0.03
  - name: "carol"
```

Run and export all formats:

```
mcprojsim validate constrained-full.yaml
mcprojsim simulate constrained-full.yaml \
  --iterations 30000 \
  --seed 42 \
  --table \
  --critical-paths 5 \
  -f json,csv,html \
  -o results/constrained-full
```

10.12 Single-pass vs two-pass assignment

10.12.1 Single-pass automatic assignment (available)

Current constrained scheduling uses deterministic single-pass assignment while respecting dependencies, resource eligibility, calendars, and absences.

You use it by defining top-level `resources` (and optionally `calendars`). No extra CLI flag is required.

10.12.2 Two-pass criticality assignment (available)

Two-pass criticality-prioritized scheduling is available via CLI and config.

How it works:

1. Pass 1 runs greedy scheduling for a subset of iterations to estimate task criticality.
2. Pass 2 re-runs constrained scheduling using criticality ranking as dispatch priority.
3. Final summary metrics come from pass 2, with a two-pass traceability delta in CLI and exports.

CLI usage:

```
mcprojsim simulate constrained-full.yaml --two-pass --pass1-iterations 1500 --seed 42 --table
```

Config usage:

```
constrained_scheduling:
  assignment_mode: criticality_two_pass
  pass1_iterations: 1500
```

Notes:

- `--two-pass` overrides config and enables `criticality_two_pass` for that run.
- `--pass1-iterations` is capped to total simulation iterations.
- If `pass1_iterations` is small, criticality ranking may be noisy.

10.13 CLI options most relevant to constrained runs

Use these with `mcprojsim simulate`:

Option	Why it matters for constrained scheduling
`-n`, `--iterations`	More iterations stabilize constrained diagnostics and tail percentiles
`-s`, `--seed`	Makes resource/sickness-driven runs reproducible
`-c`, `--config`	Apply custom uncertainty and output/reporting defaults
`--two-pass`	Enable criticality two-pass constrained scheduling for this run
`--pass1-iterations`	Set pass-1 sample budget for criticality ranking in two-pass mode
`--critical-paths`	Show more critical-path sequences for bottleneck analysis

Option	Why it matters for constrained scheduling
`-t, --table`	Easier reading of diagnostics and interval tables
`-f json,csv,html`	Export constrained diagnostics to all report channels
`-o`	Keep scenario outputs organized for side-by-side comparison
`--target-date`	Evaluate on-time probability against a concrete deadline

10.14 Where to configure constrained behavior

10.14.1 In the project file (primary for constrained scheduling)

- `resources[*].availability`
- `resources[*].productivity_level`
- `resources[*].experience_level`
- `resources[*].calendar`
- `resources[*].sickness_prob`
- `resources[*].planned_absence`
- `calendars[*].work_hours_per_day`
- `calendars[*].work_days`
- `calendars[*].holidays`
- `tasks[*].resources`
- `tasks[*].max_resources`
- `tasks[*].min_experience_level`

10.14.2 In the config file (`--config`)

Use config files for simulation/reporting defaults and shared stochastic model parameters. They do **not** replace per-resource sickness probabilities in the project file, but they can now define the shared sickness-duration distribution used by constrained scheduling.

Example:

```
constrained_scheduling:
  sickness_prob: 0.03
  assignment_mode: criticality_two_pass
  pass1_iterations: 1500

simulation:
  default_iterations: 30000

output:
  critical_path_report_limit: 5
  number_bins: 80

staffing:
  effort_percentile: 80

sprint_defaults:
  sickness:
    duration_log_mu: 1.10
    duration_log_sigma: 0.90
```

`constrained_scheduling.assignment_mode` supports:

- `greedy_single_pass` (default)
- `criticality_two_pass`

`constrained_scheduling.pass1_iterations` controls pass-1 ranking depth when two-pass mode is active.

10.15 Related chapters

- Project Files for full schema reference
- Running Simulations for command reference
- Interpreting Results for diagnostics interpretation
- Multi-Phase (Two-Pass) Simulation for criticality-prioritized scheduling

CHAPTER 11

Multi-Phase (Two-Pass) Simulation

Multi-phase simulation is an advanced scheduling strategy for resource-constrained projects. When several tasks compete for the same limited pool of resources, the order in which they are dispatched to those resources can materially affect the projected completion date.

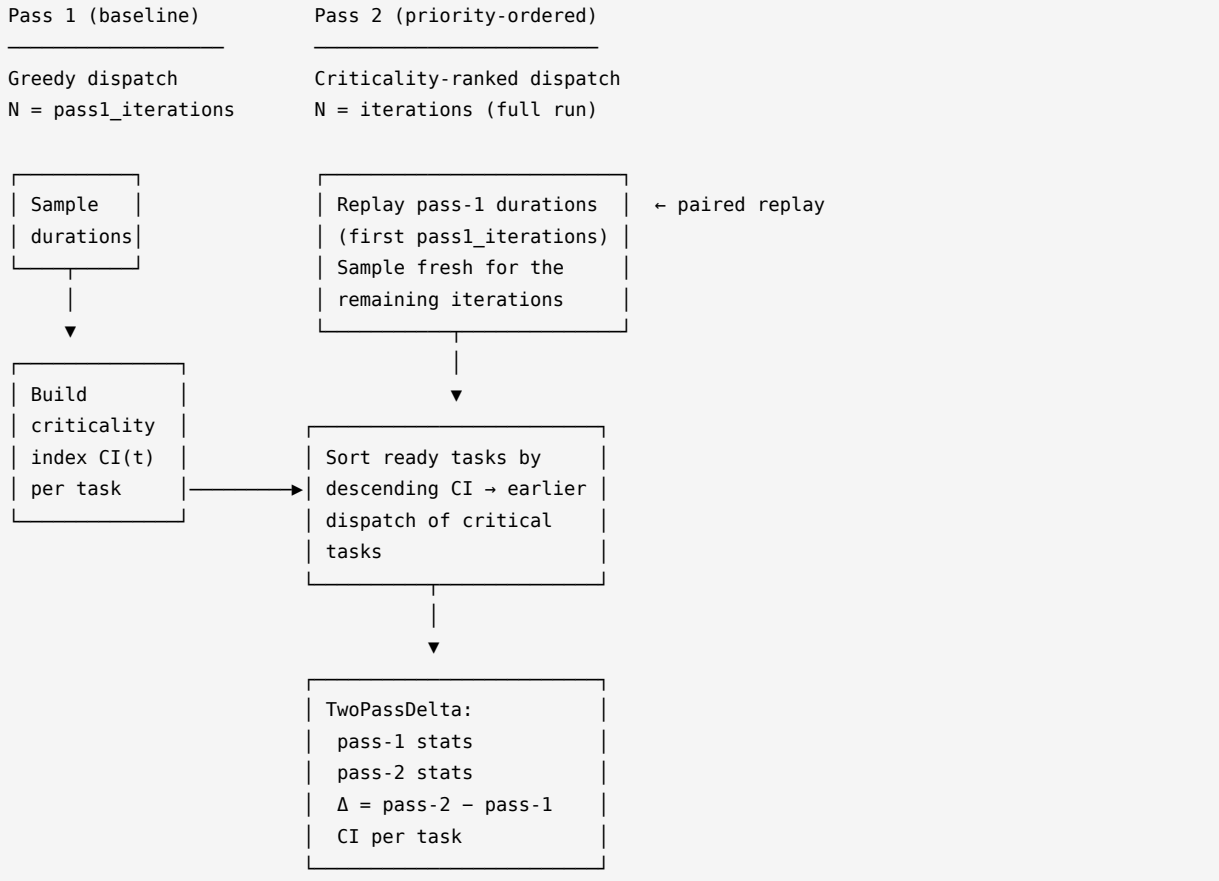
The standard scheduler uses a simple greedy policy: when multiple tasks become ready at the same moment it picks them in deterministic ID order. This works well when resources are abundant or contention is low. When resources are scarce and some tasks are strongly critical, a smarter ordering—one that dispatches the most critical tasks first—can reduce the median project duration and, more importantly, reduce tail-risk at the P80– P95 percentiles.

Two-pass simulation solves this problem with two consecutive simulation runs.

Only active with resource-constrained scheduling

Two-pass mode requires at least one named `resource` in your project file. If no resources are defined the engine falls back to the standard single-pass mode automatically. **This fallback is silent** — no warning is printed even with `--verbose`. If you use `--two-pass` and the output does not show a “Pass 1” / “Pass 2” progress line, it means no resources were found and single-pass was used instead.

11.1 How It Works



Pass 1 — criticality baseline
The engine runs `pass1_iterations` iterations using the standard greedy policy. Every iteration, it records which tasks appear on the critical path. The *criticality index* of each task is the fraction of pass-1 iterations in which it was critical (range $0 \rightarrow 1$). Task durations are also cached for paired replay.

Pass 2 — priority-ordered run
The engine runs the full `--iterations` run again. Whenever two or more tasks become ready at the same moment and there is a resource to claim, they are sorted by *descending criticality index* before dispatch. This ensures that pass-1 identified bottleneck tasks get resources first.

For the first `pass1_iterations` iterations, the exact same sampled durations from pass-1 are replayed (paired replay). This makes the comparison between pass-1 and pass-2 apples-to-apples: any difference in project duration is caused solely by the change in dispatch policy, not by different duration samples.

TwoPassDelta traceability
The results include a `two_pass_traceability` block in all output formats (console, JSON, CSV, HTML) with per-pass statistics and Δ values.

11.2 Basic Example

11.2.1 Project file

For this example we will use the provided `examples/contention.yaml` file with three tasks sharing one senior developer and a separate junior developer:

```
project:
  name: "contention-demo"
  start_date: "2025-03-01"
  hours_per_day: 8
  confidence_levels: [50, 80, 90, 95]

tasks:
  - id: "A1"
    name: "Spec review"
    estimate: { low: 9, expected: 10, high: 12 }
    resources: ["dev-senior"]
  - id: "A2"
    name: "Core implementation"
    estimate: { low: 13, expected: 15, high: 18 }
    dependencies: ["A1"]
    resources: ["dev-senior"]
  - id: "A3"
    name: "Code review"
    estimate: { low: 9, expected: 10, high: 12 }
    dependencies: ["A2"]
    resources: ["dev-senior"]
  - id: "B1"
    name: "Documentation"
    estimate: { low: 18, expected: 20, high: 24 }
    resources: ["dev-senior"]          # competes for dev-senior!
  - id: "B2"
    name: "User testing"
    estimate: { low: 9, expected: 10, high: 12 }
    dependencies: ["B1"]
    resources: ["dev-junior"]
  - id: "C1"
    name: "Integration"
    estimate: { low: 4, expected: 5, high: 7 }
    dependencies: ["A3", "B2"]
    resources: ["dev-junior"]
  - id: "C2"
    name: "Release"
    estimate: { low: 7, expected: 8, high: 10 }
    dependencies: ["C1"]
    resources: ["dev-junior"]

resources:
  - name: "dev-senior"
    experience_level: 3
  - name: "dev-junior"
    experience_level: 1
```

11.2.2 Single-pass run

```
mcprojsim % mcprojsim simulate examples/contention.yaml -n 1000 -t --seed 1234 --minimal
```

```
mcprojsim v0.15.2

Project Overview:
┌──────────────────────────────────┬──────────────────────────────────────┐
│ Field                            │ Value                                │
├──────────────────────────────────┼──────────────────────────────────────┤
│ Project                          │ contention-demo                      │
│ Start Date                       │ 2025-03-01                           │
│ Number of Tasks                  │ 7                                    │
│ Effective Default Distribution   │ triangular                           │
│ T-Shirt Category Used            │ story                                │
│ Hours per Day                    │ 8.0                                  │
│ Max Parallel Tasks               │ 1                                    │
│ Schedule Mode                    │ resource_constrained                 │
└──────────────────────────────────┴──────────────────────────────────────┘

Calendar Time Statistical Summary:
┌──────────────┬──────────────────────────────────────────────────────────┐
│ Metric       │ Value                                                    │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Mean         │ 360.44 hours (46 working days)                           │
│ Median (P50) │ 384.62 hours                                             │
│ Std Dev      │ 32.78 hours                                              │
│ Minimum      │ 314.61 hours                                             │
│ Maximum      │ 391.54 hours                                             │
└──────────────┴──────────────────────────────────────────────────────────┘

Project Effort Statistical Summary:
┌──────────────┬──────────────────────────────────────────────────────────┐
│ Metric       │ Value                                                    │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Mean         │ 80.69 person-hours (11 person-days)                      │
│ Median (P50) │ 80.62 person-hours                                       │
│ Std Dev      │ 2.18 person-hours                                        │
│ Minimum      │ 74.61 person-hours                                       │
│ Maximum      │ 87.54 person-hours                                       │
└──────────────┴──────────────────────────────────────────────────────────┘

Calendar Time Confidence Intervals:
┌──────────────┬──────────┬──────────────┬──────────────────────────────┐
│ Percentile   │    Hours │   Working Days │ Date                       │
├──────────────┼──────────┼──────────────┼──────────────────────────────┤
│ P50          │   384.62 │             49 │ 2025-05-08                 │
│ P80          │   386.51 │             49 │ 2025-05-08                 │
│ P90          │   387.45 │             49 │ 2025-05-08                 │
│ P95          │   388.43 │             49 │ 2025-05-08                 │
└──────────────┴──────────┴──────────────┴──────────────────────────────┘
```

The mean duration is ~361 hours (45 working days), with a P80 of 386 hours and a P95 of 388 hours. The critical path is dominated by the senior developer's stream (A1 → A2 → A3) but the documentation task B1 is also critical in a significant fraction of iterations, causing the junior developer to wait for it before starting B2 and C1.

11.2.3 Two-pass run

Add the `--two-pass` flag:

```
mcprojsim % mcprojsim simulate examples/contention.yaml -n 1000 --two-pass --pass1-iterations 500 -t --seed 1234
```

Output excerpt:

```
mcprojsim v0.15.2

Project Overview:
┌──────────────────────────────┬──────────────────────────────────────┐
│ Field                        │ Value                                │
├──────────────────────────────┼──────────────────────────────────────┤
│ Project                      │ contention-demo                      │
│ Start Date                   │ 2025-03-01                           │
│ Number of Tasks              │ 7                                    │
│ Effective Default Distribution │ triangular                         │
│ T-Shirt Category Used        │ story                                │
│ Hours per Day                │ 8.0                                  │
│ Max Parallel Tasks           │ 2                                    │
│ Schedule Mode                │ resource_constrained                 │
└──────────────────────────────┴──────────────────────────────────────┘

Calendar Time Statistical Summary:
┌─────────────┬────────────────────────────────────────────────────────┐
│ Metric      │ Value                                                  │
├─────────────┼────────────────────────────────────────────────────────┤
│ Mean        │ 298.42 hours (38 working days)                         │
│ Median (P50) │ 294.43 hours                                          │
│ Std Dev     │ 8.60 hours                                             │
│ Minimum     │ 288.75 hours                                           │
│ Maximum     │ 317.03 hours                                           │
└─────────────┴────────────────────────────────────────────────────────┘

Project Effort Statistical Summary:
┌─────────────┬────────────────────────────────────────────────────────┐
│ Metric      │ Value                                                  │
├─────────────┼────────────────────────────────────────────────────────┤
│ Mean        │ 80.74 person-hours (11 person-days)                    │
│ Median (P50) │ 80.73 person-hours                                    │
│ Std Dev     │ 2.22 person-hours                                      │
│ Minimum     │ 74.13 person-hours                                     │
│ Maximum     │ 88.08 person-hours                                     │
└─────────────┴────────────────────────────────────────────────────────┘

Calendar Time Confidence Intervals:
┌────────────┬──────────┬──────────────┬──────────────────────────────┐
│ Percentile │    Hours │   Working Days │ Date                       │
├────────────┼──────────┼──────────────┼──────────────────────────────┤
│ P50        │   294.43 │           37 │ 2025-04-22                   │
│ P80        │   312.31 │           40 │ 2025-04-25                   │
│ P90        │   313.21 │           40 │ 2025-04-25                   │
│ P95        │   313.83 │           40 │ 2025-04-25                   │
└────────────┴──────────┴──────────────┴──────────────────────────────┘
```

The two-pass run reduces the mean project duration from ~**361 h to ~298 h** (~8 working days) purely by prioritising the tasks that pass-1 identified as critical bottlenecks.

11.3 When Is Two-Pass Useful?

Two-pass simulation is most valuable when **all of the following** are true:

Condition	Why it matters
Named resources are defined in the project file	Without resources, dispatch order has no effect
At least two tasks compete for the same resource	No contention → dispatch order is irrelevant
Some tasks have much higher criticality than others	Uniform criticality → reordering has little effect
The project has more than ~5–10 tasks	Very small projects often have no slack to reorder

Typical use-cases

- Software delivery teams where one or two senior engineers are on the critical path and also reviewed by junior work items
- Infrastructure projects where a specialist (network architect, DBA) is a shared bottleneck across multiple workstreams
- Hardware/software co-design where a scarce test bench is contended between parallel integration streams
- Any project where you suspect the default greedy schedule is sub-optimal

When two-pass has little effect

If every task has its own dedicated resource (no contention), the delta will be close to zero. Example with `abundant-resources.yaml` (each task on its own resource):

```
Two-Pass Scheduling Traceability:
  Pass-1 iterations: 300 | Pass-2 iterations: 500
  Mean: 189.6h (pass-1) → 189.2h (pass-2) [delta -0.3h]
  P80:  194.7h (pass-1) → 194.6h (pass-2) [delta -0.1h]
  P95:  197.1h (pass-1) → 196.8h (pass-2) [delta -0.3h]
  Resource wait delta: +0.0h
```

The near-zero delta confirms that two-pass adds no distortion when resources are not a bottleneck.

11.4 Configuration Reference

11.4.1 CLI flags

Flag	Default	Description
`--two-pass`	off	Enable two-pass mode for this run
`--pass1-iterations N`	1 000	Pass-1 iteration budget for criticality ranking

Both flags are accepted by the `simulate` command. `--pass1-iterations` requires `--two-pass` to have any effect.

```
# Basic two-pass run
mcprojsim simulate project.yaml --two-pass

# Fine-tuned: pass-1 with 2 000 iterations, pass-2 with 10 000
mcprojsim simulate project.yaml --two-pass --pass1-iterations 2000 -n 10000

# Export all formats
mcprojsim simulate project.yaml --two-pass -n 5000 -f json,csv,html -o report
```

11.4.2 Configuration file

You can set the assignment mode globally in your `~/.mcprojsim/configuration.yaml` or any config file passed with `--config`:

```
constrained_scheduling:
  assignment_mode: criticality_two_pass   # default: greedy_single_pass
  pass1_iterations: 1000                  # default: 1000
```

With this config, every `simulate` invocation uses two-pass mode automatically for projects that have resources, with no need for the `--two-pass` flag.

11.5 More Complex Example: Developer Allocation

Consider a project with three workstreams all sharing a DevOps engineer for deployment tasks:

```
project:
  name: "multi-stream-release"
  start_date: "2025-06-01"
  hours_per_day: 8
  confidence_levels: [50, 80, 90, 95]

tasks:
  # --- Stream A: Backend ---
  - id: "backend-dev"
    name: "Backend development"
    estimate: { low: 60, expected: 80, high: 120 }
    resources: ["backend-dev-1"]
  - id: "backend-deploy"
    name: "Backend deployment"
    estimate: { low: 8, expected: 12, high: 16 }
    dependencies: ["backend-dev"]
    resources: ["devops"]              # shared bottleneck

  # --- Stream B: Frontend ---
  - id: "frontend-dev"
    name: "Frontend development"
    estimate: { low: 40, expected: 60, high: 90 }
    resources: ["frontend-dev-1"]
  - id: "frontend-deploy"
    name: "Frontend deployment"
    estimate: { low: 6, expected: 8, high: 12 }
    dependencies: ["frontend-dev"]
    resources: ["devops"]              # shared bottleneck

# --- Stream C: Mobile ---
  - id: "mobile-dev"
    name: "Mobile development"
    estimate: { low: 80, expected: 100, high: 140 }
    resources: ["mobile-dev-1"]
  - id: "mobile-deploy"
    name: "Mobile deployment"
    estimate: { low: 8, expected: 10, high: 14 }
    dependencies: ["mobile-dev"]
    resources: ["devops"]              # shared bottleneck
# --- Integration ---
  - id: "smoke-test"
    name: "Smoke test all streams"
    estimate: { low: 8, expected: 12, high: 20 }
    dependencies: ["backend-deploy", "frontend-deploy", "mobile-deploy"]
    resources: ["qa-1"]
  - id: "release"
    name: "Production release"
    estimate: { low: 4, expected: 6, high: 10 }
    dependencies: ["smoke-test"]
    resources: ["devops"]
resources:
  - name: "backend-dev-1"
    experience_level: 3
  - name: "frontend-dev-1"
    experience_level: 2
  - name: "mobile-dev-1"
    experience_level: 2
  - name: "devops"
    experience_level: 3
  - name: "qa-1"
    experience_level: 2
```

Run with two-pass to discover which deployment stream should be prioritised:

```
mcprojsim simulate multi-stream.yaml --two-pass --pass1-iterations 2000 \
  -n 10000 -t -f json,html -o multi-stream-report
```

The TwoPassDelta block in the JSON output will show which stream's deploy task had the highest criticality index in pass-1, and how much the end-to-end schedule improved once pass-2 gave that stream's DevOps slot priority.

11.6 Output Formats

11.6.1 Console (`-t` table mode)

A **Two-Pass Scheduling Traceability** table is printed after the constrained diagnostics when `--two-pass` is active. The table shows pass-1, pass-2, and Δ values for Mean, P80, and P95. The resource wait delta is printed on a separate line below the table.

11.6.2 JSON export (`-f json`)

The `two_pass_traceability` key is always present. It is `null` for single-pass runs and an object for two-pass runs:

```
{
  "two_pass_traceability": {
    "enabled": true,
    "pass1_iterations": 500,
    "pass2_iterations": 1000,
    "ranking_method": "criticality_index",
    "pass1": {
      "mean_hours": 519.9,
      "p50_hours": 519.8,
      "p80_hours": 530.6,
      "p90_hours": 532.7,
      "p95_hours": 533.0,
      "resource_wait_hours": 212.2,
      "resource_utilization": 0.87,
      "calendar_delay_hours": 0.0
    },
    "pass2": {
      "mean_hours": 422.8,
      "p50_hours": 422.3,
      "p80_hours": 434.3,
      "p90_hours": 436.4,
      "p95_hours": 437.1,
      "resource_wait_hours": 90.2,
      "resource_utilization": 0.91,
      "calendar_delay_hours": 0.0
    },
    "delta": {
      "mean_hours": -97.0,
      "p50_hours": -97.5,
      "p80_hours": -96.3,
      "p90_hours": -96.3,
      "p95_hours": -95.9,
      "resource_wait_hours": -122.0,
      "resource_utilization": 0.04,
      "calendar_delay_hours": 0.0
    },
    "task_criticality_index": {
      "A1": 0.0,
      "A2": 0.0,
      "A3": 0.0,
      "B1": 1.0,
      "B2": 1.0,
      "C1": 1.0,
      "C2": 1.0
    }
  }
}
```


11.6.3 CSV export (`-f csv`)

A **Two-Pass Scheduling Traceability** section is appended after the constrained diagnostics block, followed by a **Task Criticality Index (pass-1)** section that lists each task's CI value.

11.6.4 HTML export (`-f html`)

The HTML report gains a **Two-Pass Scheduling Traceability** table with pass-1, pass-2, and Δ columns for each key metric, followed by a **Task Criticality Index (Pass 1)** table.

11.7 Choosing `pass1_iterations`

Pass-1 is used only to rank tasks; it does not directly affect the final results. A few practical guidelines:

Table 11.3: Recommendations

Scenario	Recommended `pass1_iterations`
Quick exploratory run	200–500
Standard analysis	500–2 000 (default 1 000)
Rigorous / publication-quality	2 000–5 000
Noisy project with many tasks	5 000+

The default of **1 000** is a good balance between ranking stability and computational cost for most projects. If you reduce `pass1_iterations` below 100 the engine logs a warning that the criticality indices may be noisy.

`pass1_iterations` is automatically capped to `--iterations` if it is larger.

11.8 Interpreting the Delta

The delta values tell you the **effect of smarter dispatch** on each reported metric, holding all other inputs constant:

Delta sign	Meaning
Negative (e.g. `-97 h`)	Pass-2 schedule is shorter → good, the reordering helped
Near zero	No contention, or all tasks equally critical → reordering made no difference
Positive	Unusual; could indicate that the pass-1 ranking was noisy (increase `pass1_iterations`)

A large negative delta on mean/P80 combined with a large negative delta on resource wait time is strong evidence that the project suffers from avoidable resource contention, and that assigning more resources or restructuring task ownership would reduce schedule risk.

11.9 Related chapters

- Resource and Calendar Constrained Scheduling for resource setup, calendar configuration, and the full constrained scheduling walkthrough
- Running Simulations for the complete CLI reference
- Interpreting Results for understanding constrained diagnostics

CHAPTER 12

Running simulations

This chapter is a reference for the `mcprojsim` command-line interface. It covers all four commands: `generate`, `validate`, `simulate`, and `config`.

12.1 Command overview

Command	Purpose
`mcprojsim generate`	Create a project YAML file from a natural language description
`mcprojsim validate`	Check a project file for errors without running a simulation
`mcprojsim simulate`	Run a Monte Carlo simulation and produce results
`mcprojsim config`	Display the active configuration

A typical workflow runs these commands in sequence: generate (or write by hand) → validate → simulate.

12.2 `mcprojsim generate`

Converts a plain-text project description into a syntactically correct YAML project file. The parser runs locally — no AI service or network access is required.

12.2.1 Usage

```
mcprojsim generate INPUT_FILE [-o OUTPUT_FILE] [--validate-only] [-v]
```

12.2.2 Options

Option	Description
`INPUT_FILE`	Path to a plain-text file containing the project description (required)
`-o`, `--output FILE`	Write the generated YAML to a file. If omitted, the YAML is printed to stdout
`--validate-only`	Check the description for issues without generating output
`-v`, `--verbose`	Show detailed informational messages (e.g. file loaded, tasks parsed)

12.2.3 Examples

```
# Print generated YAML to the terminal
mcprojsim generate description.txt

# Save to a file
mcprojsim generate description.txt -o my_project.yaml

# Validate the description without generating
mcprojsim generate --validate-only description.txt

# Generate and then simulate in one pipeline
mcprojsim generate description.txt -o project.yaml && mcprojsim simulate project.yaml
```

12.2.4 Input format

The input file is a semi-structured text description. See MCP Server & Natural Language Project Input for the full input format reference, supported fields, and examples.

A minimal example:

```
Project name: My Project
Start date: 2026-04-01
Task 1:
- Design phase
- Size: M
Task 2:
- Implementation
- Depends on Task 1
- Size: XL
```

12.3 `mcprojsim validate`

Checks a YAML or TOML project file for structural and semantic errors.

12.3.1 Usage

```
mcprojsim validate PROJECT_FILE [-v]
```

12.3.2 Options

Option	Description
`-v`, `--verbose`	Show detailed informational messages

12.3.3 Examples

```
mcprojsim validate project.yaml
```

Successful output:

```
✓ Project file is valid!
```

If the file has errors, the validator reports the issue and aborts:

```
✗ Validation failed:
  Task 'task_003' depends on 'task_999', which does not exist
```

It is good practice to validate before every simulation, especially after editing a file or generating one with `mcprojsim generate`.

12.4 `mcprojsim simulate`

Runs the Monte Carlo simulation on a validated project file.

12.4.1 Usage

```
mcprojsim simulate PROJECT_FILE [OPTIONS]
```

12.4.2 Options

Option	Description	Default
`-n`, `--iterations NUM`	Number of simulation iterations	10000
`-c`, `--config FILE`	Custom configuration file	built-in defaults
`-s`, `--seed INT`	Random seed for reproducibility	random
`-o`, `--output PATH`	Output file base path (without extension)	project name
`-f`, `--output-format FORMATS`	Comma-separated export formats: `json`, `csv`, `html`	none
`--critical-paths N`	Number of critical paths to display	config default: 2
`--number-bins N`	Number of histogram bins for distribution charts in JSON, CSV, and HTML exports. Overrides the config file setting if specified.	50
`--target-budget AMOUNT`	Report probability of staying within the provided budget amount	none
`--full-cost-detail`	Include per-iteration task-cost arrays in JSON output	off
`--no-fx`	Disable exchange-rate fetches for secondary currency reporting	off
`-q`, `-qq`, `--quiet`	Reduce CLI output verbosity. Use `-q` to suppress detailed output, or `-qq` to suppress all normal output	off
`-v`, `--verbose`	Show detailed informational messages (config loaded, project parsed, etc.) and additional output sections: Default Uncertainty Factors and Sensitivity Analysis	off
`-p`, `--progress`	Show a progress bar while the simulation is running	off
`-S`, `--simtime`	Print elapsed simulation time and peak memory usage after each run	off
`--minheader`	Show a compact two-line header (separator + version line) instead of the full project block	off
`--noheader`	Suppress the header block entirely; output starts directly with the Project Overview section	off
`-t`, `--table`	Format tabular output sections (confidence intervals, sensitivity, slack, risk impact, staffing) as ASCII tables	off
`-m`, `--minimal`	Show minimal console output: version, project overview, calendar/effort statistical summaries, and calendar confidence intervals only	off
`--staffing`	Show full staffing analysis table with team-size recommendations per experience profile	off
`--tshirt-category CATEGORY`	Override default T-shirt category for bare values like `M` during this run	config setting

Option	Description	Default
`--target-date DATE`	Target completion date (`YYYY-MM-DD`) to calculate probability of meeting	none
`--velocity-model MODEL`	Override the sprint planning velocity model for how to use historic data (`empirical` or `neg_binomial`). Applies only when sprint planning is enabled in the project file.	project file setting
`--no-sickness`	Disable sickness modelling regardless of the project file setting. Applies only when sprint planning is enabled.	off
`--include-historic-base`	Add a `Historic Base` section to HTML reports (historic summary + committed/completed bar chart) and include matching historic baseline rows/summary in JSON under `sprint_planning.historic_base`. **Requires `-f` to include `json` or `html`.**	off
`--two-pass`	Enable criticality two-pass scheduling. Only has effect when resource-constrained scheduling is active. Overrides the config file `assignment_mode` setting.	off
`--pass1-iterations N`	Number of pass-1 iterations for criticality ranking when `--two-pass` is used. Capped to `--iterations`.	1000 (from config)
`--workers N | auto`	Number of worker processes for parallel simulation. Pass a positive integer or `auto` to use all available CPUs. Ignored when iterations or task count is below the parallel threshold.	1 (sequential)

12.4.3 Examples

```
# Default simulation
mcprojsim simulate project.yaml

# Reproducible run with a seed and progress bar
mcprojsim simulate project.yaml --seed 42 -p

# More iterations for higher precision, show simulation timne and memory usage
mcprojsim simulate project.yaml --iterations 50000 --seed 42 --simtime

# Custom configuration
mcprojsim simulate project.yaml --config my_config.yaml

# Export to all formats
mcprojsim simulate project.yaml -f json,csv,html -o results/my_project

# Custom histogram bins for more granular distribution charts
mcprojsim simulate project.yaml -f json,csv,html --number-bins 100 -o results/my_project

# Budget probability analysis
mcprojsim simulate project.yaml --target-budget 45000

# Include full task-cost arrays in JSON export
mcprojsim simulate project.yaml -f json --full-cost-detail

# Disable FX lookups (offline/CI mode)
mcprojsim simulate project.yaml --no-fx

# Quiet mode (suppress detailed simulation results output)
mcprojsim simulate project.yaml --quiet

# Fully quiet mode (suppress all normal CLI output)
mcprojsim simulate project.yaml -qq

# Verbose mode (show config/project loading details)
mcprojsim simulate project.yaml --verbose

# Table mode (ASCII tables for tabular sections)
mcprojsim simulate project.yaml --table

# Minimal output mode (core summaries only)
mcprojsim simulate project.yaml --minimal

# Show staffing recommendations
mcprojsim simulate project.yaml --staffing

# Override bare T-shirt sizes to use the epic category
mcprojsim simulate project.yaml --tshirt-category epic

# Staffing with table formatting
mcprojsim simulate project.yaml --staffing --table
```

```
# Sprint planning: switch velocity model to negative binomial
mcprojsim simulate sprint_project.yaml --velocity-model neg_binomial

# Sprint planning: disable sickness modelling
mcprojsim simulate sprint_project.yaml --no-sickness

# Include Historic Base in HTML/JSON exports
mcprojsim simulate sprint_project.yaml -f json,html --include-historic-base
```

```
# Parallel simulation: use 4 worker processes
mcprojsim simulate project.yaml --workers 4 --seed 42

# Parallel simulation: use all available CPUs
mcprojsim simulate project.yaml --workers auto --seed 42

# Force sequential execution explicitly
mcprojsim simulate project.yaml --workers 1
```

12.4.4 Output

By default, the simulation prints a plain-text summary. With `--table` (`-t`), the project overview, statistical summaries, confidence intervals, sensitivity analysis, schedule slack, and risk impact sections are all formatted as ASCII tables with column headers and box-drawing borders:

```
Calendar Time Confidence Intervals:
┌──────────────┬─────────┬──────────────┬────────────┐
│ Percentile   │   Hours │   Working Days │ Date       │
├──────────────┼─────────┼──────────────┼────────────┤
│ P25          │  522.75 │             66 │ 2026-02-02 │
│ P50          │  571.84 │             72 │ 2026-02-10 │
│ P90          │  682.94 │             86 │ 2026-03-02 │
│ P99          │  790.61 │             99 │ 2026-03-19 │
└──────────────┴─────────┴──────────────┴────────────┘
```

Staffing output

A short staffing advisory is always included at the end of the output (unless `--quiet` is active):

```
Staffing (based on mean effort): 3 people recommended (mixed team), 38 working days
  Total effort: 910 person-hours (114 person-days) | Parallelism ratio: 1.6
```

By default the staffing analysis uses **mean** effort and elapsed time. To base it on a specific confidence level, set `staffing.effort_percentile` in your configuration file:

```
staffing:
  effort_percentile: 80  # use P80 effort and elapsed time
```

If `staffing.effort_percentile` is omitted, mean effort and mean elapsed time are used. The mean effort is calculated as the average total effort across all iterations, and the mean elapsed time is the average project duration. If the distribution is symmetric, the mean and median (P50) will be close, but for skewed distributions they can differ significantly.

If the percentile is set to a higher value like 80 or 90, the staffing recommendation will be based on a more conservative estimate of effort and elapsed time, which can lead to recommending more staff to mitigate the risk of overruns. A staffing percentile of 90, for example, would recommend a team size that has a 90% chance of completing the project within the estimated effort and time, providing a buffer against uncertainty.

The reason we say both effort and elapsed time is that the staffing model considers both how much total work there is (effort) and how long the project takes (elapsed time) to determine how many people are needed. A higher effort percentile means the model assumes a higher total workload, while a higher elapsed time percentile means it assumes a longer project duration. Both factors influence the recommended team size to ensure the project can be completed within the desired confidence level.

The reason they are not separated into `staffing.effort_percentile` and `staffing.elapsed_time_percentile` is that they are inherently linked in the staffing model. The number of staff needed depends on both the total effort and the project duration, so it makes sense to use the same confidence level for both to maintain consistency in the risk tolerance. If they were set separately, it could lead to conflicting recommendations (e.g., a high effort percentile but a low elapsed time percentile might recommend more staff than necessary, or vice versa). By using a single `effort_percentile`, we ensure that the staffing recommendation is based on a coherent risk profile.

With this setting the output changes to:

```
Staffing (based on P80 effort percentile): 3 people recommended (mixed team), 42 working days
  Total effort: 1,024 person-hours (128 person-days) | Parallelism ratio: 1.6
```

Using a higher percentile produces more conservative staffing (since we want to be more confident in our recommendations because it accounts for higher-effort scenarios).

Pass `--staffing` to expand this into a full table for each experience profile (senior, mixed, junior). With `--table`:

```
Staffing Analysis (senior team, overhead=4%/person, productivity=100%):
  Effort basis: mean (910 person-hours, 571 critical-path hours)
┌─────────────┬──────────────────┬──────────────┬─────────────────┬──────────────┐
│ Team Size   │ Eff. Capacity    │ Working Days │ Delivery Date   │ Efficiency   │
├─────────────┼──────────────────┼──────────────┼─────────────────┼──────────────┤
│ 1           │ 1.00             │ 114          │ 2026-06-15      │ 100.0%       │
│ 2           │ 1.92             │ 60           │ 2026-03-27      │ 96.0%        │
│ 3 *         │ 2.76             │ 42           │ 2026-03-02      │ 92.2%        │
└─────────────┴──────────────────┴──────────────┴─────────────────┴──────────────┘
```

The * suffix in the "Team Size" column marks the recommended team size for that profile. Without `--table`, the same information appears in plain indented text with `<-- recommended` annotations. See Interpreting Results — Staffing recommendations for a detailed explanation of the model and columns.

Without `--table`, the same data is printed as indented text:

```
=== Simulation Results ===

Project Overview:
Project: Website Refresh
Hours per Day: 8.0
Max Parallel Tasks: 2
Schedule Mode: dependency_only

Calendar Time Statistical Summary:
Mean: 126.78 hours (16 working days)
Median (P50): 125.36 hours
Std Dev: 18.01 hours
Minimum: 98.40 hours
Maximum: 192.30 hours
Coefficient of Variation: 0.1421
Skewness: 0.4123
Excess Kurtosis: -0.2341

Project Effort Statistical Summary:
Mean: 232.45 person-hours (30 person-days)
Median (P50): 229.80 person-hours
Std Dev: 34.22 person-hours
...

Calendar Time Confidence Intervals:
  P50: 125.36 hours (16 working days)  (2026-04-22)
  P80: 143.08 hours (18 working days)  (2026-04-24)
  P90: 151.74 hours (19 working days)  (2026-04-27)

Most Frequent Critical Paths:
  1. task_001 -> task_002 (10000/10000, 100.0%)

Staffing (based on mean effort): 3 people recommended (mixed team), 38 working days
  Total effort: 910 person-hours (114 person-days) | Parallelism ratio: 1.6
```

When export formats are specified with `-f`, the following files are created:

Format	Content
`json`	Full machine-readable results including histogram data

Format	Content
`csv`	Tabular summary with statistics, percentiles, and critical paths
`html`	Interactive report with thermometer chart and distribution histogram

12.5 `mcprojsim config`

Displays the active configuration (uncertainty factor multipliers, T-shirt size mappings, story point mappings, simulation defaults).

12.5.1 Usage

```
mcprojsim config [-c CONFIG_FILE] [--list] [--generate]
```

12.5.2 Options

Option	Description
`-c`, `--config FILE`	Show configuration merged with a custom YAML file
`--list`, `--show`	List current configuration settings. Alias `--show` is identical. When omitted, the same output is produced by default for backward compatibility.
`--generate`	Write the built-in default configuration to `~/.mcprojsim/config.yaml`

!!! note The auto-loaded user default configuration file (applied automatically when no `-c` flag is given to any command) lives at `~/.mcprojsim/config.yaml`. Use `--generate` to create a starter template at that path.

12.5.3 Examples

```
# Show built-in defaults (all three forms are equivalent)
mcprojsim config
mcprojsim config --list
mcprojsim config --show

# Show a custom configuration merged with defaults
mcprojsim config --config my_config.yaml
mcprojsim config --list --config my_config.yaml

# Generate a default configuration file at ~/.mcprojsim/config.yaml
mcprojsim config --generate
```

12.6 Parallel simulation

By default `mcprojsim simulate` runs iterations sequentially on a single process. For large iteration counts or computationally heavy projects you can distribute the work across multiple CPU cores with `--workers`.

12.6.1 How it works

When `--workers N` is set and both the iteration count and task count both exceed an absolute minimum thresholds (500 iterations and 5 tasks respectively) but also exceeds heuristic limits that takes how heavy the execution are into account. For example, a dependency-only simulation (no resource constrains) will almost never benefit from parallel execution as each iteration is trivial and most time will be spent in the parallel overhead. Hence, only one worker will be deployed in all but the most extreme dependency-only simulations.

If the heuristics finds that several workers are beneficial the engine will:

1. Partitions the total iterations into many small micro-chunks (more chunks than workers for smooth progress and load balancing).
2. Submits chunks to a short-lived `ProcessPoolExecutor` with `N` workers, using the `spawn` start method for safety on all platforms.
3. Merges results in deterministic order after all workers finish.
4. Runs the normal post-processing (`_build_results`, statistics, sensitivity analysis) in the parent process.

For smaller workloads the engine falls back to the sequential path automatically, so `--workers 4` on a tiny project has no overhead cost.

Progress reporting still works in parallel mode. When stdout progress is enabled, updates are emitted as chunks complete. If you use `progress_callback`, it remains active even when stdout progress is disabled. Single-pass runs report `iterations` as the total; two-pass runs report `pass1_iterations + iterations` as the total so callback progress stays monotonic across both phases. Parallel mode may therefore call the callback more frequently than sequential mode.

12.6.2 Reproducibility

Results are deterministic for a fixed `(--seed, --workers)` combination. Changing the number of workers changes how the random seed is partitioned across chunks, so the exact duration values will differ between a 1-worker run and a 4-worker run even with the same seed. To reproduce results exactly, use the same `--workers` value as the original run.

> The sequential path (`--workers 1`) is unchanged by this feature. Two runs with `--workers 1` and the same `--seed` produce bit-for-bit identical output.

12.6.3 Performance expectations

Parallel execution adds fixed overhead:

- Process pool startup and teardown (~50–200 ms per run on typical hardware).
- Argument serialisation and result deserialisation across worker processes.
- Serial post-processing (statistics, sensitivity analysis) after merge.

Because of this overhead, parallel mode is most beneficial for larger workloads:

Workload	Expected speedup on 8 cores
< 500 iterations or < 5 tasks	Falls back to sequential
~10 000 iterations, ~30 tasks	Roughly 2× to 4×
Heavy workloads with minimal post-processing	Up to ~4× to 5×

Benchmark your own project with `--workers 1`, `--workers 2`, `--workers 4`, and `--workers auto` to find the best setting.

12.6.4 Usage

```
# Use 4 worker processes
mcprojsim simulate project.yaml --workers 4 --seed 42

# Use all available CPUs
mcprojsim simulate project.yaml --workers auto --seed 42

# Force sequential (default)
mcprojsim simulate project.yaml --workers 1
```

CHAPTER 13

Interpreting Results

After running a simulation, `mcprojsim` produces a set of statistics, percentiles, critical path data, and optionally an HTML report with a histogram and thermometer chart. This chapter explains what those outputs mean and how to use them for practical decision-making.

13.1 The output at a glance

A typical CLI run

```
mcprojsim simulate --seed 42 examples/sample_project.yaml
```

produces the full output seen below. The sections are explained in detail in the following parts of this chapter.

```
01: Project Overview:
02:   Project: Customer Portal Redesign
03:   Start Date: 2025-11-01
04:   Number of Tasks: 8
05:   Effective Default Distribution: triangular
06:   T-Shirt Category Used: story
07:   Hours per Day: 8.0
08:   Max Parallel Tasks: 2
09:   Schedule Mode: dependency_only
10:
```

```
Calendar Time Statistical Summary:
  Mean: 580.89 hours (73 working days)
  Median (P50): 574.73 hours
  Std Dev: 78.46 hours
  Minimum: 365.50 hours
  Maximum: 924.02 hours
  Coefficient of Variation: 0.1351
  Skewness: 0.4798
  Excess Kurtosis: 0.1518

Project Effort Statistical Summary:
  Mean: 686.35 person-hours (86 person-days)
  Median (P50): 684.08 person-hours
  Std Dev: 61.54 person-hours
  Minimum: 487.94 person-hours
  Maximum: 902.14 person-hours
  Coefficient of Variation: 0.0897
  Skewness: 0.1210
  Excess Kurtosis: -0.1140

Calendar Time Confidence Intervals:
  P25: 524.00 hours (66 working days)  (2026-02-02)
  P50: 574.73 hours (72 working days)  (2026-02-10)
  P75: 629.98 hours (79 working days)  (2026-02-19)
  P80: 645.37 hours (81 working days)  (2026-02-23)
  P85: 663.34 hours (83 working days)  (2026-02-25)
  P90: 685.64 hours (86 working days)  (2026-03-02)
  P95: 722.71 hours (91 working days)  (2026-03-09)
  P99: 789.05 hours (99 working days)  (2026-03-19)

Effort Confidence Intervals:
  P25: 643.99 person-hours (81 person-days)
  P50: 684.08 person-hours (86 person-days)
  P75: 728.42 person-hours (92 person-days)
  P80: 738.74 person-hours (93 person-days)
  P85: 751.36 person-hours (94 person-days)
  P90: 767.83 person-hours (96 person-days)
  P95: 789.35 person-hours (99 person-days)
  P99: 833.24 person-hours (105 person-days)

Schedule Slack:
  task_008: 0.00 hours (Critical)
  task_006: 0.00 hours (Critical)
  task_005: 0.00 hours (Critical)
  task_004: 0.00 hours (Critical)
  task_001: 0.00 hours (Critical)
  task_002: 0.00 hours (Critical)
  task_003: 165.18 hours (165.2h buffer)
  task_007: 356.32 hours (356.3h buffer)

Risk Impact Analysis:
  task_001: mean=3.20h, triggers=20.0%, mean_when_triggered=16.00h
  task_003: mean=5.99h, triggers=24.9%, mean_when_triggered=24.00h
  task_004: mean=12.17h, triggers=30.4%, mean_when_triggered=40.00h
  task_006: mean=11.31h, triggers=35.3%, mean_when_triggered=32.00h
  task_008: mean=4.75h, triggers=19.8%, mean_when_triggered=24.00h

Most Frequent Critical Paths:
  1. task_001 -> task_002 -> task_004 -> task_005 -> task_006 -> task_008 (9999/10000, 100.0%)
  2. task_003 -> task_005 -> task_006 -> task_008 (1/10000, 0.0%)

Staffing (based on mean effort): 2 people recommended (mixed team), 73 working days
  Total effort: 686 person-hours (86 person-days) | Parallelism ratio: 1.2
```

Every number here comes from simulating your project thousands of times with different random samples. The results describe the *distribution* of outcomes, not a single prediction.

The sections below explain each part of this output in detail.

13.2 Output modes and flags that change what you see

Several `simulate` flags change which sections are printed:

Flag	Effect on output
`--minimal`	Shows only core overview and confidence intervals; omits detailed analyses (sensitivity, slack, risk impact, etc.)
`--table`	Renders sections as ASCII tables instead of plain text lines
`--quiet`, `-q`	Reduces or suppresses most normal console output
`-qq`	Suppresses all console output except errors
`--staffing`	Adds full per-profile staffing tables (`senior`, `mixed`, `junior`, plus custom profiles)
`--critical-paths N`	Changes how many full critical path sequences are listed
`--target-date YYYY-MM-DD`	Adds probability of finishing by that date
`--include-historic-base`	Adds Historic Base payload/sections in JSON/HTML exports (requires output format including `json` or `html`)

Two additional flags affect output only when resource-constrained scheduling is active:

Flag	Effect on output
`--two-pass`	Enables criticality two-pass scheduling
`--pass1-iterations`	Sets pass-1 sample size used for two-pass ranking

Because of these switches, not every run prints every section shown in this chapter.

13.3 Project overview fields

At the top of simulation results, `mcprojsim` prints core context fields before detailed analyses.

- `Project`
- `Hours per Day`
- `Max Parallel Tasks`
- `Schedule Mode`

`Schedule Mode` tells you whether the run used dependency-only scheduling or resource/calendar-constrained scheduling. Read all downstream sections in that context.

13.4 Understanding percentiles

Percentiles are the most actionable part of the output. Each percentile answers the question: *"How many hours (or days) would be enough to finish the project X% of the time?"*

Percentile	Practical meaning
P50	The median — half the simulated outcomes finished faster, half slower. This is an aggressive target; you have a coin-flip chance of meeting it.
P75	Three out of four simulations finished by this point. A reasonable internal planning target for teams comfortable with some risk.
P80	Four out of five simulations finished here. Often used as the default commitment for internal stakeholders.

Percentile	Practical meaning
P90	Nine out of ten simulations finished. A common choice for external commitments and contractual deadlines.
P95	Only 1 in 20 simulations exceeded this. A conservative target suitable for high-stakes commitments where a miss is costly.
P99	Virtually worst-case. Useful for capacity planning or budget ceilings, but not a realistic daily target.

13.4.1 Choosing the right percentile

There is no universally correct percentile. The right choice depends on the cost of being late versus the cost of over-committing:

- **Low-stakes internal milestone**: P75–P80 is usually sufficient. If you slip, you adjust.
- **Customer-facing deadline**: P85–P90 gives a meaningful safety margin without excessive padding.
- **Contractual commitment with penalties**: P90–P95 is appropriate when the consequence of a miss is significant.
- **Budget ceiling or worst-case planning**: P95–P99 helps answer "what's the most this could possibly cost?"

A useful rule of thumb: commit externally at the P85–P90 level, plan internally at P50, and track the gap between them. If the gap is large, your project has high uncertainty — which is itself valuable information.

13.4.2 Working days and delivery dates

Each percentile also shows working days (hours ÷ hours_per_day, rounded up) and a projected delivery date that counts forward from `start_date`, skipping weekends. These are calculated from the hours figure — they are not separate estimates.

13.5 Effort confidence intervals

The **Effort Confidence Intervals** section appears immediately after the calendar confidence intervals. While the calendar percentiles answer *"how long will this project take?"*, the effort percentiles answer *"how much total work will this project require?"*

```
Effort Confidence Intervals:
  P50: 910.15 person-hours (114 person-days)
  P80: 992.18 person-hours (125 person-days)
  P90: 1046.80 person-hours (131 person-days)
  P95: 1091.24 person-hours (137 person-days)
```

13.5.1 What is total effort?

Total effort is the sum of all task durations in a single simulation iteration — the total person-hours of *work* required. This differs from calendar/elapsed time because tasks that run in parallel still each consume their full person-hours.

For example, if two 40-hour tasks run at the same time, the elapsed time is 40 hours but the total effort is 80 person-hours. In projects with high parallelism, effort is always larger than elapsed time.

13.5.2 Why effort varies across iterations

Each Monte Carlo iteration samples different task durations from the estimate ranges. Some iterations draw longer durations (more total work), others draw shorter ones. Risks that fire in some iterations add extra effort. The effort percentiles capture this variation:

- **P50 effort**: Half the simulations needed less than this much total work, half needed more.
- **P80–P90 effort**: A high-confidence estimate of total person-hours required.
- **P95–P99 effort**: Near-worst-case total work, driven by wide estimate ranges and risk events firing.

13.5.3 Effort vs. calendar time

The **parallelism ratio** connects the two:

$$\text{Parallelism ratio} = \frac{\text{Mean total effort}}{\text{Mean elapsed time}}$$

A ratio of 1.0 means all tasks are serial (effort = elapsed time). A ratio above 1.0 means tasks overlap, so effort exceeds elapsed time. The sample output shows a parallelism ratio of 1.57, meaning about 57% more work is done than the wall clock suggests because of concurrent tasks.

13.5.4 Practical use

Use case	Which metric to use
Scheduling and deadlines	Calendar confidence intervals (P80–P90 elapsed hours)
Budgeting and cost estimation	Effort confidence intervals (P80–P90 person-hours)
Staffing decisions	Effort percentiles tell you how many person-hours to fund; the staffing table maps that to team sizes
Comparing estimates to actuals	Track both: actual elapsed time against calendar P50, and actual time-sheet hours against effort P50

13.6 Minimum and maximum

The CLI reports observed minimum and maximum simulated durations as part of the **Calendar Time Statistical Summary** block, alongside mean, median, and std dev.

- **Minimum** is the fastest sampled project completion across iterations.
- **Maximum** is the slowest sampled completion.

These are useful for sanity checking tails, but they are sample extremes, not commitment targets. Use percentiles (P80/P90/P95) for planning decisions.

The **Project Effort Statistical Summary** block includes the same fields (minimum/maximum person-hours) for total work.

13.7 The thermometer chart

The HTML report includes two thermometers — vertical colour-coded bars showing elapsed time and effort (person-hours) at probability levels from 50% to 99%. They provide a quick visual: green zones are highly likely to succeed, orange zones are risky.

The colour transitions as a continuous gradient from dark orange (50%) to dark green (99%):

- **Dark orange** (50%–70%): aggressive targets. You are more likely than not to miss these.
- **Olive/amber tones** (75%–85%): moderate confidence. Reasonable for internal targets.
- **Green → dark green** (90%–99%): high confidence. Suitable for external commitments.

The thermometer has 11 fixed segments at probability levels 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, and 99. The colour of each segment is determined solely by the probability value — it is a fixed gradient, not configurable thresholds.

13.8 Mean vs. median

The **mean** (average) and **median** (P50) are often close, but they diverge when the distribution is skewed — which it almost always is in software projects, because tasks tend to overrun more than they finish early.

- When **mean > median**, the distribution has a right tail (some iterations produced long durations pulling the average up). This is normal.

- A large gap between mean and median signals high asymmetry. It suggests a few bad-case scenarios are contributing disproportionately to the average.

For planning, the **median** is generally more useful than the mean. The mean can be distorted by outliers, while the median always represents the 50th-percentile outcome.

13.9 Standard deviation and coefficient of variation

The **standard deviation** measures the spread of outcomes in hours. By itself, the number is hard to interpret — 50 hours of spread means something different for a 100-hour project than for a 1000-hour one.

The **coefficient of variation** (CV) normalises the spread so that you can compare uncertainty across projects of different sizes.

13.9.1 Definition

$$\text{CV} = \frac{\sigma}{\mu}$$

where σ is the standard deviation and μ is the mean project duration.

13.9.2 Interpretation

CV	Interpretation
< 0.15	Low uncertainty — estimates are tight and risks are well-contained
0.15–0.30	Moderate uncertainty — typical for well-scoped projects with some unknowns
0.30–0.50	High uncertainty — wide estimate ranges or significant risk exposure
> 0.50	Very high uncertainty — the project timeline is essentially unpredictable at this stage

In the sample output, `CV = 0.1353` indicates low uncertainty: the spread of outcomes is about 13.5% of the mean.

13.9.3 What to watch out for

- **CV increasing across re-estimates**: If the CV grows as you refine estimates, new information is revealing wider uncertainty than originally assumed. This is healthy — it means the model is becoming more honest.
- **Very low CV (< 0.05)**: Suspiciously tight. Check whether estimate ranges are artificially narrow (e.g., `min` and `max` almost equal) or whether risks have been omitted.
- **CV > 0.50**: The schedule is so uncertain that committing to any specific date is risky. Consider breaking the project into phases and re-estimating each one.

13.9.4 Practical use

A high CV is not necessarily bad — it is *honest*. It tells you that the inputs contain wide estimate ranges, high-probability risks, or both. Narrowing the CV means improving the inputs: tightening estimates, reducing risk exposure, or breaking large tasks into better-understood pieces. Use the CV to compare uncertainty across different project proposals: a project with CV 0.40 carries much more schedule risk than one with CV 0.15, regardless of their absolute durations.

13.10 Skewness

Skewness measures how asymmetric the distribution of project durations is. In software projects, distributions almost always skew right: there are more ways for things to go wrong than to go better than planned.

13.10.1 Definition

$$\text{Skewness} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i - \mu}{\sigma}\right)^3$$

A positive value means the distribution has a longer right tail (late finishes are further from the median than early finishes). A negative value means the opposite. Zero means perfect symmetry.

13.10.2 Interpretation

Skewness	Meaning
~ 0	Roughly symmetric — early and late outcomes are about equally spread around the median
0.1–0.5	Mild right skew — a slight tendency toward late outcomes, which is typical
0.5–1.0	Moderate right skew — the mean is noticeably higher than the median; a few bad scenarios are pulling the average up
> 1.0	Strong right skew — significant tail risk; worst-case outcomes are much worse than typical outcomes

In the sample output, `Skewness = 0.4865` indicates moderate right skew: some iterations hit notably longer durations than the median, but the effect is not extreme.

13.10.3 What to watch out for

- **Skewness > 1.0**: The project has significant tail risk. The mean is a poor planning target — use the median (P50) instead, and pay close attention to P90/P95.
- **Negative skewness**: Unusual in project estimation. It may indicate that `max` estimates are too conservative relative to `min`, or that uncertainty factors are compressing durations at the high end.
- **Skewness combined with high CV**: When both are high, the project has both wide variance and a long tail. This is a red flag for external commitments.

13.10.4 Practical use

When skewness is positive, the mean overstates a "typical" outcome. Prefer the median for internal planning. The gap between mean and median quantifies how much the tail is pulling the average: a gap of 10+ hours suggests meaningful tail risk that should be communicated to stakeholders.

13.11 Excess kurtosis

Excess kurtosis describes the "tailedness" of the distribution — whether extreme outcomes (very early or very late finishes) are more or less likely than a normal distribution would predict.

13.11.1 Definition

$$\text{Excess Kurtosis} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i - \mu}{\sigma}\right)^4 - 3$$

The -3 subtracts the kurtosis of a normal distribution, so that a normal distribution has excess kurtosis of zero.

13.11.2 Interpretation

Excess kurtosis	Meaning
~ 0	Tail behaviour similar to a normal distribution
> 0 (leptokurtic)	Heavier tails — extreme outcomes are more frequent than normal. Surprises (both good and bad) happen more often
< 0 (platykurtic)	Lighter tails — outcomes are more concentrated around the mean. Fewer surprises

In the sample output, `Excess Kurtosis = 0.2469` is slightly positive, meaning the tails are marginally heavier than a normal distribution — a small number of iterations produced notably long (or short) durations.

13.11.3 What to watch out for

- **Kurtosis > 2**: Heavy tails. The P95 and P99 values may be much higher than P90 suggests. Check the high percentiles carefully before making commitments at those levels.
- **Kurtosis < −1**: Very light tails. This may indicate that estimate ranges are too narrow or that the model lacks risks that could cause extreme outcomes.
- **Kurtosis combined with high skewness**: When both are high and positive, the distribution has a fat right tail — rare but severe overruns are plausible.

13.11.4 Practical use

Excess kurtosis guides how much buffer to add between your planning target and your worst-case commitment. When kurtosis is high, the gap between P90 and P99 is larger than usual, so committing at P90 leaves more residual risk than it would for a normal-tailed distribution. Conversely, negative kurtosis means outcomes are bunched together and P90 commitments are safer than they might first appear.

13.12 Sensitivity analysis

The sensitivity analysis section ranks tasks by how strongly their duration influences the total project duration. It answers the question: *"Which task's estimate uncertainty matters most to the overall schedule?"*

13.12.1 Definition

Sensitivity is measured using Spearman's rank correlation coefficient (ρ) between each task's sampled durations and the project's total duration across all iterations:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

where d_i is the difference between the rank of the task duration and the rank of the project duration in iteration i, and n is the number of iterations.

The sign indicates the direction of the relationship. A positive value (the norm) means that when this task takes longer, the project takes longer. The magnitude (0 to 1) indicates the strength.

13.12.2 Interpretation

Correlation	Meaning
$> +0.5$	Strong positive driver — this task's variance is a major contributor to project variance
$+0.2$ to $+0.5$	Moderate driver — contributes meaningfully to schedule uncertainty
-0.1 to $+0.2$	Weak or negligible — the task has little influence on overall duration
< -0.1	Inverse relationship — unusual and worth investigating (possibly a non-critical task that shares resources)

In the sample output, `task_004: +0.4322` is the strongest driver, meaning its duration variance contributes the most to the spread of project outcomes. Tasks near zero (like `task_007: +0.0062`) have essentially no influence — even large changes to their estimates would not move the project schedule.

13.12.3 What to watch out for

- **Several tasks above +0.3**: Schedule uncertainty is spread across multiple tasks. Reducing variance on any single task has limited effect — you need to address several.
- **One dominant task (> +0.7)**: This task essentially controls the schedule. Focus estimation refinement and risk mitigation there first.
- **Negative correlations**: Usually means the task is on a non-critical branch. When it runs long, a different branch becomes critical, and the total project time may stay the same or even decrease (rare).

13.12.4 Practical use

Use sensitivity to prioritise where to invest in better estimates. Spending a day refining the estimate for a +0.45 task reduces overall schedule uncertainty far more than refining a +0.02 task. In the HTML report, the sensitivity analysis appears as a tornado chart for visual comparison.

13.12.5 Reading the tornado chart

The tornado chart is a ranked horizontal bar chart of task sensitivity values.

How to read it:

1. **Bar length = influence strength**: longer bars mean stronger influence on total project duration.
2. **Signed value = direction**: bars to the right of zero indicate positive correlation (longer task -> longer project), while bars to the left indicate negative correlation (uncommon inverse relationship).
3. **Top bars = highest priority**: tasks are sorted by absolute correlation, so the most influential tasks are grouped as the key schedule drivers.
4. **Colour = direction cue**: positive and negative bars use different colours to make direction easier to scan quickly.

Scope note:

- The chart is a prioritisation view, not a full dump of every task. It shows the most influential tasks (up to 15) to keep the chart readable.

Interpretation rule of thumb:

- Start at the top bar and work downward.
- Focus first on tasks that are both long bars and operationally important (for example, low slack or high risk impact).
- Treat small bars near zero as lower-leverage targets, even if their raw estimates look uncertain.

13.13 Schedule slack

Schedule slack (also called total float) shows how much each task can be delayed without delaying the overall project. It is calculated using a backward-pass Critical Path Method (CPM) and then averaged across all Monte Carlo iterations.

13.13.1 Definition

For each task, the total float is:

$$\text{Slack} = LS - ES$$

where LS is the latest start time (computed via the backward pass from the project end) and ES is the earliest start time (from the forward pass). The value reported is the mean slack across all simulation iterations.

13.13.2 Interpretation

- **0.00 hours (Critical)**: The task is on the critical path in most or all iterations. Any delay to this task delays the project.
- **Positive slack (buffer)**: The task can overrun by up to this many hours without affecting the project delivery date. The larger the buffer, the less you need to worry about this task.

In the sample output, `task_003` has 165.16 hours of slack and `task_007` has 355.57 hours — these tasks could each overrun significantly without moving the project deadline. Meanwhile, six tasks show zero slack, confirming they are consistently on the critical path.

13.13.3 What to watch out for

- **All tasks showing zero slack**: The project is a single serial chain with no parallelism. Any task delay equals a project delay. Consider restructuring dependencies to create parallel paths.
- **Very small slack (< 8 hours / 1 day)**: The task is *almost* critical. It appears on the critical path in some iterations. Treat it with similar urgency to zero-slack tasks.
- **Slack changing across runs**: If the same task has very different slack with different seeds, the critical path is unstable — the project has competing bottleneck paths, and which one dominates depends on which sampled durations happen to be long.

13.13.4 Practical use

Slack tells you where you have scheduling flexibility. Tasks with large buffers are safe to deprioritise, start later, or assign to less experienced team members. Tasks with zero slack are where delays are most costly — prioritise them in resource allocation, assign your strongest people, and mitigate their risks first.

13.14 Risk impact analysis

The risk impact analysis section shows how much each task's risks contribute to the schedule across all simulation iterations. This goes beyond listing risks in the project file — it shows what actually happened during the simulation.

13.14.1 Reported metrics

For each task that has risks defined, three values are reported:

Metric	Definition
mean	Average risk impact across all iterations (in hours), including iterations where the risk did not trigger: mean $= \frac{1}{n}\sum_{i=1}^{n} \text{impact}_i$

Metric	Definition
triggers	Trigger rate — the fraction of iterations where at least one risk on this task fired: triggers = $\frac{\text{count(impact>0)}}{n}$
mean_when_triggered	Average impact in only the iterations where a risk actually fired: mean_when_triggered = $\frac{\sum_{i:\text{impact}_i>0} \text{impact}_i}{\text{count(impact>0)}}$

13.14.2 Interpretation

In the sample output:

```
task_004: mean=11.74h, triggers=29.3%, mean_when_triggered=40.00h
```

This means: in 29.3% of the 10,000 iterations, at least one of `task_004`'s risks fired. When they did, they added an average of 40 hours. Across *all* iterations (including the 70.7% where nothing happened), the average impact was 11.74 hours.

The gap between `mean` and `mean_when_triggered` reveals whether a risk is a frequent nuisance or a rare catastrophe:

- **High trigger rate, low impact when triggered**: A frequent but manageable disruption. The overall mean is close to `mean_when_triggered`.
- **Low trigger rate, high impact when triggered**: A "black swan" event. The overall mean looks low, but when it hits, it hits hard. These deserve contingency planning.

13.14.3 What to watch out for

- **High trigger rate (> 50%)**: The risk is more likely than not. Consider whether it should be modelled as part of the base estimate rather than as a risk.
- **Large `mean_when_triggered` on a critical-path task**: This combination has the highest potential to blow the schedule. If `task_004` is critical (zero slack) and adds 40 hours when its risk fires, that is a full working week added to the project almost a third of the time.
- **Tasks with no risk output**: These tasks have no risks defined. If they have high sensitivity scores, consider whether risks were overlooked.

13.14.4 Practical use

Combine risk impact analysis with sensitivity and slack to decide where to act:

1. If a task has **high sensitivity + zero slack + high trigger rate** → top priority for risk mitigation.
2. If a task has **high `mean_when_triggered` but low trigger rate** → have a contingency plan ready, but don't rearrange the whole schedule for it.
3. If a task has **large slack** → its risks are mostly absorbed by the buffer. Monitor, but don't prioritise.

13.15 Critical path analysis

13.15.1 What is the critical path?

The critical path is the longest chain of dependent tasks that determines when the project finishes. If any task on the critical path takes longer, the whole project takes longer. Tasks *not* on the critical path have slack — they can overrun without affecting the delivery date, up to a point.

13.15.2 Why Monte Carlo gives multiple critical paths

In a deterministic schedule there is one critical path. In a Monte Carlo simulation, each iteration samples different task durations, so the critical path can shift. A task that is critical in one iteration may have slack in another.

`mcprojsim` tracks this across all iterations and reports:

1. **Criticality index** per task — the fraction of iterations where that task appeared on a critical path. A criticality of 0.85 means the task was critical in 85% of simulations.
2. **Most frequent critical path sequences** — the full path (e.g., $A \to C \to E$) and how often it occurred.

13.15.3 How to use criticality

- **Criticality > 0.7**: This task is almost always on the critical path. It is a bottleneck. Focus risk mitigation, staffing, and management attention here.
- **Criticality 0.3–0.7**: Sometimes critical, sometimes not. Worth monitoring, but not the primary bottleneck.
- **Criticality < 0.3**: Rarely critical. This task has significant slack in most scenarios.

If multiple tasks share high criticality, the project has several competing bottleneck paths. That actually *reduces* schedule risk compared to a single dominant path, because no one task controls the outcome.

13.16 Max parallel tasks

The **Max parallel tasks** line reports the highest number of tasks that were scheduled to run at the same time in any single simulation iteration. The scheduler places every task at the earliest moment its dependencies allow, so two tasks without a dependency relationship can overlap.

13.16.1 How it is calculated

After each iteration schedules tasks, a sweep-line algorithm scans through the start and end events of every task. A task is considered active during the half-open interval $[\text{start}, \text{end})$, so a task ending at the exact moment another begins does **not** count as concurrent. The peak concurrent count from each iteration is recorded, and the overall maximum across all iterations is reported.

13.16.2 Interpretation

Value	Meaning
1	All tasks form a single serial chain — no parallelism.
2–3	Limited parallelism. A small team can handle the concurrent work.
4+	Significant parallelism. Verify that you actually have enough people or capacity to run this many tasks simultaneously.

In the sample output, `Max Parallel Tasks: 3` means the scheduler assumed up to three tasks running at the same time in at least one iteration.

13.16.3 Why this matters

Interpretation depends on schedule mode:

- In **dependency-only mode**, peak parallelism reflects structural parallelism implied by dependencies and assumes unlimited resource capacity.
- In **resource-constrained mode**, scheduling already accounts for resource availability and calendars.

Use this number as a sanity check:

- If **dependency-only mode** and **max parallel tasks <= team size**: the unconstrained parallelism assumption is plausible.
- If **dependency-only mode** and **max parallel tasks > team size**: the run is optimistic relative to your real capacity.
- If **resource-constrained mode**: combine this with constrained diagnostics (below) to understand where waiting and calendar effects are occurring.

13.17 Constrained schedule diagnostics

When resource/calendar constraints are active, the CLI adds:

- **Average Resource Wait (hours)**
- **Effective Resource Utilization**
- **Calendar Delay Contribution (hours)**

These quantify how much elapsed time comes from waiting for available resources and working-time calendars, not just raw task effort.

Use these diagnostics to decide whether to add capacity, rebalance assignments, or adjust calendars.

13.18 Two-pass scheduling traceability

When `--two-pass` is enabled (or equivalent config mode is active), output may include a traceability block comparing pass-1 and pass-2 results.

Reported deltas include:

- mean duration (`pass-1 -> pass-2`)
- P80 duration (`pass-1 -> pass-2`)
- P95 duration (`pass-1 -> pass-2`)
- resource wait delta

This helps explain whether two-pass prioritization materially improved (or worsened) schedule outcomes for your project structure.

13.19 Target-date probability

If you pass `--target-date YYYY-MM-DD`, the CLI prints the estimated probability of finishing by that date.

This converts an external deadline question into a direct probabilistic statement and is often the clearest communication artifact for stakeholders.

13.20 Staffing recommendations

The staffing advisory appears at the bottom of every simulation run. It answers the question: *"Given this project's parallelism and effort, how many people should work on it?"*

13.20.1 The advisory line

```
Staffing (based on mean effort): 3 people recommended (mixed team), 38 working days
  Total effort: 910 person-hours (114 person-days) | Parallelism ratio: 1.6
```

This tells you:

- **Recommended team size**: The team size that achieves the shortest calendar duration for this profile. The search stops at the first local minimum: if adding one more person would make the schedule longer (because communication overhead outweighs the extra capacity — a manifestation of Brooks's Law), the current size is chosen. If all team sizes within the display range keep improving, the largest shown size is recommended.
- **Experience profile**: The profile used for the quick advisory ("mixed" by default). See below for a comparison of all profiles.
- **Working days**: Estimated calendar duration with the recommended team, accounting for communication overhead.
- **Total effort**: The sum of mean task durations, measured in person-hours. This is the amount of *work* regardless of how many people do it.
- **Parallelism ratio**: Total effort divided by critical-path duration. A ratio of 1.0 means the project is entirely serial; higher values indicate potential for parallel work.

- **Effort basis**: Shown in parentheses after "Staffing". By default the analysis is based on **mean** effort and elapsed time. Set `staffing.effort_percentile` in configuration (e.g. `80`) to use a specific percentile instead — this produces more conservative recommendations for risk-averse planning.

13.20.2 The `--staffing` flag

Pass `--staffing` to see a detailed table for each experience profile:

```
=== Staffing Analysis ===

--- senior ---
Team Size  Eff. Capacity  Working Days  Delivery Date  Efficiency
        1           1.00           114     2026-06-15       36.9%
        2           1.92            60     2026-03-27       70.9%
        3*          2.76            42     2026-03-02      100.0%   <-- recommended

--- mixed ---
Team Size  Eff. Capacity  Working Days  Delivery Date  Efficiency
        1           0.85           134     2026-07-14       37.9%
        2           1.60            72     2026-04-13       71.2%
        3*          2.24            51     2026-03-16      100.0%   <-- recommended

--- junior ---
Team Size  Eff. Capacity  Working Days  Delivery Date  Efficiency
        1           0.65           176     2026-10-08       39.7%
        2           1.20            96     2026-06-12       73.0%
        3*          1.64            70     2026-05-06      100.0%   <-- recommended
```

Each column is explained below.

13.20.3 Communication overhead model (Brooks's Law)

Adding people to a project increases communication overhead — a concept famously expressed in Fred Brooks's *The Mythical Man-Month*. `mcprojsim` models this with a simple formula:

$$E(n) = n \cdot \max(p_{\min},\ 1 - c \cdot (n-1)) \cdot f$$

where:

Symbol	Meaning	Default values
n	Team size	—
c	Per-person communication overhead	0.04 (senior), 0.06 (mixed), 0.08 (junior)
f	Productivity factor for the experience profile	1.00, 0.85, 0.65
$p_{\min}$	Floor on individual productivity	0.25
$E(n)$	Effective capacity in "ideal-person" units	—

The individual productivity of one person on an n-person team is $\max(p_{\min},\ 1 - c \cdot (n-1))$. This decreases linearly with team size until it hits the floor.

Calendar duration is then:

$$T(n) = \max\left(T_{\text{CP}},\ \frac{W}{E(n)}\right)$$

where W is total effort (person-hours) and T_{CP} is the critical-path duration — an irreducible lower bound that no amount of parallelism can shorten.

13.20.4 Understanding the table columns

Column	Meaning
Team Size	Number of people
Eff. Capacity	$E(n)$ — the team's output in "ideal-person" equivalents after overhead
Working Days	Calendar working days at this team size
Delivery Date	Projected date (weekends excluded), or blank if no `start_date`
Efficiency	How close this team size is to the fastest possible schedule for this profile, expressed as a percentage. Calculated as min calendar hours/this row's calendar hours. **100 %** means this team achieves the minimum calendar duration (it is the optimal size or tied at the critical-path floor). Values below 100 % indicate the team is either too small (parallelism is underutilised) or too large (communication overhead has exceeded the gain from extra capacity).

The row marked with * or `<-- recommended` is the optimal team size for that profile.

13.20.5 Experience profiles

Three built-in profiles model different team compositions:

Profile	Productivity factor (f)	Overhead (c)	Description
senior	1.00	0.04	Experienced team — high output, low overhead
mixed	0.85	0.06	Typical blend of senior and junior — moderate overhead
junior	0.65	0.08	Less experienced team — higher coordination cost

Profiles are fully configurable. In a YAML config, override or add profiles under `staffing.experience_profiles`:

```
staffing:
  effort_percentile: 80  # use P80 effort for conservative staffing (omit for mean)
  min_individual_productivity: 0.25
  experience_profiles:
    senior:
      productivity_factor: 1.0
      communication_overhead: 0.04
    mixed:
      productivity_factor: 0.85
      communication_overhead: 0.06
    my_custom_team:
      productivity_factor: 0.90
      communication_overhead: 0.05
```

13.20.6 Choosing an effort basis

By default, the staffing model operates on **mean** values — mean total effort and mean elapsed (critical-path) time. This gives a "most likely" recommendation.

For risk-averse planning, set `staffing.effort_percentile` to a higher percentile (e.g. 80 or 90). When set, the model uses:

- **P*N* total effort** instead of mean total effort
- **P*N* elapsed time** instead of mean elapsed time

This results in more people or more calendar time, but protects against schedules that only work in the average case. The chosen basis is always shown in the output so there is no ambiguity.

Setting	Behaviour	Use when
(omitted)	Mean effort and elapsed time	You want the most likely recommendation
`effort_percentile: 50`	Median effort / elapsed time	Similar to mean for symmetric distributions
`effort_percentile: 80`	P80 effort / elapsed time	Moderate risk aversion
`effort_percentile: 90`	P90 effort / elapsed time	Conservative / high-stakes projects

13.20.7 How efficiency is calculated

The **Efficiency** column answers: *"How close is this team size to the best achievable schedule for this profile?"*

$$\text{Efficiency}(n) = \frac{T_{\min}}{T(n)}$$

where $T_{\min}$ is the shortest calendar duration found across all team sizes for this profile, and $T(n)$ is the calendar duration at team size n.

- **100 %**: This team size achieves the minimum calendar duration. The recommended team (*) always shows 100 %.
- **Below 100 %**: The schedule is longer than the optimum. Two different situations can cause this:
  - **Too few people** (left of the optimal): The project is *effort-bound* — parallelism is underutilised and adding more people would shorten the schedule.
  - **Too many people** (right of the optimal): The project is in the *Brooks's Law diminishing-returns zone* — communication overhead now outweighs the benefit of extra capacity, so the schedule actually lengthens.

13.20.8 Interpreting the profile tables together

A typical profile shows a "U-shaped" efficiency curve (when viewed from left to right): low efficiency at team size 1, rising as team size grows, peaking at 100 % at the recommended size, then potentially falling if the overhead regime kicks in.

Efficiency pattern	What it means
Monotonically rising to 100 % (no drop)	The project never enters the overhead regime within the displayed range. All shown team sizes are effort-bound; the last is the fastest.
Peaks at 100 % then drops	The optimal team is in the middle of the range. Teams beyond the peak are actively slowed by overhead.
All rows near 100 %	The project is dominated by the critical path; adding people makes little difference to calendar time.

- **Recommended size = 1**: The project is almost entirely serial (parallelism ratio near 1.0) or the total effort is small. Adding people just adds overhead.
- **Recommended size = max parallel tasks**: The project has enough parallelism to keep everyone busy. This is the ideal scenario.

- **Senior team recommends more people than junior**: A senior team has lower overhead per person, so it can grow larger before the overhead outweighs the benefit.
- **Efficiency drops below 70%**: Adding more people is subject to strongly diminishing returns. Consider splitting the project into independent work streams instead.

13.20.9 Caveats

- The model assumes all team members can work on any task. In practice, specialisation may constrain who does what.
- It does not account for ramp-up time: a new team member joining mid-project may take weeks to reach full productivity.
- The communication overhead is linear in team size. Real-world overhead grows faster (some models use $n(n-1)/2$ communication channels); the linear model is a conservative simplification.
- The simulation already assumes unlimited parallelism, so the staffing recommendation does not change the simulated durations — it is an advisory layer on top of the existing results.

13.21 Combining the analyses

The real power of these metrics comes from reading them together. Here is a decision framework:

Situation	Action
High sensitivity + zero slack + high risk trigger rate	**Top priority.** This task drives the schedule, sits on the critical path, and frequently gets hit by risk events. Invest in mitigation, tighter estimates, or splitting the task.
High sensitivity + zero slack + low risk trigger rate	**Monitor closely.** The task drives the schedule through estimate variance, not risk. Focus on refining the estimate range.
High sensitivity + large slack	**Unusual.** The task's duration varies a lot but it isn't critical. Investigate whether dependencies are correctly modelled.
Low sensitivity + zero slack	The task is on the critical path but doesn't contribute much variance. It is a stable bottleneck — duration is predictable but the path still runs through it.
Low sensitivity + large slack	**Low priority.** This task has both predictable duration and scheduling flexibility.

13.22 Reducing volatility

If the spread between P50 and P90 is uncomfortably wide, consider:

- **Splitting large tasks**: A single task with `low: 5, high: 40` contributes more variance than two tasks with `low: 3, high: 20` each, especially when they are on different dependency paths.
- **Tightening estimate ranges**: If subject-matter experts can narrow the gap between `min` and `max` after investigation, uncertainty drops directly.
- **Mitigating high-impact risks**: Removing or reducing the probability of risks on critical tasks has the largest effect.
- **Adding parallel paths**: Restructuring dependencies so that fewer tasks are in a single sequential chain reduces the chance of cascading delays.
- **Addressing high-sensitivity tasks first**: Use the sensitivity analysis to focus effort where it matters most.

13.23 Limitations of the simulation

Monte Carlo simulation is a powerful tool, but it has boundaries. Understanding them prevents over-reliance on the numbers.

13.23.1 What the simulation does well

- Quantifies the *range* of plausible outcomes given uncertain inputs
- Identifies which tasks and risks contribute most to schedule uncertainty
- Provides probabilistic confidence levels instead of false-precision single-number estimates
- Handles complex dependency structures and mixed estimation methods

13.23.2 What the simulation does not do

- **It does not improve bad inputs**. If your estimate ranges are systematically optimistic — if the real `max` is higher than what you entered — the simulation will be optimistic too. The output is only as good as the input.
- **It assumes task independence**. Each task's duration is sampled independently. In reality, if one task overruns because the technology is harder than expected, related tasks may also overrun. The simulation does not model these correlations.
- **Dependency-only runs ignore resource constraints**. If you do not model `resources`/`calendars`, parallelism is constrained only by dependencies. For resource-constrained planning, use the constrained scheduling inputs and read the constrained diagnostics section.
- **It does not model learning or momentum**. Teams often speed up as they gain context, or slow down from fatigue. The simulation treats each task in isolation.
- **Risks are binary**. Each risk either fires or doesn't, at a fixed probability. In reality, risk severity exists on a spectrum and probabilities shift as the project evolves.

13.23.3 Calibration matters

The most useful thing you can do over time is *calibrate*: compare the simulation's P50 and P90 with actual outcomes, and adjust your estimation habits accordingly. If your P90 is consistently exceeded, your `max` values are too low. If P50 is consistently beaten, your `min` values may be too conservative — or your uncertainty factors are inflating estimates.

13.24 Quick reference

Output	What it tells you	Primary use
P50 (median)	The 50/50 point	Internal planning baseline
P80–P90	High-confidence estimates	External commitments
Std dev / CV	How spread out the outcomes are	Gauging overall uncertainty
Skewness	How asymmetric the distribution is (tail risk)	Deciding between mean and median for planning
Excess kurtosis	Whether extreme outcomes are more/less likely than normal	Sizing the buffer between P90 and P99
Sensitivity	Which tasks drive overall schedule variance	Prioritising estimation and risk-mitigation effort
Schedule slack	How much buffer each task has before delaying the project	Resource allocation and task prioritisation
Risk impact	How often and how much each task's risks fire	Identifying high-impact risk events to mitigate
Criticality index	How often a task is on the critical path	Focusing management attention
Critical path sequences	Which chains of tasks dominate	Dependency restructuring

Output	What it tells you	Primary use
Max parallel tasks	Peak number of tasks running simultaneously	Validating resource assumptions
Effort confidence intervals	Total person-hours at each probability level	Budgeting, cost estimation, staffing
Staffing advisory	Recommended team size per experience profile	Team sizing, hiring decisions
Thermometer	Visual probability-to-effort mapping	Stakeholder communication
Delivery dates	Calendar dates at each percentile	Scheduling and milestone setting

13.25 Sprint-planning mode outputs (when enabled)

If a project uses sprint-planning mode, output sections differ from dependency-schedule runs and include sprint-specific summaries such as:

- Sprint planning summary
- Sprint count statistical summary
- Sprint count confidence intervals
- Optional historical diagnostics (series, ratios, correlations)
- Burn-up percentiles

Treat these as the sprint-forecast analogue of calendar/effort percentiles in dependency-schedule mode.

CHAPTER 14

Configuration

This page documents the application configuration file used by `mcprojsim`.

The configuration file controls:

- uncertainty multipliers,
- symbolic estimate mappings such as T-shirt sizes and Story Points,
- default simulation settings,
- output and reporting defaults,
- staffing-analysis behavior.

14.1 Configuration file format

The configuration file is currently loaded as **YAML**.

Typical usage:

```
mcprojsim simulate project.yaml --config config.yaml
```

You can also load it directly from Python:

```
from mcprojsim import SimulationEngine
from mcprojsim.config import Config

config = Config.load_from_file("config.yaml")
engine = SimulationEngine(iterations=10000, config=config)
```

14.2 How configuration loading works

When `mcprojsim` loads a configuration file, it does **not** replace the built-in defaults wholesale. Instead, it:

1. starts from the built-in default configuration,
2. reads your YAML file,
3. recursively merges your values into those defaults,
4. validates the merged result against the `Config` model.

This means you can override only the values you care about.

For example, if you define only `t_shirt_sizes.story.M`, the built-in mappings for the remaining sizes and categories remain available.

14.3 Top-level configuration structure

The current configuration schema supports these top-level keys:

- `uncertainty_factors`
- `t_shirt_sizes`
- `t_shirt_size_default_category`
- `t_shirt_size_unit`
- `story_points`
- `story_point_unit`
- `simulation`
- `lognormal`
- `output`
- `staffing`
- `constrained_scheduling`
- `sprint_defaults`

14.4 Full example

```
uncertainty_factors:
  team_experience:
    high: 0.90
    medium: 1.0
    low: 1.30
  requirements_maturity:
    high: 1.0
    medium: 1.15
    low: 1.40
  technical_complexity:
    low: 1.0
    medium: 1.20
    high: 1.50
  team_distribution:
    colocated: 1.0
    distributed: 1.25
  integration_complexity:
    low: 1.0
    medium: 1.15
    high: 1.35

t_shirt_sizes:
  story:
    XS:
      low: 3
      expected: 5
      high: 15
    M:
      low: 40
      expected: 60
      high: 120
  epic:
    M:
      low: 200
      expected: 480
      high: 1200
t_shirt_size_default_category: "story"
t_shirt_size_unit: "hours"
story_points:
  1:
    low: 0.5
    expected: 1
    high: 3
  5:
    low: 3
    expected: 5
    high: 8
  8:
    low: 5
    expected: 8
    high: 15
story_point_unit: "days"

simulation:
  default_iterations: 10000
  random_seed: 42
  max_stored_critical_paths: 20
lognormal:
  high_percentile: 95

output:
  formats: ["json", "csv", "html"]
  include_histogram: true
  number_bins: 50
```

14.5 Uncertainty factors

Uncertainty factors are multipliers applied to base task estimates. A value of `1.0` is the baseline. Values below `1.0` make work faster; values above `1.0` make work slower.

The built-in configuration defines these factor names:

- `team_experience`
- `requirements_maturity`
- `technical_complexity`
- `team_distribution`
- `integration_complexity`

14.5.1 Defaults

Team experience

- **high (`0.90`)**: experienced team, about 10% faster than baseline
- **medium (`1.0`)**: baseline
- **low (`1.30`)**: inexperienced team, about 30% slower than baseline

Requirements maturity

- **high (`1.0`)**: stable, well-defined requirements
- **medium (`1.15`)**: moderate ambiguity
- **low (`1.40`)**: substantial ambiguity and likely change

Technical complexity

- **low (`1.0`)**: straightforward technology or architecture
- **medium (`1.20`)**: moderate complexity
- **high (`1.50`)**: difficult or novel technology

Team distribution

- **colocated (`1.0`)**: team in one place
- **distributed (`1.25`)**: coordination and communication slow delivery

Integration complexity

- **low (`1.0`)**: little external integration
- **medium (`1.15`)**: moderate integration effort
- **high (`1.35`)**: many integration points or fragile dependencies

14.5.2 Notes

- The configuration model accepts nested dictionaries here.
- The current project-file model uses the built-in factor names listed above.
- Extra factor names in the configuration file are not useful unless the source model and simulation logic also reference them.

14.6 Symbolic estimate mappings

`mcprojsim` supports two symbolic estimate systems:

- `t_shirt_size`
- `story_points`

Tasks using those symbolic forms are resolved through the active configuration.

If you override only part of either mapping table, the remaining built-in defaults stay available.

14.6.1 T-shirt sizes

Tasks may use `t_shirt_size` instead of explicit `low` / `expected` / `high` values.

Canonical configuration shape:

```
t_shirt_sizes:
  story:
    XS: {low: 3, expected: 5, high: 15}
    M: {low: 40, expected: 60, high: 120}
  bug:
    M: {low: 3, expected: 8, high: 24}
  epic:
    M: {low: 200, expected: 480, high: 1200}

t_shirt_size_default_category: story
```

Project-file values can be:

- bare size: `M` (resolved via `t_shirt_size_default_category`)
- qualified size: `epic.M`
- long-form aliases: `Medium`, `Epic.Large`

Built-in category keys:

- `bug`
- `story`
- `epic`
- `business`
- `initiative`

Built-in size keys (per category):

- `XS`
- `S`
- `M`
- `L`
- `XL`
- `XXL`

Built-in `story` defaults:

Size	low	expected	high
XS	3	5	15
S	5	16	40
M	40	60	120
L	160	240	500
XL	320	400	750
XXL	400	500	1200

Canonical override example:

```
t_shirt_sizes:
  story:
    M:
      low: 45
      expected: 65
      high: 130
  epic:
    M:
      low: 240
      expected: 520
      high: 1400

t_shirt_size_default_category: story
```

Backward compatibility:

- legacy flat maps under `t_shirt_sizes` are accepted and migrated to `t_shirt_sizes.<default_category>`
- transitional alias key `t_shirt_size_categories` is accepted as input
- defining both `t_shirt_sizes` and `t_shirt_size_categories` in one file is rejected

14.6.2 `t_shirt_size_default_category`

This field controls how bare T-shirt size tokens are resolved when the project file uses values like `M` instead of qualified values like `story.M` or `epic.M`.

Default: `story`

That means:

- `t_shirt_size: M` resolves to `story.M` unless you override the default category,
- legacy flat `t_shirt_sizes` maps are migrated into `t_shirt_sizes.<default_category>` during config loading,
- changing this value changes both bare-token resolution and the target category used for legacy flat-map normalization.

Example:

```
t_shirt_sizes:
  story:
    M: {low: 40, expected: 60, high: 120}
  epic:
    M: {low: 120, expected: 240, high: 400}

t_shirt_size_default_category: story
```

With that configuration, project-file `t_shirt_size: M` resolves to `story.M`.

14.7 Shifted log-normal configuration

When an estimate uses `distribution: "lognormal"`, `mcprojsim` fits a **shifted** log-normal from the already familiar `low`, `expected`, and `high` fields:

- `low` is treated as the hard shift/minimum,
- `expected` is treated as the mode,
- `high` is treated as a chosen percentile of the distribution.

That percentile is configured here:

```
lognormal:
  high_percentile: 95
```

Allowed values are:

- `70`
- `75`
- `80`

- 85
- 90
- 95
- 99

The default is `95`, meaning the `high` value is interpreted as the P95 point of the shifted log-normal distribution.

14.7.1 `t_shirt_size_unit`

This field controls the unit used for all values in `t_shirt_sizes`.

Supported values:

- `"hours"`
- `"days"`
- `"weeks"`

Default: `"hours"`

Example:

```
t_shirt_size_unit: "days"
```

14.7.2 Story Points

Tasks may also use `story_points` for agile-style relative sizing.

Built-in point values:

- 1
- 2
- 3
- 5
- 8
- 13
- 21

Built-in defaults:

Points	low	expected	high
1	0.5	1	3
2	1	2	4
3	1.5	3	5
5	3	5	8
8	5	8	15
13	8	13	21
21	13	21	34

Example override:

```
story_points:
  5:
    low: 4
    expected: 6
    high: 9
  8:
    low: 6
    expected: 9
    high: 16
```

14.7.3 `story_point_unit`

This field controls the unit used for all values in `story_points`.

Supported values:

- `"hours"`
- `"days"`
- `"weeks"`

Default: `"days"`

14.7.4 Important note

Story Point and T-shirt-size estimates in the **project file** must **not** include their own `unit` field. The unit comes from the configuration via `story_point_unit` and `t_shirt_size_unit`.

See Task Estimation and Task Estimation for more guidance.

14.8 Simulation settings

The `simulation` section controls default run behavior.

Field	Type	Default	Meaning
`default_iterations`	integer	`10000`	Default number of Monte Carlo iterations
`random_seed`	integer or `null`	`null`	Seed for reproducible runs
`max_stored_critical_paths`	integer	`20`	Number of full critical path sequences retained in memory

Example:

```
simulation:
  default_iterations: 20000
  random_seed: 42
  max_stored_critical_paths: 50
```

14.8.1 Critical path reporting

The simulator can retain full critical path sequences, not just task-level criticality.

- `simulation.max_stored_critical_paths` controls how many of the most common full paths are retained in the results object.
- `output.critical_path_report_limit` controls how many of those stored paths are shown in CLI summaries and exports by default.

Example:

```
simulation:
  default_iterations: 10000
  random_seed: null
  max_stored_critical_paths: 50

output:
  formats: ["json", "html"]
  include_histogram: true
  number_bins: 50
  critical_path_report_limit: 3
```

With that configuration, the simulation stores up to 50 unique full critical path sequences and shows the top 3 in the CLI, JSON, CSV, and HTML outputs.

You can still override the report count from the CLI:

```
mcprojsim simulate project.yaml --config config.yaml --critical-paths 5
```

That affects reporting only. Storage remains controlled by `simulation.max_stored_critical_paths`.

14.9 Output settings

The `output` section controls reporting and export defaults.

Field	Type	Default	Meaning
`formats`	list of strings	`["json", "csv", "html"]`	Default export formats
`include_histogram`	boolean	`true`	Include histogram data where supported
`number_bins`	integer	`50`	Number of bins used for histograms in JSON, CSV, and HTML exports
`critical_path_report_limit`	integer	`2`	Number of stored full critical paths shown by default

14.9.1 Histogram bins

The `number_bins` setting controls the granularity of duration distribution charts in exported reports. Higher bin counts produce finer-grained histograms with narrower bars, while lower counts produce coarser histograms with wider bars.

- **Typical range**: 20–100 bins
- **Default**: 50 bins
- **CLI override**: Use `--number-bins N` to override the config file setting for a single run

The histogram data is included in: - HTML reports (visual distribution chart) - JSON exports (histogram array) - CSV exports (histogram table)

Configuration example:

```
output:
  formats: ["json", "html"]
  include_histogram: true
  number_bins: 80
  critical_path_report_limit: 5
```

CLI override example:

```
# Use 100 bins for this run, regardless of the config file
mcprojsim simulate project.yaml --number-bins 100 -f json,html
```

When `--number-bins` is specified on the command line, it overrides the value in the config file for that simulation run only.

14.10 Staffing settings

The `staffing` section controls the staffing analysis shown in CLI output and exports.

Field	Type	Default	Meaning
`effort_percentile`	integer (optional)	omitted	Use a specific effort percentile such as P80 instead of the mean

Field	Type	Default	Meaning
`min_individual_productivity`	float	`0.25`	Lower bound on per-person productivity after communication overhead
`experience_profiles`	mapping	built-in defaults	Named team profiles used for staffing recommendations

14.10.1 `effort_percentile`

When `effort_percentile` is omitted, staffing uses mean total effort and mean elapsed time.

When it is set, staffing uses the matching percentile for both:

- total effort, and
- elapsed time / critical-path basis.

For example, `80` means the staffing recommendation is based on P80 effort and P80 elapsed time.

This is useful when you want conservative staffing guidance rather than a mean-based plan.

14.10.2 `min_individual_productivity`

The staffing model assumes each additional person introduces communication overhead. Raw per-person productivity declines with team size and is then floored by `min_individual_productivity`.

Conceptually:

$$P(n) = \max(P_{min}, 1 - c(n-1))$$

where:

- n is team size,
- c is the profile's `communication_overhead`,
- P_{min} is `min_individual_productivity`.

This prevents the model from predicting unrealistically close-to-zero productivity for very large teams.

Lower values penalize oversized teams more aggressively. Higher values produce a softer diminishing-returns curve.

14.10.3 `experience_profiles`

Each named profile contains:

Field	Type	Meaning
`productivity_factor`	float	Base multiplier for that team's effectiveness
`communication_overhead`	float	Per-person overhead penalty as team size grows

Built-in profiles:

Profile	`productivity_factor`	`communication_overhead`
`senior`	1.00	0.04
`mixed`	0.85	0.06
`junior`	0.65	0.08

Example:

```
staffing:
  effort_percentile: 80
  min_individual_productivity: 0.30
  experience_profiles:
    senior:
      productivity_factor: 1.0
      communication_overhead: 0.03
    contractor:
      productivity_factor: 0.75
      communication_overhead: 0.05
```

14.11 Sprint planning defaults

The `sprint_defaults` section provides company-wide defaults for sprint-planning simulations. These values are used when the project file does not set the corresponding `project.sprint_planning` field and no CLI flag overrides it.

Several of these settings change simulation behavior directly, not just reporting:

- `planning_confidence_level` changes which percentile the sprint plan targets,
- `velocity_model` changes how sprint capacity is sampled,
- `removed_work_treatment` changes whether removed work is modeled as churn only or as actual backlog reduction,
- `volatility_disruption_*` settings control whether disruption multipliers are sampled,
- `spillover_*` settings control how often work spills over and how much remains,
- `sprint_defaults.sickness.*` affects sprint sickness modeling and also constrained-scheduling sickness duration.

14.11.1 Field reference

1. `planning_confidence_level`
   - **Type:** float
   - **Default:** `0.8`
   - **Allowed:** `0 < value < 1`
   - **Effect:** Target confidence used when planning sprint outcomes
2. `removed_work_treatment`
   - **Type:** string
   - **Default:** `churn_only`
   - **Allowed:** `churn_only`, `reduce_backlog`
   - **Effect:** Controls whether removed work is treated as churn only or also reduces backlog
3. `velocity_model`
   - **Type:** string
   - **Default:** `empirical`
   - **Allowed:** `empirical`, `neg_binomial`
   - **Effect:** Controls how historical capacity is sampled
4. `volatility_disruption_probability`
   - **Type:** float
   - **Default:** `0.0`
   - **Allowed:** `0 <= value <= 1`
   - **Effect:** Probability that disruption multipliers apply
5. `volatility_disruption_multiplier_low`
   - **Type:** float
   - **Default:** `1.0`
   - **Allowed:** `value >= 0`
   - **Effect:** Low bound for sampled disruption multiplier
6. `volatility_disruption_multiplier_expected`
   - **Type:** float

- **Default:** `1.0`
- **Allowed:** `value >= 0`
- **Effect:** Expected disruption multiplier

7. `volatility_disruption_multiplier_high`
   - **Type:** float
   - **Default:** `1.0`
   - **Allowed:** `value >= 0`
   - **Effect:** High bound for sampled disruption multiplier
8. `spillover_model`
   - **Type:** string
   - **Default:** `table`
   - **Allowed:** `table`, `logistic`
   - **Effect:** Controls how spillover probability is modeled
9. `spillover_size_reference_points`
   - **Type:** float
   - **Default:** `5.0`
   - **Allowed:** `value > 0`
   - **Effect:** Reference size used by spillover probability models
10. `spillover_size_brackets`
    - **Type:** list
    - **Default:** built-in table
    - **Allowed:** bracket list
    - **Effect:** Size-to-probability table used when `spillover_model: table`
11. `spillover_consumed_fraction_alpha`
    - **Type:** float
    - **Default:** `3.25`
    - **Allowed:** `value > 0`
    - **Effect:** Beta-shape parameter controlling typical consumed fraction
12. `spillover_consumed_fraction_beta`
    - **Type:** float
    - **Default:** `1.75`
    - **Allowed:** `value > 0`
    - **Effect:** Beta-shape parameter controlling typical remaining spillover
13. `spillover_logistic_slope`
    - **Type:** float
    - **Default:** `1.9`
    - **Allowed:** `value > 0`
    - **Effect:** Slope of the logistic spillover probability curve
14. `spillover_logistic_intercept`
    - **Type:** float
    - **Default:** `-1.9924301646902063`
    - **Allowed:** any float
    - **Effect:** Intercept of the logistic spillover probability curve
15. `sickness.enabled`
    - **Type:** boolean
    - **Default:** `false`
    - **Allowed:** `true`, `false`
    - **Effect:** Enables sprint sickness modeling
16. `sickness.probability_per_person_per_week`
    - **Type:** float
    - **Default:** `0.058`
    - **Allowed:** `0 < value < 1`
    - **Effect:** Weekly sickness probability per team member
17. `sickness.duration_log_mu`
    - **Type:** float
    - **Default:** `0.693`
    - **Allowed:** any float

- **Effect:** Log-normal sickness duration location parameter

18. `sickness.duration_log_sigma`
    - **Type:** float
    - **Default:** `0.75`
    - **Allowed:** `value > 0`
    - **Effect:** Log-normal sickness duration scale parameter

14.11.2 Notes on behavioral impact

- `velocity_model: empirical` re-samples directly from historical data; `neg_binomial` fits a negative binomial model to historical throughput.
- `removed_work_treatment: churn_only` treats removed work as diagnostic churn; `reduce_backlog` allows removed work to reduce remaining backlog.
- `spillover_model: table` uses the configured size brackets directly; `logistic` uses `spillover_logistic_slope`, `spillover_logistic_intercept`, and `spillover_size_reference_points`.
- The `volatility_disruption_multiplier_*` values define the sampled disruption range when `volatility_disruption_p` is non-zero.

14.12 Viewing current configuration

```
# Show effective configuration (built-in defaults, or auto-loaded user config)
mcprojsim config

# Show configuration from a specific file
mcprojsim config --config config.yaml

# Generate a default configuration file at ~/.mcprojsim/config.yaml
mcprojsim config --generate
```

When no `--config` is given, `mcprojsim` automatically loads `~/.mcprojsim/config.yaml` if it exists. Use `--generate` to produce a starter config file at that location that you can then customise.

The `config` output includes the most relevant effective settings, including:

- default iteration count,
- random seed,
- max stored critical paths,
- output histogram settings,
- critical path report limit,
- staffing settings,
- sprint defaults.

!!! note “Derived lognormal parameters in config output” For each T-shirt size and story point value, `mcprojsim config` also shows computed `lognormal params` (`mu`, `sigma`, `z-score`). These are derived from the configured `low`/`expected`/`high` values and are displayed for verification purposes only. They are not configurable fields — changing the underlying size values automatically recomputes them.

14.13 Sprint planning precedence

Sprint-planning values are resolved in this order (highest to lowest priority):

1. CLI flags
2. Project `sprint_planning` fields
3. Global `sprint_defaults`
4. Built-in defaults

This lets you keep company-wide defaults in `sprint_defaults`, set project-specific behavior in the project file, and still override per run with CLI flags.

For sprint-planning field details and examples, see Sprint planning.

14.14 Constrained scheduling defaults

The `constrained_scheduling` section controls defaults for resource-constrained scheduling runs.

These settings affect scheduling behavior directly:

- `sickness_prob` supplies the per-resource sickness fallback when the project file omits `resources[*].sickness_prob`,
- `assignment_mode` selects single-pass greedy scheduling or two-pass criticality-aware scheduling,
- `pass1_iterations` controls how much pass-1 sampling is used to rank tasks when two-pass scheduling is active.

14.14.1 Field reference

1. `sickness_prob`
   - **Type:** float
   - **Default:** `0.0`
   - **Allowed:** `0 <= value <= 1`
   - **Effect:** Fallback per-resource sickness probability
2. `assignment_mode`
   - **Type:** string
   - **Default:** `greedy_single_pass`
   - **Allowed:** `greedy_single_pass`, `criticality_two_pass`
   - **Effect:** Selects constrained scheduling strategy
3. `pass1_iterations`
   - **Type:** integer
   - **Default:** `1000`
   - **Allowed:** `value > 0`
   - **Effect:** Number of pass-1 iterations used to compute criticality ranking in two-pass mode

14.14.2 `sickness_prob`

`constrained_scheduling.sickness_prob` sets the default sickness probability for resources in constrained scheduling when a resource does not specify `sickness_prob` in the project file.

Precedence:

1. `resources[*].sickness_prob` in project file
2. `constrained_scheduling.sickness_prob` in config
3. built-in default `0.0`

Example:

```
constrained_scheduling:
  sickness_prob: 0.03
```

If you omit this key, resources without `sickness_prob` still behave as `0.0`.

14.14.3 `assignment_mode`

This field controls how tasks are prioritized when resource constraints are active.

- `greedy_single_pass`: the default. Tasks are dispatched in deterministic greedy order.
- `criticality_two_pass`: the simulator first runs a baseline pass to estimate task criticality, then re-runs the schedule using those criticality rankings as priority.

Two-pass mode affects the simulation itself, not just the output. The final results use pass-2 statistics, and the simulator includes a traceability block describing the delta between pass 1 and pass 2.

Example:

```
constrained_scheduling:
  assignment_mode: criticality_two_pass
  pass1_iterations: 1500
```

14.14.4 `pass1_iterations`

This field matters only when `assignment_mode: criticality_two_pass`.

- It controls how many iterations are used in pass 1 to estimate criticality indices.
- If it is larger than total simulation iterations, the simulator caps it to the total iteration count.
- Very low values can produce noisy criticality rankings.

As a practical rule, use enough pass-1 iterations to get stable rankings before relying on two-pass deltas for decision-making.

14.15 Sickness duration defaults

The keys under `sprint_defaults.sickness` now serve two related purposes:

- sprint-planning sickness modeling,
- resource-constrained scheduler sickness duration modeling.

Specifically:

- `probability_per_person_per_week` is used by sprint planning,
- `enabled` turns sprint sickness simulation on or off,
- `duration_log_mu` and `duration_log_sigma` define the shared log-normal duration model for sickness episodes,
- per-resource `sickness_prob` still belongs in the project file because it is resource-specific.

Example:

```
sprint_defaults:
  sickness:
    enabled: false
    probability_per_person_per_week: 0.058
    duration_log_mu: 0.693
    duration_log_sigma: 0.75
```

If you raise `duration_log_mu` or `duration_log_sigma`, constrained schedules with non-zero `resources[*].sickness_prob` will tend to experience longer sickness-driven absences.

14.16 Sprint default logistic spillover parameters

The global keys `sprint_defaults.spillover_logistic_slope` and `sprint_defaults.spillover_logistic_intercept` control the logistic spillover probability curve used when project spillover model is `logistic`.

The planner computes probability in log-space from planning story points:

$$z = \max\left(\frac{x}{r}, 10^{-6}\right), \quad \ell = s \cdot \ln(z) + b, \quad p = \frac{1}{1 + e^{-\ell}}$$

where:

- x = item planning story points
- r = `spillover_size_reference_points`
- s = `spillover_logistic_slope`
- b = `spillover_logistic_intercept`

At the reference size ($x = r$), baseline probability is:

$$p(x = r) = \frac{1}{1 + e^{-b}}$$

For full spillover workflow details (including Beta-sampled consumed fraction and carryover construction), see Sprint planning.

`sprint_defaults.spillover_consumed_fraction_alpha` and `sprint_defaults.spillover_consumed_fraction_beta`

are the Beta-distribution shape parameters for the consumed fraction during spillover events. Their expected consumed fraction is:

$$\mathbb{E}[f] = \frac{\alpha}{\alpha + \beta}$$

Increasing `alpha` raises typical consumed fraction; increasing `beta` lowers it.

14.17 Validation rules

The configuration model validates these constraints directly:

- `t_shirt_size_unit` must be one of `hours`, `days`, or `weeks`,
- `story_point_unit` must be one of `hours`, `days`, or `weeks`,
- `t_shirt_sizes` accepts either a nested `<category>: <size>: {low, expected, high}` map or a legacy flat size map,
- if `t_shirt_size_categories` is used as a transitional alias, it cannot appear together with `t_shirt_sizes`,
- T-shirt size tokens in config must normalize to one of `XS`, `S`, `M`, `L`, `XL`, or `XXL`,
- all configured estimate ranges require positive `low`, `expected`, and `high`,
- `lognormal.high_percentile` must be one of `70`, `75`, `80`, `85`, `90`, `95`, or `99`,
- `simulation.default_iterations` must be greater than 0,
- `simulation.max_stored_critical_paths` must be greater than 0,
- `output.formats` must be non-empty and may contain only `json`, `csv`, and `html`,
- `output.number_bins` must be greater than 0,
- `output.critical_path_report_limit` must be greater than 0,
- `staffing.effort_percentile`, when set, must be between 1 and 99,
- `staffing.min_individual_productivity` must be greater than 0 and at most 1,
- `experience_profiles[*].productivity_factor` must be greater than 0,
- `experience_profiles[*].communication_overhead` must be between 0 and 1,
- `constrained_scheduling.sickness_prob` must be between 0 and 1.
- `constrained_scheduling.pass1_iterations` must be greater than 0,
- `sprint_defaults.planning_confidence_level` must be between 0 and 1,
- `sprint_defaults.removed_work_treatment` must be either `churn_only` or `reduce_backlog`,
- `sprint_defaults.velocity_model` must be either `empirical` or `neg_binomial`,
- `sprint_defaults.volatility_disruption_probability` must be between 0 and 1,
- `sprint_defaults.volatility_disruption_multiplier_low`, `volatility_disruption_multiplier_expected`, and `volatility_disruption_multiplier_high` must be greater than or equal to 0,
- `sprint_defaults.spillover_model` must be either `table` or `logistic`,
- `sprint_defaults.spillover_size_reference_points` must be greater than 0,
- `sprint_defaults.spillover_consumed_fraction_alpha` and `sprint_defaults.spillover_consumed_fraction_beta` must be greater than 0,
- `sprint_defaults.spillover_logistic_slope` must be greater than 0,
- `sprint_defaults.sickness.probability_per_person_per_week` must be greater than 0 and less than 1,
- `sprint_defaults.sickness.duration_log_sigma` must be greater than 0.

14.18 Related settings in the project file

Some important settings are **not** part of `config.yaml`. They belong in the project file instead.

For example, `probability_red_threshold` and `probability_green_threshold` are project-level fields stored in the simulation results and available for programmatic use:

```
project:
  name: "My Project"
  start_date: "2025-01-01"
  probability_red_threshold: 0.50
  probability_green_threshold: 0.90
```

Default values:

- `probability_red_threshold: 0.50`
- `probability_green_threshold: 0.90`

!!! note These thresholds are stored in simulation results and exported in JSON output, but they do **not** control the colour of the HTML thermometer chart. The thermometer uses a fixed gradient from dark orange at 50 % to dark green at 99 %, computed solely from the probability value of each segment.

14.19 Calibrating configuration values

To calibrate the configuration for your organisation:

1. collect historical estimate-versus-actual data,
2. derive realistic uncertainty multipliers,
3. adjust symbolic estimate mappings to match your team's sizing habits,
4. tune staffing profiles to reflect your real communication overhead,
5. iterate over several completed projects.

Examples:

- if experienced teams historically finish at about 85% of baseline effort, set `team_experience.high: 0.85`,
- if distributed teams typically take about 30% longer, set `team_distribution.distributed: 1.30`,
- if your team uses Story Point 8 for bigger work than the defaults assume, widen the `story_points.8` range.

CHAPTER 15

Natural Language Input

The `mcprojsim` natural language parser accepts a wide range of input formats — from structured `Task N:` headers to plain bullet lists copied from a planning tool. This page is the single reference for every input pattern the parser understands.

Quick rule of thumb

If it looks like a task list a human would write, the parser will probably understand it. When in doubt, run `mcprojsim generate` and inspect the output.

15.1 Introductory example

As an introduction the following "human" project description can be used:

```
Project: New feature launch
Start date: next Monday

1. Design database schema (about a week)
2. Implement backend REST API (probably 2–4 days, depends on the DB work)
3. UI design , about ~ 2 weeks
4. Frontend integration testing (around 3 weeks)
5. Deployment and smoke tests (a few days), depends on UI and backend being ready
6. Post-launch monitoring (a sprint),  after deployment
7. Delivery report , about 2-3 days
```

Save this as `proj1.txt` . Then run

```
mcprojsim generate proj1.txt
```

It will then generate the full project specification as:

```yaml
project:
  name: "New feature launch"
  start_date: "2026-04-13"
  confidence_levels: [50, 80, 90, 95]
tasks:
  - id: "task_001"
    name: "Design database schema ()"
    estimate:
      t_shirt_size: "M"
    dependencies: []
  - id: "task_002"
    name: "Implement backend REST API (probably"
    estimate:
      low: 2
      expected: 3
      high: 4
      unit: "days"
    dependencies: ["task_001"]
  - id: "task_003"
    name: "UI design"
    estimate:
      low: 1.5
      expected: 2
      high: 5
      unit: "weeks"
    dependencies: []
  - id: "task_004"
    name: "Frontend integration testing ()"
    estimate:
      low: 2.1
      expected: 3
      high: 5.4
      unit: "weeks"
    dependencies: []
  - id: "task_005"
    name: "Deployment and smoke tests ()"
    estimate:
      t_shirt_size: "S"
    dependencies: ["task_002", "task_003", "task_004"]
  - id: "task_006"
    name: "Post-launch monitoring ()"
    estimate:
      t_shirt_size: "L"
    dependencies: ["task_005"]
  - id: "task_007"
    name: "Delivery report"
    estimate:
      low: 2
      expected: 2.5
      high: 3
      unit: "days"
    dependencies: []
```

Review generated task names

Task names ending with `()` — such as `"Design database schema ()"` — are parser artifacts that occur when the input contains unmatched or empty parentheses. They are harmless but untidy. Always review the generated YAML and remove or correct these fragments before using the file in a simulation.

The parser supports two task-definition modes. They cannot be mixed in the same description.

Mode	Trigger	Example
Explicit headers	`Task N:` followed by bullet properties	`Task 1:` / `- Size: M`
Auto-detected lists	Plain numbered or bullet lists with no `Task N:` headers	`1. Design phase` / `- Backend API`

Auto-detected lists activate automatically when the parser sees numbered or bulleted items and no `Task N:` header has appeared. Once a `Task N:` header is encountered, auto-detection is disabled for the rest of the description.

15.2 Project-level metadata

These lines can appear anywhere outside a task section:

```
Project name: Website Redesign
Description: Q3 infrastructure work
Start date: 2026-06-01
Starting in 2 weeks
Start date: next Monday
Hours per day: 7.5
Confidence levels: 50, 80, 90, 95
```

`Start date` also accepts relative phrases such as `next Monday`, `in 5 days`, `in 2 weeks`, `beginning of May`, and `May 2026`.

All fields are optional. Defaults: name = "Untitled Project", hours = 8.0, confidence = [50, 80, 90, 95].

15.3 Explicit task headers (`Task N:`)

The original structured format. Each task starts with a numbered header, followed by indented bullet properties:

```
Task 1: Design the login flow
- Size: M

Task 2: Implement backend
- Depends on Task 1
- Estimate: 5/8/15 days

Task 3: QA and testing
- Depends on Task 2
- Story points: 8
```

15.3.1 Bullet properties

Property	Patterns	Example
Task name	`Name:` or first unmatched bullet	`- Name: Backend API`
T-shirt size	`Size: M`, `Size XL`, `Size. XL`	`- Size: L`
Story points	`Story points: 5`, `Points: 8`	`- Story points: 13`
Explicit estimate	`Estimate: low/expected/high [unit]`	`- Estimate: 3/5/10 days`
Dependencies	`Depends on Task 1, Task 3`	`- Depends on Task 2`
Resources	`Resources: Alice, Bob`	`- Resources: Alice`
Max resources	`Max resources: 2`	`- Max resources: 2`
Min experience	`Min experience: 2`	`- Min experience: 3`

Property	Patterns	Example
Description	Second unmatched bullet	`- Involves DB migration`

Separators between keyword and value can be `:`, `.`, `=`, or a space. All are equivalent and case-insensitive.

15.4 Auto-detected task lists

When no `Task N:` headers are present, the parser automatically detects tasks from plain lists. This is ideal for copy-paste from planning tools, meeting notes, or email threads.

15.4.1 Supported list formats

Plain numbered lists — task numbers are preserved from the source:

```
Project name: Backend Migration
Start date: 2026-05-01

1. Design database schema
2. Implement REST API
3. Write integration tests
4. Deploy to staging
```

Parenthesis numbered lists:

```
1) Authentication module
2) User management
3) Reporting
```

Bracket numbered lists:

```
[1] Authentication
[2] User management
[3] Reporting module
```

Bullet lists — task numbers are auto-assigned (1, 2, 3, …):

```
- Discovery and requirements
- Database design
- Backend implementation
- Frontend
- QA
- Deployment
```

Bullets can use `-`, `*`, or `•`.

Hash numbered lists (rare but supported):

```
# 1 First task
# 2 Second task
```

15.4.2 Continuation lines

Indented lines under an auto-detected task are treated as bullet properties, exactly like the explicit-header format:

```
1. Design database schema
  Size: M
2. Implement REST API
  Size: XL
  Depends on Task 1
3. Write integration tests
  Estimate: 3/5/10 days
```

15.5 Inline properties on task lines

Both auto-detected and continuation lines can carry properties directly on the task name line. The parser extracts them and cleans the task name.

15.5.1 Bracketed or parenthesized sizes

```
- Backend API [XL]
- Frontend (M)
1. QA testing [S]
```

15.5.2 Fuzzy size hints

Natural phrasing like "probably an M" or "assume S" is recognized:

```
- Backend refactoring, probably an M
- Frontend overhaul, likely an L
- Quick patch, assume S
```

Supported qualifiers: `probably`, `likely`, `assume`, `estimated as`.

15.5.3 Inline estimate ranges

Numeric ranges on the task line are parsed as explicit estimates:

```
- QA: 3–5 days
- Quick fix 2-4 hours
- Backend migration 2–4 weeks
- Implementation 3 to 5 days
```

The parser calculates `expected = (low + high) / 2` automatically.

15.5.4 Approximate point estimates

Approximate single-value estimates on the task line are expanded into a three-point range automatically:

```
- Quick fix about 10 hours
- Frontend integration around 3 weeks
- Rollout verification roughly 6 hours
```

For these phrases, the parser keeps the stated value as `expected` and synthesizes:

- `low = expected * 0.7`
- `high = expected * 1.8`

So `about 10 hours` becomes `low=7`, `expected=10`, `high=18`, `unit=hours`.

When the estimate includes `~` (for example `about ~2 weeks` or `~10 hours`), the parser treats it as a higher-uncertainty signal and widens the synthesized range:

- `low = expected * 0.75`
- `high = expected * 2.5`

So `about ~2 weeks` becomes `low=1.5`, `expected=2`, `high=5`, `unit=weeks`.

15.5.5 Natural duration phrases

Common prose-style duration phrases are mapped directly to T-shirt sizes when no explicit numeric estimate is present:

```
- Quick patch takes a couple of days
- Design database schema about a week
- Frontend integration will take a few weeks
- Migration effort is a month or so
- Post-launch monitoring takes a sprint
```

Examples:

- `a couple of days` -> `S`
- `about a week` -> `M`
- `a few weeks` -> `L`
- `a month or so` -> `L`
- `a sprint` -> derived from sprint cadence when sprint planning is available

Numeric expressions still take precedence. For example, `around 3 weeks` is treated as an approximate point estimate in weeks, not as a T-shirt phrase.

15.5.6 Inline dependencies

```
1. Design database schema
2. Implement REST API depends on Task 1
```

The parser also understands prose-style dependency connectors in task lines and bullets:

```
1. Database schema design
2. Authentication service implementation (builds on the database schema)
3. API endpoints (after task 2)
4. UI integration (once authentication is done)
5. Deployment checks (requires the API endpoints)
```

Supported connectors include `after`, `following`, `blocked by`, `requires`, `needs`, `depends on`, `builds on`, and `based on`.

The parser also resolves natural references to previous tasks when the text does not name an explicit `Task N` reference:

```
1. Design database schema
2. Implement backend REST API (probably 2–4 days, depends on the DB work)
3. Frontend integration testing (around 3 weeks)
4. Deployment and smoke tests (a few days), depends on frontend and backend being ready
```

Supported fuzzy references include terms such as `DB`, `API`, `frontend`, `backend`, `auth`, `deployment`, and `QA/testing`, matched against previously parsed task names.

Name-based matching only looks at tasks that were parsed earlier in the file.

15.5.7 Combined example

Multiple inline properties can appear on the same line:

```
- Backend API [XL] 3–5 days
```

This sets both the T-shirt size and the estimate range.

15.6 T-shirt size aliases

The parser normalizes many size labels to the six canonical sizes:

Canonical	Accepted inputs
`XS`	`XS`, `Extra Small`, `Extrasmall`
`S`	`S`, `Small`
`M`	`M`, `Medium`, `Med`
`L`	`L`, `Large`
`XL`	`XL`, `Extra Large`, `Extralarge`
`XXL`	`XXL`, `Extra Extra Large`, `2XL`

Matching is case-insensitive.

15.7 Complete examples

15.7.1 Example 1: Numbered list with inline sizes

```
Project name: Backend Migration
Start date: 2026-05-01

1. Design database schema [M]
2. Implement REST API [XL] depends on Task 1
3. Write integration tests [L] depends on Task 2
4. Deploy to staging [S] depends on Task 3
5. Production cutover [S] depends on Task 4
```

15.7.2 Example 2: Bullet list with fuzzy sizes

```
Project name: Mobile App MVP
Start date: 2026-06-01

- Discovery and requirements (S)
- UX wireframes (M)
  Depends on Task 1
- Backend API, probably an XL
  Depends on Task 2
- iOS frontend (XL)
  Depends on Task 3
- Android frontend (XL)
  Depends on Task 3
- QA and bug fixes, likely an L
  Depends on Task 4, Task 5
- App store submission (S)
  Depends on Task 6
```

15.7.3 Example 3: Bracket list with inline ranges

```
Project name: Data Pipeline Rebuild
Start date: 2026-07-01

[1] Audit existing ETL jobs [S]
[2] Design new pipeline architecture [M] depends on Task 1
[3] Implement ingestion layer 3–5 days depends on Task 2
[4] Build transformation engine [XL] depends on Task 3
[5] Set up monitoring and alerts [M] depends on Task 4
[6] Migration and cutover, assume S, depends on Task 5
```

15.7.4 Example 4: Mixed inline properties

```
Project name: Auth Service Rewrite
Description: Replace legacy auth with OAuth2/OIDC
Start date: 2026-08-01
1. Evaluate identity providers 2–4 days
2. Design token flow and session management [M]
  Depends on Task 1
3. Implement OAuth2 authorization server, probably an XL
  Depends on Task 2
4. Migrate user database (L)
  Depends on Task 2
5. Integration testing [L]
  Depends on Task 3, Task 4
6. Security audit, assume M
  Depends on Task 5
7. Staged rollout [S]
  Depends on Task 6
```

15.7.5 Example 5: Traditional structured format (still fully supported)

```
Project name: Website Redesign
Start date: 2026-04-15

Task 1: Gather requirements
- Size: S

Task 2: Create wireframes
- Depends on Task 1
- Size: M

Task 3: Build frontend
- Depends on Task 2
- Size: XL
```

15.7.6 Example 6: Relative dates and prose-style estimates

```
Project: New feature launch
Start date: next Monday

1. Design database schema (about a week)
2. Implement backend REST API (probably 2–4 days, depends on the DB work)
3. Frontend integration testing (around 3 weeks)
4. Deployment and smoke tests (a few days), depends on frontend and backend being ready
5. Post-launch monitoring (a sprint)
```

15.7.7 Example 7: Approximate point estimates

```
Project name: Incident Remediation
Starting in 2 weeks

- Quick patch about 10 hours
- Auth hardening around 3 to 5 days
- Database cleanup 2-4 hours
- Rollout verification roughly 6 hours
```

15.8 Resources, calendars, and sprint planning

These sections use the same structured format regardless of whether tasks use explicit headers or auto-detection. See the MCP Server page and the API reference for full details on resource, calendar, and sprint planning input patterns.

15.9 Mixing estimation methods

Different tasks can use different estimation methods within the same project:

```
1. Well-understood work
  Estimate: 3/5/8 days
2. Vaguely scoped work [XL]
3. Agile team estimate
  Story points: 8
```

The parser handles this correctly, and `mcprojsim` resolves each estimate type using the appropriate configuration mapping at simulation time.

15.10 What is NOT supported in NL input

- **Uncertainty factors**, add these manually to the YAML
- **Project-level risks**, add these manually to the YAML
- **Circular dependency detection**, caught later by `mcprojsim validate`
- **Volatility-overlay and spillover calibration**, edit directly in YAML
- Mixing `Task N:` headers and auto-detected lists in the same description

CHAPTER 16

MCP Server & NL Input

Writing a full YAML project file by hand is straightforward once you know the format, but it takes time — especially when you are in an early planning session and just want to sketch out tasks quickly. The `mcprojsim` **MCP server** bridges that gap. It lets an AI assistant (such as Claude, GitHub Copilot, or any MCP-compatible client) convert a rough, natural-language project description into a syntactically correct project specification file that `mcprojsim` can simulate immediately.

This chapter explains what the MCP server is, how to install and configure it, how the natural language parser works, and how to get the best results from it.

16.1 What is the MCP server?

The **Model Context Protocol (MCP)** is an open standard that lets AI assistants call external tools in a structured way. The mcprojsim MCP server exposes the following tools grouped by input type and action:

Command class	Tool	Purpose
NL -> YAML generation	`generate_project_file`	Convert natural language project description into a valid mcprojsim YAML file
NL validation	`validate_project_description`	Lightweight parser-level checks on the description
NL strict validation	`validate_generated_project_yaml`	Full parser/model validation of generated YAML using runtime defaults and overrides
NL simulation	`simulate_project`	Generate YAML from description and run simulation in one step
YAML strict validation	`validate_project_yaml`	Validate existing YAML directly (no NL regeneration)
YAML simulation	`simulate_project_yaml`	Simulate directly from existing YAML with runtime options
YAML update/transform	`update_project_yaml`	Apply NL update instructions to existing YAML (optionally replace tasks)

When an MCP client connects to the server, the AI assistant can invoke these tools on your behalf. You can now work in either direction:

- describe in natural language and generate/simulate immediately,
- or keep an existing YAML project as the source of truth and validate/simulate/update it directly.

16.2 Prerequisites and installation

The MCP server has one additional dependency beyond the core `mcprojsim` package: the `mcp` Python library. It is declared as an optional dependency group so it does not affect users who do not need the server.

16.3 Ways to install the server

There are two main ways to install the server

1. [Recommended] Ask your assistant to download and install the server from GitHub
2. Download the project, setup the development environment and build the server locally your self.

16.4 Ask your assistant

Use the following example prompt to get your assistant to install `mcprojsim`:

```
Download and install the latest mcprojsim MCP server from GitHub Releases. Follow the README.md for installation instructions.
```

Depending on which assistant you use (e.g. `copilot-cli`, `claude-desktop`, …) and you general instructions you might get different questions to approve various file-operations in order to setup the server.

16.5 Download project and install manually

16.5.1 Step 1: Install mcprojsim with MCP support

If you already have the project checked out and use Poetry you can install it with:

```
poetry install --with mcp
```

This installs the `mcp` package alongside the existing dependencies. The `--with mcp` flag pulls in only the MCP group without affecting other optional groups like `docs` or `dev`.

16.5.2 Step 2: Verify the installation

mcprojsim-mcp does not accept –help

`mcprojsim-mcp` is a **long-running server process**. When started from a terminal it immediately begins listening for MCP protocol messages over stdio. It does not accept `--help` or any command-line flags — running `mcprojsim-mcp --help` will appear to hang, because the process is waiting for a client. Press `Ctrl+C` to exit.

To verify that the entry point and all MCP dependencies are correctly installed, use the Python import check instead:

```
from mcprojsim.mcp_server import main
print("MCP server installed correctly")
```

If this import succeeds without an `ImportError`, the MCP dependencies are correctly installed. You can also run this as a one-liner from the terminal:

```
python -c "from mcprojsim.mcp_server import main; print('OK')"
```

With Poetry:

```
poetry run python -c "from mcprojsim.mcp_server import main; print('OK')"
```

A successful run prints `OK` and exits immediately. Any `ImportError` or `ModuleNotFoundError` means the `mcp` dependency group was not installed — re-run `poetry install --with mcp` to fix it.

16.5.3 Step 3: Configure your MCP client

Every MCP client has its own configuration format. Below are examples for common clients.

Claude Desktop — edit `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "mcprojsim": {
      "command": "mcprojsim-mcp"
    }
  }
}
```

VS Code (GitHub Copilot) — add to your `.vscode/mcp.json` or user settings:

```
{
  "servers": {
    "mcprojsim": {
      "command": "mcprojsim-mcp"
    }
  }
}
```

!!! note If you installed `mcprojsim` inside a Poetry virtual environment, you may need to specify the full path to the entry point. You can find it with `poetry run which mcprojsim-mcp`.

16.6 A basic example of a prompt that uses the MCP Server

After restarting the assistant (needed to load the new MCP Server) ask for a simple project simulation:

```
Simulate a project that starts 2025-05-01 and has two M-size tasks that depends on each-other. Show the result for all comp
```

An example output would be

```
● Here are the simulation results for the two-task project (sequential M→M, starting 2025-05-01):

  ┌────────────┬───────┬──────────────┬───────────────┐
  │ Percentile │ Hours │ Working Days │ Delivery Date │
  ├────────────┼───────┼──────────────┼───────────────┤
  │ P50        │ 230.8 │ 29           │ 2025-06-11    │
  ├────────────┼───────┼──────────────┼───────────────┤
  │ P80        │ 265.3 │ 34           │ 2025-06-18    │
  ├────────────┼───────┼──────────────┼───────────────┤
  │ P90        │ 283.4 │ 36           │ 2025-06-20    │
  ├────────────┼───────┼──────────────┼───────────────┤
  │ P95        │ 298.7 │ 38           │ 2025-06-24    │
  └────────────┴───────┴──────────────┴───────────────┘

  Key stats: Mean ~233h (30 days) · Std dev ~38h ·
  Both tasks are on the critical path 100% of the time.
```

You can then continue to explore more of the statistics with another prompt

```
Show the summary statistics for this simulation in a table.
```

which could result in something simlar to this table:

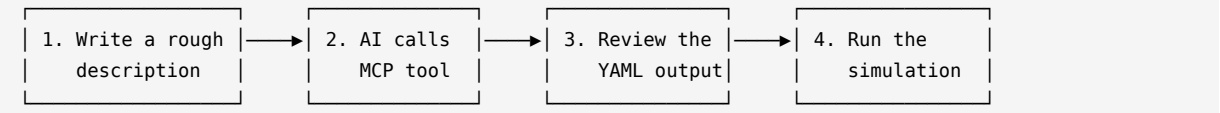
```
● From the previous simulation results:

┌──────────────────────────┬──────────────────────────────────┐
│ Statistic                │ Value                            │
├──────────────────────────┼──────────────────────────────────┤
│ Mean                     │ 232.64 hours (30 working days)   │
├──────────────────────────┼──────────────────────────────────┤
│ Median (P50)             │ 230.82 hours (29 working days)   │
├──────────────────────────┼──────────────────────────────────┤
│ Std Dev                  │ 37.97 hours                      │
├──────────────────────────┼──────────────────────────────────┤
│ Coefficient of Variation │ 0.1632                           │
├──────────────────────────┼──────────────────────────────────┤
│ Skewness                 │ 0.2703                           │
├──────────────────────────┼──────────────────────────────────┤
│ Excess Kurtosis          │ -0.3336                          │
├──────────────────────────┼──────────────────────────────────┤
│ Iterations               │ 10,000                           │
├──────────────────────────┼──────────────────────────────────┤
│ Hours per Day            │ 8.0                              │
└──────────────────────────┴──────────────────────────────────┘
```

16.7 Workflow overview

There are now four practical workflows depending on whether you start from natural language or existing YAML.

16.7.1 Two-step workflow: generate then simulate



1. **Write a description.** Open your MCP client (e.g., Claude Desktop) and describe the project in natural language. You do not need to worry about YAML syntax or exact field names — the parser handles that.
2. **The AI calls `generate_project_file`.** The assistant invokes the MCP tool with your description as input. The natural language parser extracts project metadata, tasks, sizes, dependencies, and estimates.
3. **Review the generated YAML.** The tool returns a complete YAML file. Review it for correctness — the parser captures structure faithfully, but you may want to add risks, uncertainty factors, or adjust task names.
4. **Run the simulation.** Save the YAML to a file and simulate:

```
mcprojsim simulate my_project.yaml --iterations 10000 --seed 42
```

16.7.2 One-step workflow: describe and simulate directly

If you want results immediately without reviewing the YAML first, ask the assistant to simulate directly:

> "Simulate this project with 10000 iterations and seed 42: Project name: API Migration Start date: 2026-07-01 Task 1: Design — Size: M Task 2: Build — Depends on Task 1 — Size: XL"

The assistant calls `simulate_project`, which generates the YAML internally and runs the simulation in a single step. The response includes both the generated YAML and the full simulation results (statistics, confidence intervals, delivery dates, critical paths).

16.7.3 YAML-first workflow: validate/simulate existing project files

If your team already stores project YAML in version control, you can use YAML-native MCP tools directly:

- `validate_project_yaml` for strict validation
- `simulate_project_yaml` to run simulation with runtime options

This avoids re-generating the project from natural language when you only need execution or checks.

16.7.4 YAML update workflow: apply natural-language change requests

Use `update_project_yaml` when you already have a project file and want the assistant to apply changes from natural-language instructions (for example, update sprint settings, change project metadata, or replace tasks).

This is useful for iterative planning sessions where the YAML evolves over time.

`simulate_project` parameters

Parameter	Type	Default	Description
`description`	string	required	Natural language project description
`iterations`	integer	10000	Number of Monte Carlo iterations
`seed`	integer	none	Random seed for reproducible results
`config_yaml`	string	none	Custom configuration YAML content (as a string)
`velocity_model`	string	none	Sprint velocity override: `empirical` or `neg_binomial`
`no_sickness`	boolean	`false`	Disable sprint sickness modeling for this run
`two_pass`	boolean	`false`	Enable criticality two-pass constrained scheduling
`pass1_iterations`	integer	none	Pass-1 iteration override when `two_pass` is enabled
`critical_paths_limit`	integer	none	Override number of full critical paths shown in output

The `config_yaml` parameter lets you pass custom T-shirt size mappings, uncertainty factor values, or simulation settings inline without needing a separate file. Runtime options such as `velocity_model`, `no_sickness`, and `two_pass` mirror the corresponding CLI behavior.

`simulate_project_yaml` parameters

`simulate_project_yaml` accepts the same runtime simulation options as `simulate_project`, but takes `project_yaml` instead of `description`.

Parameter	Type	Default	Description
`project_yaml`	string	required	Existing mcprojsim YAML project content
`iterations`	integer	10000	Number of Monte Carlo iterations
`seed`	integer	none	Random seed for reproducible results
`config_yaml`	string	none	Custom configuration YAML content (as a string)
`velocity_model`	string	none	Sprint velocity override: `empirical` or `neg_binomial`
`no_sickness`	boolean	`false`	Disable sprint sickness modeling for this run

Parameter	Type	Default	Description
`two_pass`	boolean	`false`	Enable criticality two-pass constrained scheduling
`pass1_iterations`	integer	none	Pass-1 iteration override when `two_pass` is enabled
`critical_paths_limit`	integer	none	Override number of full critical paths shown in output

Validation tools: when to use which

Tool	Input	Validation depth
`validate_project_description`	NL description	Parser-level warnings and obvious issues
`validate_generated_project_yaml`	NL description	Full parser/model validation on generated YAML
`validate_project_yaml`	Existing YAML	Full parser/model validation on provided YAML

`validate_project_description` is fast and useful during drafting. Use `validate_generated_project_yaml` or `validate_project_yaml` when you want strict behavior equivalent to normal YAML parsing and simulation setup.

`validate_generated_project_yaml` parameters

Parameter	Type	Default	Description
`description`	string	required	Natural language project description
`config_yaml`	string	none	Custom configuration YAML content (as a string)
`velocity_model`	string	none	Sprint velocity override: `empirical` or `neg_binomial`
`no_sickness`	boolean	`false`	Disable sprint sickness modeling for this validation

`validate_project_yaml` parameters

Parameter	Type	Default	Description
`project_yaml`	string	required	Existing mcprojsim YAML project content
`config_yaml`	string	none	Custom configuration YAML content (as a string)
`velocity_model`	string	none	Sprint velocity override: `empirical` or `neg_binomial`
`no_sickness`	boolean	`false`	Disable sprint sickness modeling for this validation

`update_project_yaml` parameters

Parameter	Type	Default	Description
`existing_yaml`	string	required	Existing mcprojsim project YAML content to update
`update_description`	string	required	Semi-structured NL instructions describing the changes to apply

Parameter	Type	Default	Description
`replace_tasks`	boolean	`false`	When `true`, replaces all existing tasks with tasks parsed from the update description. When `false` (default), existing tasks are preserved and only project metadata, sprint planning, resources, and calendars are updated

16.8 Translating NL intent to simulation options

One of the biggest benefits of MCP is that assistants can map user intent to runtime options.

16.8.1 Example 1: Sprint velocity override

User intent:

```
Simulate this backlog using negative binomial velocity modeling.
```

Assistant maps to MCP call:

- `simulate_project(..., velocity_model="neg_binomial")`

16.8.2 Example 2: Disable sickness for a scenario

User intent:

```
Run the same sprint simulation but ignore sickness effects.
```

Assistant maps to MCP call:

- `simulate_project(..., no_sickness=true)`

16.8.3 Example 3: Constrained scheduling with two-pass

User intent:

```
Use two-pass constrained scheduling with 2000 pass-1 iterations.
```

Assistant maps to MCP call:

- `simulate_project(..., two_pass=true, pass1_iterations=2000)`

16.8.4 Example 4: Existing YAML, no NL regeneration

User intent:

```
Validate and simulate this existing project YAML with seed 42.
```

Assistant maps to MCP calls:

- `validate_project_yaml(project_yaml=...)`
- `simulate_project_yaml(project_yaml=..., seed=42)`

16.8.5 Example 5: Apply update instructions to an existing YAML

User intent:

```
Update this project: change start date, add sprint planning history, keep existing tasks.
```

Assistant maps to MCP call:

- `update_project_yaml(existing_yaml=..., update_description=..., replace_tasks=false)`

If the user says to replace tasks entirely, the assistant can set `replace_tasks=true`.

You can also ask the assistant to validate first:

> "Validate this project description before generating the file."

The assistant will call `validate_project_description` and report any warnings (missing estimates, undefined dependencies) before you commit to generating the file.

16.9 Natural language input format

The parser is intentionally tolerant of formatting variations. It understands semi-structured text that follows a simple pattern: project metadata at the top, then numbered tasks with bullet-point attributes. It also accepts plain numbered lists, bullet lists, and inline properties — see Natural Language Input for the comprehensive reference.

The parser also supports sprint-planning phrasing, including sprint history and future sprint capacity override sections.

16.9.1 Project-level fields

Field	Pattern	Example
Project name	`Project name:` or `Project:`	`Project name: API Migration`
Start date	`Start date:` followed by `YYYY-MM-DD`	`Start date: 2026-07-01`
Description	`Description:`	`Description: Migrate legacy REST API to GraphQL`
Hours per day	`Hours per day:`	`Hours per day: 6`
Confidence levels	`Confidence levels:`	`Confidence levels: 50, 80, 90, 95`

All project-level fields are optional. If omitted, sensible defaults are used (project name defaults to "Untitled Project", confidence levels default to 50, 80, 90, 95).

16.9.2 Task definitions

Tasks are introduced with a header line (`Task N:`) followed by bullet points. The task number determines the generated task ID (`task_001`, `task_002`, etc.).

Each bullet can contain one of the following:

Attribute	Patterns recognized	Example
Task name	Any unrecognized bullet (first one becomes the name)	`- Design the new login flow`
T-shirt size	`Size: M`, `Size M`, `Size. XL`	`- Size: L`
Story points	`Story points: 5`, `Points: 8`	`- Story points: 13`
Explicit estimate	`Estimate: min/likely/max [unit]`	`- Estimate: 3/5/10 days`
Dependencies	`Depends on Task N`, `Depends on Task 1, Task 3`	`- Depends on Task 2`
Description	Second unrecognized bullet	`- Involves database migration`

16.9.3 Sprint-planning definitions (NL)

The NL parser can extract sprint planning information from sections such as:

- `Sprint planning:`
- `Sprint history SPR-001:`
- `Future sprint override 4:`

Supported sprint-planning bullets include:

Attribute	Patterns recognized	Example
Sprint length	`Sprint length: 2`, `2-week sprints`	`- Sprint length: 2`
Capacity mode	`Capacity mode: story points`, `Capacity mode: tasks`	`- Capacity mode: story points`
Planning confidence	`Planning confidence level: 80%`	`- Planning confidence level: 80%`
Removed work treatment	`Removed work treatment: churn only` / `reduce backlog`	`- Removed work treatment: churn only`
Velocity model	`Velocity model: empirical` / `negative binomial`	`- Velocity model: negative binomial`
Sprint history completed	`Done: 20 points`, `Delivered: 9 tasks`	`- Done: 20 points`
Sprint history spillover	`Carryover: 3 points`, `Rolled over: 2 tasks`	`- Carryover: 3 points`
Sprint history added/removed	`Scope added: 2 points`, `Scope removed: 1 points`	`- Scope added: 2 points`
Holiday factor	`Holiday factor: 90%`	`- Holiday factor: 90%`
Future override locator	`Future sprint override 4`, `start date: 2026-07-01`	`Future sprint override 4:`
Future override multiplier	`Holiday factor: 80%`, `Capacity multiplier: 0.9`	`- Capacity multiplier: 0.9`
Sprint sickness toggles	`Sickness: enabled`, `No sickness`	`- Sickness: enabled`

16.9.4 T-shirt size aliases

The parser normalizes a variety of size labels to standard sizes:

Input	Normalized to
`XS`, `Extra Small`	`XS`
`S`, `Small`	`S`
`M`, `Medium`, `Med`	`M`
`L`, `Large`	`L`
`XL`, `Extra Large`	`XL`
`XXL`, `Extra Extra Large`, `2XL`	`XXL`

Size matching is case-insensitive: `size: medium`, `Size: MEDIUM`, and `Size Medium` all resolve to `M`.

16.10 Examples

The following examples show different levels of detail and formatting styles, all of which the parser handles correctly.

16.10.1 Example 1: Minimal description with T-shirt sizes

The simplest useful input — just a project name, a start date, and tasks with sizes:

```
Project name: Website Redesign
Start date: 2026-04-15
Task 1:
- Gather requirements
- Size: S
Task 2:
- Create wireframes
- Depends on Task 1
- Size: M
Task 3:
- Build frontend
- Depends on Task 2
- Size: XL
Task 4:
- QA and launch
- Depends on Task 3
- Size: M
```

Generated YAML:

```yaml
project:
  name: "Website Redesign"
  start_date: "2026-04-15"
  confidence_levels: [50, 80, 90, 95]

tasks:
  - id: "task_001"
    name: "Gather requirements"
    estimate:
      t_shirt_size: "S"
    dependencies: []

  - id: "task_002"
    name: "Create wireframes"
    estimate:
      t_shirt_size: "M"
    dependencies: ["task_001"]

  - id: "task_003"
    name: "Build frontend"
    estimate:
      t_shirt_size: "XL"
    dependencies: ["task_002"]

  - id: "task_004"
    name: "QA and launch"
    estimate:
      t_shirt_size: "M"
    dependencies: ["task_003"]
```

16.10.2 Example 2: Story points with multiple dependencies

Using story points and tasks with fan-in dependencies:

```
Project: Mobile App v2
Start date: 2026-09-01
Task 1:
- API design
- Story points: 5
Task 2:
- iOS development
- Depends on Task 1
- Story points: 13
Task 3:
- Android development
- Depends on Task 1
- Story points: 13
Task 4:
- Integration testing
- Depends on Task 2, Task 3
- Story points: 8
```

Generated YAML:

```yaml
project:
  name: "Mobile App v2"
  start_date: "2026-09-01"
  confidence_levels: [50, 80, 90, 95]

tasks:
  - id: "task_001"
    name: "API design"
    estimate:
      story_points: 5
    dependencies: []

  - id: "task_002"
    name: "iOS development"
    estimate:
      story_points: 13
    dependencies: ["task_001"]

  - id: "task_003"
    name: "Android development"
    estimate:
      story_points: 13
    dependencies: ["task_001"]
  - id: "task_004"
    name: "Integration testing"
    estimate:
      story_points: 8
    dependencies: ["task_002", "task_003"]
```

16.10.3 Example 3: Explicit estimates with different units

When your team prefers numeric ranges over symbolic sizes:

```
Project name: Database Migration
Start date: 2026-05-01
Hours per day: 7
Task 1:
- Schema analysis
- Estimate: 2/3/5 days
Task 2:
- Write migration scripts
- Depends on Task 1
- Estimate: 5/8/15 days
Task 3:
- Data validation
- Depends on Task 2
- Estimate: 8/16/32 hours
Task 4:
- Production cutover
- Depends on Task 3
- Estimate: 1/2/3 days
```

Generated YAML:

```yaml
project:
  name: "Database Migration"
  start_date: "2026-05-01"
  hours_per_day: 7
  confidence_levels: [50, 80, 90, 95]

tasks:
  - id: "task_001"
    name: "Schema analysis"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"
    dependencies: []

  - id: "task_002"
    name: "Write migration scripts"
    estimate:
      low: 5
      expected: 8
      high: 15
      unit: "days"
    dependencies: ["task_001"]

  - id: "task_003"
    name: "Data validation"
    estimate:
      low: 8
      expected: 16
      high: 32
      unit: "hours"
    dependencies: ["task_002"]
  - id: "task_004"
    name: "Production cutover"
    estimate:
      low: 1
      expected: 2
      high: 3
      unit: "days"
    dependencies: ["task_003"]
```

16.10.4 Example 4: Sloppy formatting (typos, inconsistent punctuation)

The parser is designed to handle real-world input with minor typos and formatting inconsistencies. All of the following variations parse correctly:

```
Project name: Rework Web Interface
Start date: 2026-06-02
Task 1:
- Analyse existing UI
- Size: M
Task 2:
- Refine requirements
- Depends on Task1
- Size XL
Task 3:
- Design solution
- Depends on Task 2
- Size. XL
```

Notice: `- Depends on Task1` — no space between "Task" and "1" (still works) - `Size XL` — no colon after "Size" (still works) - `Size. XL` — period instead of colon (still works)

The parser normalizes all of these to the correct output.

16.10.5 Example 5: Inline task names and descriptions

You can put the task name directly on the header line, and add a description bullet:

```
Project: Platform Upgrade
Start date: 2026-08-01
Task 1: Upgrade framework
- Update all packages to latest versions
- Size: L
Task 2: Fix breaking changes
- Address deprecated APIs and removed features
- Depends on Task 1
- Size: XL
Task 3: Regression testing
- Depends on Task 2
- Size: M
```

Generated YAML:

```
project:
  name: "Platform Upgrade"
  start_date: "2026-08-01"
  confidence_levels: [50, 80, 90, 95]
tasks:
  - id: "task_001"
    name: "Upgrade framework"
    description: "Update all packages to latest versions"
    estimate:
      t_shirt_size: "L"
    dependencies: []

  - id: "task_002"
    name: "Fix breaking changes"
    description: "Address deprecated APIs and removed features"
    estimate:
      t_shirt_size: "XL"
    dependencies: ["task_001"]

  - id: "task_003"
    name: "Regression testing"
    estimate:
      t_shirt_size: "M"
    dependencies: ["task_002"]
```

16.10.6 Example 6: Verbose size labels

Teams unfamiliar with T-shirt abbreviations can use full words:

```
Project: Sprint 12
Start date: 2026-03-15
Task 1:
- Bug triage
- Size: Extra Small
Task 2:
- Feature implementation
- Depends on Task 1
- Size: Extra Large
Task 3:
- Documentation
- Size: Small
```

`Extra Small` becomes `XS`, `Extra Large` becomes `XL`, and `Small` becomes `S` in the generated YAML.

16.10.7 Example 7: Auto-detected lists with inline properties

You don't need `Task N:` headers at all. Plain numbered lists, bullet lists, and bracket-numbered lists are automatically detected as tasks. Sizes, estimates, and dependencies can appear inline on the same line:

```
Project name: Mobile App MVP
Start date: 2026-06-01

- Discovery and requirements (S)
- UX wireframes (M)
  Depends on Task 1
- Backend API, probably an XL
  Depends on Task 2
- iOS frontend (XL)
  Depends on Task 3
- QA and bug fixes, likely an L
  Depends on Task 4
- App store submission (S)
  Depends on Task 5
```

The parser auto-numbers the bullet items (1, 2, 3, …), extracts parenthesized sizes like `(S)` and `(M)`, and recognizes fuzzy phrasing like `probably an XL` and `likely an L`. Indented continuation lines are parsed as task properties.

Numbered lists also work — and preserve the original numbering:

```
Project name: Data Pipeline Rebuild
Start date: 2026-07-01

[1] Audit existing ETL jobs [S]
[2] Design new pipeline architecture [M] depends on Task 1
[3] Implement ingestion layer 3–5 days depends on Task 2
[4] Build transformation engine [XL] depends on Task 3
```

See Natural Language Input for the full range of supported list formats and inline properties.

16.11 Using the parser from the command line

You do not need an MCP client or an AI service to use the natural language parser. The `mcprojsim generate` command runs the same parser directly from the terminal:

```
mcprojsim generate description.txt -o my_project.yaml
```

See Running Simulations — `mcprojsim generate` for the full command reference and all options.

16.12 Using the MCP tools directly from Python

While the MCP server is the primary way to use the parser through an AI assistant, you can also call the parser directly from Python code:

```
from mcprojsim.nl_parser import NLProjectParser

description = """
Project name: My Project
Start date: 2026-01-15
Task 1:
- Design phase
- Size: M
Task 2:
- Implementation
- Depends on Task 1
- Size: XL
"""

parser = NLProjectParser()

# Parse and generate YAML in one step
yaml_output = parser.parse_and_generate(description)
print(yaml_output)

# Or parse first, inspect, then generate
project = parser.parse(description)
print(f"Project: {project.name}")
print(f"Tasks: {len(project.tasks)}")
for task in project.tasks:
    print(f"  Task {task.number}: {task.name} ({task.t_shirt_size})")

yaml_output = parser.to_yaml(project)

# Save to file
with open("my_project.yaml", "w") as f:
    f.write(yaml_output)
```

16.13 Tips & tricks

16.13.1 Start simple, then enrich

The parser generates a minimal but valid project file. After generating the initial YAML, you can manually add:

- **Uncertainty factors** (`team_experience`, `requirements_maturity`, etc.)
- **Task-level risks** (e.g., "API complexity higher than expected")
- **Project-level risks** (e.g., "Key team member leaves")
- **Custom confidence levels** or threshold overrides

This two-step workflow — generate the skeleton, then enrich — is often faster than writing everything from scratch.

16.13.2 Use the validation tool first

Before generating a file, ask your AI assistant to validate the description. This catches common problems early:

> "Validate this project description for me."

The `validate_project_description` tool will flag:

- Missing project name or start date
- Tasks without any estimate
- Dependencies referencing non-existent tasks

16.13.3 Be consistent with task numbering

When using `Task N:` headers, the parser uses these numbers to resolve dependencies. If you write `Task 1`, `Task 2`, `Task 5` (skipping 3 and 4), the parser will still work — but a dependency like `Depends on Task 3` will not resolve to anything.

When using auto-detected lists (plain numbered or bullets), the same numbering rules apply. Bullet lists are auto-numbered starting from 1.

16.13.4 Mix estimation methods across tasks

Different tasks can use different estimation methods within the same project. For example:

```
Task 1:
- Well-understood work
- Estimate: 3/5/8 days
Task 2:
- Vaguely scoped work
- Size: XL
Task 3:
- Agile team estimate
- Story points: 8
```

The parser handles this correctly, and `mcprojsim` will resolve each estimate type using the appropriate configuration mapping.

16.13.5 Use a configuration file for T-shirt sizes and story points

The generated YAML uses symbolic estimates (`t_shirt_size: "M"` or `story_points: 5`). These are resolved to numeric ranges at simulation time using a configuration file. If you want custom mappings, create a `config.yaml`:

```
t_shirt_sizes:
  story:
    M:
      low: 40
      expected: 60
      high: 120
  epic:
    M:
      low: 200
      expected: 480
      high: 1200

t_shirt_size_default_category: epic
```

Then run the simulation with:

```
mcprojsim simulate my_project.yaml --config config.yaml
```

See Configuration for full details on customizing the mappings.

16.13.6 Generating files for quick "what-if" scenarios

The MCP server is particularly useful for rapid scenario comparison. You can ask your AI assistant:

> "Generate a project file with 5 tasks, all size M, in a chain of dependencies — then a second version where tasks 2 and 3 run in parallel."

Compare the simulation results side by side to see how parallelism affects the schedule distribution.

16.13.7 Handling the generated YAML output

When the AI assistant returns the generated YAML, you can:

1. **Copy it into a file** and run `mcprojsim validate` to double-check.
2. **Ask the assistant to save it** if your MCP client supports file operations.
3. **Pipe it into the simulation** if you prefer a scripted workflow:

```
echo '<paste YAML here>' > project.yaml && mcprojsim simulate project.yaml
```

16.13.8 Known limitations

The natural language parser is a pattern-based parser, not a full natural language understanding system. Keep these points in mind:

- **Uncertainty factors** are not extracted from natural language input. Add them to the YAML file after generation.
- **Resource definitions** and **calendar constraints** are supported, but only through recognized semi-structured patterns (not arbitrary prose).
- **Project-level risks** are not extracted from the description. Add them manually.
- The parser expects either **numbered task headers** (`Task 1:`, `Task 2:`) or **auto-detected lists** (plain numbered, bullet, or bracket lists). The two modes cannot be mixed in the same description. See Natural Language Input for details.
- **Circular dependencies** are not detected by the parser — they will be caught when you load the file with `mcprojsim validate`.
- Advanced structures such as full volatility-overlay and spillover calibration blocks are still best edited directly in YAML after generation.

CHAPTER 17

Examples

- Examples
  - Basic Project with Explicit Estimates
  - T-Shirt Sizing
  - Story Points
  - Complex Project with Risks
  - Log-Normal Distribution
  - Constrained Scheduling with `team_size`
  - Constrained Scheduling with Explicit Resources and Calendars
  - Natural Language Project Generation
    - Basic text input (dependency-only)
    - Text input with resources and calendars (constrained)
  - Sprint Planning
    - Minimal story-point forecast
    - Advanced sprint forecast
    - Tasks mode and external history
  - Running Examples
    - Common CLI options
    - Additional example files

This page provides practical examples of project definitions, progressing from simple to complex. Each example includes real simulation output. For complete specification details, see the Formal Grammar.

All outputs below were generated with `--minimal --seed 42` for reproducibility. Use `--seed` to get identical results on your machine.

17.1 Basic Project with Explicit Estimates

A simple project with two sequential tasks using three-point (min / most likely / max) estimates. This is the simplest useful project definition.

```
project:
  name: "Website Refresh"
  description: "Small example project"
  start_date: "2026-04-01"
  confidence_levels: [50, 80, 90]
  distribution: "lognormal"

tasks:
  - id: "task_001"
    name: "Design updates"
    estimate:
      low: 2
      expected: 3
      high: 5
      unit: "days"

  - id: "task_002"
    name: "Frontend changes"
    estimate:
      low: 4
      expected: 6
      high: 10
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"

  - id: "task_003"
    name: "Backend changes"
    estimate:
      low: 4
      expected: 6
      high: 20
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"```

```bash
mcprojsim simulate examples/quickstart_example.yaml --minimal --seed 42
```
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Website Refresh
  Start Date: 2026-04-01
  Number of Tasks: 3
  Effective Default Distribution: lognormal
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 2
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 126.41 hours (16 working days)
  Median (P50): 111.31 hours
  Std Dev: 53.21 hours
  Minimum: 68.61 hours
  Maximum: 1115.92 hours

Project Effort Statistical Summary:
  Mean: 187.83 person-hours (24 person-days)
  Median (P50): 173.24 person-hours
  Std Dev: 57.56 person-hours
  Minimum: 110.54 person-hours
  Maximum: 1199.36 person-hours

Calendar Time Confidence Intervals:
  P50: 111.31 hours (14 working days)  (2026-04-21)
  P80: 147.18 hours (19 working days)  (2026-04-28)
  P90: 181.17 hours (23 working days)  (2026-05-04)
```

Key observations: two sequential tasks, no parallelism (`Max Parallel Tasks: 1`), `dependency_only` scheduling.

See `examples/quickstart_example.yaml` in the repository.

17.2 T-Shirt Sizing

For quick estimation using relative sizes (`XS`, `S`, `M`, `L`, `XL`, `XXL`). T-shirt sizes map to default effort ranges that are configurable via the configuration file.

```yaml
project:
  name: "Mobile App Development"
  description: "Example project using T-shirt sizing for effort estimates"
  start_date: "2025-11-01"
  confidence_levels: [10, 50, 75, 80, 85, 90, 95, 99]
  probability_red_threshold: 0.50
  probability_green_threshold: 0.90

tasks:
  - id: "design_ui"
    name: "UI/UX Design"
    estimate:
      t_shirt_size: "M"
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "high"

  - id: "setup_backend"
    name: "Backend Infrastructure Setup"
    estimate:
      t_shirt_size: "L"
    dependencies: []
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"

  - id: "api_development"
    name: "REST API Development"
    estimate:
      t_shirt_size: "XL"
    dependencies: ["setup_backend"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "high"
    risks:
      - id: "api_complexity"
        name: "API complexity higher than expected"
        probability: 0.30
        impact:
          type: "absolute"
          value: 5
          unit: "hours"

  - id: "mobile_app_dev"
    name: "Mobile App Development"
    estimate:
      t_shirt_size: "XXL"
    dependencies: ["design_ui", "api_development"]
    uncertainty_factors:
      team_experience: "low"
      technical_complexity: "high"

  - id: "integration_testing"
    name: "Integration Testing"
    estimate:
      t_shirt_size: "M"
    dependencies: ["mobile_app_dev"]
    uncertainty_factors:
      requirements_maturity: "high"

  - id: "deployment"
    name: "Deployment and Go-Live"
    estimate:
      t_shirt_size: "S"
    dependencies: ["integration_testing"]
```

```
mcprojsim simulate examples/tshirt_sizing_project.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Mobile App Development
  Start Date: 2025-11-01
  Number of Tasks: 6
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 2
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 2635.93 hours (330 working days)
  Median (P50): 2584.12 hours
  Std Dev: 436.95 hours
  Minimum: 1653.07 hours
  Maximum: 4602.84 hours

Project Effort Statistical Summary:
  Mean: 2622.12 person-hours (328 person-days)
  Median (P50): 2578.96 person-hours
  Std Dev: 384.08 person-hours
  Minimum: 1721.88 person-hours
  Maximum: 3990.15 person-hours

Calendar Time Confidence Intervals:
  P10: 2107.45 hours (264 working days)  (2026-11-05)
  P50: 2584.12 hours (324 working days)  (2027-01-28)
  P75: 2921.42 hours (366 working days)  (2027-03-29)
  P80: 3006.12 hours (376 working days)  (2027-04-12)
  P85: 3099.16 hours (388 working days)  (2027-04-28)
  P90: 3227.73 hours (404 working days)  (2027-05-20)
  P95: 3411.59 hours (427 working days)  (2027-06-22)
  P99: 3818.90 hours (478 working days)  (2027-09-01)
```

Key observations: two independent starting tasks yield `Max Parallel Tasks: 2`. The wide P10–P99 spread (938–1590 working days) reflects the inherent uncertainty of T-shirt sizing for large effort items.

Note: T-shirt size and story point estimates must **not** include a `unit` field in the project file. The unit is controlled by the configuration.

See `examples/tshirt_sizing_project.yaml` in the repository.

17.3 Story Points

For agile-style relative estimation using calibrated story point mappings. Default mappings are configurable via the configuration file.

```yaml
project:
  name: "Tiny Landing Page"
  description: "Story Point sizing example"
  start_date: "2026-03-01"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Design page"
    estimate:
      story_points: 2
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "high"

  - id: "task_002"
    name: "Build page"
    estimate:
      story_points: 5
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      technical_complexity: "medium"

  - id: "task_003"
    name: "Deploy page"
    estimate:
      story_points: 1
    dependencies: ["task_002"]
```

```
mcprojsim simulate examples/story_points_walkthrough_project.yaml --minimal --table --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
┌──────────────────────────────┬──────────────────────────────────────┐
│ Field                        │ Value                                │
├──────────────────────────────┼──────────────────────────────────────┤
│ Project                      │ Tiny Landing Page                    │
│ Start Date                   │ 2026-03-01                           │
│ Number of Tasks              │ 3                                    │
│ Effective Default Distribution │ triangular                         │
│ T-Shirt Category Used        │ story                                │
│ Hours per Day                │ 8.0                                  │
│ Max Parallel Tasks           │ 1                                    │
│ Schedule Mode                │ dependency_only                      │
└──────────────────────────────┴──────────────────────────────────────┘

Calendar Time Statistical Summary:
┌─────────────┬────────────────────────────────────────────────────────┐
│ Metric      │ Value                                                  │
├─────────────┼────────────────────────────────────────────────────────┤
│ Mean        │ 79.98 hours (10 working days)                          │
│ Median (P50) │ 79.44 hours                                           │
│ Std Dev     │ 11.65 hours                                            │
│ Minimum     │ 47.31 hours                                            │
│ Maximum     │ 121.18 hours                                           │
└─────────────┴────────────────────────────────────────────────────────┘

Project Effort Statistical Summary:
┌─────────────┬────────────────────────────────────────────────────────┐
│ Metric      │ Value                                                  │
├─────────────┼────────────────────────────────────────────────────────┤
│ Mean        │ 79.98 person-hours (10 person-days)                    │
│ Median (P50) │ 79.44 person-hours                                    │
│ Std Dev     │ 11.65 person-hours                                     │
│ Minimum     │ 47.31 person-hours                                     │
│ Maximum     │ 121.18 person-hours                                    │
└─────────────┴────────────────────────────────────────────────────────┘

Calendar Time Confidence Intervals:
┌────────────┬─────────┬──────────────┬──────────────────────────────┐
│ Percentile │   Hours │   Working Days │ Date                       │
├────────────┼─────────┼──────────────┼──────────────────────────────┤
│ P50        │   79.44 │           10 │ 2026-03-13                   │
│ P80        │   90.19 │           12 │ 2026-03-17                   │
│ P90        │   95.57 │           12 │ 2026-03-17                   │
└────────────┴─────────┴──────────────┴──────────────────────────────┘
```

Key observations: small project with low uncertainty — the CV is only 0.15 and P50 to P90 spans just 3 working days.

See `examples/story_points_walkthrough_project.yaml` in the repository.

17.4 Complex Project with Risks

A realistic project with 8 tasks, complex dependencies, uncertainty factors, and both task-level and project-level risks.

```yaml
project:
  name: "Customer Portal Redesign"
  description: "Next-generation customer portal with enhanced features"
  start_date: "2025-11-01"
  confidence_levels: [25, 50, 75, 80, 85, 90, 95, 99]
  # Probability thresholds for thermometer visualization
  probability_red_threshold: 0.50    # Below 50% shown as red (high risk)
  probability_green_threshold: 0.90  # Above 90% shown as green (low risk)

project_risks:
  - id: "risk_001"
    name: "Key developer leaves"
    probability: 0.15
    impact:
      type: "percentage"
      value: 20
    description: "Risk of losing senior developer mid-project"

  - id: "risk_002"
    name: "Requirements change"
    probability: 0.30
    impact:
      type: "absolute"
      value: 10
      unit: "days"

tasks:
  - id: "task_001"
    name: "Database schema design"
    description: "Design normalized schema for customer data"
    estimate:
      low: 3
      expected: 5
      high: 10
      unit: "days"
    dependencies: []
    uncertainty_factors:
      team_experience: "high"
      requirements_maturity: "medium"
      technical_complexity: "low"
      team_distribution: "colocated"
      integration_complexity: "low"
    risks:
      - id: "task_risk_001"
        name: "Schema migration issues"
        probability: 0.20
        impact:
          type: "absolute"
          value: 2
          unit: "days"

  - id: "task_002"
    name: "API endpoint implementation"
    description: "Implement RESTful API endpoints"
    estimate:
      low: 5
      expected: 8
      high: 15
      unit: "days"
    dependencies: ["task_001"]
    uncertainty_factors:
      team_experience: "medium"
      requirements_maturity: "high"
      technical_complexity: "medium"
      team_distribution: "colocated"
      integration_complexity: "medium"
```

```
mcprojsim simulate examples/sample_project.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Customer Portal Redesign
  Start Date: 2025-11-01
  Number of Tasks: 8
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 2
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 580.89 hours (73 working days)
  Median (P50): 574.73 hours
  Std Dev: 78.46 hours
  Minimum: 365.50 hours
  Maximum: 924.02 hours

Project Effort Statistical Summary:
  Mean: 686.35 person-hours (86 person-days)
  Median (P50): 684.08 person-hours
  Std Dev: 61.54 person-hours
  Minimum: 487.94 person-hours
  Maximum: 902.14 person-hours

Calendar Time Confidence Intervals:
  P25: 524.00 hours (66 working days)  (2026-02-02)
  P50: 574.73 hours (72 working days)  (2026-02-10)
  P75: 629.98 hours (79 working days)  (2026-02-19)
  P80: 645.37 hours (81 working days)  (2026-02-23)
  P85: 663.34 hours (83 working days)  (2026-02-25)
  P90: 685.64 hours (86 working days)  (2026-03-02)
  P95: 722.71 hours (91 working days)  (2026-03-09)
  P99: 789.05 hours (99 working days)  (2026-03-19)
```

Key observations:

- Effort and calendar time differ (`86 person-days` effort vs `73 working days` calendar) because some tasks run in parallel.
- The positive skewness (0.48) shows a right-skewed distribution — risks and uncertainty create a longer tail toward delays.

See `examples/sample_project.yaml` in the repository.

17.5 Log-Normal Distribution

For tasks where extreme overruns are more probable than a triangular distribution predicts:

```yaml
tasks:
  - id: "research"
    name: "Research new technology"
    estimate:
      distribution: "lognormal"
      low: 2
      expected: 5
      high: 14
      unit: "days"
    dependencies: []
```

The shifted log-normal distribution produces a heavier right tail, making it suitable for research, exploration,

or tasks with high uncertainty about upper bounds.

17.6 Constrained Scheduling with `team_size`

The simplest way to activate resource-constrained scheduling: add `team_size` to the project metadata. This auto-generates default resources.

```
project:
  name: "Team Size Demo"
  start_date: "2026-04-01"
  hours_per_day: 8
  team_size: 10

tasks:
  - id: "task_001"
    name: "Task 1"
    estimate: { low: 8, expected: 16, high: 24 }
  - id: "task_002"
    name: "Task 2"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]
```

```
mcprojsim simulate examples/team_size_demo_with_team_size.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Team Size Demo
  Start Date: 2026-04-01
  Number of Tasks: 2
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: resource_constrained

Calendar Time Statistical Summary:
  Mean: 335.59 hours (42 working days)
  Median (P50): 338.16 hours
  Std Dev: 46.63 hours
  Minimum: 196.00 hours
  Maximum: 486.77 hours

Project Effort Statistical Summary:
  Mean: 82.69 person-hours (11 person-days)
  Median (P50): 82.16 person-hours
  Std Dev: 11.85 person-hours
  Minimum: 52.00 person-hours
  Maximum: 118.77 person-hours

Calendar Time Confidence Intervals:
  P10: 291.31 hours (37 working days)  (2026-05-22)
  P25: 314.03 hours (40 working days)  (2026-05-27)
  P50: 338.16 hours (43 working days)  (2026-06-01)
  P75: 363.12 hours (46 working days)  (2026-06-04)
  P80: 365.33 hours (46 working days)  (2026-06-04)
  P85: 384.05 hours (49 working days)  (2026-06-09)
  P90: 386.92 hours (49 working days)  (2026-06-09)
  P95: 390.98 hours (49 working days)  (2026-06-09)
  P99: 461.04 hours (58 working days)  (2026-06-22)
```

Compare with the same project **without** `team_size` (dependency-only mode):

```
mcprojsim simulate examples/team_size_demo_base.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Team Size Demo
  Start Date: 2026-04-01
  Number of Tasks: 2
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 82.66 hours (11 working days)
  Median (P50): 82.16 hours
  Std Dev: 11.94 hours
  Minimum: 49.42 hours
  Maximum: 116.52 hours

Project Effort Statistical Summary:
  Mean: 82.66 person-hours (11 person-days)
  Median (P50): 82.16 person-hours
  Std Dev: 11.94 person-hours
  Minimum: 49.42 person-hours
  Maximum: 116.52 person-hours

Calendar Time Confidence Intervals:
  P10: 66.98 hours (9 working days)  (2026-04-14)
  P25: 74.24 hours (10 working days)  (2026-04-15)
  P50: 82.16 hours (11 working days)  (2026-04-16)
  P75: 91.13 hours (12 working days)  (2026-04-17)
  P80: 93.49 hours (12 working days)  (2026-04-17)
  P85: 95.91 hours (12 working days)  (2026-04-17)
  P90: 98.90 hours (13 working days)  (2026-04-20)
  P95: 103.09 hours (13 working days)  (2026-04-20)
  P99: 108.98 hours (14 working days)  (2026-04-21)
```

Key observations:

- Adding `team_size` switches from `dependency_only` to `resource_constrained`.
- Calendar time increases significantly (P50: 17 → 67 working days) because resources are assigned one at a time by default (`max_resources: 1`), and working calendars now account for weekends and non-working hours.
- Effort remains the same — only elapsed calendar time changes.

See `examples/team_size_demo_base.yaml` and `examples/team_size_demo_with_team_size.yaml` in the repository.

17.7 Constrained Scheduling with Explicit Resources and Calendars

For full control, define resources with individual skill levels, productivity, sickness probability, and planned absences. Attach resources to named calendars with custom work patterns.

```yaml
project:
  name: "Onboarding Portal"
  description: "Constrained scheduling example with explicit resources and calendars"
  start_date: "2026-04-01"
  hours_per_day: 8
  confidence_levels: [50, 80, 90, 95]

tasks:
  - id: "task_001"
    name: "Requirements"
    estimate: { low: 8, expected: 16, high: 24 }

  - id: "task_002"
    name: "Implementation"
    estimate: { low: 40, expected: 64, high: 96 }
    dependencies: ["task_001"]
    resources: ["alice", "bob"]
    max_resources: 2
    min_experience_level: 2

  - id: "task_003"
    name: "Testing"
    estimate: { low: 16, expected: 24, high: 40 }
    dependencies: ["task_002"]

resources:
  - name: "alice"
    experience_level: 3
    productivity_level: 1.1
    sickness_prob: 0.02
    planned_absence: ["2026-04-22"]

  - name: "bob"
    calendar: "part_time"
    experience_level: 2
    productivity_level: 0.9
    availability: 0.8
    sickness_prob: 0.04

calendars:
  - id: "default"
    work_hours_per_day: 8
    work_days: [1, 2, 3, 4, 5]
    holidays: ["2026-04-10"]

  - id: "part_time"
    work_hours_per_day: 6
    work_days: [1, 2, 3, 4]
```

```
mcprojsim simulate examples/constrained_portal.yaml --minimal --seed 42 --iterations 200
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Onboarding Portal
  Start Date: 2026-04-01
  Number of Tasks: 3
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: resource_constrained

Calendar Time Statistical Summary:
  Mean: 377.05 hours (48 working days)
  Median (P50): 363.68 hours
  Std Dev: 50.00 hours
  Minimum: 288.97 hours
  Maximum: 531.94 hours

Project Effort Statistical Summary:
  Mean: 108.80 person-hours (14 person-days)
  Median (P50): 108.03 person-hours
  Std Dev: 12.31 person-hours
  Minimum: 74.53 person-hours
  Maximum: 137.90 person-hours

Calendar Time Confidence Intervals:
  P50: 363.68 hours (46 working days)  (2026-06-04)
  P80: 391.05 hours (49 working days)  (2026-06-09)
  P90: 462.23 hours (58 working days)  (2026-06-22)
  P95: 482.20 hours (61 working days)  (2026-06-25)
```

Key observations:

- `resources` on `task_002` restricts it to only `alice` and `bob`.
- `max_resources: 2` allows both to work in parallel on that task.
- `min_experience_level: 2` filters out any resource below that skill tier.
- `sickness_prob` introduces stochastic sick days that vary across iterations.
- `planned_absence` blocks specific dates deterministically.
- Bob's `part_time` calendar (6 hours/day, Mon–Thu) reduces his available capacity.
- Calendar time (78 working days) is much larger than effort (22 person-days) due to calendar constraints, weekends, holidays, and sickness.

For a full constrained walkthrough with incremental complexity, see the Constrained Scheduling Guide.

17.8 Natural Language Project Generation

The `generate` command converts plain-text project descriptions into valid YAML project files. This lets you sketch a project quickly and iterate.

17.8.1 Basic text input (dependency-only)

```
Project name: Rework Web Interface
Start date: 2026-06-02
Task 1:
- Analyse existing UI
- Size: M
Task 2:
- Refine requirements
- Depends on Task1
- Size XL
Task 3:
- Design solution
- Depends on Task 2
- Size. XL
```

```
mcprojsim generate examples/nl_example.txt -o .build/gen-examples/nl_project.yaml
mcprojsim simulate .build/gen-examples/nl_project.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Rework Web Interface
  Start Date: 2026-06-02
  Number of Tasks: 3
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 1052.49 hours (132 working days)
  Median (P50): 1042.54 hours
  Std Dev: 132.88 hours
  Minimum: 736.96 hours
  Maximum: 1533.41 hours

Project Effort Statistical Summary:
  Mean: 1052.49 person-hours (132 person-days)
  Median (P50): 1042.54 person-hours
  Std Dev: 132.88 person-hours
  Minimum: 736.96 person-hours
  Maximum: 1533.41 person-hours

Calendar Time Confidence Intervals:
  P50: 1042.54 hours (131 working days)  (2026-12-02)
  P80: 1167.75 hours (146 working days)  (2026-12-23)
  P90: 1233.46 hours (155 working days)  (2027-01-05)
  P95: 1285.85 hours (161 working days)  (2027-01-13)
```

17.8.2 Text input with resources and calendars (constrained)

The `generate` command also supports resource definitions, calendar definitions, and task-level resource constraints:

```
Project name: Platform Migration
Start date: 2026-05-01
Hours per day: 8

Resource 1: Alice
- Experience: 3
- Productivity: 1.1
- Sickness: 0.02
- Absence: 2026-05-15

Resource 2: Bob
- Experience: 2
- Productivity: 1.0
- Availability: 0.8
- Sickness: 0.03

Resource 3: Carol
- Experience: 2
- Calendar: part_time
- Availability: 0.75
- Productivity: 0.9
- Sickness: 0.04
- Absence: 2026-06-01, 2026-06-02

Calendar: default
- Work hours: 8
- Work days: 1, 2, 3, 4, 5
- Holidays: 2026-05-25

Calendar: part_time
- Work hours: 6
- Work days: 1, 2, 3, 4

Task 1: Architecture design
- Estimate: 16/24/40 hours
- Min experience: 2

Task 2: Core implementation
- Estimate: 80/120/180 hours
- Depends on Task 1
- Resources: Alice, Bob, Carol
- Max resources: 2
- Min experience: 2

Task 3: Data migration
- Estimate: 40/64/96 hours
- Depends on Task 2
- Resources: Alice
- Min experience: 3

Task 4: Verification and rollout
- Estimate: 24/40/64 hours
- Depends on Task 3
- Resources: Alice, Bob, Carol
- Max resources: 2
```

```
mcprojsim generate examples/nl_constrained_example.txt -o .build/gen-examples/nl_constrained_project.yaml
mcprojsim simulate .build/gen-examples/nl_constrained_project.yaml --minimal --seed 42 --iterations 200
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Platform Migration
  Start Date: 2026-05-01
  Number of Tasks: 4
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 1
  Schedule Mode: resource_constrained

Calendar Time Statistical Summary:
  Mean: 840.04 hours (106 working days)
  Median (P50): 819.88 hours
  Std Dev: 90.25 hours
  Minimum: 630.72 hours
  Maximum: 1272.02 hours

Project Effort Statistical Summary:
  Mean: 264.61 person-hours (34 person-days)
  Median (P50): 263.89 person-hours
  Std Dev: 24.51 person-hours
  Minimum: 206.48 person-hours
  Maximum: 326.63 person-hours

Calendar Time Confidence Intervals:
  P50: 819.88 hours (103 working days)  (2026-09-23)
  P80: 918.51 hours (115 working days)  (2026-10-09)
  P90: 942.87 hours (118 working days)  (2026-10-14)
  P95: 991.48 hours (124 working days)  (2026-10-22)
```

The generated YAML includes full `resources:` and `calendars:` sections. Use `--validate-only` to check your description before generating:

```
mcprojsim generate examples/nl_constrained_example.txt --validate-only
```

```
✓ Valid: 'Platform Migration' with 4 task(s)
```

See `examples/nl_example.txt` and `examples/nl_constrained_example.txt` in the repository.

17.9 Sprint Planning

Sprint-planning examples combine the normal task simulation with a sprint-based forecast built from historical sprint results.

17.9.1 Minimal story-point forecast

```
project:
  name: "Sprint Planning Minimal"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Foundation"
    estimate:
      story_points: 3
  - id: "task_002"
    name: "Backend API"
    estimate:
      story_points: 5
  - id: "task_003"
    name: "Frontend"
    estimate:
      story_points: 8
  - id: "task_004"
    name: "Integration"
    estimate:
      story_points: 5
  - id: "task_005"
    name: "Release prep"
    estimate:
      story_points: 3

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  planning_confidence_level: 0.8
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 10
      spillover_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 9
      spillover_story_points: 2
      added_story_points: 1
    - sprint_id: "SPR-003"
      completed_story_points: 11
      spillover_story_points: 1
      removed_story_points: 1
```

```
mcprojsim simulate examples/sprint_planning_minimal.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Sprint Planning Minimal
  Start Date: 2026-05-04
  Number of Tasks: 5
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 5
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 74.92 hours (10 working days)
  Median (P50): 72.58 hours
  Std Dev: 16.38 hours
  Minimum: 41.37 hours
  Maximum: 119.56 hours

Project Effort Statistical Summary:
  Mean: 210.51 person-hours (27 person-days)
  Median (P50): 209.49 person-hours
  Std Dev: 21.91 person-hours
  Minimum: 146.41 person-hours
  Maximum: 290.15 person-hours

Calendar Time Confidence Intervals:
  P50: 72.58 hours (10 working days)  (2026-05-18)
  P80: 89.98 hours (12 working days)  (2026-05-20)
  P90: 98.82 hours (13 working days)  (2026-05-21)

Sprint Planning Summary:
Sprint Length: 2 weeks
Planning Confidence Level: 80%
Removed Work Treatment: churn_only
Velocity Model: empirical
Planned Commitment Guidance: 7.55
Historical Sampling Mode: matching_cadence
Historical Observations: 3
Carryover Mean: 0.00
Aggregate Spillover Rate: 0.0000
Observed Disruption Frequency: 0.0000

Sprint Count Statistical Summary:
Mean: 3.00 sprints
Median (P50): 3.00 sprints
Std Dev: 0.00 sprints
Minimum: 3.00 sprints
Maximum: 3.00 sprints
Coefficient of Variation: 0.0000

Sprint Count Confidence Intervals:
  P50: 3 sprints  (2026-06-01)
  P80: 3 sprints  (2026-06-01)
  P90: 3 sprints  (2026-06-01)
```

Key observations:

- Uses `story_points` as the sprint-capacity unit.
- Three historical sprint rows are enough to start forecasting.
- The sprint summary shows commitment guidance and sprint-count percentiles alongside the regular project forecast.

17.9.2 Advanced sprint forecast

```
project:
  name: "Sprint Planning Advanced"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Discovery"
    estimate:
      story_points: 3
  - id: "task_002"
    name: "Core API"
    estimate:
      story_points: 8
  - id: "task_003"
    name: "UI flow"
    estimate:
      story_points: 5
  - id: "task_004"
    name: "Reporting"
    estimate:
      story_points: 8
    spillover_probability_override: 0.55
  - id: "task_005"
    name: "Hardening"
    estimate:
      story_points: 5
  - id: "task_006"
    name: "Rollout"
    estimate:
      story_points: 3

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  planning_confidence_level: 0.85
  removed_work_treatment: reduce_backlog
  future_sprint_overrides:
    - sprint_number: 2
      capacity_multiplier: 0.7
      notes: "Shared release sprint"
    - start_date: "2026-06-29"
      holiday_factor: 0.8
      notes: "Summer vacation period"
  volatility_overlay:
    enabled: true
    disruption_probability: 0.25
    disruption_multiplier_low: 0.6
    disruption_multiplier_expected: 0.8
    disruption_multiplier_high: 1.0
  spillover:
    enabled: true
    model: logistic
    size_reference_points: 5
    consumed_fraction_alpha: 3.25
    consumed_fraction_beta: 1.75
  history:
    - sprint_id: "SPR-001"
      completed_story_points: 12
      spillover_story_points: 1
      added_story_points: 1
    - sprint_id: "SPR-002"
      completed_story_points: 10
```

```
mcprojsim simulate examples/sprint_planning_advanced.yaml --minimal --table --seed 42 --iterations 200
```

```
mcprojsim v0.15.2

Project Overview:
┌──────────────────────────────┬──────────────────────────────────────┐
│ Field                        │ Value                                │
├──────────────────────────────┼──────────────────────────────────────┤
│ Project                      │ Sprint Planning Advanced             │
│ Start Date                   │ 2026-05-04                           │
│ Number of Tasks              │ 6                                    │
│ Effective Default Distribution │ triangular                         │
│ T-Shirt Category Used        │ story                                │
│ Hours per Day                │ 8.0                                  │
│ Max Parallel Tasks           │ 6                                    │
│ Schedule Mode                │ dependency_only                      │
└──────────────────────────────┴──────────────────────────────────────┘

Calendar Time Statistical Summary:
┌─────────────┬────────────────────────────────────────────────────────┐
│ Metric      │ Value                                                  │
├─────────────┼────────────────────────────────────────────────────────┤
│ Mean        │ 86.31 hours (11 working days)                          │
│ Median (P50) │ 85.93 hours                                           │
│ Std Dev     │ 14.55 hours                                            │
│ Minimum     │ 53.43 hours                                            │
│ Maximum     │ 118.88 hours                                           │
└─────────────┴────────────────────────────────────────────────────────┘

Project Effort Statistical Summary:
┌─────────────┬────────────────────────────────────────────────────────┐
│ Metric      │ Value                                                  │
├─────────────┼────────────────────────────────────────────────────────┤
│ Mean        │ 287.15 person-hours (36 person-days)                   │
│ Median (P50) │ 287.50 person-hours                                   │
│ Std Dev     │ 27.30 person-hours                                     │
│ Minimum     │ 227.60 person-hours                                    │
│ Maximum     │ 369.30 person-hours                                    │
└─────────────┴────────────────────────────────────────────────────────┘

Calendar Time Confidence Intervals:
┌──────────────┬──────────┬──────────────┬──────────────────────────────┐
│ Percentile   │    Hours │   Working Days │ Date                       │
├──────────────┼──────────┼──────────────┼──────────────────────────────┤
│ P50          │    85.93 │           11 │ 2026-05-19                   │
│ P80          │    99.93 │           13 │ 2026-05-21                   │
│ P90          │   106.82 │           14 │ 2026-05-22                   │
└──────────────┴──────────┴──────────────┴──────────────────────────────┘

Sprint Planning Summary:
┌──────────────────────────────┬──────────────────────────────────────┐
│ Field                        │ Value                                │
├──────────────────────────────┼──────────────────────────────────────┤
│ Sprint Length                │ 2 weeks                              │
│ Planning Confidence Level    │ 85%                                  │
│ Removed Work Treatment       │ reduce_backlog                       │
│ Velocity Model               │ empirical                            │
│ Planned Commitment Guidance  │ 7.13                                 │
│ Historical Sampling Mode     │ matching_cadence                     │
│ Historical Observations      │ 4                                    │
│ Carryover Mean               │ 2.10                                 │
│ Aggregate Spillover Rate     │ 0.1981                               │
│ Observed Disruption Frequency │ 0.7250                              │
└──────────────────────────────┴──────────────────────────────────────┘

Sprint Count Statistical Summary:
┌─────────────────┬────────────────────────────────────────────────────┐
│ Metric          │ Value                                              │
├─────────────────┼────────────────────────────────────────────────────┤
```

Key observations:

- `future_sprint_overrides` model known upcoming capacity reductions.
- `volatility_overlay` adds random sprint-level disruption.
- `spillover` plus `spillover_probability_override` let larger items partially carry into later sprints.

17.9.3 Tasks mode and external history

Tasks mode is useful for service or maintenance backlogs where items are intentionally kept to similar size.

```yaml
project:
  name: "Sprint Planning Tasks Mode"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Bug fix A"
    estimate:
      low: 4
      expected: 6
      high: 8
  - id: "task_002"
    name: "Bug fix B"
    estimate:
      low: 4
      expected: 6
      high: 8
  - id: "task_003"
    name: "Bug fix C"
    estimate:
      low: 4
      expected: 6
      high: 8
  - id: "task_004"
    name: "Bug fix D"
    estimate:
      low: 4
      expected: 6
      high: 8
  - id: "task_005"
    name: "Bug fix E"
    estimate:
      low: 4
      expected: 6
      high: 8
  - id: "task_006"
    name: "Bug fix F"
    estimate:
      low: 4
      expected: 6
      high: 8

sprint_planning:
  enabled: true
  sprint_length_weeks: 1
  capacity_mode: tasks
  planning_confidence_level: 0.8
  history:
    - sprint_id: "SPR-001"
      completed_tasks: 4
      spillover_tasks: 1
    - sprint_id: "SPR-002"
      completed_tasks: 5
      spillover_tasks: 0
      added_tasks: 1
    - sprint_id: "SPR-003"
      completed_tasks: 4
      spillover_tasks: 1
      removed_tasks: 1
```

```
mcprojsim simulate examples/sprint_planning_tasks.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Sprint Planning Tasks Mode
  Start Date: 2026-05-04
  Number of Tasks: 6
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 6
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 7.03 hours (1 working days)
  Median (P50): 7.06 hours
  Std Dev: 0.46 hours
  Minimum: 5.35 hours
  Maximum: 7.99 hours

Project Effort Statistical Summary:
  Mean: 35.99 person-hours (5 person-days)
  Median (P50): 35.98 person-hours
  Std Dev: 1.98 person-hours
  Minimum: 27.99 person-hours
  Maximum: 43.33 person-hours

Calendar Time Confidence Intervals:
  P50: 7.06 hours (1 working days)  (2026-05-05)
  P80: 7.46 hours (1 working days)  (2026-05-05)
  P90: 7.63 hours (1 working days)  (2026-05-05)

Sprint Planning Summary:
Sprint Length: 1 weeks
Planning Confidence Level: 80%
Removed Work Treatment: churn_only
Velocity Model: empirical
Planned Commitment Guidance: 2.28
Historical Sampling Mode: matching_cadence
Historical Observations: 3
Carryover Mean: 0.00
Aggregate Spillover Rate: 0.0000
Observed Disruption Frequency: 0.0000

Sprint Count Statistical Summary:
Mean: 2.00 sprints
Median (P50): 2.00 sprints
Std Dev: 0.00 sprints
Minimum: 2.00 sprints
Maximum: 2.00 sprints
Coefficient of Variation: 0.0000

Sprint Count Confidence Intervals:
  P50: 2 sprints  (2026-05-11)
  P80: 2 sprints  (2026-05-11)
  P90: 2 sprints  (2026-05-11)
```

The same sprint-planning workflow can load history from external data files:

```yaml
project:
  name: "Sprint Planning External JSON"
  start_date: "2026-05-04"
  confidence_levels: [50, 80, 90]

tasks:
  - id: "task_001"
    name: "Foundation"
    estimate:
      story_points: 3
  - id: "task_002"
    name: "API"
    estimate:
      story_points: 5
  - id: "task_003"
    name: "UI"
    estimate:
      story_points: 8
  - id: "task_004"
    name: "Release"
    estimate:
      story_points: 3

sprint_planning:
  enabled: true
  sprint_length_weeks: 2
  capacity_mode: story_points
  history:
    format: json
    path: sprint_planning_history.json
```

```
{
  "metricDefinitions": {
    "committed_StoryPoints": "Sum of story points for issues present at sprint start; issues added during the sprint are exclude
    "completed_StoryPoints": "Sum of story points for issues completed during the sprint.",
    "addedIntraSprint_StoryPoints": "Sum of story points for issues added during the sprint.",
    "removedInSprint_StoryPoints": "Sum of story points removed from the sprint during execution.",
    "spilledOver_StoryPoints": "Sum of story points not completed at sprint close."
  },
  "sprints": [
    {
      "sprintUniqueID": "2026:Q1 Sprint 1",
      "startDate": "2026-01-06T09:00:00.000+01:00",
      "endDate": "2026-01-20",
      "metrics": {
        "committed_StoryPoints": 95.0,
        "completed_StoryPoints": 78.0,
        "addedIntraSprint_StoryPoints": 12.0,
        "removedInSprint_StoryPoints": 6.0,
        "spilledOver_StoryPoints": 11.0
      }
    },
    {
      "sprintUniqueID": "2026:Q1 Sprint 2",
      "endDate": "2026-02-03",
      "metrics": {
        "committed_StoryPoints": 88.0,
        "completed_StoryPoints": 82.0
      }
    },
    {
      "sprintUniqueID": "2026:Q1 Sprint 3",
      "metrics": {
        "committed_StoryPoints": 91.0,
        "completed_StoryPoints": 85.0,
        "spilledOver_StoryPoints": 4.0
      }
    }
  ]
}
```

```
[project]
name = "Sprint Planning External CSV"
start_date = "2026-05-04"
confidence_levels = [50, 80, 90]

[sprint_planning]
enabled = true
sprint_length_weeks = 1
capacity_mode = "story_points"

[sprint_planning.history]
format = "csv"
path = "sprint_planning_history.csv"

[[tasks]]
id = "task_001"
name = "Support ticket A"
planning_story_points = 3

[tasks.estimate]
low = 4
expected = 6
high = 8

[[tasks]]
id = "task_002"
name = "Support ticket B"
planning_story_points = 3

[tasks.estimate]
low = 4
expected = 6
high = 8

[[tasks]]
id = "task_003"
name = "Support ticket C"
planning_story_points = 3

[tasks.estimate]
low = 4
expected = 6
high = 8

[[tasks]]
id = "task_004"
name = "Support ticket D"
planning_story_points = 3

[tasks.estimate]
low = 4
expected = 6
high = 8

[[tasks]]
id = "task_005"
name = "Support ticket E"
planning_story_points = 3

[tasks.estimate]
low = 4
expected = 6
high = 8
```

```
sprintUniqueID,committed_StoryPoints,completed_StoryPoints,addedIntraSprint_StoryPoints,removedInSprint_StoryPoints,spilledOver_
2026:Q2 Sprint 1,32,28,4,2,2,2026-05-04T09:00:00.000+02:00,2026-05-11T09:00:00.000+02:00
2026:Q2 Sprint 2,30,29,3,1,1,2026-05-11T09:00:00.000+02:00,2026-05-18T09:00:00.000+02:00
```

Natural-language input also supports sprint-planning sections:

```
Project: Sprint Planning from Text
Start date: 2026-05-04
Task 1:
- Discovery
- Story points: 3
Task 2:
- API implementation
- Story points: 5
Task 3:
- Frontend integration
- Story points: 8
Sprint planning:
- Sprint length: 2
- Capacity mode: story points
- Planning confidence level: 80%
Sprint history SPR-001:
- Done: 10 points
- Carryover: 1 points
Sprint history SPR-002:
- Done: 9 points
- Carryover: 2 points
- Scope added: 1 points
```

```
mcprojsim generate examples/sprint_planning_nl.txt -o .build/gen-examples/sprint_planning_nl.yaml
mcprojsim simulate .build/gen-examples/sprint_planning_nl.yaml --minimal --seed 42
```

```
mcprojsim v0.15.2

Project Overview:
  Project: Sprint Planning from Text
  Start Date: 2026-05-04
  Number of Tasks: 3
  Effective Default Distribution: triangular
  T-Shirt Category Used: story
  Hours per Day: 8.0
  Max Parallel Tasks: 3
  Schedule Mode: dependency_only

Calendar Time Statistical Summary:
  Mean: 74.67 hours (10 working days)
  Median (P50): 72.60 hours
  Std Dev: 16.50 hours
  Minimum: 40.24 hours
  Maximum: 118.79 hours

Project Effort Statistical Summary:
  Mean: 142.54 person-hours (18 person-days)
  Median (P50): 141.08 person-hours
  Std Dev: 19.49 person-hours
  Minimum: 90.20 person-hours
  Maximum: 212.98 person-hours

Calendar Time Confidence Intervals:
  P50: 72.60 hours (10 working days)  (2026-05-18)
  P80: 89.64 hours (12 working days)  (2026-05-20)
  P90: 98.61 hours (13 working days)  (2026-05-21)
  P95: 105.02 hours (14 working days)  (2026-05-22)

Sprint Planning Summary:
Sprint Length: 2 weeks
Planning Confidence Level: 80%
Removed Work Treatment: churn_only
Velocity Model: empirical
Planned Commitment Guidance: 7.15
Historical Sampling Mode: matching_cadence
Historical Observations: 2
Carryover Mean: 0.00
Aggregate Spillover Rate: 0.0000
Observed Disruption Frequency: 0.0000

Sprint Count Statistical Summary:
Mean: 2.00 sprints
Median (P50): 2.00 sprints
Std Dev: 0.00 sprints
Minimum: 2.00 sprints
Maximum: 2.00 sprints
Coefficient of Variation: 0.0000

Sprint Count Confidence Intervals:
  P50: 2 sprints  (2026-05-18)
  P80: 2 sprints  (2026-05-18)
  P90: 2 sprints  (2026-05-18)
```

17.10 Running Examples

17.10.1 Common CLI options

```
# Basic simulation
mcprojsim simulate examples/quickstart_example.yaml

# Minimal output for quick overview
mcprojsim simulate examples/sample_project.yaml --minimal

# Reproducible results with a seed
mcprojsim simulate examples/sample_project.yaml --seed 42

# More iterations for higher accuracy
mcprojsim simulate examples/sample_project.yaml --iterations 50000

# With custom config (uncertainty factors, output settings)
mcprojsim simulate examples/sample_project.yaml \
  --config examples/sample_config.yaml

# Tabular output format
mcprojsim simulate examples/sample_project.yaml --table

# Check probability of hitting a target date
mcprojsim simulate examples/sample_project.yaml \
  --target-date 2026-03-15

# Show top 5 critical path sequences
mcprojsim simulate examples/sample_project.yaml \
  --critical-paths 5

# Export to multiple formats
mcprojsim simulate examples/sample_project.yaml \
  -f json,csv,html -o results/portal
```

17.10.2 Additional example files

File	Demonstrates
`quickstart_example.yaml`	Basic explicit estimates
`tshirt_sizing_project.yaml`	T-shirt size estimation with risks
`story_points_walkthrough_project.yaml`	Story point estimation
`sample_project.yaml`	Complex project with risks and dependencies
`project_with_custom_thresholds.yaml`	Custom probability thresholds
`team_size_demo_base.yaml`	Dependency-only baseline
`team_size_demo_with_team_size.yaml`	`team_size` constrained scheduling
`resource_cap_small_task.yaml`	Auto-capping on short tasks
`resource_cap_large_task.yaml`	Global coordination cap on large tasks
`sprint_planning_minimal.yaml`	Minimal story-point sprint forecast
`sprint_planning_advanced.yaml`	Advanced sprint forecast with spillover, volatility, and overrides
`sprint_planning_tasks.yaml`	Throughput-style sprint planning in `tasks` mode
`sprint_planning_external_json.yaml`	YAML project loading external JSON sprint history
`sprint_planning_external_csv.toml`	TOML project loading external CSV sprint history
`sprint_planning_nl.txt`	Natural-language sprint planning input
`nl_example.txt`	Natural language input (basic)
`nl_constrained_example.txt`	Natural language input with resources
`sample_config.yaml`	Custom configuration file